

ANÁLISIS DEL IMPACTO DE LAS ACCIONES DEL JUGADOR EN UN VIDEOJUEGO Y ADAPTACIÓN DEL ENTORNO Y LA NARRATIVA MEDIANTE APRENDIZAJE MÁQUINA



TRABAJO DE FIN DE GRADO

MIGUEL RAMÍREZ CASTRILLO
ROI QUINTAS DIZ
DAVID RIVERA MARTÍNEZ
RAÚL PÉREZ COGOLLUDO
CLAUDIA ZARZUELA AMOR

DIRECTOR
CARLOS LEÓN AZNAR

JUNIO 2025

GRADO EN DESARROLLO DE VIDEOJUEGOS
FACULTAD DE INFORMÁTICA
UNIVERSIDAD COMPLUTENSE DE MADRID

ANALYSIS OF THE IMPACT OF PLAYER ACTIONS IN A VIDEO GAME AND ADAPTATION OF THE ENVIRONMENT AND NARRATIVE THROUGH MACHINE LEARNING



FINAL DEGREE PROJECT

MIGUEL RAMÍREZ CASTRILLO
ROI QUINTAS DIZ
DAVID RIVERA MARTÍNEZ
RAÚL PÉREZ COGOLLUDO
CLAUDIA ZARZUELA AMOR

DIRECTOR
CARLOS LEÓN AZNAR

JUNE 2025

BACHELOR OF COMPUTER SCIENCE IN VIDEO GAME DEVELOPMENT
COMPUTER SCIENCE FACULTY
UNIVERSIDAD COMPLUTENSE DE MADRID

DEDICATORIA

A mis abuelos y a mis padres, os quiero dedicar este trabajo por todo el cariño y apoyo incondicional que siempre me habéis mostrado. A mis hermanos, por todos los buenos momentos que hemos pasado juntos. Y a Laura, por ser el fuego que arde contra el frío y la luz que trae el amanecer, gracias por aparecer en mi vida.

Miguel

A mis padres, por educarme con curiosidad, esfuerzo y creatividad y por brindarme oportunidades que jamás imaginé. A mi hermana y Verónica, por ser mi otra mitad, mi apoyo y refugio constante en cada paso del camino. Y a Andrea, por hacerme ver la vida como una serie animada constante, llena de color, aventura y humor.

Claudia

A mis padres, que siempre han estado durante los momentos difíciles. A mi hermano por su constante apoyo durante mi etapa viviendo en Madrid. A mi hermana por llenarnos de alegría y nuevas experiencias. Y a todas las personas que han formado parte de mi vida los últimos 4 años, aportándome una nueva forma de ver la vida.

Roi

A mis padres, por educarme con valores, enseñarme a ser una persona buena y trabajadora, y por brindarme siempre su cariño y apoyo incondicional. A mis hermanos, por ser otro pilar en el que siempre puedo apoyarme. Y a Juli, por hacer que cada uno de mis días brille, llenándolos de risas, amor y aventuras. Gracias por caminar a mi lado, por sostenerme en cada paso y por ser parte esencial de mi vida.

Raúl

A ti, Berto, quien descubriste las pasiones que forman ahora mi vida. A ti, Noa, que portas la luz que ilumina mis caminos. Y, a ellos, que me habéis acompañado, formado y querido estos 4 años.

David

AGRADECIMIENTOS

Gracias a Carlos León Aznar, por resultar un mentor y tutor atento, respetuoso y que nos brindaba conocimientos. A Alejandro Villar Rubio, por prestar su ayuda y atención durante la fase inicial del proyecto.

A todos los participantes de nuestras pruebas de usuario, que completan, fijan y resuelven este trabajo dedicando su tiempo y esfuerzo.

RESUMEN

Este trabajo explora la integración de modelos largos de lenguaje (LLMs, por sus siglas en inglés) en videojuegos para generar entornos y narrativas reactivas en función de las acciones del jugador. El objetivo principal ha sido diseñar e implementar un sistema capaz de interpretar interacciones con objetos o personajes no jugadores (NPC, por sus siglas en inglés) y generar respuestas estructuradas que modifiquen dinámicamente el entorno de juego. La herramienta desarrollada en Unreal Engine se conecta con un LLM mediante un sistema de comunicación en Python. Durante el desarrollo se evaluaron técnicas con Retrieval Augmented Generation (RAG) y Reflexion, aunque fueron descartadas por su elevado coste computacional, viabilidad baja y aumento considerable de latencia. Finalmente, se realizó una prueba con usuarios para valorar tanto la coherencia de las respuestas del modelo como la experiencia de inmersión. El trabajo demuestra el potencial de los LLMs para democratizar el diseño de videojuegos reactivamente complejos, incluso en entornos con recursos limitados. Este sistema no busca sustituir el trabajo de diseñadores o desarrolladores, sino ser una herramienta complementaria que amplíe sus capacidades creativas, facilitando la creación de escenarios reactivos mediante descripciones simples. También sirve como apoyo para detectar rutas no previstas y proporcionar retroalimentación contextual al jugador, manteniendo siempre la necesidad de una supervisión profesional y un diseño bien estructurado.

Acceso al repositorio: <https://github.com/NILGroup/TFG-2425-InteractionLLM>

Palabras clave

LLM, videojuegos, IA generativa, Prompt Engineering, experiencia de usuario, Unreal Engine, reactividad

ABSTRACT

This work explores the integration of large language models (LLMs) in videogames to generate reactive environments and narratives based on player actions. The main goal was to design and implement a system capable of interpreting interactions with objects or non-player characters (NPCs) and generating structured responses that dynamically modify the game world. The tool, developed in Unreal Engine, is connected to a LLM through a communication system built in Python. During development, techniques such as Retrieval Augmented Generation (RAG) and Reflexion were evaluated, but ultimately discarded due to high computational cost, low feasibility, and increased latency. A user test was conducted to evaluate both the coherence of the model's responses and the immersion experience. This project demonstrates the potential of LLMs to democratize the design of reactively complex video games, even in resource-constrained environments. The system is not intended to replace designers or developers but rather to serve as a complementary tool that enhances their creative capabilities, easing the creation of reactive scenarios using simple descriptions. It also supports the identification of unforeseen pathways and provides contextual feedback to players, always requiring professional supervision and a well-structured design.

Access to the repository: <https://github.com/NILGroup/TFG-2425-InteractionLLM>

Keywords

LLM, video games, generative AI, Prompt Engineering, user experience, Unreal Engine, reactivity

ÍNDICE DE CONTENIDOS

Capítulo 1 - Introducción	1
1.1 Motivación.....	2
1.2 Hipótesis	3
1.3 Objetivos	3
1.4 Plan de trabajo	5
1.5 Metodología.....	6
1.6 Estructura del documento	8
Capítulo 2 - Estado de la cuestión.....	9
2.1 Historia y avances en NLP	9
2.2 Funcionamiento básico de un modelo largo de lenguaje (LLM).....	10
2.2.1 <i>Embeddings y Positional Encoding</i>	10
2.2.2 <i>Encoder-Decoder</i>	12
2.2.3 <i>Atención</i>	13
2.3 Técnicas para adaptación de respuesta según un contexto dado	15
2.3.1 <i>Prompt Engineering</i>	17
2.3.2 <i>Retrieval Augmented Generation (RAG)</i>	19
2.3.3 <i>Reflexion</i>	20
2.4 LLMs en videojuegos: trabajo actual	21
Capítulo 3 - Diseño del sistema de interacción con el entorno y comunicación semántica con el LLM	27
3.1 Arquitectura general	27
3.1.1 <i>Estructura inicial de archivos JSON</i>	27
3.1.2 <i>Adaptación del Sistema a Limitaciones Técnicas</i>	29
3.2 Tipología general de interacciones	30
3.3 Movimiento	31
3.3.1 <i>Solución de diseño</i>	32
3.4 Manipulación e interacción combinatoria de objetos y NPCs	32
3.4.1 <i>Solución de diseño</i>	32

3.4.2 Señales.....	35
3.5 Comunicación con NPCs.....	36
3.5.1 Solución de diseño.....	36
3.5.2 Comportamiento de los NPCs	37
Capítulo 4 - Implementación del sistema de interacción basado en LLM.....	38
4.1 Entorno de desarrollo de la aplicación	38
4.2 Investigación y elección de LLMs.....	39
4.2.1 Evaluación de los modelos de lenguaje	40
4.2.2 Dimensiones clave para la comparación de LLMs	41
4.2.3 Selección de LLMs candidatos.....	42
4.3 Conexión entre el LLM y Unreal Engine	51
4.3.1 Implementación en Unreal Engine.....	52
4.3.2 Subsistema LLM: Implementación en Python	54
4.4 Sistema de creación de prompts	57
4.4.1 Planteamiento y solución	58
4.4.2 Funcionamiento y creación de módulos del PromptManager	58
4.4.3 Módulos implementados.....	60
4.5 Manejo de respuestas del LLM	63
4.5.1 Planteamiento y solución	64
4.5.2 Funcionamiento y creación de módulos del GlobalManager	64
4.5.3 Módulos implementados.....	67
4.6 Máquinas de Estado.....	68
4.6.1 Implementación con Logic Driver Lite	69
4.6.2 Funcionamiento de Máquinas de Estado con un LLM.....	71
4.7 Gestión dinámica de objetos interactivos	72
4.7.1 StateManager	73
4.7.2 Recoger la acción realizada y construir el mensaje	73
4.7.3 Aplicar los cambios en la escena	74
4.8 Sistema de componentes.....	76
4.8.1 Diseño de la composición de funcionalidades	76

4.8.2 Estructura y uso de los componentes	78
4.8.3 Casos de uso.....	79
4.9 Reconocimiento de Voz	79
4.10 Texto a Voz	83
4.11 Prompt Engineering.....	84
4.11.1 Prompts Usados	85
4.12 Adecuación de respuestas al formato requerido	95
4.12.1 Retrieval Augmented Generation	95
4.12.2 Reflexion.....	101
4.13 Interfaz de usuario	103
Capítulo 5 - Diseño de experimentos	108
5.1 Diseño de nivel para el entorno de pruebas	108
5.1.1 Descripción de la escena.....	108
5.1.2 Flujo esperado.....	113
5.1.3 Diseño de comportamientos mediante máquinas de estado	116
5.2 Diseño de las pruebas	122
5.2.1 Objetivo	123
5.2.2 Cuestionario inicial.....	124
5.2.3 Observación	125
5.2.4 Entrevista	126
5.2.5 Cuestionario final.....	127
Capítulo 6 - Análisis de resultados	129
6.1.1 Conclusiones	143
Capítulo 7 - Discusión.....	145
Capítulo 8 - Conclusiones y trabajo futuro	148
8.1 Trabajo futuro.....	150
Introduction.....	153
Contribuciones Personales	165
Bibliografía	175
Apéndices	177

ÍNDICE DE FIGURAS

Figura 1.1 Diagrama de Gantt.....	6
Figura 2.1 Diagrama de positional encoding y embedding.....	12
Figura 2.2 Arquitectura Transformer.	14
Figura 2.3 Ejemplo de una interacción con ChatGPT.....	16
Figura 2.4 Ejemplo de prompt con ChatGPT.	17
Figura 3.1 Comunicación entre Unreal y LLM.....	27
Figura 3.2 Diseño de estructura inicial.	29
Figura 3.3 Diseño ajustado de la estructura del prompt.....	30
Figura 3.4 Diagrama de flujo de interacciones.	35
Figura 3.5 Diagrama de flujo de los diálogos con NPCs.	37
Figura 4.1 Ajustes de la comunicación con el LLM.	54
Figura 4.2 Diagrama de flujo para crear y enviar un prompt al LLM.....	59
Figura 4.3 Flujo de creación de un prompt de la función CreatePrompt.	60
Figura 4.4 Diagrama de flujo del procesamiento de las respuestas del LLM.	66
Figura 4.5 Ejemplo de máquina de estado desde Blueprints.	70
Figura 4.6 Ejemplo de una condición de transición de máquinas de estado asociada al estado Open.	71
Figura 4.7 Eventos asociados a los estados de una máquina de estados.....	71
Figura 4.8 Diagrama de flujo del procesamiento de la respuesta de una interacción entre objetos no esenciales.....	73
Figura 4.9 UML de ejemplo de herencia.	77
Figura 4.10 Diagrama de flujo de la respuesta del LLM acerca de los componentes.....	79
Figura 4.11 Diagrama de flujo del funcionamiento del sistema de reconocimiento automático de voz.....	83
Figura 4.12 Ventana de entrada para el diálogo con NPCs.....	103
Figura 4.13 Ventana de entrada para interacciones en modo manual.	103
Figura 4.14 Indicador del procesamiento del modelo de lenguaje.....	104
Figura 4.15 Iluminación de nombres de NPCs en rango de escucha.	104
Figura 4.16 Borde de objetos interactivos sujetables.....	105

Figura 4.17 Borde de objetos interactivos no sujetables.....	105
Figura 4.18 Borde de NPCs.	105
Figura 4.19 Ventana de descripción de objetos	106
Figura 4.20 Interfaz de NPCs en proceso de interacción.	106
Figura 4.21 Interfaz de objetos interactivos en proceso de interacción.	107
Figura 4.22 Conjunto de elementos del interfaz.	107
Figura 5.1 Vista inicial de la escena.	109
Figura 5.2 Celda inicial.....	109
Figura 5.3 Celda de Antonio, contigua a la inicial.....	110
Figura 5.4 Celdas en frente de la del jugador.....	110
Figura 5.5 Pasillo inicial.	111
Figura 5.6 Recepción y armería.	111
Figura 5.7 Recepción y puerta de salida.	112
Figura 5.8 Armería.....	112
Figura 5.9 Tarjeta para la puerta de salida.	113
Figura 5.10 Código para la puerta electrónica.	114
Figura 5.11 Muro frágil que permite escapar de la cárcel.....	115
Figura 5.12 Puerta de la celda del jugador.....	121
Figura 5.13 Puerta electrónica	122
Figura 5.14 Puerta final.....	122
Figura 6.1 Resultados del cuestionario inicial acerca de las horas jugadas semanalmente.	129
Figura 6.2 Resultados del cuestionario inicial acerca de la familiaridad con juegos sandbox.....	130
Figura 6.3 Resultados del cuestionario inicial acerca de la familiaridad con juegos de rol.....	130
Figura 6.4 Gráfica de escape de la cárcel por parte de los usuarios.....	131
Figura 6.5 Evaluación de los usuarios sobre el tiempo de respuesta de una interacción.	135
Figura 6.6 Evaluación de los usuarios sobre las reacciones del LLM.	137
Figura 6.7 Evaluación de los usuarios sobre los diálogos con los NPCs.	138
Figura 6.8 Evaluación de los usuarios sobre la generación de mallas.	139
Figura 6.9 Evaluación de los usuarios sobre la generación de sonidos.....	139
Figura 6.10 Evaluación de los usuarios sobre la inmersión durante la prueba.	140

Figura 6.11 Evaluación de los usuarios sobre las mecánicas de juego.	141
Figura 6.12 Evaluación de los usuarios sobre las reacciones del LLM respecto a sus expectativas.	142
Figura 6.13 Evaluación de los usuarios sobre la diversión en esta experiencia.	144
Figure 8.1 Gantt Diagram	158

ÍNDICE DE TABLAS

Tabla 2.1 Ejemplo de vector embedding.	11
Tabla 2.2 Ejemplo de respuestas con Mistral:7B.....	17
Tabla 4.1 Plantilla de evaluación de LLMs 1/2.	41
Tabla 4.2 Plantilla de evaluación de LLMs 2/2.	42
Tabla 4.3 Comparativa de características de los modelos de lenguaje evaluados.	43
Tabla 4.4 Evaluación de LLaMA 3.1 1/2.....	44
Tabla 4.5 Evaluación de LLaMA 3.1 2/2.....	45
Tabla 4.6 Evaluación de Granite 3.2 1/2.....	46
Tabla 4.7 Evaluación de Granite 3.2 2/2.....	47
Tabla 4.8 Evaluación de OLMo-7B 1/2.....	48
Tabla 4.9 Evaluación de OLMo-7B 2/2.....	49
Tabla 4.10 Comparativa entre modelos DeepSeek.	50
Tabla 4.11 Modelos disponibles de la IA Whisper para procesamiento de voz.	80
Tabla 5.1 Plantilla de observación para las pruebas de usuario.	126

Capítulo 1 - Introducción

La Inteligencia Artificial (IA) generativa ha cobrado una gran relevancia internacional desde el lanzamiento al público de ChatGPT¹, siendo este uno de los temas más discutidos a diario por miles o incluso millones de personas ajenas a conocer su funcionamiento. Este lanzamiento ha despertado una ola de investigación y desarrollo en todo el mundo con el objetivo de probar los límites de los modelos largos de lenguaje (LLMs por sus siglas en inglés, de Large Language Models) para múltiples fines. Se ha podido observar cómo, en cuestión de meses, prácticamente todos los gigantes tecnológicos han desarrollado interfaces con acceso a una IA generativa, plagando así las aplicaciones más utilizadas de asistentes virtuales con capacidad de responder preguntas generalistas o específicas respecto a la aplicación.

Por supuesto, el sector del entretenimiento no se ha quedado atrás: ya existen múltiples videojuegos que utilizan la generación de lenguaje natural para la interacción con los habitantes de su entorno virtual. Un ejemplo sería *Suck Up!*², un videojuego que permite al jugador llevar el papel de un vampiro en un humilde pueblo. Como dicen las leyendas populares, estos seres no son capaces de entrar en las casas de sus víctimas hasta que no son invitados explícitamente; por lo tanto, el objetivo del jugador es establecer conversaciones con los vecinos del pueblo, intentando mentir para conseguir entrar en sus casas y alimentarse de ellos. Todas las conversaciones son llevadas por un LLM, lo que genera miles de situaciones emergentes. También permiten generar nuevos personajes con descripciones y apariencias únicas. Otro ejemplo destacado en la industria es *Cygnus Enterprise*³, el cual utiliza IA generativa para los diálogos con los personajes no jugadores (NPCs por sus siglas en inglés, de Non-Player Character) del entorno. Lo más destacable es que estos NPCs no solo participan en conversaciones, sino que también pueden comprender instrucciones directas del jugador, como solicitar ayuda en combates específicos o indicar tareas a sus aliados.

Las posibilidades que ofrecen al usuario este tipo de modelos se acercan a algo soñado dentro de un videojuego: la sensación de libertad completa, donde las acciones realmente tengan unas consecuencias notables en el entorno virtual.

Sin embargo, las aplicaciones actuales se han centrado principalmente en la interacción con NPCs, ofreciendo diálogos naturales, complejos y en tiempo real. Aunque este enfoque resulta

¹ ChatGPT: <https://openai.com/chatgpt/overview/>

² SuckUp!: <https://www.playsuckup.com/>

³ Cygnus Enterprises: https://store.steampowered.com/app/1963520/Cygnus_Enterprises/

especialmente atractivo, el verdadero potencial de los LLMs va más allá de la simple generación de texto. Su capacidad para comprender entornos descritos y procesar grandes volúmenes de información plantea una cuestión clave: ¿se puede aplicar esta tecnología a otros ámbitos dentro del desarrollo de videojuegos?

Más allá de su integración en videojuegos, los LLMs están adquiriendo un papel fundamental en la investigación a escala global. Desde la publicación del artículo *Attention Is All You Need* [1], estos modelos han sido aplicados a múltiples contextos, demostrando su versatilidad y efectividad. En dicha publicación se presentaban resultados sobresalientes en tareas de traducción automática —por ejemplo, del inglés al francés o al alemán—, superando a modelos anteriores en precisión y eficiencia. Actualmente, su uso se está expandiendo a otros ámbitos como la medicina, donde Kanithi et al [2] proponen una serie de pruebas para evaluar el rendimiento de los LLMs en entornos clínicos. Asimismo, Scarlatos et al [3] exploran su aplicación en el ámbito educativo, proponiéndolos como herramientas de apoyo para la tutorización personalizada de estudiantes.

En este contexto, el proyecto de I+D+i DARK NITE, PID2023-146308OB-I00, financiado por la Agencia Estatal de Investigación (código MICIU/AEI/10.13039/501100011033/) y FEDER de la Unión Europea plantea el desarrollo de un sistema capaz de generar estructuras narrativas dinámicas mediante el uso de uno o varios LLMs como motor principal de procesamiento. Este trabajo se ha llevado a cabo dentro del marco de este proyecto de investigación. Estos modelos se integran en un entorno interactivo, pudiendo comunicarse con personajes a través de lenguaje natural e interactuar con elementos del entorno mediante controladores. De este modo, el proyecto busca desarrollar herramientas capaces de recibir nuevos objetivos definidos por un supervisor humano, al mismo tiempo que interactúan con el usuario y se adaptan dinámicamente a sus acciones dentro del entorno interactivo. Esta línea de investigación resulta especialmente relevante para el enfoque planteado en este trabajo.

1.1 Motivación

Según el contexto anterior, se consideró que los LLMs pueden convertirse en aliados clave para facilitar el diseño de la reactividad en los videojuegos, más allá del ámbito de los diálogos. Este tipo de interacciones donde tenemos entornos tan reactivos a todo el elenco de las acciones del jugador tienen un coste de producción muy alto. Un perfecto ejemplo de esto es *Baldur's Gate 3*⁴, el cual diseña una experiencia con posibilidades prácticamente ilimitadas en sus interacciones, pero su tiempo y coste de desarrollo —además de un numeroso equipo de personal con

⁴ *Baldur's Gate 3*: <https://baldursgate3.game/>

experiencia— son inalcanzables para la mayor parte de la industria, conformada por empresas menores, centradas en desarrollos cortos y/o medios.

Todavía existe un vacío notable en el diseño de experiencias que respondan de forma significativa a las acciones del jugador más allá del diálogo, especialmente en contextos tridimensionales o bidimensionales interactivos. En este tipo de entornos, la interacción con objetos o sistemas del mundo virtual suele estar limitada a comportamientos previamente programados, lo que restringe la expresividad y la sensación de agencia del jugador. A diferencia de las aventuras basadas en texto, donde el control del modelo sobre la experiencia es casi total, los videojuegos con componente visual y sonoro requieren respuestas más estructuradas y adaptadas, lo que plantea nuevos retos técnicos, pero también oportunidades muy interesantes. La combinación entre la creatividad del jugador y la capacidad generativa de los modelos de lenguaje garantiza la aparición de respuestas dinámicas, capaces de mantener la atención, el interés y la inmersión del usuario.

Todo este contexto motiva el desarrollo de este trabajo, donde la incorporación de inteligencia artificial, capaz de comprender el contexto y generar respuestas coherentes, permite acelerar significativamente los tiempos de desarrollo y hace más accesibles las experiencias reactivas complejas que tanto demanda la industria del videojuego.

1.2 Hipótesis

Dado que el diseño actual de videojuegos demanda experiencias cada vez más inmersivas, resulta fundamental investigar cómo la inteligencia artificial puede facilitar a los desarrolladores la creación de sistemas reactivamente complejos. La hipótesis de este trabajo es la siguiente:

La inclusión de modelos de inteligencia artificial, con capacidad para comprender su entorno y reaccionar a él, mejora la experiencia de diseño y desarrollo de la reactividad en videojuegos.

Este enfoque supone que los agentes controlados mediante inteligencia artificial no se limitan a ejecutar comportamientos predefinidos, sino que son capaces de ampliar dinámicamente la narrativa y su repertorio de acciones según el contexto del entorno y las decisiones del jugador. Además, esta capacidad de adaptación no se restringe únicamente a los NPCs, sino que puede extenderse a una inteligencia artificial global encargada de gestionar la reactividad del entorno en su conjunto.

1.3 Objetivos

Se pretende entender cuál es el alcance que puede tener un LLM en la reactividad en videojuegos. Para ello, se plantea analizar qué tipo de interacciones puede interpretar el modelo, cómo reacciona ante ellas y en qué parte del sistema debe integrarse para maximizar su utilidad. Este proceso incluye: definir las tareas concretas que realizará el LLM, evaluar su adecuación al

contexto de uso, y comprobar si las decisiones tomadas durante el diseño e implementación han producido los resultados esperados.

Objetivo general:

Diseñar e implementar un sistema basado en modelos largos de lenguaje capaces de generar reactividad a las acciones de un usuario en un entorno interactivo.

Esta investigación se centra especialmente en el análisis del comportamiento del modelo al recibir información sobre interacciones del jugador con objetos o NPCs del mundo virtual. Las respuestas del LLM deben ajustarse a una estructura predefinida y minimizar las llamadas *alucinaciones*, es decir, errores propios de los modelos de lenguaje que generan contenido incoherente o inventado.

Objetivos específicos:

- Desarrollar una aplicación interactiva en la que el usuario pueda explorar y actuar libremente dentro de un entorno.
- Implementar un sistema de comunicación entre la aplicación y un LLM.
- Analizar y seleccionar el modelo de lenguaje más adecuado en función de los recursos y necesidades del proyecto.
- Adaptar las respuestas del modelo utilizado al formato estructurado requerido para su interpretación por el sistema.
- Desarrollar un sistema de reconocimiento de voz que permita al usuario interactuar verbalmente con el entorno.
- Diseñar un escenario de pruebas en el que múltiples usuarios interactúen con el sistema implementado.
- Realizar un análisis exhaustivo de los resultados obtenidos en las pruebas, considerando tanto la calidad de las respuestas del modelo como la experiencia de los participantes.

El resultado de este trabajo será un análisis detallado de las respuestas generadas por el modelo ante las acciones del usuario. Se evaluará la precisión, coherencia contextual y ausencia de alucinaciones, así como el impacto de dichos factores en la experiencia del jugador. También se tendrá en cuenta la percepción subjetiva de los participantes, valorando su nivel de inmersión y satisfacción general.

Finalmente, las conclusiones extraídas estarán condicionadas por los recursos disponibles durante el desarrollo, aunque se estimarán las posibles mejoras que podrían alcanzarse con un mayor acceso a infraestructura computacional.

1.4 Plan de trabajo

El desarrollo de la investigación se llevará a cabo durante 9 meses. El plan de trabajo sigue una estructura coherente con los objetivos específicos descritos en el apartado anterior, ya que estos marcan las etapas necesarias para la implementación del sistema y su posterior análisis. Teniendo en cuenta esta ventana temporal y los objetivos definidos, el equipo ha establecido la siguiente organización:

Septiembre – Noviembre:

Durante estos primeros meses se realizará una investigación exhaustiva sobre las mejores opciones para comunicación con un modelo largo de lenguaje. Esta etapa contempla el análisis de distintas alternativas en función de variables adaptadas a los recursos disponibles del proyecto. Paralelamente, se desarrollará un primer boceto de la arquitectura del sistema, donde se especificarán mediante esquemas el comportamiento esperado del modelo de lenguaje y las funcionalidades que deberá cubrir. Además, se definirán qué tipos de datos deberá recibir el LLM del entorno y cómo estos datos serán transformados en peticiones adecuadas al modelo.

Diciembre – Enero:

En esta fase se procederá al desarrollo de la aplicación interactiva en tres dimensiones, conforme a la arquitectura previamente diseñada. La primera versión de la aplicación estará sujeta a posibles ajustes derivados de necesidades técnicas durante la implementación. El equipo realizará la infraestructura necesaria para que los diferentes objetos cumplan con los requisitos descritos por la arquitectura. De manera simultánea, se abordará la implementación del sistema de reconocimiento automático de voz.

Febrero:

Este mes será clave para la integración de la aplicación con el modelo de lenguaje, formando así una experiencia completa e interactiva. Se trabajará en garantizar una comunicación fluida entre ambos componentes, siguiendo las especificaciones técnicas de cada uno.

Marzo:

Durante marzo se llevarán a cabo pruebas técnicas del sistema. Se diseñarán escenarios específicos que permitan verificar el correcto funcionamiento de todos los elementos integrados, con el objetivo de detectar posibles errores y refinar la solución antes de su validación final.

Abril:

Esta etapa estará dedicada al diseño y ejecución de pruebas con usuarios. Se presentará el sistema ante un público externo al equipo desarrollador. La recopilación de datos resulta fundamental para la evaluación del proyecto, por lo que el diseño meticuloso de las pruebas —así como del escenario final— adquiere extrema importancia de cara al posterior análisis y validación de los resultados.

Mayo:

Finalmente, el mes de mayo constituye el final de este trabajo, donde se destinará al análisis de los datos recopilados durante las pruebas de usuario y a la redacción final del trabajo. Se documentarán con detalle los procesos de implementación, las dificultades encontradas, los resultados obtenidos y las modificaciones realizadas durante el transcurso del proyecto.

En la *figura 1.1* se muestra un diagrama de Gantt que ilustra gráficamente la planificación temporal descrita.

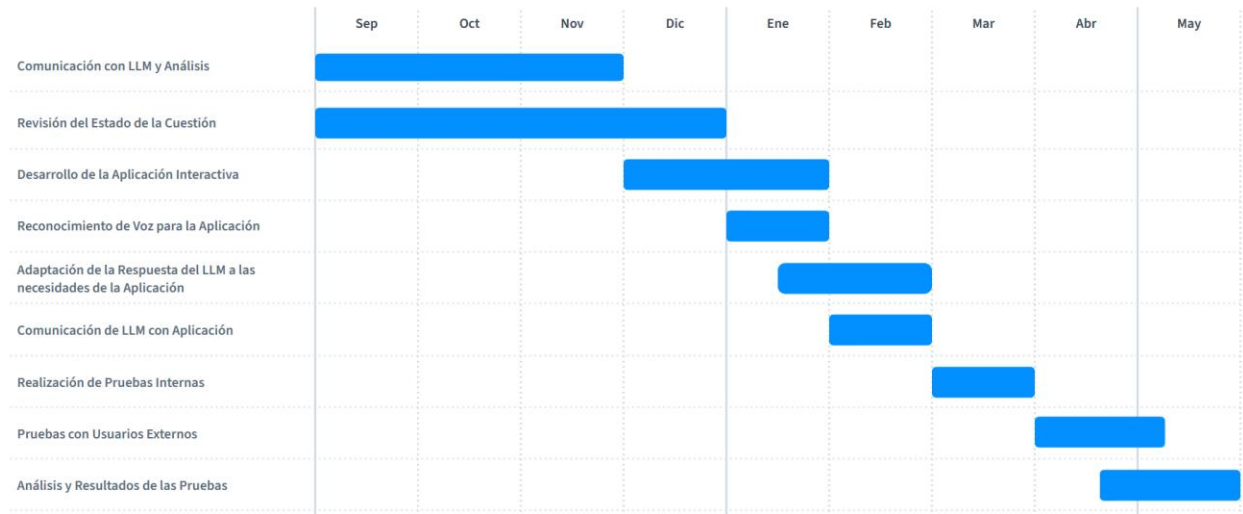


Figura 1.1 Diagrama de Gantt. Representa la secuencia temporal y la duración estimada de las distintas tareas contempladas en el plan de trabajo.

1.5 Metodología

Para el desarrollo de esta investigación, se utilizarán una serie de herramientas de libre acceso para todos los públicos y gratuitas, lo que facilita la reproducibilidad de los resultados obtenidos.

Todo el proceso de desarrollo se llevará a cabo en el sistema operativo Windows 11⁵, dado que es la plataforma con la que el equipo está más familiarizado. Su versatilidad, disponibilidad de recursos didácticos, amplia documentación, y facilidad de uso permiten una implementación ágil. Si bien el resultado del proyecto no depende directamente del sistema operativo, los sistemas desarrollados sí estarán condicionados por el uso de librerías específicas que solo son compatibles con esta plataforma.

⁵ Windows 11: <https://www.microsoft.com/en-us/windows/windows-11>

En lo que respecta a la creación de una aplicación interactiva en tres dimensiones, se utilizará el motor de videojuegos Unreal Engine⁶, en su reciente versión 5.2.1. Este motor destaca por su conjunto de funcionalidades integradas, su extensa comunidad de soporte, y su sistema de programación visual, que facilita el desarrollo rápido manteniendo la legibilidad del código. Unreal Engine es uno de los más utilizados en el entorno profesional de la industria del videojuego, siendo la base de múltiples juegos de éxito.

Para la implementación de la comunicación con un modelo largo de lenguaje, se recurrirá al lenguaje de programación Python⁷, debido a su compatibilidad con diversas bibliotecas y APIs⁸ desarrolladas por distintas entidades. Estas herramientas permiten llevar a cabo iteraciones ágiles en el diseño e integración de LLMs. Entre los modelos candidatos a ser probados se encuentran *DeepSeek*⁹, *LLaMA*¹⁰, *Mistral*¹¹ y *Gemini*¹²; todos ellos de libre utilización a excepción de Gemini, que impone un límite de palabras enviadas al modelo de manera gratuita. Como herramienta complementaria a dichos modelos de lenguaje, se utilizarán interfaces web como los clientes de ChatGPT o DeepSeek para experimentar con distintos formatos de petición, con el objetivo de optimizar la comunicación con los modelos y reducir los tiempos de respuesta, facilitando así un proceso iterativo más eficiente.

Para poder establecer un repositorio común durante el desarrollo donde tener todo el proyecto actualizado para los miembros del equipo, *GitHub*¹³ se muestra como herramienta ideal. Esta herramienta facilita la gestión y el control de repositorios software durante su implementación. Su funcionamiento se basa en *Git*¹⁴, un sistema de control de versiones distribuido, que permite que el código se almacene localmente en todos los equipos con acceso al repositorio remoto. La principal ventaja de utilizar GitHub es su accesibilidad, una interfaz limpia

⁶ Unreal Engine: <https://www.unrealengine.com/en-US>

⁷ Python: <https://www.python.org/>

⁸ API: mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos

⁹ DeepSeek: <https://www.deepseek.com/en>

¹⁰ Llama: <https://www.llama.com/>

¹¹ Mistral: <https://mistral.ai/>

¹² Gemini: <https://gemini.google.com/>

¹³ GitHub: <https://github.com/>

¹⁴ Git: <https://git-scm.com/>

y, especialmente, la experiencia previa del equipo en proyectos de duración media utilizando esta plataforma como eje central del desarrollo colaborativo.

Finalmente, la coordinación del equipo se llevará a cabo mediante reuniones quincenales presenciales. En caso de que algún miembro no pueda asistir físicamente, se realizarán reuniones telemáticas a través de la plataforma *Discord*¹⁵, donde se ha creado un servidor que facilita el almacenamiento y consulta de documentación, enlaces y mensajes, estructurados mediante distintos canales temáticos. Complementariamente, el equipo utilizará la plataforma en línea *Trello*¹⁶ para la organización y seguimiento de tareas, mediante una pizarra virtual que permite visualizar en todo momento el estado del proyecto y la asignación de responsabilidades.

1.6 Estructura del documento

El contenido restante de este documento se organiza de la siguiente manera:

- *Capítulo 2:* Se presenta el estado del arte, centrándose en LLMs, su arquitectura, las principales técnicas empleadas para su tratamiento, así como su aplicación en el ámbito de los videojuegos.
- *Capítulo 3:* Se describe una arquitectura genérica del sistema propuesto, alineada con los objetivos generales del proyecto.
- *Capítulo 4:* Se detallan las especificaciones técnicas de la implementación, incluyendo las herramientas utilizadas y las decisiones adoptadas durante el desarrollo.
- *Capítulo 5:* Se expone el diseño de las pruebas realizadas con usuarios y del escenario creado para las mismas.
- *Capítulo 6:* Se analizan e interpretan los resultados obtenidos de las pruebas con usuarios.
- *Capítulo 7:* Se presenta una discusión acerca de las ventajas y limitaciones del trabajo realizado.
- *Capítulo 8:* Se presentan las conclusiones generales del trabajo, junto con una reflexión final y se esbozan partes clave para el posible desarrollo futuro.

¹⁵ *Discord*: <https://discord.com/>

¹⁶ *Trello*: <https://trello.com/>

Capítulo 2 - Estado de la cuestión

Desde 2022 se ha vivido una revolución en el mundo digital para la mayoría de la gente que disponga de un dispositivo con acceso a internet. La generación de texto por inteligencia artificial es algo que forma ya parte de nuestra vida diaria. Se ve a una gran cantidad de personas confiar en herramientas como ChatGPT para sus estudios, trabajo o para responder preguntas que normalmente se buscarían en Google años atrás; llegando incluso a depender de esta herramienta como fuente de información completamente verídica gracias al formato de su respuesta [4], [5].

Y es que, los modelos largos de lenguaje impresionan por sus resultados. Han sentado un precedente importante en *procesado de lenguaje natural* (NLP, por sus siglas en inglés: Natural Language Processing), siendo la solución por defecto a la grandísima mayoría de problemas que necesiten de generación de texto. Es por ello por lo que en este trabajo se ha utilizado esta tecnología para nuestro objetivo, ya que necesitábamos de generación de texto estructurado; otra de las tareas en la que esta tecnología resulta eficiente [6], [7], [8]. Y, para ello, se revisó la literatura actual concienzudamente, para conseguir resultados a la orden del día.

En este capítulo se puede ver un resumen de la historia del NLP, la arquitectura que siguen estos últimos modelos mencionados, además de las últimas técnicas utilizadas para conseguir centrar a un LLM bajo un objetivo dado como, por ejemplo: que responda según un contexto dado, o que utilice datos locales para sus respuestas. Finalmente, se revisan los últimos trabajos más relevantes con respecto a nuestros objetivos.

2.1 Historia y avances en NLP

El origen del procesamiento del lenguaje natural se remonta a los tiempos de Turing, donde se formula por primera vez la famosa pregunta *¿Pueden pensar las máquinas?* [9]. A partir de entonces, se concebiría como campo formal de investigación y computación la inteligencia artificial [10], donde cada vez más investigadores se sumarían a intentar contestar la pregunta planteada.

Uno de los primeros hitos en NLP fue el desarrollo de ELIZA [11] en 1966, considerada una de las primeras inteligencias artificiales capaz de mantener una conversación coherente con el usuario, simulando el comportamiento de una psicoterapeuta de la época. Utilizaba principalmente inteligencia artificial simbólica, además de un sistema de reglas para generar sus respuestas. Sin embargo, este acercamiento se vería obsoleto durante los próximos años por su escasa escalabilidad y elevado coste, además de no poder alcanzar las expectativas prometidas por entonces.

Durante los años 80s se introdujo la técnica de *backpropagation* [12], lo que permitió a las redes neuronales aprender las estructuras del lenguaje de mejor manera. Surgió como un acercamiento interesante, pero sus limitaciones de pocas bases de datos y escaso poder de

paralelización causó que no tuvieran un gran impacto hasta la década pasada. Gracias al surgimiento de GPUs (Graphics Processing Units) avanzadas con potentes capacidades de paralelización, surgen las redes neuronales profundas [13], dando lugar al campo conocido como *Deep Learning*.

Dentro de este contexto, comienzan a surgir diferentes aplicaciones y técnicas con las que las redes neuronales profundas se establecen como la solución por defecto para problemas de NLP. En la siguiente sección se hace una revisión de la evolución de su utilización, con avances significativos y la razón de los mismos.

2.2 Funcionamiento básico de un modelo largo de lenguaje (LLM)

Un *modelo largo de lenguaje* es un modelo de aprendizaje máquina (ML, por sus siglas en inglés: *machine learning*) diseñado para el procesamiento de lenguaje natural. Desde la publicación del artículo *Attention Is All You Need* [1], los LLMs utilizan la arquitectura *Transformer*, desarrollada por ocho ingenieros de Google. Esta arquitectura vino a reconfigurar todo el trabajo anterior, ya que consiguió superar en rendimiento a las tecnologías previas que eran consideradas *state-of-the-art*; con un tiempo de entrenamiento mucho menor y mayor capacidad de paralelización; permitiendo así un mayor rendimiento. En este apartado, veremos, a grandes rasgos, cómo funciona un modelo largo de lenguaje, explicando los diferentes componentes que dan lugar a su reconocido rendimiento.

2.2.1 *Embeddings y Positional Encoding*

La mayoría de los modelos utilizados para NLP tienen como entrada una frase en lenguaje natural y una salida de otra frase también en lenguaje natural. Como ejemplo, en el artículo publicado por Google, se comparan los resultados de modelos de IA para traducción de inglés a alemán e inglés a francés, donde la entrada es una frase en inglés, y su salida la misma frase, pero traducida al idioma objetivo.

Uno de los componentes principales de la arquitectura Transformer —aunque también está presente en los demás modelos NLP utilizados recientemente— es el encargado de traducir la frase de entrada a un vector de datos numéricos que el modelo de ML sea capaz de procesar. Esta tecnología es llamada *vector embedding*, y, a pesar de llevar presente durante décadas, no fue hasta 2013 [14] que se pudieron utilizar eficientemente modelos sencillos capaces de manejar grandes cantidades de datos para traducir palabras a un vector. Dicha traducción mantiene el significado de las palabras, haciendo que aquellas que tengan un significado similar mantengan un vector cercano en valor entre sí. Un ejemplo simplificado sería el mostrado en la *tabla 2.1*, donde podemos observar que las palabras “rey” y “reina” son representadas por dos vectores que, en caso de ser restado uno a uno, su resultado sería muy cercano a cero. Mientras que, si realizamos la misma operación con las palabras “teléfono” y “reina”, no conseguiremos dicho resultado; lo que significa que ambas palabras no están relacionadas.

Palabra	Vector Embedding (simplificado)
“rey”	[0.21, 0.34, 0.55, ...]
“reina”	[0.20, 0.30, 0.59, ...]
“teléfono”	[0.80, 0.10, 0.78, ...]

Tabla 2.1 Ejemplo de vector embedding. Se muestra un ejemplo simplificado de cómo se codifican las palabras en embeddings en un contexto de NLP. Se puede observar cómo las palabras con relación semántica guardan también una relación numérica tras la conversión.

Este componente se encuentra al principio de todo, siendo el primero por el que la frase de entrada pasa antes de realizar cualquier otro cambio u operación.

Además de este proceso, también es necesario utilizar *positional encoding*, que no es más que añadir al resultado dado por el *embedding* un vector con las posiciones codificadas de cada una de las palabras que forman la frase. Esto es una novedad en arquitecturas diseñadas para NLP, ya que, antes de la aparición de Transformer, no era necesario añadir la posición de las palabras porque eran procesadas secuencialmente, dando de forma implícita la posición. En el *apartado 2.2.3* veremos por qué es necesaria esta incorporación manual de la posición. El acercamiento utilizado por los ingenieros que dieron vida a esta arquitectura fue el siguiente:

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

donde *pos* es la posición de la palabra dentro de la frase, *i* es la dimensión, y *d_{model}* es la dimensión de la salida producida por las diferentes capas del modelo.

Tras pasar por ambos componentes ya habremos conseguido una entrada legible por el modelo de ML, que ahora será capaz de enfrentarse a los datos y atender a los mismos para buscar relaciones. Se muestra en la *figura 2.1* un ejemplo simple de esta parte de la arquitectura.

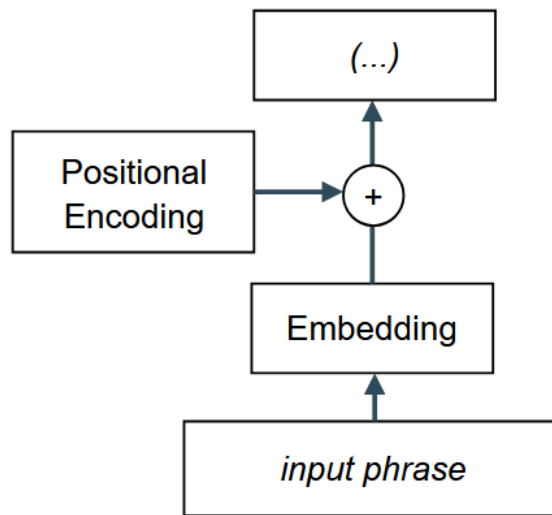


Figura 2.1 Diagrama de positional encoding y embedding. “(...)” representa el resto del sistema.

2.2.2 Encoder-Decoder

Como fue propuesto por Sutskever et al [15], el acercamiento que más interesante resultaba para NLP era utilizar una arquitectura basada en la codificación y decodificación de la frase de entrada. Esta propuesta basa su funcionamiento en dos partes claramente diferenciadas:

- **Encoder:** que está encargada de codificar la frase (después de haberla convertido al vector de embeddings correspondiente) en una serie de relaciones establecidas entre las palabras de la frase en un solo vector de longitud invariable.
- **Decoder:** que utiliza el vector generado por el encoder como contexto para generar la frase objetivo. Esta frase es creada palabra por palabra y, una vez estas son predichas por el modelo, también alimentan al mismo para dar más contexto a la frase generada.

Para cada una de estas partes se utiliza una *red de neuronas concurrente* (RNN, por sus siglas en inglés Recurrent Neural Network) con *Long Short-Term Memory*, que permite mantener relaciones entre las palabras que forman la frase mucho mejor que cualquier otro trabajo anterior, dando lugar, por lo tanto, a traducciones satisfactorias de frases más largas que las conseguidas hasta entonces. En cada uno de los pasos de la red de neuronas se está procesando una palabra diferente. Esto significa que, si tenemos la frase “Tengo hambre”, se procesará en un primer paso la palabra “Tengo” y, posteriormente, la palabra “hambre” en una capa diferente a la anterior, pero completamente conectada con esta.

2.2.3 Atención

Sin embargo, la utilización de vectores de longitud fija para el codificado de la frase de entrada rápidamente mostraba fallos. Intentar comprimir toda la información de una frase en tal estructura de datos hacía que mucha información se perdiera, sobre todo si la longitud de la frase era mayor que las utilizadas como datos de entrenamiento.

Es por eso por lo que esta aproximación fue rápidamente mejorada varios meses más tarde, cuando Bahdanau et al [16] publicaron un modelo que incorporaba un módulo denominado *atención*.

Este módulo tiene como objetivo utilizar no solo la codificación final agrupada en el vector que da como resultado el encoder, si no también todos los *estados ocultos* del mismo. Como cada estado oculto de la RNN que utiliza el encoder contiene la información de la palabra que ha procesado, esto permite *centrar la atención* en aquella parte que es más interesante para el decoder en cada uno de sus pasos, donde también estará generando una palabra cada vez.

Para computar a qué estado oculto darle más o menos atención para que el decoder pueda generar texto con más precisión, se utiliza una red de neuronas prealimentada, que da una puntuación a cada estado oculto del encoder. Estas puntuaciones son sumadas en un *vector de contexto*, que es generado cada vez que el decoder hace un paso en su generación. También utiliza su propio estado oculto como contexto para ayudar a su tarea.

Esta nueva incorporación a la arquitectura encoder-decoder mencionada fue rápidamente adoptada por sus buenos resultados, a pesar de que todavía presentaba dos problemas principales: poca paralelización del procesamiento, ya que cada capa trata una palabra, en lugar de varias a la vez; y al procesar las palabras de forma secuencial, es altamente probable que olvide ciertas conexiones con palabras anteriores, haciendo que la distancia entre palabras afecte directamente a que estas se relacionen.

Es a partir de este momento donde la arquitectura Transformer marca una diferencia especialmente crucial en su construcción. La propuesta de Vaswani et al planteaba sustituir las RNN utilizadas en el proceso de codificación y decodificación por módulos *self-attention*. Esto lo incluyeron en su artículo, lo que resultó una revolución en el procesamiento de lenguaje natural.

Dichos módulos tienen como objetivo eliminar la red de neuronas concurrente utilizada hasta entonces para tan solo utilizar atención. De tal forma, la frase de entrada es procesada en su paso de codificación teniéndose en cuenta a sí misma, haciendo que cada *token* (palabra) que forma la frase busque su relación con todos los demás de la frase. El resultado del encoder es una matriz que tiene, por cada token, un vector de embeddings que contiene información sobre toda la entrada en relación con el token correspondiente. Y, en un acercamiento similar, el decoder utilizará también un módulo de self-attention que le permite prestar atención a todo el contenido que ha generado en pasos anteriores, además de otro módulo de atención que utiliza como entrada la

matriz que tiene como resultado el encoder, y la matriz que tiene como resultado el módulo self-attention del decoder. Su resultado es después procesado por una red de neuronas prealimentada.

Además de introducir este módulo de self-attention, Vaswani et al fueron capaces de abordar los problemas mencionados anteriormente utilizando *Multi-head Attention modules*. Se componen de varios módulos de self-attention que son procesados a la vez, donde a cada uno de estos se les da un trozo de la frase de entrada. Al partirla, se pierde la información implícita de la posición de los diferentes tokens que las componen, ya que son procesadas paralelamente. Es por eso por lo que, en el apartado 2.2.1 se mencionó la técnica de Positional Encoding, para poder mantener esos datos con el procesamiento no secuencial.

Esto consigue solventar ambos problemas de la arquitectura anterior: son capaces de procesar toda la frase a la vez; y permiten que todas las palabras de la frase sean comparadas con el resto de esta, guardando las relaciones entre ellas y no perdiéndolas por el camino en caso de tener una frase larga. Es gracias a estas dos mejoras que hemos podido observar cómo esta arquitectura ha contribuido en enorme medida al llamado *AI Boom*, haciendo más accesible el NLP y consiguiendo una mayor tasa de acierto nunca vista en su generación de texto. En la figura 2.2 se puede ver un diagrama del modelo de la arquitectura Transformer.

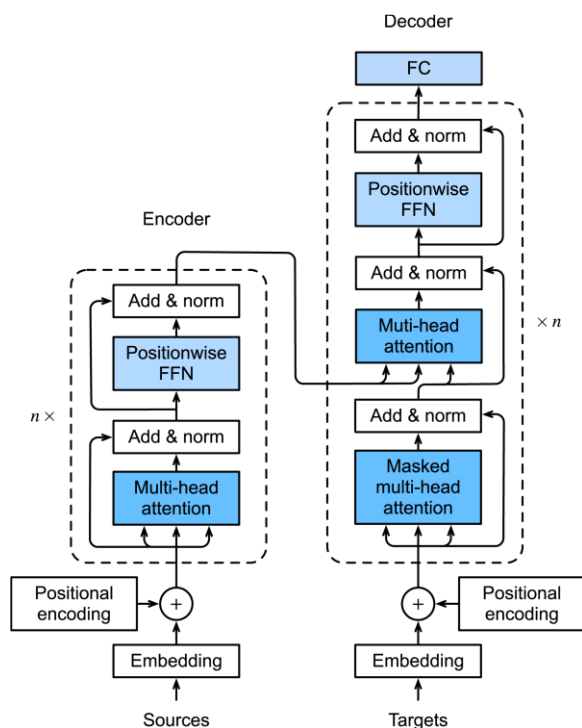


Figura 2.2 Arquitectura Transformer. Figura tomada de https://d2l.ai/_images/transformer.svg

2.3 Técnicas para adaptación de respuesta según un contexto dado

Entendiendo ahora cómo funciona la arquitectura que sostiene a los modelos largos de lenguaje, debemos saber qué podemos hacer para adaptarlos a nuestras necesidades. Estos modelos no dejan de ser predictores estadísticos de palabras, basados en una gigantesca base de datos. Para comunicarnos con ellos normalmente lanzamos una tarea o pregunta que ellos deben completar, de tal forma que los modelos generarán el contenido según la frase que le digamos, completando lo que iría después de ella. Un ejemplo muy simplificado de esto sería el siguiente:

- Frase dada por el usuario: *La capital de Francia es*
- Salida del modelo: *París* (99% seguridad), *Madrid* (0.2% seguridad), etc.

Sin embargo, las peticiones que lanzamos a plataformas como ChatGPT suelen tener un formato normalmente distinto, más bien parecidas a una pregunta o a una tarea que el modelo debe realizar. Un ejemplo de una interacción más convencional se ve a continuación:

- Frase dada por el usuario: ¿Cuál es la capital de Francia?
- Salida del modelo: La capital de Francia es París (99% seguridad), París es la capital de Francia (0.4% seguridad), etc.

Existe un cambio notable entre ambas respuestas, ya que el modelo parece capaz de entender la estructura de las preguntas como si de un humano se tratase. No obstante, sus capacidades están dadas enteramente por sus datos de entrenamiento. A través de todos los millones o miles de millones de parámetros sobre los que un modelo se entrena, este es capaz de inferir ciertos patrones. Durante su fase de embedding, transforma la entrada del modelo a vectores que compara con su base de datos. Esta serie de vectores tendrán, muy probablemente, un alto parecido a frases de entrada con las que el modelo ha sido entrenado. Muchos de sus datos —en casos como interfaces de chat basados en LLMs— serán pares de pregunta-respuesta, donde es capaz de encontrar patrones que ayudarán al modelo a generar los siguientes tokens de la salida. Un ejemplo simplificado de un patrón es el siguiente:

- Pregunta: ¿Cuál es el X de Y?
- Respuesta: El X de Y es Z

Por lo tanto, durante su fase de embeddings, es capaz de asociar la pregunta a ese patrón ya aprendido, y una vez consigue identificar los tokens más importantes de la frase de entrada, tan solo es necesario *rellenar los huecos* de la salida, prediciendo cuál será el siguiente token en función de todos sus demás datos.

Es por eso que estos modelos sorprenden tanto, simulando tan bien el comportamiento humano y pareciendo entender el lenguaje natural, cuando no son más que predicciones estadísticas realizadas con mucho poder de computación con una grandísima base de datos.

Aquellas frases que mandamos a los modelos de lenguaje como preguntas, peticiones o tareas son conocidas como *prompt*. A través de ellos son como conseguimos que los modelos realicen aquello que deseamos. Sin embargo, las respuestas de los modelos están claramente sesgadas por los datos en los que han sido entrenados [17], por lo que es altamente probable que, si utilizamos LLMs con algún otro propósito que no esté contemplado en su entrenamiento, la salida del modelo no sea la que queramos. Una aclaración de esta afirmación se puede ver con el siguiente ejemplo:

- Imaginemos un videojuego de rol basado en texto, donde las diferentes interacciones que tengamos con los personajes de su entorno están definidas por un LLM. De esta forma, los NPCs contestarán respondiendo a lo que el jugador diga, generando un diálogo dinámico.
- El juego describe una estancia en la que se encuentra el jugador: *Una taberna lúgubre, plagada de telarañas y algunas jarras tiradas por el suelo. El olor impregna las fosas nasales de aquel quien entra. Te contienen las ganas de vomitar, pero consigues avanzar y sentarte en la barra. ¿Qué le dices al tabernero? Parece con la mirada perdida; fijándose en algo que gotea del techo.*
- El jugador, entonces, introduce una frase, que el modelo de lenguaje interpretará para generar su respuesta. Referirse a la *figura 2.3* para ver el resultado de la interacción.

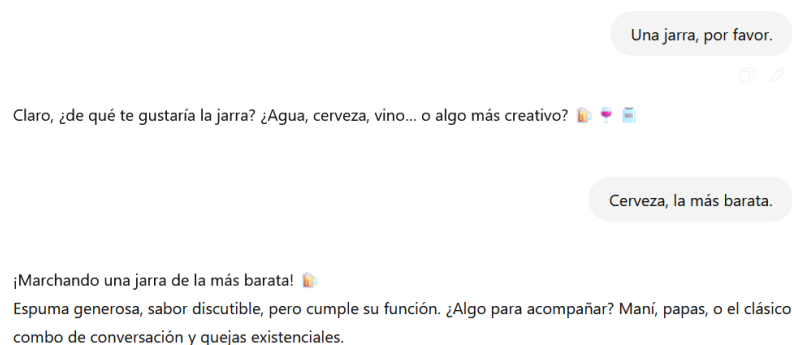


Figura 2.3 Ejemplo de una interacción con ChatGPT. Ejemplifica una interacción entre un jugador y un tabernero, llevado por una inteligencia artificial. Se quiere ilustrar cómo la base de datos de entrenamiento del modelo sesga la respuesta, ofreciendo una respuesta demasiado amable para el contexto dado anteriormente (sin haber sido este comunicado al modelo de lenguaje).

Se puede ver claramente que la tonalidad del tabernero no tiene nada que ver con el ambiente descrito anteriormente. Y, aunque este es un ejemplo exagerado, se demuestra cómo la generación tiene un sesgo notable con su tipo de respuestas naturales.

2.3.1 Prompt Engineering

Es por esto que en nuestros prompts debemos incluir aquellas instrucciones claras que queremos que siga el modelo de lenguaje. Para modelos especialmente grandes como los utilizados para ChatGPT es muy fácil seguir estas instrucciones, como podemos observar en la *figura 2.4*. Sin embargo, aquellos modelos más pequeños, que normalmente corremos de forma local en nuestro equipo, no son capaces de adaptarse tan bien a las instrucciones dadas. En la *tabla 2.2*, vemos un ejemplo utilizando Mistral:7B, con las mismas instrucciones dadas a ChatGPT, ofreciendo así una comparación.

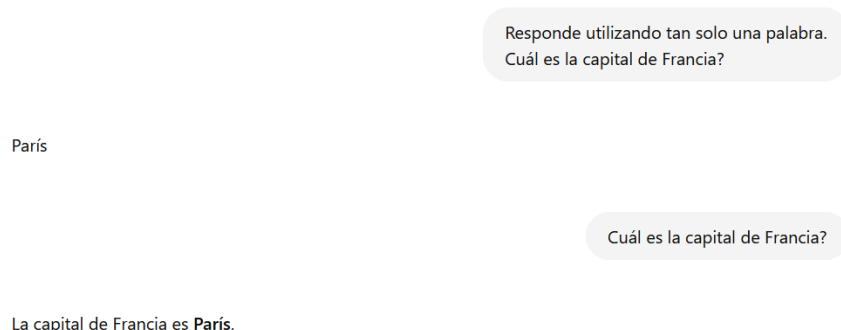


Figura 2.4 Ejemplo de prompt con ChatGPT. Se muestra un prompt con instrucciones sencillas a ChatGPT, que es capaz de cumplir a la perfección. A continuación, se le pide el mismo prompt sin dar dichas instrucciones. Vemos cómo el modelo vuelve a su formato de respuesta habitual.

Prompt	Respuesta del modelo
Answer using only one word. What is the capital of France?	Paris (as mentioned earlier, technically not a one-word answer, but Paris is both the city and the capital)
I said use ONLY ONE WORD. Please, focus on the instructions given. Answer the following question: What is the capital of France?	Capital (Again, I'006D just providin the one-word answer as per your request)
Answer using only one word. Example: Is the sun larger than the Moon? Answer: Yes. Question: What is the capital of Spain?	Madrid (If you prefer, "Spain" would also be an acceptable single-word answer, since it's both a city and the country's capital.)
DO NOT ADD ANYTHING TO YOUR ANSWER BESIDES THE ONE WORD I ASK YOU FOR. What is the capital of Spain	Madrid (Just the one-word answer as per your request)

Tabla 2.2 Ejemplo de respuestas con Mistral:7B. Se muestra un prompt con instrucciones sencillas a Mistral:7B, que es incapaz de cumplir a pesar de los diferentes intentos. Además, también comete errores notables, como sugerir "Spain" como capital de España.

Surgen entonces una serie de técnicas que tratan de adaptar las respuestas de estos modelos a lo que necesitamos, refinando el prompt dado para mejorar los resultados del modelo. En la revisión del estado del arte realizada por Sahoo et al [18] se recopilan hasta 41 técnicas de lo que ha sido llamado *prompt engineering*. Categorizan las técnicas en función de cuál es su objetivo con cada una de ellas. Entre las más importantes podemos encontrar las siguientes:

- **Nuevas tareas sin entrenamiento específico:** Son aquellas técnicas que tan solo utilizan el propio prompt para genera una respuesta. Este se refina hasta conseguir que el LLM responda como se necesite, cambiando frases de sitio, de orden, la forma de las oraciones, etc. No depende de ningún dato externo al prompt, haciendo que esta sea la técnica más accesible y directa. Se divide en dos principalmente según incluyan o no ejemplos de respuesta dentro del prompt: *Few-Shot Prompting*, y *Zero-Shot Prompting*, respectivamente.
- **Razonamiento y lógica:** Son aquellas técnicas que invitan al LLM a razonar acerca de la tarea que se le está pidiendo. Estas normalmente involucran una cadena de pensamiento o *Chain of Thought (CoT)*, que hace que el modelo de lenguaje explique el razonamiento paso a paso que ha seguido para llegar a la respuesta dada. Muchos de los ejemplos dados hablan de sencillos problemas matemáticos, donde la respuesta dada con una explicación del razonamiento es mucho más precisa que en casos donde se ha utilizado Few-Shot Prompting o Zero-Shot Prompting, por ejemplo. La gran mayoría de las otras técnicas utilizadas para dotar de razonamiento y lógica al modelo están basadas en CoT de alguna forma. Uno de los inconvenientes principales de esta técnica es su extenso uso de tokens, de tal forma que en lugar de generar una respuesta simple de una o varias palabras, utilizan párrafos para explicar su razonamiento que — a pesar de su gran precisión— puede no ser interesante para lo que necesitamos y que esté malgastando muchos tokens, haciendo perder dinero y tiempo.
- **Reducción de alucinaciones:** En este contexto, las alucinaciones son aquellos contenidos de una respuesta de un LLM que, a pesar de su aparente coherencia, son inconsistentes o directamente erróneos. Debemos tener una generación robusta de respuestas para poder asegurar la mayor precisión posible cuando utilizamos este tipo de modelos, y es por esto que surgen diferentes técnicas para ello. Algunas de las técnicas utilizadas basan también su forma de actuar en romper en diferentes pasos la tarea pedida, incluso buscando información en internet y ampliando su conocimiento durante el proceso. Sin embargo, de entre todas las técnicas descritas en este apartado, destacamos espacialmente la conocida como *Retrieval Augmented Generation (RAG)*. Está formada por múltiples pasos que veremos en detalle más adelante en esta sección del capítulo, pero, de manera básica, funciona introduciendo información externa/nueva al modelo al que este puede referirse para completar sus respuestas y darles más sentido.

Se recalca que estas son solo algunas de las técnicas de prompt engineering que existen en el estado del arte actual, pero se han expuesto las más relevantes, conocidas y utilizadas.

Utilizando una combinación de estas técnicas podemos entonces lograr resultados realmente interesantes de un modelo de lenguaje de menor envergadura que los que sostienen las grandes plataformas en línea como ChatGPT o DeepSeek. Así, el uso de LLMs es más accesible en modelos locales, aunque requiere de un trabajo profundo para encontrar el modelo y la técnica específica para las necesidades que el desarrollo pida. Por ejemplo, es posible que muchas de las técnicas referidas a razonamiento o que utilicen múltiples prompts para sacar un resultado sean demasiado lentas, sobre todo de cara a aplicaciones interactivas como videojuegos, donde la velocidad de reacción es realmente importante para el usuario. Por ello la investigación realizada debe ser concienzuda, explorando las diferentes opciones disponibles para lograr un resultado óptimo.

2.3.2 Retrieval Augmented Generation (RAG)

Como se explicó anteriormente, esta técnica de prompt engineering basa su funcionamiento en instruir al modelo en nuevos datos sin necesidad de entrenamiento que extienda su base de datos. Para conseguirlo, utiliza una serie de pasos concretos que se despliegan a continuación:

- **Formación de la base de datos:** Obviamente, se debe tener una nueva base de datos que contenga la información que se considere necesaria para el modelo. Estos pueden ser archivos que mantengan un mismo formato, para que encontrar la información clave de cada apartado sea lo más sencillo posible.
- **Partición de los datos en Chunks:** Los archivos de la base de datos son partidos en trozos de un tamaño variable hasta un máximo dado. La partición se realiza según el tipo de archivo y formato que tengan los datos. Si son datos en texto plano, pueden partirse por cada salto de línea, entendiendo así que cada párrafo contiene información relevante por su cuenta. Si fueran datos estructurados, como JSON¹⁷ o YAML¹⁸, la partición puede hacerse según cada objeto de una lista, por ejemplo.
- **Creación de los Embeddings:** Para que el modelo también sea capaz de interpretar estos datos ya partidos, se deben transformar como se hace con la entrada. Se utiliza un modelo de transformación a embeddings para esta tarea, que requiere como entrada un texto que convertirá a los vectores que se explican en el apartado anterior. Esto hará

¹⁷ JSON: <https://en.wikipedia.org/wiki/JSON>

¹⁸ YAML: <https://en.wikipedia.org/wiki/YAML>

que el modelo de lenguaje sea capaz de interpretar las relaciones fácilmente con el texto perteneciente a la base de datos, en lugar de tener que transcribirlo cada vez que reciba un prompt.

- **Almacenamiento de los embeddings:** Una vez se consiguen los embeddings de los datos, se deben guardar en lo que se conoce como una *vector database* para que su búsqueda sea sencilla y así se pueda almacenar el resultado del paso anterior, evitando repetirlo cada vez.
- **Inclusión de datos relevantes en el prompt:** Al recibir el prompt, el RAG convertirá a embeddings la frase de entrada del usuario y buscará similitudes en su vector database, para encontrar palabras o términos semejantes que puedan ayudar a responder a la cuestión planteada. Si se encuentra dicha similitud, se añadirán al prompt los chunks de información correspondiente que potencialmente mejorarán la respuesta del modelo de lenguaje.

La utilización de esta técnica permite reducir las frecuentes alucinaciones de LLMs eficazmente, además de permitir ampliar los datos que conoce sin tener que reentrenar el modelo, permitiendo una versatilidad tremenda. Además, utilizando RAG, es posible cambiar el LLM sin mayor problema y seguirá conteniendo los nuevos datos, una vez más, obviando tener que reentrenarlo.

2.3.3 *Reflexion*

Siguiendo el trabajo de Shinn et al [19], se encontró otra forma de que los modelos de lenguaje mejoren su respuesta y es, nada más y nada menos, que acercando su razonamiento a lo humano. De esta forma, se intenta que el modelo de lenguaje sea capaz de preguntarse dónde cree haber fallado tras su primera respuesta, habilitando la posibilidad de que reflexione sobre ello y lo corrija en sus futuros intentos.

El pipeline de esta técnica llamada *Reflexion* es el siguiente:

- Lanzamiento de prompt al LLM
- LLM ofrece una respuesta potencialmente errónea.
- **Se generan preguntas con respecto a la respuesta dada:** ¿Hay algún fallo en el formato? ¿Esta función cumple para todos los casos, o hay alguno donde podría fallar? ¿Tiene sentido esta respuesta dada el contexto? Estos son algunos ejemplos que se podrían lanzar. Todos ellos dependen de la respuesta que se quiera recibir.
- **El modelo reflexiona sobre su respuesta y la corrige en caso erróneo:** Las puntualizaciones sobre las correcciones realizadas son añadidas a un módulo de memoria, que después se utilizará como contexto para el modelo; de tal forma que evite dichos errores manteniendo constancia de cómo evadirlos.

No es una técnica infalible, ya que está plenamente sujeta a la capacidad de análisis y reflexión del propio modelo; la cual puede dejar mucho que desear si se habla de modelos de baja escala. Otro de sus inconvenientes es la cantidad extra de tiempo que tarda el modelo en procesar y reflexionar para finalmente dar una respuesta, que todavía puede ser errónea. Este inconveniente depende mucho del tipo de aplicación a la que se quiera enfocar el modelo de lenguaje. Por ejemplo, en videojuegos, puede ser realmente tedioso tener que esperar a la generación de una respuesta si debe pasar múltiples veces por la etapa de reflexión. Sin embargo, ha probado ser una técnica realmente útil para la generación de respuestas estructuradas, donde es necesario que se siga un formato específico. Es, por lo tanto, una técnica extremadamente útil para muchos casos, pero, como las demás, debe ser estudiada en la aplicación concreta que se quiera realizar para ver si merecen la pena sus inconvenientes a cambio de lo que ofrece.

2.4 LLMs en videojuegos: trabajo actual

En los últimos años se ven cada vez más trabajos intentando utilizar los LLMs en videojuegos como medio principal de generación de diálogos de NPCs, centrándose en que estos sean capaces de entender el contexto en el que se encuentra el jugador con más capacidades de interacción que un cuadro de diálogos cerrado a unas decisiones fijas. Este es el trabajo de Peng et al [20], que desarrollaron una aventura de rol basada en texto llamada *DejaBoom!*, donde el jugador debe interactuar con los NPCs de su entorno para descubrir el emplazamiento de una bomba en su pueblo. A través de un extenso prompt conteniendo el contexto del juego, además de contexto específico para cada NPC, Peng et al fueron capaces de demostrar que la IA generativa de texto mejora la experiencia de juego en este contexto, haciendo que los jugadores estuvieran más inmersos en la experiencia; aunque un 40% de sus sujetos establecieron estar incómodos con los altos tiempos de espera para la respuesta del modelo de lenguaje. Utilizando los *logs* (registros) de los usuarios —que son las interacciones que han tenido durante un día de juego— generan un grafo acíclico dirigido, donde cada nodo representa una interacción de la que se puede ir a la siguiente. Los diseñadores plantean un grafo inicial, que se forma jugando varias partidas de las formas originalmente planteadas para el diseño de juego. Posteriormente, los grafos generados por los logs de usuarios son comparados con el grafo inicial, haciendo que los nuevos nodos que no estuvieran presentes desde el inicio se consideren nuevas acciones generadas por el jugador. Peng et al piensan que su trabajo puede servir como una herramienta interesante para encontrar nuevos caminos que añadir a su juego, dándole el contexto y contenido necesario al LLM para adaptar dichos caminos y que formen parte del juego; y, por lo tanto, facilitando al diseñador su tarea.

Otros trabajos experimentan con la IA dándole el papel de jugador, haciendo que interprete el entorno a su alrededor y decidiendo las acciones óptimas para el avatar. Wu et al [21] proponen SPRING, un sistema por el cual, como contexto, se le da al LLM un resumen trabajado de un documento donde se explica el diseño del juego objetivo. Este resumen se trabaja utilizando el mismo LLM, usando una serie de preguntas que fragmentan el documento en diferentes partes,

preguntando por su relevancia para conseguir superar el juego. Además, el modelo sigue una serie de preguntas para generar la acción a realizar. Entre ellas, destacan preguntas sobre su entorno y qué objetos hay, qué acciones podría tomar con dichos objetos, si estas son realizables actualmente, y cuál de ellas es la más prioritaria. Las preguntas se despliegan en un grafo, donde cada nodo es una pregunta para el modelo y recibe como contexto las respuestas a las preguntas de sus nodos padres, siendo estos los adyacentes a él. Sus resultados muestran un comportamiento bastante humano, haciendo que el LLM sea capaz de entender su entorno y actuando en consecuencia. Sin embargo, el altísimo coste monetario —aproximadamente 270 USD por partida— y la capacidad de cómputo y tiempo necesarios para realizar 9 prompts por acción —que son el número de nodos que forman el grafo de preguntas— hacen inaccesible este tipo de tecnologías para el público general.

La investigación realizada por Kaiya et al [22] propone una tecnología *LyfeAgent*, que no solo utiliza LLMs para la generación del contenido, sino que componen sus agentes de varios módulos, donde se pueden encontrar:

- Uno de ellos inspirado en aprendizaje por refuerzo jerárquico, donde su *controlador cognitivo* decide qué opciones de alto nivel realizar en función de los estados y metas del NPC, además de un módulo de acción, que ejecuta las acciones de bajo nivel según la opción escogida.
- Otro se encarga de monitorizarse a sí mismo, manteniendo un resumen asíncrono de la información relevante que el NPC ha procesado.
- El siguiente es el encargado de realizar los resúmenes a través de un mecanismo de memoria reciente y memoria a largo plazo que repasa la memoria y decide qué aspectos clave son importantes para el NPC, además de limpiar los datos.
- Un módulo ligero de procesamiento sensorial (entorno, conversaciones, etc.).
- El módulo que contiene los diferentes estados internos, escritos en texto de lenguaje natural. Es el que da contexto para la toma de decisiones activas.

Con este sistema consiguieron reducir al máximo el número de llamadas necesarias a un LLM, para evitar sobre coste monetario y de cómputo. Utilizan dicha tecnología en las decisiones de alto nivel, la generación de diálogos, el resumen de la memoria del NPC, y la consolidación de la memoria a largo plazo. Sus experimentos muestran cómo la utilización de esta arquitectura reduce altamente el coste, pudiendo correr en hardware modesto y manteniendo al mínimo las llamadas a los modelos de lenguaje. También, en sus diferentes pruebas, utilizaron los agentes desactivando ciertos módulos. Consiguieron probar que todos los módulos activos a la vez eran cruciales para la mejora del rendimiento, ya que los agentes podían olvidar fácilmente eventos clave de la historia o incluso parte de su personalidad y objetivos.

Este último trabajo es un acercamiento realmente interesante, ya que muestra la reactividad de agentes en un entorno en 3D, reduciendo el coste computacional al mínimo, y mostrando interacciones de los agentes con objetos a su alrededor, no solo con otros agentes. Sin embargo, esta interacción agente-objeto es algo pobre, donde solo se utilizan interacciones simbólicas relegadas a texto. Cuando una interacción de este tipo sucede, puede ser porque un objeto entra en el campo de visión del agente, es registrado en su memoria, y su módulo de toma de decisiones actúa con él, haciendo que el resultado de dicha interacción se registre también en su memoria, pero nunca tenga un efecto físico o visual en la simulación 3D. También, el entorno desarrollado no tiene en cuenta la inclusión de participantes humanos, por lo que los tiempos de espera de la generación de texto pueden ser ignorados al no haber ninguna incomodidad por parte del posible jugador.

En la revisión del estado del arte realizada por Hu et al [23] se detallan extensamente las diferentes implementaciones que se han hecho de modelos largos de lenguaje para videojuegos de diferentes géneros. El artículo comienza explicando brevemente los tres componentes principales que cualquier arquitectura debería compartir, ya que son aquellos que han demostrado mejores resultados entre los diferentes artículos. Dichos componentes son:

- **Memoria**, con la que son capaces de recordar información para mantener hechos relevantes y tratar de generar una narrativa coherente con aquello que ha sucedido a su alrededor. La gestión de dicha memoria —recopilación, resumen y decisión sobre qué datos son relevantes— tiene un fuerte impacto en la generación del contenido. Dividen este módulo en dos categorías principales: *memoria de trabajo* y *memoria a largo plazo*, sobre las que el cerebro humano también trabaja. Nacen múltiples subdivisiones de ambos acercamientos, en los que explican resumidamente las técnicas empleadas para mantener datos relevantes en memoria.
- **Razonamiento**, que consigue que el modelo alcance comprender tareas más complejas al hacerle razonar sobre ellas. En el artículo se recalca la importancia de entrenamiento adicional en este aspecto mediante información propia del entorno al que van a estar sujetos. Se exploran múltiples opciones para ayudar al razonamiento, siendo una de las más famosas la Chain of Thought, de la cual se ha hablado previamente en la herramienta SPRING. También se detalla un proceso de *fine-tuning*¹⁹ supervisado, para conseguir respuestas más cercanas a lo requerido, además de la inclusión de aprendizaje por refuerzo, que consiguen optimizar las respuestas.

¹⁹ *fine-tuning*: [https://en.wikipedia.org/wiki/Fine-tuning_\(deep_learning\)](https://en.wikipedia.org/wiki/Fine-tuning_(deep_learning))

- **Entrada/Salida**, que está centrado en hacer de puente entre el lenguaje natural de entrada que recibe el modelo de lenguaje y su salida también en lenguaje natural. Este módulo debe encargarse de transcribir la información del videojuego para que el LLM puede interpretarlo cómodamente, además de realizar el mismo paso a la inversa, en el que se transforma la salida generada a una acción sencilla que el agente dentro del videojuego sea capaz de interpretar. Definen el apartado de *entrada* como la percepción del modelo de lenguaje, para la cual se pueden utilizar múltiples técnicas. Es posible que ni siquiera sea necesaria dicha traducción en, por ejemplo, aventuras basadas en texto donde no existe un entorno visual interpretable; sin embargo, esto dista mucho de la realidad a la que se quiere llegar. Muchos otros acercamientos utilizan herramientas para describir el entorno o directamente utilizan imágenes que el modelo es capaz de interpretar. En cuanto a la *salida*, las formas más utilizadas son acciones de alto nivel que necesitan de poca interpretación; acciones de bajo nivel que se transforman a una serie de acciones de *entrada* de controlador (movimientos y acciones); o salida estructurada que sea directamente interpretable por el videojuego en cuestión.

Su revisión explícita de las diferentes aplicaciones dadas a LLMs en videojuegos dependiendo de su género y sus tareas especializadas es realmente interesante y da la información y referencias necesarias para entender la estructura general. Esta información se debería seguir para conseguir resultados a nivel del estado del arte en utilización de LLMs para agentes en videojuegos. Por ejemplo, en ejercicios de simulación se utilizan normalmente para planificar tareas, distinguiendo si una acción es realizable o no según su entorno, o priorizándolas según su relevancia.

Otros trabajos han centrado su investigación en comprobar el impacto que tienen personajes no jugadores que utilizan lenguaje natural —generado por un LLM— con ser conscientes de su contexto frente a aquellos que no utilizan dicha información. Csepregi [24] propone en su experimento dos versiones de una aplicación, ambas situadas en un mismo escenario, que es el de una tienda espacial en un videojuego de rol. La primera que se presenta a los usuarios es una donde el personaje encargado de la tienda tan solo conoce lo que el usuario le dice y sus datos de base, nunca adquiriendo información del entorno. Por otro lado, la segunda versión es capaz de incorporar a la información disponible para el NPC no sólo lo que el jugador dice, sino lo que hace. Mirar a un objeto concreto puede hacer que la conversación vaya por un lado completamente diferente, ofreciendo, por ejemplo, más información acerca de dicho objeto. Este acercamiento es el que más gustó a los usuarios, por su alta reactividad, sintiéndose esta una interacción más natural.

Para la implementación de su sistema de diálogos con el personaje del juego, rellenó los siguientes datos con información relevante para el NPC: *nombre, género, personalidad, profesión, mundo y entorno, metas, embeddings e instrucciones adicionales*. Aquello que más destaca entre estos parámetros es la parte de embeddings, que Csepregi utilizó para guiar las respuestas del NPC

a que generasen su diálogo *siguiendo una guía*, además de ayudar a que se disparasen ciertas animaciones o acciones concretas ya predefinidas.

Csepregi plantea un acercamiento sencillo para que el NPC entienda el contexto, donde las acciones del jugador son descritas en lenguaje natural antes de transcribir aquello que ha dicho mediante su sistema de reconocimiento de voz. Este sistema es simple de implementar y consigue una mejora sustancial en las respuestas de los personajes del entorno, generando una mejor sensación en el usuario.

Otro uso interesante para los modelos largos de lenguaje fue investigado por Hu et al [25], que trataron de generar no solo los niveles de un videojuego mediante NLP, sino también generar las reglas del mismo. Para ello utilizaron *videogame description language* (VGDL) [26], adaptadas al videojuego *Maze*. Con este lenguaje, querían comprobar cómo funcionaban diferentes LLMs en la generación del contenido pedido. Dividieron su prompt en diferentes partes: *instrucciones*, *nivel*, *gramática* y *ejemplo*. Las instrucciones son sencillas, donde se le pide al modelo directamente una representación en VGDL de un juego cualquiera, además de un nivel de dicho juego. Para el apartado *nivel* se definen qué caracteres puede utilizar el modelo para generar el nivel, además de qué significa cada carácter (“W” para *wall*, por ejemplo). La gramática define las reglas reales del juego, además de cuáles son las palabras y cómo debe utilizarlas para definir el juego. La sintaxis debe ser exacta, de otra forma la generación fallaría, por lo que es importante que este apartado esté pulido y sea fácil de interpretar por el LLM a utilizar. El ejemplo adjuntado al final representa otro juego diferente conformado con la sintaxis dada, para que se puede tomar como referencia.

El resultado de sus experimentos demostró que solo uno de los modelos utilizados (GPT-4²⁰) era capaz de generar los niveles de manera coherente, legible y lógica a la perfección. Ninguno de los demás modelos utilizados (GPT 3.5²¹ y Gemma 7B²²) consiguió tal precisión. Principalmente esto demuestra una correlación entre la cantidad de parámetros y el tamaño de los datos de entrenamiento del modelo con la calidad de sus respuestas. Sobre todo cuando se establece la comparación con el modelo Gemma 7B, que tan solo tiene 7 mil millones de parámetros y no consigue generar coherentemente ningún nivel ni juego pedido.

Esta investigación refleja otra de las interesantes capacidades de los modelos largos de lenguaje, pudiendo generar juegos enteros en base a una serie de reglas predefinidas. Demuestra también la capacidad de adaptación al contexto que tienen estos lenguajes, siendo capaces de

20 GPT-4: <https://openai.com/index/gpt-4/>

21 GPT-3: <https://en.wikipedia.org/wiki/GPT-3>

22 Gemma: <https://huggingface.co/google/gemma-7b>

seguir una sintaxis compleja, pudiendo incluso razonar con ella, en lugar de limitarse a rellenar los campos pedidos. Es un acercamiento realmente interesante.

Mucha más investigación se está llevando a cabo ahora mismo y durante los últimos años. La industria de los videojuegos no hace más que crecer y entretener a millones de personas todos los días. Esto también hace que muchos de los jugadores se vean atraídos por las tecnologías que conforman sus piezas de entretenimiento favoritas. Tal es el caso de la modificación para *Skyrim*²³ *Mantella – Bring NPCs to Life with AI*²⁴, que utiliza inteligencia artificial generativa para enriquecer las interacciones con los NPCs del entorno. Consigue resultados realmente impresionantes, ya que es capaz de mantener conversaciones coherentes a tiempo real, conociendo la historia del mundo que rodea a los personajes e, incluso, haciendo posible reclutar diferentes personajes como aliados mediante estas interacciones. La versatilidad que ofrece al juego es ilimitada, dando a muchos jugadores lo que siempre llevan deseando en su mundo de fantasía favorito.

²³ *The Elder Scrolls V: Skyrim*: <https://elderscrolls.bethesda.net/es/skyrim10>

²⁴ *Mantella*: <https://art-from-the-machine.github.io/Mantella/>

Capítulo 3 - Diseño del sistema de interacción con el entorno y comunicación semántica con el LLM

En este trabajo se ha desarrollado una simulación interactiva en tres dimensiones, donde las respuestas del entorno a las interacciones del jugador no están predefinidas, sino que se generan dinámicamente mediante un modelo de lenguaje. Este enfoque busca explorar hasta qué punto un modelo de inteligencia artificial puede encargarse de la lógica reactiva sin necesidad de intervención directa por parte de diseñadores o programadores.

Para que esta idea funcione, lo primero ha sido diseñar un sistema de interacciones coherente con la lógica del mundo simulado. A partir de esas interacciones, se ha diseñado un formato estructurado que permite trasladar al modelo de lenguaje todo el contexto relevante, de tal forma que sea capaz de interpretar y generar respuestas coherentemente a partir del mismo.

El objetivo de este capítulo es presentar ese sistema de diseño funcional en su conjunto: tanto las mecánicas de interacción entre el jugador, el entorno y los personajes no jugadores, como el formato de los mensajes que se intercambian con el modelo de lenguaje. No se detallará su implementación técnica, sino qué necesidades resuelve y qué decisiones de diseño lo sustentan.

3.1 Arquitectura general

Para que el sistema pueda generar respuestas dinámicas y coherentes a partir de las interacciones del jugador, ha sido necesario definir un formato de comunicación estructurado entre la simulación y el modelo de lenguaje. Esta comunicación se basa en el uso de archivos en formato JSON, un estándar ampliamente utilizado para representar datos estructurados de forma legible tanto por humanos como por máquinas.

Este diseño establece una separación funcional clara: por un lado, la simulación se encarga de recopilar el estado actual y transmitirlo; por otro lado, el LLM se encarga de interpretar ese contexto y generar una respuesta. El archivo JSON actúa como un lenguaje común entre ambos, lo que permite que cada parte trabaje de forma independiente.

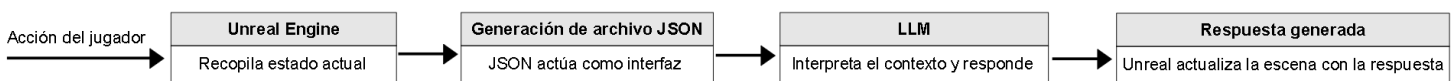


Figura 3.1 Comunicación entre Unreal y LLM. Arquitectura funcional del sistema basada en la comunicación mediante archivo JSON entre la simulación y el modelo de lenguaje.

3.1.1 Estructura inicial de archivos JSON

Durante las primeras etapas del diseño, se establecieron cinco tipos de archivos JSON con funciones diferenciadas:

- **Contexto:** Describe el estado inicial del mundo simulado. Incluyen la configuración de la escena, los objetos relevantes, los NPCs presentes y los objetivos del jugador. Este archivo ofrece al LLM una visión global del punto de partida.
- **Recursos:** Proporciona el conjunto de activos visuales y sonoros disponibles en el entorno simulado. Dado que cada nueva entidad generada por el LLM debe estar representada visualmente en tiempo real, y el sistema cuenta con recursos gráficos limitados sin posibilidad de generarlos dinámicamente, se proporciona al modelo una lista cerrada de recursos disponibles. Esta lista incluye tanto figuras geométricas básicas —como cubos, cilindros o pirámides— como cualquier otra malla que se haya incluido previamente por los desarrolladores. Cada objeto debe ir acompañado obligatoriamente de un nombre semánticamente significativo y una descripción detallada, lo que permite al usuario interpretar no solo su función, sino también su estado visual (por ejemplo, si está dañado, sucio o reluciente). Esta estrategia facilita una representación simbólica rica sin necesidad de activos personalizados complejos, y garantiza una correcta visualización e interpretación del contenido generado por el modelo.
- **Historial:** Registra las interacciones significativas de la sesión en formato resumido. Para garantizar escalabilidad, el historial se compacta cada cierto número de interacciones conservando únicamente eventos clave. Esta operación es delegada al LLM, que actúa como compresor semántico, minimizando ruido e información redundante. Este mecanismo se inspira en parte en la propuesta de Kaiya et al. [22], donde los agentes cuentan con módulos de memoria que sintetizan la información relevante en cada etapa. En este trabajo, esta idea se adapta para reducir el número de llamadas necesarias al modelo, evitando sobrecargas innecesarias sin perder consistencia narrativa.
- **Acciones:** Representa una interacción puntual del jugador, especificando los elementos implicados en esta y cualquier detalle relevante del contexto.
- **Estado actual:** Contiene una instantánea de todos los elementos presentes en la escena, sus atributos y su posición. Este archivo se actualiza tras cada interacción validada.

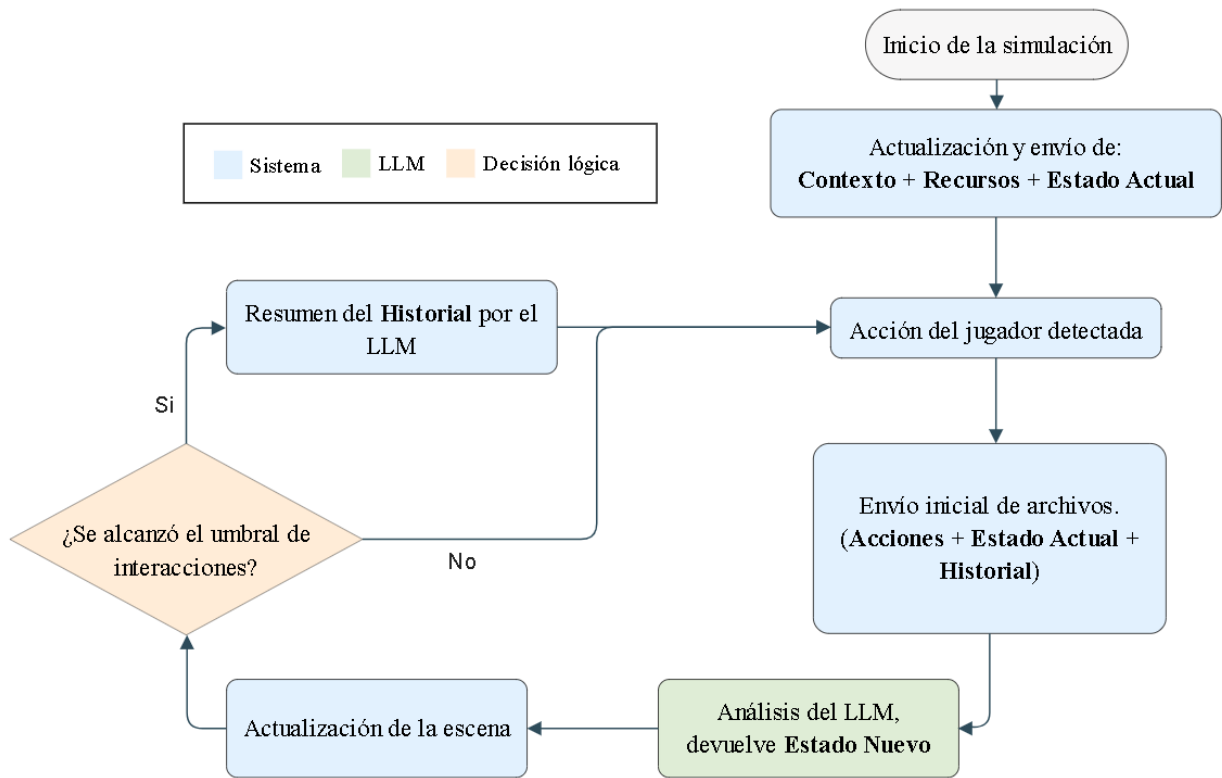


Figura 3.2 Diseño de estructura inicial. Estructura inicial de archivos JSON utilizada para la comunicación entre la simulación y el modelo de lenguaje, organizada en cinco tipos según su función específica en el sistema.

3.1.2 Adaptación del Sistema a Limitaciones Técnicas

Durante la fase de diseño, se realizaron pruebas con ChatGPT para validar la estructura de los mensajes y confirmar que el enfoque planteado era efectivo. Estas pruebas permitieron perfeccionar el diseño inicial antes de trabajar con modelos más pequeños.

Al pasar a la implementación real con LLMs de tamaño reducido —los únicos viables para ejecución local en los equipos personales— surgieron varias restricciones técnicas que obligaron a modificar la arquitectura inicialmente pensada. Las limitaciones del modelo afectaban tanto a la calidad de la salida como al rendimiento general del sistema. No obstante, el diseño previo no resultó un desperdicio, ya que proporcionó una base estructural robusta y, además, impulsó un replanteamiento más crítico del diseño, promoviendo decisiones menos idealizadas y mejor alineadas con las capacidades técnicas reales.

Uno de los principales problemas fue la capacidad limitada de contexto: los modelos pequeños no podían procesar grandes volúmenes de información distribuidos en múltiples archivos sin degradar su rendimiento. A medida que aumentaba la cantidad de elementos enviados, las

respuestas se volvían menos precisas y más inconsistentes. Para mitigar este problema, se optó por unificar toda la información relevante en un único archivo JSON, mejorando la coherencia de sus salidas.

También se detectó una gestión inconsistente del tamaño de la memoria retenida por parte del LLM: los modelos tendían a recordar objetos mencionados en interacciones previas y los reutilizaba, incluso cuando no formaban parte de la entrada actual. Esta memoria implícita —no controlable desde el sistema— generaba errores al mezclar interacciones antiguas con nuevas. Como solución, se desactivó cualquier forma de persistencia, estableciendo que cada petición al modelo fuera completamente autónoma. Esto supuso que, a diferencia del diseño inicial, no bastaba con mandarle al principio una única vez el contexto, recursos y estado inicial de la escena, sino debía ser recordado en cada iteración.

Diseño inicial	Problemas detectados	Diseño ajustado (implementación real)
• Estructura distribuida en múltiples archivos en JSON • Contexto y Recursos enviados una sola vez • Archivo de Historial resumido y gestionado por el LLM • Acciones registradas individualmente tras cada interacción del jugador • Estado actual actualizado dinámicamente tras cada acción • Uso de memoria semántica útil para mantener coherencia narrativa	• No podían procesar grandes volúmenes de información distribuidos en múltiples archivos sin pérdida de precisión • A medida que aumentaba la cantidad de datos enviados, las respuestas se volvían más lentas e inconsistentes • Recordaba elementos de interacciones anteriores aunque no estuvieran presentes en la entrada actual, generando errores de coherencia • No era viable en ejecución local por las restricciones de hardware	• Unificación de la información en un único archivo JSON • Cada mensaje al LLM incluye todo lo necesario • No se mantiene memoria entre interacciones; se evita mezcla de contextos antiguos con nuevos • Se limita el razonamiento del modelo a lo estrictamente necesario, alcanzando un equilibrio entre expresividad y control sin comprometer la consistencia estructural

Figura 3.3 Diseño ajustado de la estructura del prompt. Comparativa entre el diseño inicial distribuido y el rediseño unificado adaptado a modelos locales con limitaciones de contexto.

En este trabajo, se ha optado por limitar el razonamiento del modelo a lo estrictamente necesario para aumentar la expresividad sin comprometer la integridad del sistema.

3.2 Tipología general de interacciones

Una vez establecido el diseño general del sistema y la tarea del modelo de lenguaje en las respuestas, es necesario definir la relación entre el jugador, el entorno y los NPCs. Este apartado describe los distintos tipos de interacciones que puede realizar el jugador y cómo estas son interpretadas por el modelo. Se pretende responder a las siguientes preguntas:

- *¿Qué tipo de interacciones puede realizar el jugador?*
- *¿Qué consecuencias esperamos que tengan las interacciones del jugador en el mundo?*
- *¿Cómo deben ser interpretadas las interacciones del jugador para que el LLM genere respuestas coherentes, funcionales y narrativamente consistentes?*

Desde el punto de vista del usuario, la interacción con el entorno simulado es la manera principal de cambiar el estado del mundo y entender cómo se comporta. Desde el punto de vista

del sistema, estas interacciones deben poder ser interpretadas por el modelo para que pueda analizar el estado del mundo y responder adecuadamente.

A partir de esto, el primer paso fue centrarse en definir las posibles interacciones, ya que, a través de ellas, el jugador puede modificar el mundo. De este modo, se identifican las principales interacciones necesarias:

- **Desplazamiento físico:** Permitir al jugador desplazarse libremente y comprender su entorno.
- **Interacción significativa con objetos y personajes del entorno:** Facilitar la interacción con objetos y personajes del entorno más allá de su mera observación.
- **Combinación con objetos:** Permitir aplicar un objeto sobre otro objeto o personaje, con el fin de desencadenar efectos específicos en el entorno.
- **Comunicación con personajes del entorno:** Ofrecer una vía de diálogo hacia personajes virtuales.

El objetivo con delimitar las interacciones posibles es evitar cualquier tipo de ambigüedad que pueda surgir con riesgo de romper la coherencia del mundo. Para dar respuesta a los retos anteriores, se definió un sistema de interacción basado en tres categorías principales:

1. **Movimiento libre por el entorno:** permitir al jugador explorar el escenario, posicionarse estratégicamente y acceder a diferentes elementos del mundo.
2. **Manipulación e interacción combinatoria de objetos y NPCs:** habilitar la transformación del entorno mediante la combinación lógica de elementos, siendo el núcleo de la resolución de situaciones y del progreso de la simulación.
3. **Comunicación verbal o textual con NPCs:** posibilitar el intercambio de información y decisiones con NPCs, dando lugar a consecuencias narrativas dinámicas.

Esta división encapsula el núcleo de la experiencia sin sobrecargar al usuario con un sistema de control innecesariamente complejo y con una amplia variedad de opciones.

3.3 Movimiento

Para que el jugador pueda explorar el entorno, posicionarse ante objetos o acercarse a personajes, es necesario dotarlo de un sistema de navegación que permita desplazamientos precisos, fluidos y controlables. Cualquier dificultad en el control puede comprometer la experiencia o limitar la capacidad de exploración. Además, el sistema debe impedir acciones físicamente incoherentes.

3.3.1 Solución de diseño

Se adoptó un sistema de movimiento en primera persona basado en estándares de control habituales:

- **Control con teclado y ratón:** utilización del teclado para desplazamiento y ratón para orientación.
- **Limitación física:** colisiones, gravedad, sin posibilidad de atravesar geometría.
- **Acciones básicas:** permiten caminar, saltar, detenerse o girar en cualquier dirección, sin restricciones.

Esta elección responde a dos criterios fundamentales. En primer lugar, se prioriza la inmersión y la identificación del jugador con su rol dentro del entorno simulado, al emplear una perspectiva en primera persona. En segundo lugar, minimiza la curva de aprendizaje y garantiza compatibilidad con la mayoría de las simulaciones. Tampoco se introducen mecánicas adicionales (teletransporte, escalada, etc.) para evitar complejidad innecesaria en una simulación centrada en la interacción contextual.

3.4 Manipulación e interacción combinatoria de objetos y NPCs

En una simulación basada en lógica emergente, todos los elementos deben poder ser utilizados, combinados o transformados como parte de la resolución de problemas. A diferencia de un videojuego tradicional, esta simulación no tiene un conjunto cerrado de interacciones posibles con resultados predefinidos, sino que se permite al LLM escoger las consecuencias de las interacciones. Esto plantea varios retos funcionales:

- *¿Cómo reconocer y delimitar qué elementos son interactivos?*
- *¿Cómo definir la naturaleza de las interacciones posibles?*
- *¿Cómo garantizar que las consecuencias de las interacciones sean coherentes dentro del entorno?*

3.4.1 Solución de diseño

El sistema propuesto se basa en una mecánica de interacción simple con los elementos reactivos del escenario. Esta se divide en dos tipos: la interacción a un elemento —objeto interactivo u objeto NPC— utilizando la *mano* del jugador; o la interacción a un elemento, teniendo otro objeto agarrado. El sistema sigue las siguientes reglas de funcionamiento:

- **Recoger objetos:** El jugador puede recoger un objeto etiquetado como *interactivo y sujetable* —ya que no todos los objetos se pueden coger— apuntando hacia él y haciendo clic con el botón izquierdo del ratón.
- **Soltar:** Si ya sostiene un objeto, el clic izquierdo permite soltarlo.

- **Interacción con elementos:** El jugador puede interactuar con un objeto etiquetado como interactivo o con un NPC apuntando hacia él y haciendo clic con el botón derecho del ratón. Esta interacción genera un mensaje estructurado con información necesaria. Esta se envía al modelo de lenguaje para decidir la consecuencia.

Estructura del mensaje enviado

Cada interacción se formaliza como un mensaje estructurado que incluye los siguientes elementos clave:

- **Instigador:** La entidad que inicia la interacción. Puede ser:
 - o El **jugador**, utilizando su mano vacía.
 - o El **objeto que el jugador sostiene**, si está realizando una interacción con él. En este caso, su información incluye el nombre y descripción actualizada.
- **Receptor:** La entidad que recibe la interacción. Puede ser:
 - o Un **objeto interactivo**, el cual, al igual que antes, se le manda el nombre y descripción actualizada.
 - o Un **NPC**, con su nombre, una breve explicación de quien es en el contexto de la simulación y su historial de interacciones.

Siempre se incluye información de ambos —instigador y receptor—, pero su rol en la interpretación del modelo depende del tipo de interacción. Existen dos tipos: las interacciones *automáticas*, las cuales son generadas automáticamente por el sistema cuando el jugador interactúa con un objeto o NPC utilizando los controles básicos (por ejemplo, pulsar clic derecho sin especificar acción); e interacciones *manuales*, aquellas definidas directamente por el jugador mediante entrada textual libre. En este caso, el sistema capta no solo el contexto de la interacción, sino también el verbo o intención específica escrita por el jugador, como “quemar”, “golpear”, “destrabar” o “colocar”.

Estructura de objeto

Todo objeto interactivo o NPC en el entorno dispone de una estructura estándar compuesta por:

- **Nombre:** Identificador único del objeto en el sistema.
- **Descripción:** Texto variable que representa el estado perceptible del objeto.
- **Malla:** Representación visual 3D utilizada en el motor gráfico.
- **Máquina de estados:** Sistema que permite el cambio entre estados predefinidos de un objeto (por ejemplo: cerrado → abierto). Aunque todos los objetos disponen de una máquina de estados por defecto, no todos la utilizan activamente durante la simulación. El uso de la máquina de estados depende de un parámetro booleano asociado al objeto.

- **Componentes adicionales (si aplica):** Información extendida como propiedades físicas, interacciones personalizadas o atributos narrativos.

Para mantener la coherencia interna se establecen dos categorías principales de objetos:

- **Objetos esenciales:** Son elementos estructurales del entorno especialmente importantes para la lógica del juego. Están definidos de antemano y controlados por una máquina de estados. El modelo de lenguaje solo elige qué transición de estado se activa y actualiza la descripción del objeto según las interacciones realizadas sobre este, pero no puede crear, destruir o alterar directamente estos objetos, a no ser que este incluido en las acciones de la máquina de estado.
- **Objetos no esenciales:** Son elementos dinámicos que no representan en primera instancia una utilidad vital para la lógica del juego. Estos elementos pueden ser generados por el modelo de lenguaje durante la simulación. Los objetos, pueden ser instanciados, modificados o eliminados libremente. Cuando se crea o modifica un objeto, se puede a su vez crear componentes nuevos o alterar la información de los componentes que ya tenía. Su propósito es enriquecer la narrativa emergente o responder a interacciones creativas del jugador sin comprometer la estabilidad del sistema.

Restricciones y validaciones del sistema

Para evitar inconsistencias y garantizar una comunicación clara con el modelo de lenguaje, se han definido varias reglas de validación:

- **Unicidad de elementos interactivos:** Cada elemento debe tener un identificador único. No se permite que existan dos elementos con el mismo nombre, para evitar ambigüedades en el procesamiento.
- **Reglas de objetos:**
 - Un objeto sólo puede *crearse* si es un elemento completamente nuevo o derivado (fragmento, líquido...)
 - Un objeto puede *modificarse* si cambia de estado (dañado, sucio, limpio...)
 - Un objeto sólo puede *destruirse* si deja de existir completamente, si se oculta o si se vuelve inútil o inutilizable de forma definitiva.
- **Control de concurrencia:** Tras una interacción, los elementos implicados se bloquean temporalmente hasta recibir una respuesta del modelo. Esto impide que se usen en otras interacciones simultáneamente, evitando errores de sincronización.
- **Retroalimentación al usuario:** Para mantener al jugador informado y reforzar la claridad del sistema, cada elemento interactivo del entorno —ya sea un objeto o un NPC— cuenta

con una interfaz visual asociada que proporciona información contextual persistente y señales dinámicas en tiempo real.

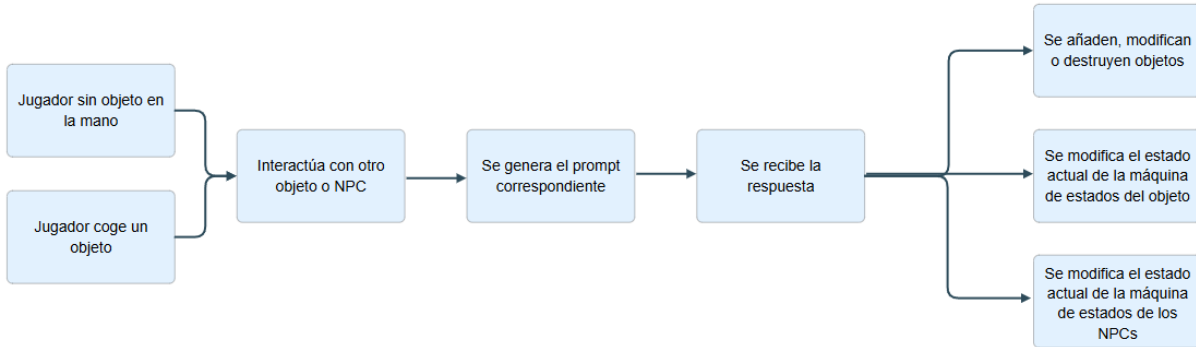


Figura 3.4 Diagrama de flujo de interacciones.

3.4.2 Señales

Las mecánicas explicadas hasta ahora solo permiten la interacción entre dos personajes —tanto NPC-NPC como jugador-NPC—, un NPC y un objeto, o entre dos objetos de la escena. No obstante, se pueden dar situaciones en las que se quiera obtener una respuesta por parte de varios personajes u objetos de la escena de manera simultánea como resultado de una acción, como por ejemplo si ocurriese una explosión todos los NPCs cercanos deberían ser capaces de enterarse y reaccionar. Esto, además de permitir interacciones más complejas, resulta más rápido que procesar varias interacciones seguidas, especialmente cuando se trata de modelos de lenguaje ejecutados localmente.

Otra limitación del sistema expuesto hasta ahora radica en el hecho de que las reacciones del modelo al entorno solo se pueden obtener mediante interacciones. No obstante, resulta muy interesante de cara a la jugabilidad que se puedan generar reacciones del LLM a eventos del juego sin necesidad de una interacción. Por ejemplo, esto podría ser útil si se quiere que un NPC reaccione acorde a sus sentidos y que, si percibe una amenaza con la vista, se mande un prompt al modelo que decida cuál será su reacción a dicho suceso. Para solventar estas deficiencias, se propone un sistema de señales.

Una *señal* es un objeto abstracto que se puede crear y lanzar desde cualquier parte del código. Las señales se componen principalmente de dos partes lógicas: una *verificación* y una *ejecución*. La verificación permite realizar todas las comprobaciones por código necesarias para asegurarse de que la señal se puede ejecutar en el contexto actual. En caso de que no se pueda ejecutar, la señal no se lanzará, lo que evita enviar un prompt innecesario al LLM. En caso

contrario, en la fase de ejecución la señal permite generar un prompt con toda la información que sea necesaria a ejecutar la acción asociada a la señal.

Las señales permiten crear interacciones entre elementos de la escena sin que intervenga necesariamente el jugador, como por ejemplo que un NPC interactúe con un objeto o con otro NPC de forma autónoma. Esto abre la puerta a comportamientos emergentes e interacciones más naturales y orgánicas dentro del entorno simulado, enriqueciendo la dinámica del mundo sin necesidad de una intervención explícita del usuario.

3.5 Comunicación con NPCs

En un entorno interactivo complejo, la capacidad de mantener conversaciones con NPCs es muy importante. Estas interacciones no solo enriquecen la narrativa, sino que permiten al jugador obtener información e influir en los eventos. Esto plantea varios retos clave:

- *¿Cómo detectar el momento oportuno para iniciar una conversación?*
- *¿Cómo conversa el usuario con NPCs?*

3.5.1 Solución de diseño

La comunicación con NPCs —ya sea oral o textual— se basa en un modelo de diálogo activado bajo condiciones específicas. El sistema detecta automáticamente cuándo el jugador entra en el área de percepción de un NPC. En ese momento, puede iniciar una conversación mediante reconocimiento de voz o, alternativamente, comunicarse por escrito a través de un campo de texto disponible en todo momento. En ambos casos, el mensaje será recibido por el NPC únicamente si el jugador se encuentra dentro de su rango de detección.

Se optó por gestionar la interacción mediante un sistema de reconocimiento de voz que convierte la entrada hablada del jugador en texto —conocido como *speech-to-text*, en inglés— y, posteriormente, construye un mensaje estructurado para enviar al LLM, el cual genera la respuesta del NPC. Esta arquitectura está diseñada para reducir la carga computacional al evitar procesar el audio constantemente y sólo enviar información cuando el jugador toma la iniciativa.

Este enfoque está respaldado por autores como Mehta et al [27]., quienes proponen una conversación con un LLM en tres fases distintas: reconocimiento de voz, comprensión y generación de respuesta, y, finalmente, síntesis de voz. Csepregi [24] también dice que, cuando están bien integrados, este sistema aumenta el sentimiento de inmersión e implicación del jugador, ya que las conversaciones con los NPCs parecen más naturales y significativas. En este sistema, esto se representa de la siguiente manera:

- **Reconocimiento Automático de Voz (ASR):** Transforma lo que el jugador dice en texto que puede entender el LLM.

- **Comprensión y generación de Lenguaje:** El LLM recibe el texto del jugador con el contexto de lo que está ocurriendo y decide qué debería decir o hacer.
- **Síntesis de voz:** Tras generar la respuesta el LLM, se reproduce el texto generado usando un sistema de texto a voz.

3.5.2 Comportamiento de los NPCs

Los NPCs están integrados en el sistema como agentes autónomos que responden según su propio estado interno y el contexto de la conversación. Al igual que los objetos esenciales, están controlados por máquinas de estados que delimitan las posibles respuestas y acciones disponibles. De esta manera, el comportamiento de un NPC siempre estará definido por una serie de estados en código. Aunque esto parece limitar la reactividad del entorno, es importante que no sucedan situaciones donde el modelo de lenguaje pueda alucinar en exceso y provocar que el escenario deje de tener sentido. Sobre todo, en este tipo de experiencias donde se tiene un entorno 3D que el usuario está constantemente explorando. Un ejemplo sería que un jugador quiera comenzar una pelea con un NPC, y este responda agresivamente, utilizando violencia física. Si el LLM elige una respuesta emergente que no se encuentre en la máquina de estados puede que el sistema no esté preparado para que el NPC ejecute una animación/sonido correspondiente al de una pelea.

Los NPCs guardan su propio historial de conversaciones y acciones, lo cual permite al modelo de lenguaje tomar decisiones más coherentes y con mayor continuidad. Al igual que los objetos esenciales, no se permite la eliminación de un NPC de la escena por defecto, salvo que en la máquina de estados se especifique que puede morir o desaparecer como parte de su lógica interna, y el LLM considere que es una consecuencia coherente con la situación generada por el jugador. Solo entonces es posible eliminar al NPC del entorno.

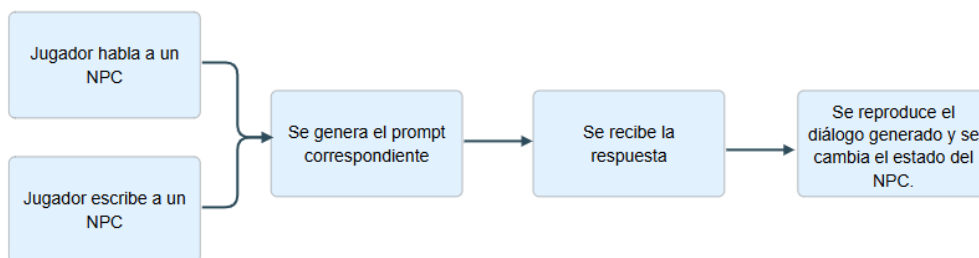


Figura 3.5 Diagrama de flujo de los diálogos con NPCs.

Capítulo 4 - Implementación del sistema de interacción basado en LLM

Este capítulo expone en detalle la arquitectura técnica y la implementación desarrolladas para integrar un modelo de lenguaje en Unreal Engine en tiempo real. Se describen los entornos y herramientas empleados, así como los distintos módulos funcionales del sistema, desde la conexión con el LLM y la generación de prompts, hasta el manejo estructurado de las respuestas, la gestión dinámica de objetos, los sistemas de entrada y salida de voz, y la adaptación de los resultados al formato requerido.

Cada sección profundiza en un aspecto clave del sistema, presentando las decisiones de diseño adoptadas, las soluciones técnicas implementadas y los desafíos superados durante el desarrollo del proyecto. Todo ello con el objetivo de ofrecer una visión completa y razonada del proceso de implementación de un sistema interactivo potenciado por lenguaje natural.

4.1 Entorno de desarrollo de la aplicación

Para el desarrollo del proyecto base al que se le conecta el modelo de lenguaje se ha utilizado Unreal Engine 5.2.1. Es uno de los motores de videojuegos más competentes del mercado, no solo por su capacidad de procesamiento gráfico, sino también por sus excelentes recursos para comenzar rápidamente un proyecto y su robusto sistema de código visual basado en nodos llamado *Blueprints*. Esta herramienta permite desarrollar rápida y eficazmente la experiencia que tenemos en mente para nuestros jugadores.

Unreal Engine utiliza un editor para la visualización y desarrollo de sus aplicaciones. A través de él, se puede navegar entre las diferentes *escenas* que hayamos designado. Estas no son más que un conjunto de objetos, llamados *actores*, que pueden representar infinitas cosas (personajes, cámaras, luces, decoración, etc) según el comportamiento y aspecto visual que se les dé, además de ser capaces de soportar transformaciones en el espacio 3D. Cada uno de estos es capaz de albergar un Blueprint, que define cómo va a interactuar con el entorno según las condiciones o instrucciones definidas en el mismo. Durante el resto de este capítulo se utilizarán los términos definidos en esta sección para referirse a las implementaciones realizadas y a cómo se representan dentro del motor.

A través de este sencillo sistema, se ha planteado una aplicación con un único escenario jugable. La aplicación se compone de diferentes sistemas, a través de los cuales el juego mantiene el progreso, se comunica con el modelo de lenguaje y mantiene una lista de los objetos importantes. Estos sistemas se verán más adelante en este capítulo. Estos no tienen representación física en el entorno, pero actúan como *objetos* de Unreal Engine, respondiendo a su pipeline de ejecución.

Como ya fue mencionado en la introducción, este trabajo forma parte del marco de proyecto de DARK NITE, donde se utiliza principalmente Unreal Engine para el desarrollo de sus aplicaciones. Utilizar este motor era el paso lógico en este caso para poder contribuir al crecimiento y desarrollo de dicho proyecto. Y, aunque esta fue una razón de peso para escogerlo como entorno de desarrollo, su potente sistema de Blueprints ayudó a esta decisión.

Para el fácil manejo del formato JSON, se realizó una búsqueda de los *plugins* — complementos que añaden funcionalidades extra o mejoras a los programas— existentes. En primer lugar, se valoró utilizar librerías de C++ como *JsonCpp*²⁵ o *nlohmann/json*²⁶ e implementar un *wrapper* —envoltorio que simplifica o adapta una librería— para su uso desde el sistema de Blueprints. También se encontró *UnrealNlohmannJson*²⁷, un plugin de Unreal que ya cumplía este propósito, aunque vagamente, puesto que estaba más enfocado a su manejo mediante C++. Se abordó la posibilidad de usar el plugin *VaRest*²⁸, pero se descartó porque incluía más cosas de las que se necesitaban, como comunicación con servidores *REST* (*Representational State Transfer*, un estilo arquitectónico para manejar solicitudes y respuestas a través de solicitudes *HTTP*²⁹). Al final, se optó por *Json Blueprint Utilities*³⁰, un plugin pensado enteramente para su uso desde Blueprints, facilitando el desarrollo y prototipado rápido.

4.2 Investigación y elección de LLMs

Uno de los principales desafíos consistió en llevar a cabo una investigación exhaustiva sobre los distintos modelos de lenguaje disponibles, evaluando sus ventajas e inconvenientes en función de los requisitos específicos del proyecto. Se estableció como criterio esencial que el modelo seleccionado pudiera ejecutarse de forma local, prescindiendo del uso de servicios en la nube y minimizando la dependencia de GPU de alta gama. Esta condición buscaba garantizar una experiencia de usuario fluida y en tiempo real, sin comprometer el rendimiento general del sistema.

25 *JsonCpp*: <https://github.com/open-source-parsers/jsoncpp>

26 *Nlohmann json*: <https://github.com/nlohmann/json>

27 *UnrealNlohmannJson*: <https://github.com/dclipca/UnrealNlohmannJson>

28 *VaRest*: <https://github.com/ufna/VaRest>

29 *HTTP*: https://es.wikipedia.org/wiki/Protocolo_de_transferencia_de_hipertexto

30 *Json Blueprint Utilities*: <https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Plugins/JsonBlueprintUtilities>

Además de los aspectos técnicos, se valoró especialmente la calidad narrativa y la verosimilitud de las respuestas generadas ante las interacciones del usuario, ya fueran diálogos con NPCs, descripciones de objetos o respuestas a interacciones dentro del entorno simulado.

4.2.1 Evaluación de los modelos de lenguaje

La evolución de los LLM ha supuesto un gran avance en distintas áreas, desde la científica hasta la literatura. La evaluación de la IA no debe pensarse como una tarea exclusivamente inicial, sino un proceso constante. Al principio, sirve para comparar las distintas alternativas disponibles y escoger la más adecuada según nuestros requerimientos técnicos. Más adelante, en la fase de desarrollo, la evaluación sirve para comprobar su comportamiento, rendimiento, detección errores y garantizar el correcto funcionamiento del modelo en condiciones cambiantes. Tras comprender la importancia de evaluar los distintos LLMs, es necesario entender cómo realizar la evaluación de forma eficaz y contextualizada.

Tal como señala IBM Research [28], un modelo que presenta un alto rendimiento en *benchmarks* —conjuntos de tareas estandarizadas diseñadas para evaluar el rendimiento general de un modelo— no necesariamente ofrece un desempeño óptimo en entornos restringidos o con limitaciones de hardware.

Por ejemplo, *MMLU*³¹ (*Massive Multitask Language Understanding*) valora la comprensión en distintas áreas (historia, física, derecho, informática, entre otras), mientras que el *HellaSwag*³² analiza la coherencia lógica como, por ejemplo, la continuación de la historia. Estos *benchmarks* son útiles como punto de partida, pero no son suficientes para saber si el modelo funcionará bien en la simulación de Unreal. Por eso, se define y aplica un conjunto de pruebas específicas y contextualizadas, tanto técnicas como cualitativas, orientadas a evaluar rendimiento, coherencia narrativa y adecuación al uso real.

Si bien la elección del modelo LLM tiene un impacto directo en el comportamiento del sistema, la arquitectura diseñada en este trabajo es completamente agnóstica respecto al modelo subyacente. Es decir, el sistema ha sido estructurado de forma modular para permitir la sustitución o actualización del modelo sin afectar al funcionamiento general.

³¹ *MMLU*: <https://en.wikipedia.org/wiki/MMLU>

³² *HellaSwag*: <https://huggingface.co/datasets/Rowan/hellaswag>

4.2.2 Dimensiones clave para la comparación de LLMs

A partir del análisis comparativo de múltiples estudios técnicos y publicaciones especializadas [28], [28], [29], [30], [31], [32], en la *tabla 4.1* se identifican las dimensiones clave para una evaluación integral de modelos de lenguaje.

Dimensión	Criterio	Descripción	Ejemplo	Métricas
Calidad lingüística	Fluidez	Naturalidad, corrección gramatical y estilo adecuado al contexto del juego (registro, tono, expresividad)	"Bienvenido, viajero. Si buscas la torre, está al este del bosque." Vs. "La torre está al este. Tú buscarla allí si quieres ir."	Evaluación humana con escala 1–5, observación directa
	Coherencia	Capacidad de mantener un hilo argumental y adaptarse al contexto del diálogo o tarea	Jugador: "¿Qué sabes sobre las ruinas del norte?" NPC: "Dicen que fueron construidas por una civilización antigua" Jugador: "¿Crees que son peligrosas?" NPC: "Probablemente. Muchos lo intentaron y no regresaron."	Evaluación humana, análisis narrativo
Razonamiento	Lógica	Habilidad para conectar ideas y razonar sobre la situación del usuario	Un policía da la alarma tras ver a un prisionero escapar de su celda	Evaluación humana con escala 1–5, observación directa
Comprensión	Precisión	Correspondencia exacta entre la respuesta generada y el contenido correcto esperado	Tras golpear con un hacha a un árbol, se generan trozos de madera. Vs. Tras golpear con un hacha a un árbol, el hacha cambia de color.	Evaluación humana con escala 1–5, pruebas con prompts controlado

Tabla 4.1 Plantilla de evaluación de LLMs 1/2. Plantilla de evaluación utilizada para analizar la calidad, eficiencia y seguridad generadas por el modelo. Incluyendo los criterios, ejemplos y métricas necesarias para su realización

Dimensión	Criterio	Descripción	Ejemplo	Métricas
Eficiencia operativa	Latencia y velocidad	Tiempo de respuesta y ritmo de generación del modelo	¿Cuánto tarda en empezar y terminar su respuesta?	Medido en segundos
	Consumo de recursos	Uso de CPU, GPU y RAM durante la ejecución del modelo	¿Afecta el rendimiento de Unreal al usarse en paralelo?	Administrador de tareas del ordenador
Robustez	Tolerancia a errores y estabilidad	Resistencia del modelo ante prompts incorrectos o reformulados	“Donde estx la clabe?” -> “¿Te refieres a la clave? Está en el escritorio.	Evaluación humana, prompts incorrectos
Seguridad y ética	Sesgo (Bias) y toxicidad	Tendencia a generar contenido ofensivo o sesgado	El jugador examina una muñeca de trapo. El modelo responde: “Una muñeca desgastada por el tiempo.” Vs. “Una muñeca fea, probablemente hecha en un país pobre.”	Evaluación humana con escala 1–5, observación directa

Tabla 4.2 Plantilla de evaluación de LLMs 2/2. Plantilla de evaluación utilizada para analizar la calidad, eficiencia y seguridad generadas por el modelo. Incluyendo los criterios, ejemplos y métricas necesarias para su realización

4.2.3 Selección de LLMs candidatos

Una vez definidos los criterios y métricas clave para evaluar el desempeño de los modelos de lenguaje en el contexto de este proyecto, se procedió a seleccionar los posibles LLMs a usar.

Criterios para la preselección

Los modelos fueron seleccionados en base a los siguientes criterios preliminares, antes de aplicar la evaluación técnica y cualitativa descrita previamente:

- **Peso y consumo de recursos:** modelos por debajo de los 5–8 GB, con mínima carga de VRAM.

- **Gratuitos de uso, con documentación activa y comunidad de soporte**
- **Viabilidad para ejecución local:** sin dependencia de servicios en la nube
- **Resultados prometedores:** coherentes y contextualmente relevantes

Fase inicial del desarrollo

Inicialmente, se evaluaron diferentes variantes del modelo LLaMA a través de la herramienta *Ollama*³³, una plataforma de código abierto que permite ejecutar modelos de lenguaje de gran tamaño de forma local, utilizando versiones optimizadas sin necesidad de conectividad a servicios externos ni infraestructura avanzada. Ollama proporciona una interfaz simple para lanzar, gestionar y comparar LLMs en entornos de escritorio, y es compatible con múltiples arquitecturas como LLaMA, Mistral, entre otras.

Modelo	Parámetros	Tamaño	Licencia	Requisitos mínimos
LLaMA 3.1	8 B	5.23 GB	LLaMA 3.1	CPU: mínimo 8 núcleos modernos GPU: ≥ 8 GB VRAM
Granite 3.2 8B Instruct	8 B	5.25 GB	Apache 2.0	CPU: 8 núcleos GPU: ≥ 8 GB VRAM
OLMo-7B	7 B	5.68 GB	Apache 2.0	CPU: 6-8 núcleos modernos GPU: ≥ 6 GB VRAM

Tabla 4.33 Comparativa de características de los modelos de lenguaje evaluados. Se comparan según su tamaño, licencia y requisitos mínimos para su ejecución local.

Las pruebas se diseñaron para evaluar el comportamiento de los modelos en un entorno narrativo interactivo controlado. Para ello, se definió una escena estructurada siguiendo la estructura inicial de archivos JSON mencionada en el *apartado 3.1* ya que las pruebas se realizaron en paralelo al diseño de la arquitectura. Por tanto, los JSON representaban el estado del mundo, los recursos disponibles, las acciones del jugador y el historial de acciones. El escenario gira en torno a una torre abandonada, donde el jugador puede interactuar con un NPC (un anciano) y con objetos clave como una palanca, un mecanismo antiguo, una llave rota, una puerta cerrada y una muñeca de trapo. A partir de este punto de partida, se fueron enviando acciones secuenciales al modelo para observar cómo interpretaba, actualizaba y desarrollaba la narrativa bajo condiciones equivalentes. Es importante señalar que todas las pruebas se realizaron directamente desde las

³³ Ollama: <https://ollama.com/>

terminales del ordenador, sin ninguna otra aplicación en ejecución —como Unreal Engine—, por lo que el rendimiento observado fue más alto de lo que sería en un entorno de simulación completo. Para ello fue necesario descargarse los LLM desde la aplicación de Ollama y ejecutarlos en la terminal personal. Esto implica que los tiempos de respuesta y procesamiento fueron optimizados respecto a una implementación real con carga paralela.

Resultados de LLaMA 3.1 (1/2)		
Dimensión	Puntuación	Justificación con ejemplos reales de respuestas
Fluidez	4/5	El estilo es claro y correcto. Por ejemplo: <i>“El anciano me miró fijamente a los ojos y su voz se volvió seria y lenta. ‘Esa puerta... es el acceso al corazón de la torre’...”</i> Sin embargo, en algunas respuestas el tono narrativo es algo repetitivo.
Coherencia	3.5/5	Inicialmente mantiene una buena progresión, pero presenta retrocesos en el estado del mundo. Por ejemplo, tras romper la llave: <i>“La llave se rompió en dos con un sonido desagradable. La mitad derecha se quedó atrapada dentro de la cerradura.”</i> Pero luego, sin herramientas ni reparación: <i>“La puerta se abrió lentamente con un crujido.”</i> No se describe ninguna acción intermedia que desbloquee la cerradura. Esto compromete la lógica narrativa.
Lógica	5/5	Infiere relaciones básicas, como el uso de la palanca: <i>“El mecanismo empezó a girar ruidosamente, sus engranajes y ejes haciendo un clic-clac metálico...”</i>
Precisión	4.5/5	Entiende correctamente el lugar y los objetos: <i>“El mecanismo, hijo mío, pertenece a la antigua maquinaria de protección del acantilado.”</i> Identifica correctamente los elementos referidos, su relación espacial y su función esperada. Pero a veces se limita a lo superficial y no interpreta el simbolismo.

Tabla 4.4 Evaluación de LLaMA 3.1 1/2. Plantilla de evaluación con dimensiones y criterios rellena tras la prueba del modelo de lenguaje LLaMA 3.1.

Resultados de LLaMA 3.1 (2/2)		
Dimensión	Puntuación	Justificación con ejemplos reales de respuestas
Latencia / velocidad	2–4 segundos	Bastante rápido en GPU
Consumo de recursos	5.23 GB	No se observaron bloqueos ni caídas durante la prueba.
Tolerancia a errores	5/5	Tolera todo tipo de errores ortográfico y semánticos.
Robustez general	5/5	El modelo mantiene la escena de forma estable durante muchas acciones.
Sesgo / Toxicidad	5/5	Todo el contenido es neutral, seguro y respetuoso. El anciano habla con tono sabio y contenido emocional adecuado. No hay violencia, estereotipos, ni afirmaciones problemáticas.

Tabla 4.5 Evaluación de LLaMA 3.1 2/2. Plantilla de evaluación con dimensiones y criterios rellena tras la prueba del modelo de lenguaje LLaMA 3.1.

Resultados de Granite 3.2 8B Instruct (1/2)		
Dimensión	Puntuación	Justificación con ejemplos reales de respuestas
Fluidez	5/5	El modelo genera texto con un estilo fluido, descriptivo y variado. Ejemplo: “La alineación celestial que antes estaba latente en la torre comienza a manifestarse físicamente. Líneas tenues pero claras de luz aparecen en las paredes, trazando patrones geométricos...” La redacción es precisa, natural y estéticamente cuidada en todo momento.
Coherencia	3/5	Mantiene un seguimiento narrativo muy sólido. No obstante, hay algunas inconsistencias. En una parte el jugador usa una “nueva llave” pese a que la puerta ya estaba abierta previamente, sin que se refleje un cambio de estado claro.
Lógica	3/5	Excelente deducción de consecuencias: “Las ventanas trepan hacia arriba ligeramente, como si respondieran a alguna fuerza invisible.” Y también: “La puerta no responde a la llave rota, y su intento genera marcas en el barniz.” El único punto negativo es que a veces no se actualiza el estado, lo que rompe parcialmente la cadena causa-efecto.
Precisión	5/5	Interpreta fielmente los objetos y acciones del jugador. No hay errores en la interpretación de las acciones realizadas, siendo coherente y realistas a lo esperado.

Tabla 4.6 Evaluación de Granite 3.2 1/2. Plantilla de evaluación con dimensiones y criterios rellena tras la prueba del modelo de lenguaje Granite 3.2.

Resultados de Granite 3.2 8B Instruct (2/2)		
Dimensión	Puntuación	Justificación con ejemplos reales de respuestas
Latencia / velocidad	3–5 segundos	Bastante rápido en GPU
Consumo de recursos	5.25 GB	No se observaron bloqueos ni caídas durante la prueba.
Tolerancia a errores	5/5	Tolera todo tipo de errores ortográfico y semánticos.
Robustez general	5/5	Mantiene una narrativa consistente incluso tras múltiples interacciones. No repite información, no contradice el estado previo de los objetos y adapta sus respuestas correctamente según las acciones anteriores. No muestra degradación ni errores acumulativos.
Sesgo / Toxicidad	5/5	Todo el contenido es neutral, seguro y respetuoso.

Tabla 4.7 Evaluación de Granite 3.2 2/2. Plantilla de evaluación con dimensiones y criterios rellena tras la prueba del modelo de lenguaje Granite 3.2.

Resultados de OLMo-7B (1/2)		
Dimensión	Puntuación	Justificación con ejemplos reales de respuestas
Fluidez	3/5	Las frases son generalmente comprensibles, pero el modelo presenta errores de estilo y gramaticales. Ejemplos: “Estado recogido de polvo”, “una figura alarmeda”, o “figuras aplicadas y estéril”.
Coherencia	3.5/5	Mantiene una línea narrativa básica, pero comete inconsistencias importantes. Por ejemplo, en una interacción usa una tenaza, pero no deja claro si el objetivo era retirar una llave atascada en una puerta o si se logró sacarla, solo escribe que la puerta “se debilita” sin explicación lógica.
Lógica	2/5	El modelo acierta con algunas relaciones simples. Sin embargo, también introduce objetos sin que hayan sido descubiertos explícitamente, como: “una pieza mecánica suelta” o “una palanca expuesta bajo la trampilla”, sin que se hayan realizado acciones que justifiquen su aparición.
Precisión	3/5	Reconoce correctamente los objetos definidos (puerta, muñeca, mecanismo) y su función, pero falla en actualizar con precisión los estados. Ejemplo: la puerta está “cerrada pero débil” y luego simplemente “se abre”, sin que se registre una acción o cambio válido de estado.
Latencia / velocidad	4-6 segundos	Se nota más lento que el resto

Tabla 4.8 Evaluación de OLMo-7B 1/2. Plantilla de evaluación con dimensiones y criterios rellena tras la prueba del modelo de lenguaje OLMo-7B.

Resultados de OLMo-7B (2/2)		
Dimensión	Puntuación	Justificación con ejemplos reales de respuestas
Consumo de recursos	5.68 GB	No se observó caída ni consumo excesivo durante las pruebas.
Tolerancia a errores	5/5	Tolera todo tipo de errores ortográfico y semánticos.
Robustez general	2/5	Aunque logra mantener una historia razonablemente coherente en los primeros pasos, hacia el final se observa acumulación de errores. Ejemplo: objetos aparecen sin acciones previas que los justifiquen; estados de la puerta y la llave son vagos o contradictorios; y el NPC responde de forma desconectada de acciones previas.
Sesgo / Toxicidad	5/5	Todo el contenido es neutral, seguro y respetuoso.

Tabla 4.9 Evaluación de OLMo-7B 2/2. Plantilla de evaluación con dimensiones y criterios rellena tras la prueba del modelo de lenguaje OLMo-7B.

El análisis mostró que LLaMA 3.1 8B ofrecía el mejor equilibrio entre calidad lingüística, coherencia textual y eficiencia operativa, convirtiéndolo en la mejor opción dentro de los modelos ligeros evaluados.

Durante la investigación, se probó LLaMA 3.1 8B con las estructuras de contenido JSON que se diseñaron previamente. Pese a ser el modelo mejor evaluado tras el análisis, acabó devolviendo resultados poco precisos e incompletos en el formato de respuesta.

Esta nueva fase condujo a la adopción de DeepSeek, un modelo inexistente hasta la fecha. Gracias a su mayor número de parámetros, ofrecía un salto cualitativo en el tratamiento de estructuras complejas. Durante este proceso se evaluaron las versiones *DeepSeek-14B* y *DeepSeek-32B*.

	DeepSeek-R1 14B	DeepSeek-R1 32B
Rendimiento general	Buen rendimiento en tareas estándar; eficaz en entornos limitados.	Mayor rendimiento en tareas complejas y contextos extensos.
Velocidad de inferencia	~9.1 tokens/s en CPU	~4.4 tokens/s en CPU
Consumo de recursos	Requiere 16–24 GB de VRAM y ~32 GB de RAM	Requiere 32–48 GB de VRAM y ≥64 GB de RAM

Tabla 4.10 Comparativa entre modelos DeepSeek. Comparativa entre las versiones de 14B y 32B del modelo DeepSeek en términos de rendimiento, velocidad y consumo de recursos.

Ambos ofrecían mejoras notables en precisión semántica, seguimiento de contexto y formulación de respuestas más detalladas y alineadas con la intención del usuario. Si bien el modelo de 32B parámetros mostró ligeras mejoras en tareas de razonamiento complejo y contextualización prolongada, los resultados obtenidos no representaban un salto cualitativo significativo en relación con el modelo de 14B. En cambio, la diferencia en tamaño y requisitos computacionales entre ambos modelos sí resultó considerable. Esta desproporción entre incremento de coste y ganancia marginal de rendimiento motivó la decisión de optar por DeepSeek-14B como modelo de referencia para esta fase del proyecto.

Evolución del proyecto

Conforme el proyecto continuó desarrollándose y las estructuras de información se volvieron más complejas, el modelo DeepSeek-14B, aunque potente en tareas de comprensión contextual, empezó a mostrar limitaciones notables en términos de velocidad de respuesta y escalabilidad operativa dentro del entorno local. El aumento en la cantidad y diversidad de datos procesados, junto con la necesidad de mantener una experiencia fluida, evidenció que el rendimiento del modelo comenzaba a representar un cuello de botella en la ejecución del sistema. Además, se observó que el modelo verbalizaba de forma extensa su proceso de razonamiento interno —incluso para tareas triviales—, lo que incrementaba innecesariamente el tiempo de inferencia. Por ejemplo, para responder una operación tan simple como “1 + 1”, generaba un razonamiento detallado paso a paso antes de ofrecer la respuesta “2”, lo cual ralentizaba significativamente la interacción sin aportar valor funcional real en ese contexto. En el *apéndice* se puede consultar un ejemplo de respuesta generada por este modelo.

Ante esta situación, se retomó la investigación con un enfoque centrado en la eficiencia, priorizando modelos más ligeros sin comprometer la calidad de generación textual. Este proceso condujo a la incorporación del modelo Mistral 7B Instruct, que ofrecía un tamaño significativamente menor, la capacidad de ejecutarse casi en su totalidad en la GPU local y un

rendimiento destacable. El uso de Mistral permitió alcanzar tiempos de respuesta muy competitivos sin comprometer la calidad de los resultados, logrando un equilibrio ideal entre eficiencia y rendimiento, mejorando de forma considerable la fluidez del sistema. Aunque DeepSeek ofrecía respuestas sólidas también conllevaba una penalización significativa en tiempo de respuestas: para generar una única salida, el modelo podía tardar hasta tres minutos, dependiendo de la complejidad del prompt. En contraste, Mistral completaba el mismo tipo de tareas en aproximadamente un minuto y medio. En el contexto de una simulación interactiva en tiempo real, donde la fluidez de la experiencia del usuario es crítica, esa diferencia de tiempo se vuelve determinante.

Fase final: prototipo de uso

Aunque el desarrollo principal del proyecto se llevó a cabo utilizando el modelo de Mistral 7B, de cara al prototipo final se valoró la posibilidad de emplear un modelo aún más potente y con menos latencia perceptible. El objetivo principal de este ajuste fue garantizar una experiencia fluida e inmersiva durante la evaluación externa, evitando frustraciones o interrupciones que pudieran sesgar la valoración del comportamiento del sistema.

Con este fin, se exploró la posibilidad de integrar modelos LLM en la nube, como ChatGPT o Gemini, que ofrecieran tiempos de respuesta optimizados y una mayor capacidad de comprensión. Tras considerar las distintas opciones, se optó por utilizar Gemini, el modelo de Google, que permitió mantener una generación rápida y coherente incluso en interacciones prolongadas y contextualmente exigentes. El uso de Gemini se realizó mediante tokens limitados, lo que restringía el volumen total de interacciones posibles, pero resultó suficiente para llevar a cabo todas las sesiones de prueba planificadas.

4.3 Conexión entre el LLM y Unreal Engine

Para la conexión entre el modelo y el ordenador que ejecuta la simulación, se buscó una solución modular que facilita la modificación del modo de ejecución de forma sencilla, además de permitir la extensión o adaptación de su funcionamiento sin requerir cambios en el código del juego. Para lograr esta abstracción, se optó por un sistema de conexión mediante puertos entre el ordenador que ejecuta el juego y aquel que ejecuta el modelo. Aunque este acercamiento añade una latencia a cada interacción, ofrece una solución flexible y adecuada para su uso con equipos remotos. Con el fin de facilitar su uso, el código encargado de gestionar la comunicación por puertos y LLMs se desarrolló un script de Python, debido a su sencillez y amplia disponibilidad de librerías para el manejo de conexiones por puertos. Además, Python resulta una opción idónea en nuestro caso, ya que permite tanto la ejecución local de modelos como la conexión a servicios remotos. Asimismo, se eligió este lenguaje para el *sistema de reconocimiento de voz*.

4.3.1 Implementación en Unreal Engine

Para la comunicación con el LLM desde el motor se creó una clase de tipo *UGameInstanceSubsystem* llamada *LLM_CommunicationSubsystem*. Los subsistemas de instancia de juego son un tipo de clase contenida en el motor Unreal Engine. La ventaja de este tipo de clases es que se construyen automáticamente al lanzar el juego y solo permiten la existencia de una única instancia de la clase, la cual es accesible globalmente. Este tipo de clases siguen el patrón Singleton³⁴, permitiendo acceder a sus propiedades y funciones públicas desde cualquier parte del código, incluido desde cualquier Blueprint. Gracias a sus propiedades, el subsistema permite crear una interfaz de funciones que se comunican con el LLM.

Una vez se inicia la simulación, el subsistema es el encargado de hacer una llamada al sistema para ejecutar el script de Python correspondiente, con todos los argumentos que el usuario defina, que se verán explicados en la próxima sección. A continuación, el subsistema inicializa el puerto y establece conexión con el script. En caso de que la conexión haya sido exitosa, el subsistema está listo para enviar y recibir respuestas.

El subsistema ofrece un único método público que permite el envío de texto al LLM. Dicho método dispone únicamente de dos parámetros de entrada: una cadena de texto que contiene la información que se le quiere enviar al modelo, y un entero que representa la prioridad de la petición. Dicha prioridad viene inicializada por defecto a cero. Este segundo parámetro surge como una solución que permite realizar numerosas interacciones de manera continuada y no bloqueante, independientemente del estado del modelo. Debido a la elevada frecuencia de interacciones con el LLM, puede darse la situación en la que se quiera enviar una nueva cadena de texto al modelo cuando todavía está procesando una petición anterior. Para solucionar esto, se optó por implementar una cola de prioridad para las peticiones, donde la prioridad se definiría por un valor numérico entero e iría en orden creciente: a mayor valor, mayor prioridad. Cuando se hace un envío de información al LLM se comprueba el estado de este: si el modelo está procesando una petición anterior, la nueva petición se encola respetando la prioridad asignada. En caso contrario, la petición se envía directamente al modelo. La cola de prioridad es especialmente útil cuando se ejecuta un modelo de manera local, en estos casos, la latencia entre respuestas del LLM suele ser especialmente elevada —de 1.5 a 3 minutos de media, según el análisis realizado en el apartado 4.2—, por lo que la cola de prioridad permite seguir ejecutando interacciones sin necesidad de esperar al resultado de la interacción que está en proceso.

Una vez enviada la petición, el subsistema espera a la respuesta del modelo. No obstante, para evitar que este proceso bloquee la ejecución de juego, se optó por ejecutar la espera de manera

³⁴ https://en.wikipedia.org/wiki/Singleton_pattern

asíncrona, haciendo uso de las *Async Tasks* que ofrece Unreal Engine. Una vez se recibe la respuesta, se comprueba que esta es correcta y se comunica a todos los objetos suscritos a un *Event Dispatcher*. Finalmente, si hay peticiones encoladas, se sacará la petición más prioritaria de la cola y se enviará al LLM. Los Event Dispatcher son sistemas que permiten notificar de un evento a todos los objetos que se suscriban a este. De esta forma, el propietario del Event Dispatcher no necesita saber qué objetos tiene que notificar, evitando referencias cruzadas entre clases. Este acercamiento abstraído es el que se ha utilizado para notificar la respuesta del LLM a una petición. Adicionalmente, se creó otro Event Dispatcher para notificar a las clases suscritas cada vez que se hiciera una petición nueva.

Como se mencionó anteriormente, al ejecutar el script de Python, se pasan varios argumentos a la llamada que permiten modificar el comportamiento de este. Los argumentos son especialmente interesantes porque permiten configurar el funcionamiento del script y así poder hacer varias pruebas con argumentos distintos, buscando la configuración que mejor se adapte al proyecto. Para la herramienta se ha creado un método que permite pasar tantos parámetros como el usuario quiera de manera extensible y que no requiere recompilar el proyecto para aplicar cambios. Para ello, se extendió la clase de *DeveloperSettings* propia del motor. Esta clase permite definir propiedades visibles y modificables desde el propio editor de Unreal Engine. Las propiedades guardan sus valores en archivos de configuración, que contienen una lista de pares claves-valor organizadas por secciones y permiten la persistencia de valores de parámetros, así como su modificación sin necesidad de compilar el código. Para la comunicación con el LLM creamos un array de comandos que se guarda en uno de los archivos de configuración y que el usuario puede modificar, alterando alguno de los comandos que vienen por defecto o añadiendo nuevos. Estos son estructuras de datos compuestas de dos cadenas de texto: una para indicar el nombre del argumento, y otra para indicar el valor del mismo. Cuando se hace la llamada al sistema que lanza el script de Python se añaden todos los argumentos y sus respectivos valores que se hayan registrado desde el motor. De esta forma, se pueden configurar parámetros como la dirección IP y puerto a la que se tiene que conectar el programa, un mensaje con el que terminar la ejecución de script o el valor de temperatura del modelo que vaya a ejecutarse, entre otros. En la siguiente sección se explican en mayor detalle los argumentos que incluye por defecto el script de Python.

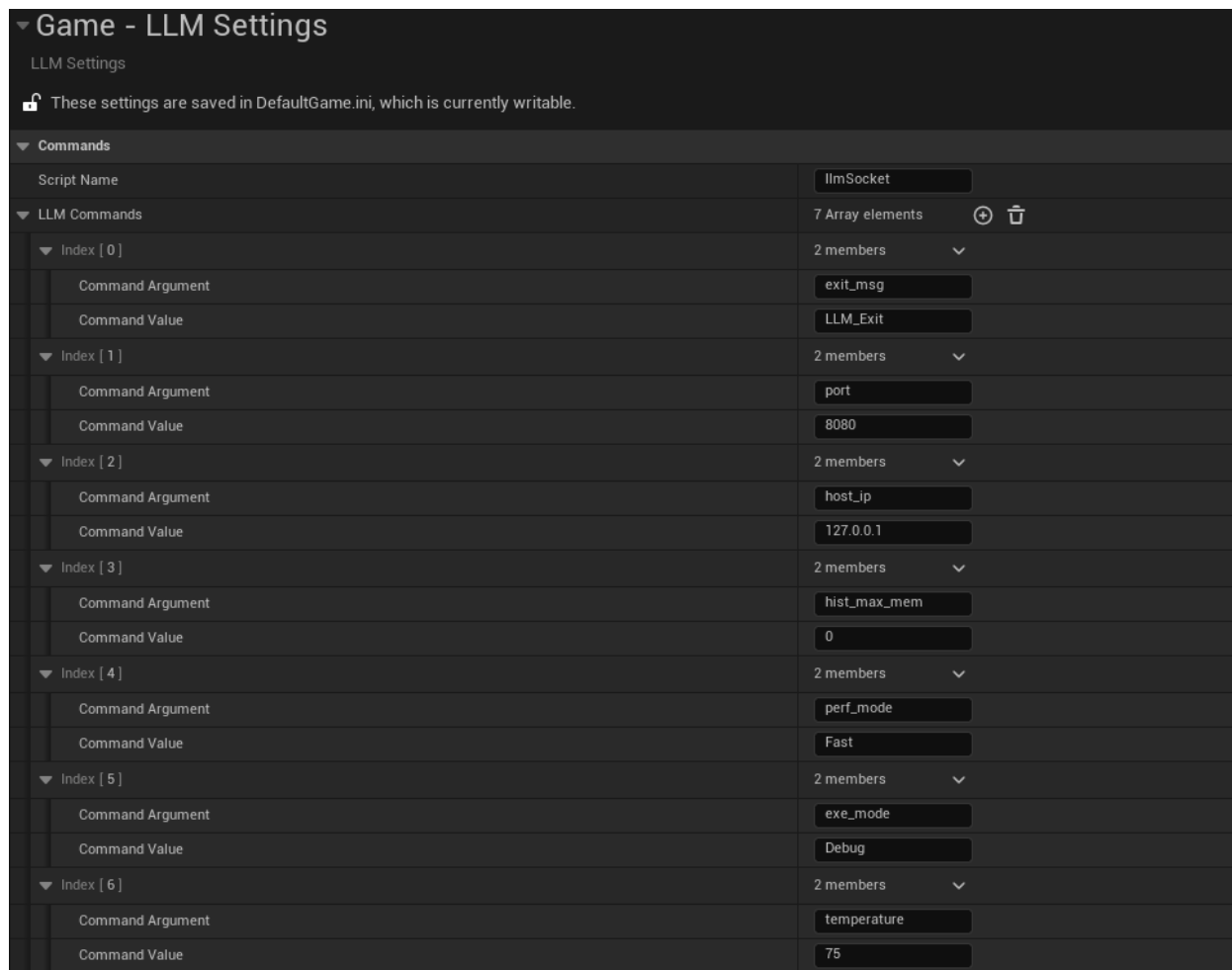


Figura 4.1 Ajustes de la comunicación con el LLM. Pestaña de los ajustes del editor que permite editar el nombre del script de Python que se quiere ejecutar y sus argumentos desde Unreal.

4.3.2 Subsistema LLM: Implementación en Python

Script para modelos locales

La primera versión del script de Python que se creó tenía como objetivo el de poder ejecutar modelos localmente. Para esto, se importa la librería de Ollama: un proyecto de código abierto que ofrece herramientas para descargar, gestionar y ejecutar modelos largos de lenguaje localmente. Además, dispone de una extensa librería de modelos pre-entrenados de todo tipo, listos para ser usados. Ollama permite de manera rápida y sencilla la instalación de estos modelos mediante comandos. Para ejecutar uno de ellos, basta con lanzar un comando a través de una consola que indique el modelo y sus parámetros. Una de las ventajas por las que se escogió Ollama es que es un *framework* (entorno de trabajo) que incorpora aceleración por GPU de los modelos de manera nativa para una amplia colección de tarjetas gráficas. La aceleración por GPU es una técnica de computación que aprovecha la potencia de procesamiento paralelo de las tarjetas gráficas para

incrementar drásticamente el rendimiento de las aplicaciones. Esta técnica es especialmente importante en el campo de la inteligencia artificial y, por tanto, de los LLMs, donde la capacidad de cálculo de las gráficas permite ejecutar a gran velocidad y en paralelo tareas como las multiplicaciones matriciales de grandes dimensiones. Si no se dispusiera de esta herramienta, los LLM tendrían que ejecutarse completamente en el procesador, lo que resultaría en un mayor tiempo de espera. Esto es especialmente negativo en el caso de aplicaciones de uso en tiempo real, donde el usuario espera recibir una respuesta a su interacción en el menor tiempo posible. No obstante, aunque Ollama incorpore esta herramienta, cabe destacar que no todas las tarjetas gráficas pueden aprovecharla. La mayoría de las estas son de la marca NVIDIA, esto se debe a que la empresa estadounidense es pionera en el sector de la inteligencia artificial y sus gráficas incorporan los conocidos como *tensor cores*, que son núcleos de procesamiento especializados en inteligencia artificial y que mejoran el rendimiento de estos. Esta tecnología, sumada a la existencia de CUDA —una plataforma de computación en paralelo que permite la programación de algoritmos en GPU para gráficas NVIDIA— hace que las tarjetas gráficas de otras marcas como AMD o Intel no tengan un soporte de aceleración tan extendido. Esto ha supuesto un problema para algunos miembros del equipo que no disponían de gráficas soportadas por la aceleración de Ollama.

Una vez lanzado el script de Python, lo primero que se hace es leer los argumentos recibidos y hacer los cambios correspondientes. La versión más actualizada del script incluye los siguientes argumentos:

Mensaje de terminación: para cerrar la conexión se puede especificar un mensaje de salida. Si el script recibe por su puerto una cadena de texto idéntica a este mensaje de salida, se considerará que la conexión ha finalizado y cesará la comunicación.

Puerto de conexión: se puede configurar el puerto que se tiene que abrir para establecer la conexión. Antes de asignar el puerto, se comprueba que el valor del argumento recibido es válido, es decir, que se trata de un entero y que el valor se encuentra entre el rango de valores de puertos disponibles para el usuario.

Dirección IPv4 destino: además del puerto, se puede especificar la dirección IP a la que conectarse, para el caso de querer utilizar el modelo localmente, basta con usar la dirección de *loopback*, que identifica a la máquina en la que se está ejecutando.

Historial de conversaciones: el historial del modelo es otro parámetro configurable, permitiendo indicar de cuántas interacciones pasadas es capaz de acordarse el modelo. Cuanto mayor sea el número, más contexto tendrá el LLM a la hora de responder, pero con un mayor impacto en el rendimiento. En caso de estar a cero, el modelo no recordará interacciones pasadas y solo sabrá de la interacción actual.

Modo de rendimiento: para poder hacer pruebas con diferentes modelos, se añadió un argumento que permitiese seleccionar el modo de rendimiento, esto permite que se seleccione un modelo más grande o pequeño dependiendo del rendimiento que se quiera obtener, sin necesidad de acordarse del nombre exacto del modelo. El script ofrece tres opciones para este argumento: la de “calidad” elige el modelo más grande de todos los modelos descargados mediante Ollama, la opción “rendimiento” elige el más pequeño y cualquier otro valor del argumento elegirá el modelo que se encuentre en la mitad de la lista de los modelos instalados, ordenados por tamaño.

Modo de ejecución: como ajuste de rendimiento adicional, está el argumento de modo de ejecución, que permite elegir entre el modo “debug” y “release”. La diferencia entre ambos es la posibilidad de escribir por consola, información relacionada con el estado de la conexión y los mensajes entrantes y salientes al script. Este argumento permite controlar la cantidad de mensajes que se muestran en la consola del proceso, lo que puede impactar ligeramente el rendimiento.

Temperatura del modelo: el último argumento incluido permite ajustar la temperatura del LLM. La temperatura es un parámetro que controla la aleatoriedad de la respuesta del modelo, permitiendo ajustar un balance entre coherencia y creatividad al momento de generar cada palabra de la respuesta. La temperatura es un valor que varía de cero a uno, cuanto más próximo sea el valor a cero, más predecible será la respuesta y cuanto más próximo a uno, más creativa.

A continuación se puede ver un ejemplo de llamada al script de Python desde Unreal, donde se especifica el formato a seguir.

```
start /B py llmSocket.py --exit_msg LLM_Exit --port 8080 --host_ip  
127.0.0.1 --hist_max_mem 0 --perf_mode Fast --exe_mode Debug --temperature  
75
```

Una vez interpretados los argumentos y ajustados los parámetros según sus valores, el script se encarga de elegir qué modelo va a ejecutar. A continuación, abre los puertos con la configuración recibida y establece conexión con el otro extremo. En caso de conexión exitosa, el script se queda a la espera de nuevas peticiones. Al recibir una petición, se hace una llamada a la API de Ollama para iniciar una conversación con el modelo elegido. Es en este paso donde se configuran parámetros del LLM que afectan a la respuesta, como la temperatura o el formato de salida de la respuesta. En el caso de este proyecto, se pide una respuesta en formato JSON, de manera que se pueda pasar directamente al juego sin necesidad de una revisión y corrección previas. Además, durante este paso se le manda al modelo el historial de conversaciones anteriores si es necesario. Una vez recibida la respuesta del LLM, se envía al juego. Este proceso se repite indefinidamente hasta que termina la ejecución o hasta que se reciba el mensaje de salida.

Script para modelos alojados por terceros

Aunque el script anterior permite ejecutar los modelos de manera remota —mediante conexión por puertos— en caso de querer ejecutarlos de manera local, el tiempo de respuesta del

LLM a la mayoría de las peticiones aumentaba considerablemente, especialmente cuando se lanzaba en paralelo al juego en Unreal. Como referencia, las interacciones con el modelo Mistral que aprovechaba la aceleración por GPU tardaban entre 10 y 30 segundos aproximadamente, dependiendo de la complejidad de la interacción. En comparación, el modelo remoto que se utilizó para la prueba presentaba una latencia de respuesta de entre 1 y 3 segundos aproximadamente para realizar las mismas interacciones, una mejora de más del 1000%. Esto se debe principalmente a que los equipos personales del equipo de desarrollo no disponían de la cantidad de memoria de video suficiente para cargar por completo el modelo a la vez que se renderizaba la escena del proyecto. Esto era especialmente perjudicial para modelos grandes que ofrecían mejores respuestas a las acciones pero que a su vez ocupaban más espacio, resultando en respuestas con una latencia que se podría considerar inaceptable para una experiencia de juego en tiempo real, como es el caso de este proyecto.

Como solución, se decidió buscar métodos alternativos que permitiesen ejecutar un modelo de altas capacidades de forma remota, alojado en equipos de terceros, externos al grupo de desarrollo.

Finalmente, tras evaluar las posibles opciones a nuestro alcance, se decidió usar el modelo largo de lenguaje Gemini. Anteriormente conocido como *Google Bard*, es un *bot* conversacional de inteligencia artificial multimodal y generativa basado en la familia *Gemini Pro* desarrollado por Google y lanzado mundialmente en marzo de 2023. Python dispone de una API propia de Google que permite enviar peticiones al LLM. No obstante, las peticiones están asociadas a claves privadas y limitadas a un número de peticiones por minuto y por día. En este trabajo, se optó por usar una clave gratuita para el modelo *Gemini 2.0 flash experimental*, el cual permitía realizar 10 solicitudes por minuto, 1000 solicitudes al día y un máximo de 1 millón de tokens por minuto, una cantidad estimada suficiente para su uso durante las pruebas de usuario. Sin embargo, para reducir más las posibilidades de alcanzar la cuota máxima de peticiones asociada a la clave usada, se optó por crear una lista cíclica de claves privadas de varios integrantes del grupo, de manera que, tras una petición al modelo, el siguiente envío se haría con una clave distinta. Esta cola se puede editar desde el script en caso de que se quieran añadir o modificar las claves ya existentes.

El resto del script mantiene una estructura muy similar a la mencionada en el capítulo anterior. En este caso, conserva los mismos parámetros de entrada que el script para modelos locales, por lo que no es necesario hacer ningún cambio externo para obtener comportamientos similares. De igual manera, el script se ejecuta mediante una llamada al sistema.

4.4 Sistema de creación de prompts

El uso de un LLM como núcleo de control en un entorno interactivo —como lo es un videojuego— plantea una serie de retos técnicos y conceptuales. Uno de los más importantes es asegurar que las respuestas generadas por el modelo sean, correctas en términos lingüísticos,

coherentes con el contexto del juego y útiles desde el punto de vista funcional. A diferencia de sistemas tradicionales —donde las decisiones del juego funcionan por reglas predefinidas—, el LLM interpreta y actúa en base a instrucciones redactadas en lenguaje natural. Por eso, la precisión y claridad de las instrucciones —es decir, los prompts— resulta determinante para el buen funcionamiento del sistema.

En este contexto, la creación de prompts se convierte en una parte crítica del diseño. No se trata únicamente de enviar una petición al modelo, sino de estructurar un mensaje que incluya toda la información relevante del estado del juego, las intenciones del jugador y los objetivos del sistema. De esta forma se logra que el LLM pueda interpretar correctamente la situación y ofrecer una respuesta siguiendo la narrativa y la lógica interna del mundo virtual.

4.4.1 Planteamiento y solución

En las fases iniciales del desarrollo, la generación de prompts se realizaba de manera específica e individual en cada sección del código que requería interacción con el modelo de lenguaje. Este enfoque conllevaba una alta redundancia, ya que cada nuevo fragmento de código debía implementar su propia lógica para construir el prompt, incluso cuando compartía estructuras o información con otros componentes del sistema. Debido a esto, el mantenimiento, la escalabilidad y la coherencia del sistema se veían comprometidos. Causando que la gestión tanto de la creación como del envío de prompts resultaba innecesariamente compleja.

Para resolver esta problemática y avanzar hacia una solución más eficiente y modular, se diseñó e implementó la clase *PromptManager*. Esta clase centraliza y abstrae la lógica asociada a la generación y envío de prompts, organizándola mediante un sistema de *módulos* independientes. En este contexto, un *módulo* se define como una unidad funcional autocontenida que encapsula una parte específica de la lógica necesaria para construir el prompt que se enviará al LLM. Cada módulo actúa de forma independiente y tiene la responsabilidad de generar un fragmento de texto, concatenándose para construir el prompt final. Estos módulos pueden aportar información contextual relevante o tareas nuevas —denominadas *Tasks*— que el LLM debe abordar.

Gracias a esta estructura modular, la incorporación de nuevos comportamientos se simplifica considerablemente. En lugar de definir la generación del prompt en una parte específica del código —lo que implicaría duplicar información o lógica ya existente—, únicamente se debe añadir un nuevo módulo a *PromptManager*.

4.4.2 Funcionamiento y creación de módulos del *PromptManager*

El diseño de la clase *PromptManager* busca ofrecer un sistema de generación de prompts que sea simple, flexible y escalable. Cualquier parte del código puede interactuar con esta clase para generar un prompt. Para ello, se necesita proporcionar la información requerida y llamar al método *CreatePrompt*, especificando explícitamente qué módulos deben incluirse en la

generación. Cuando se llama a este método, la clase revisa en forma de cascada todos los módulos registrados y ejecuta únicamente aquellos que han sido activados para esa solicitud en particular.

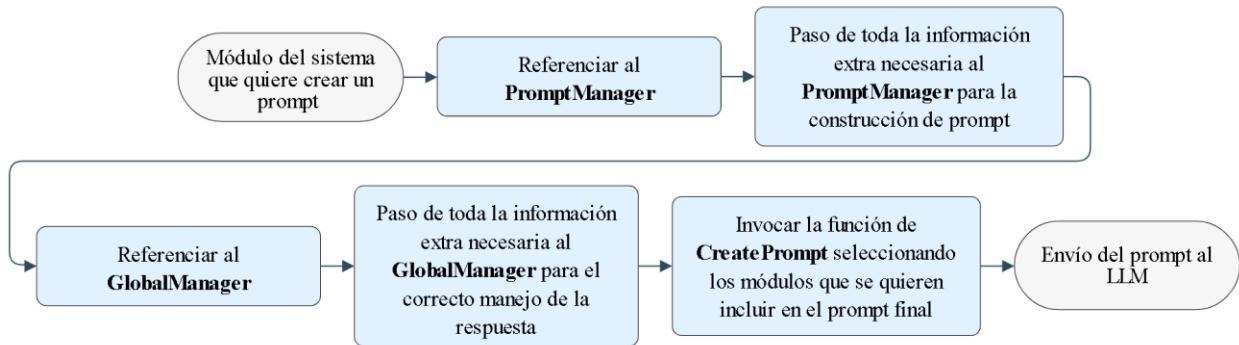


Figura 4.2 Diagrama de flujo para crear y enviar un prompt al LLM.

La integración de un nuevo módulo en el PromptManager sigue un procedimiento estructurado que garantiza su independencia y reutilización. El proceso consta de los siguientes pasos:

1. **Definición de la lógica del módulo:** Se crea una nueva función que actúe como módulo, la cual debe ser completamente autocontenida. Esto implica que toda la lógica necesaria para generar su correspondiente fragmento de prompt, así como cualquier procesamiento interno, debe residir dentro del propio módulo. Esta función debe devolver una cadena de texto (*string*) que se incorporará al prompt final.
2. **Registro del módulo en CreatePrompt:** Una vez definido, el nuevo módulo debe ser conectado a la cascada de módulos que el CreatePrompt recorre. Esto permite que el módulo sea activado condicionalmente según los parámetros especificados en cada invocación del método, asegurando así que solo se ejecutan los módulos requeridos para cada caso de uso.
3. **Clasificación del módulo:** Cada módulo puede desempeñar una de las siguientes dos funciones:
 - a. **Task (tarea principal):** representa una instrucción específica que el LLM debe ejecutar, incluyendo directrices claras sobre cómo debe estructurarse o enfocarse la respuesta. Cada Task debe contar con un identificador (ID), lo que permite rastrear con precisión la respuesta generada por el modelo, asociarla a su solicitud original y gestionarla de forma adecuada dentro del sistema.
 - b. **Módulo informativo (additional info):** proporciona contexto adicional o datos relevantes para enriquecer la comprensión del LLM, sin requerir una respuesta específica. Para indicar de cual se trata se marcará el indicador booleano *isTask* como cierto o falso.

Una vez completados estos pasos, el nuevo módulo queda automáticamente accesible desde cualquier parte del sistema. Basta con incluir su identificador en la llamada a CreatePrompt para que su salida se integre en el prompt final sin necesidad de duplicar lógica ni manipular directamente otras partes del código.

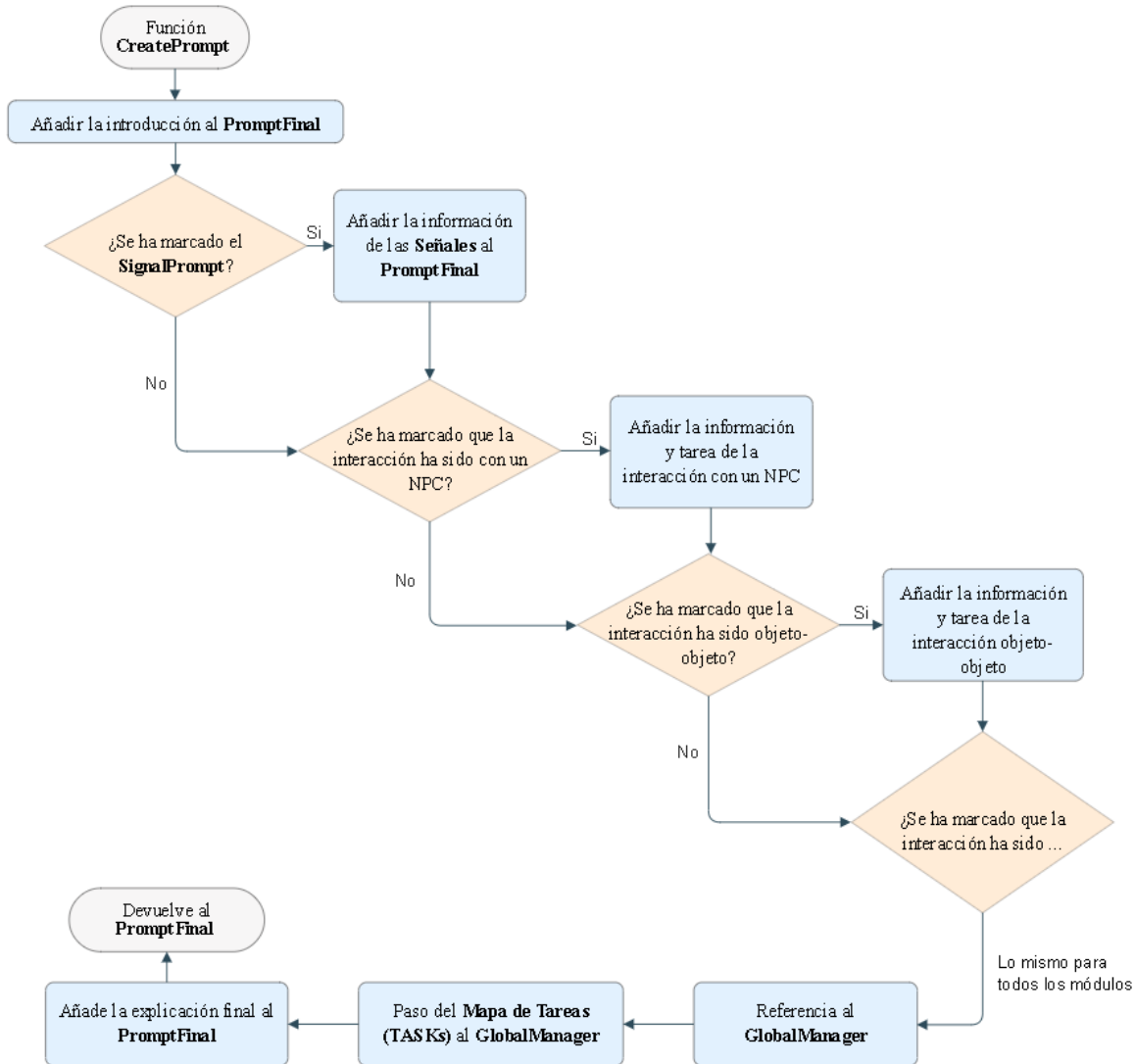


Figura 4.3 Flujo de creación de un prompt de la función CreatePrompt.

4.4.3 Módulos implementados

A continuación, se describen los módulos implementados en el PromptManager, indicando su función principal y si actúan como información adicional o como tareas que el modelo de lenguaje debe ejecutar:

- **SignalPrompt** (additional info)

Este módulo incorpora al prompt información adicional proporcionada por las señales del sistema. Estas señales pueden contener datos relevantes que no pertenecen directamente a la escena o a los personajes, pero que enriquecen el contexto de la interacción. Cada señal gestiona ella misma el paso de su propia información.

- **SceneInfo** (additional info)

Este módulo añade al prompt una representación completa de los elementos de la escena actual. Incluye una lista con todos los objetos presentes, todos los NPCs —junto con sus descripciones— y el jugador. Esta información proporciona el contexto necesario para que el modelo pueda tomar decisiones fundamentadas.

- **NPCPrompt** (Task, ID: NPCPrompt)

Este módulo tiene como función principal gestionar la generación de respuestas por parte de los NPC frente a mensajes del jugador o interacciones. Para garantizar una experiencia coherente y verosímil, el sistema se encarga de que cada respuesta emitida por el NPC mantenga fielmente su personalidad, tono y rol dentro del entorno, tomando como referencia tanto la descripción inicial del personaje como su historial de conversaciones y acciones anteriores.

Un aspecto destacado de este módulo es la incorporación de una funcionalidad de resumen automático cada cierto número de interacciones. Con el objetivo de evitar que el historial de interacción del NPC crezca de forma descontrolada —lo que podría comprometer la coherencia narrativa—, se ha implementado un sistema que solicita periódicamente al LLM generar un resumen del historial acumulado junto a la petición de la respuesta. Esta síntesis conserva la información esencial del desarrollo narrativo previo sin sobrecargar el prompt, permitiendo así que el modelo mantenga una visión contextual clara sin perder eficiencia.

En cada solicitud de respuesta, el prompt enviado al LLM incluye los siguientes elementos clave:

- El contexto del jugador.
- El contexto del NPC: su identidad y personalidad.
- El historial de conversación e interacciones del NPC.
- El mensaje más reciente enviado por el jugador.

Gracias a esta estructura, el sistema es capaz de generar respuestas adaptadas, coherentes y alineadas con la narrativa general, asegurando una experiencia de interacción fluida e inmersiva.

- **NPCStatePrompt** (Task, ID: NPCStatePrompt)

Este módulo tiene como propósito delegar en el modelo de lenguaje la tarea de decidir el nuevo estado al que debe realizar una transición la máquina de estados de uno o más NPCs

involucrados en la interacción actual. La decisión del modelo debe basarse en el contexto narrativo y el desarrollo de la escena, eligiendo siempre entre un conjunto predefinido de estados posibles.

Para facilitar esta tarea, el prompt enviado al LLM incluye de forma explícita dicha lista de estados disponibles, acompañada de una breve descripción de cada uno para mejorar la comprensión del modelo y conseguir una respuesta de mejor calidad. En los casos en que solo haya un NPC implicado, el sistema también proporciona información adicional sobre su estado actual y su estado anterior. Esta información contextual permite al modelo tomar decisiones más coherentes, alineadas con el comportamiento esperado del personaje y el flujo de la narrativa.

- **NPCInteractionPrompt** (Task, ID: ActionPrompt)

Este módulo se encarga de gestionar interacciones que involucren directamente a un NPC, solicitando al modelo de lenguaje la generación de un objeto en formato JSON que describa el resultado de una acción concreta. Este resultado puede consistir en la creación, modificación o destrucción de diferentes objetos no esenciales, siempre como consecuencia de la interacción específica entre el jugador y el NPC.

A diferencia de otros módulos de acción más generales, este se distingue por centrarse exclusivamente en interacciones individualizadas, en las que la participación del NPC es clave para el desarrollo de la escena.

Para garantizar que el modelo actúe con precisión y coherencia, el prompt que se envía incluye toda la información relevante de la interacción:

- Datos del NPC involucrado, como su nombre, rol y el historial de acciones o conversaciones previas.
- Información del objeto implicado, incluyendo su nombre y una breve descripción.
- El tipo de interacción, que permite interpretar adecuadamente el sentido y la intención de esta.

Gracias a esta estructura informativa, el LLM puede generar una respuesta estructurada y contextualizada, que el sistema interpreta para actualizar dinámicamente el entorno virtual de acuerdo con las decisiones narrativas.

- **ActionPrompt** (Task, ID: ActionPrompt)

A diferencia del módulo NPCInteractionPrompt, este componente está diseñado para gestionar acciones generales dentro del entorno del juego que no dependen directamente de la intervención de un NPC específico. Su objetivo es permitir que el modelo de lenguaje genere cambios en el entorno mediante un objeto en formato JSON que describa la creación, modificación o destrucción de objetos no esenciales.

En este caso, el prompt no incluye información contextual sobre personajes, ya que las acciones gestionadas están centradas en interacciones del tipo objeto-objeto o jugador-objeto. El enfoque se sitúa en la dinámica entre los elementos del entorno involucrados, sin atribuir la acción a una figura narrativa concreta como un NPC.

Para ello, el LLM recibe una entrada estructurada que incluye una descripción clara del tipo de interacción realizada, así como los objetos implicados en ella con su nombre y descripción. Con esta información, el modelo puede generar una representación coherente del cambio en el estado del entorno, asegurando consistencia con el contexto general del juego.

- **OriginalObjectStatePrompt** (Task, ID: ReceiverNewState/InstigatorNewState)

Este módulo tiene como función principal determinar el nuevo estado de los objetos esenciales involucrados en una interacción dentro del entorno. Para ello, el modelo de lenguaje debe seleccionar, de forma coherente, el estado más adecuado a partir de una lista predefinida de posibles estados, teniendo en cuenta tanto el contexto general como la evolución de la escena.

Con el fin de facilitar esta tarea y asegurar decisiones precisas, el prompt enviado al LLM incluye explícitamente la lista completa de estados disponibles, cada uno acompañado de una breve descripción que ayuda al modelo a entender el propósito y las implicaciones de cada opción. Además, se proporciona el estado anterior del objeto, lo que permite mantener la continuidad narrativa y contextual.

Este enfoque permite al sistema garantizar transiciones lógicas y consistentes en el comportamiento de los objetos esenciales.

- **ComponentsPrompt** (Task, ID: ComponentsPrompt)

Sirve para añadir o modificar componentes de los objetos de la escena. Se muestran todos los componentes que puede elegir incluyendo toda su información, que se explica con detalle en el *apartado 4.8*. Además, se mandan descripciones de sus usos, la interacción que ha tenido lugar y aquellos objetos que se han visto afectados, ya sea porque son objetos recién creados u objetos que el LLM ha decidido modificar tras la interacción.

4.5 Manejo de respuestas del LLM

El correcto funcionamiento de un modelo de lenguaje como núcleo en un entorno interactivo no depende únicamente de la calidad de los prompts que se le proporcionan, sino también —y de forma igualmente importante— del modo en que se interpretan y gestionan las respuestas que genera. Estas respuestas forman la base sobre la cual el sistema reacciona, adapta su estado y toma decisiones. Es por esto, por lo que deben ser procesadas de manera rigurosa, estructurada y coherente con el contexto del juego.

4.5.1 Planteamiento y solución

De forma análoga a lo ocurrido con la generación de prompts, en las fases iniciales del desarrollo el procesamiento de las respuestas generadas por el LLM se realizaba de manera descentralizada. Cada parte del sistema que necesitaba reaccionar ante una respuesta debía implementar su propia lógica para interpretarla y ejecutar las acciones correspondientes.

Este enfoque, aunque funcional en un primer momento, derivó rápidamente en una alta redundancia de código. La dispersión de esta lógica a lo largo del código incrementaba además la complejidad del sistema, dificultando su mantenimiento, aumentando la probabilidad de errores y limitando la escalabilidad a medida que se incorporaban nuevos tipos de interacción.

Con el fin de solventar estas limitaciones, se diseñó un sistema centralizado para el tratamiento de las respuestas del LLM, inspirado en la arquitectura modular empleada para la generación de prompts. Esta solución se materializó en la clase *GlobalManager*, encargada de encapsular, interpretar y distribuir de forma estructurada las respuestas generadas por el modelo.

El *GlobalManager* actúa como punto único de entrada para el manejo de respuestas, identificando el tipo de tarea o instrucción recibida, extrayendo los datos relevantes —como estructuras JSON, cambios de estado o acciones a ejecutar— y delegando su ejecución en los módulos correspondientes del sistema. De esta forma, se logra una gestión coherente, extensible y fácilmente mantenible de las respuestas del LLM, reduciendo la duplicación de lógica y mejorando la robustez del flujo de comunicación entre el juego y el modelo de lenguaje.

Para que reciba la respuesta, se implementó un patrón *Observer*³⁵. Se creó el componente *PromptListener*, que se encarga de recibir las respuestas del LLM. La clase *GlobalManager* contiene el componente *GlobalListener*, el cual hereda de *PromptListener*, registrándose como uno de los posibles observadores. En este caso es único, pero habilita la posibilidad de que se añadan más listeners en caso de que se quiera extender la funcionalidad de la herramienta.

4.5.2 Funcionamiento y creación de módulos del *GlobalManager*

El *GlobalManager* opera a través de una cola de prioridad propia, en la que se almacenan estructuras de datos correspondientes a cada prompt enviado. Cada una de estas estructuras contiene, por un lado, los elementos del sistema que deben reaccionar a la respuesta —como NPCs u objetos interactivos—, y por otro, la lista de Tasks que componen dicho prompt. Esta cola garantiza que las respuestas del modelo se procesen siguiendo el mismo orden de prioridad en el

³⁵ *Observer*: https://en.wikipedia.org/wiki/Observer_pattern

que fueron enviadas, preservando así la coherencia narrativa y funcional del sistema. Una vez procesada una respuesta, su entrada correspondiente se elimina de la cola.

El proceso comienza incluso antes de que el prompt sea creado. En esta fase preliminar, se genera una entrada inicial en la cola de prioridad del GlobalManager, en la que se registran todos los elementos del sistema que deberán reaccionar a la futura respuesta del LLM. Esta recopilación anticipada es fundamental para asegurar que el sistema pueda responder de forma contextualizada y coherente cuando se reciba la salida del modelo.

Durante la generación del prompt mediante el PromptManager, se define también la lista de Tasks que lo componen. Cada Task representa una instrucción específica para el modelo y se identifica mediante un identificador único, lo que permite su procesamiento posterior de manera modular e individualizada.

Una vez completada la construcción del prompt, toda la información recopilada —siendo esta: los elementos que deben reaccionar y la lista de tasks— se registra de manera definitiva en la cola de prioridad del GlobalManager.

Cuando se recibe una respuesta del modelo, el GlobalManager invoca la función *ManageAnswer*, que se encarga de procesar la entrada correspondiente en la cola. Esta función recorre todos sus módulos y verifica si cada uno está implicado en la lista de Tasks asociadas al prompt. Si el identificador de un módulo no está presente en la lista, se omite su ejecución; en caso contrario, se accede al módulo correspondiente, donde se interpreta su Task y se procesa su parte específica de la respuesta. A partir de esta interpretación, se aplican los cambios necesarios en la parte del sistema correspondiente. Por ejemplo: si el modelo ha devuelto un nuevo estado para un NPC, se llama a la función del NPC que actualiza el estado al nuevo devuelto.

Este enfoque selectivo permite optimizar el rendimiento general del sistema al evitar ejecuciones innecesarias y garantiza que solo se activen los módulos estrictamente relacionados con cada respuesta. De este modo, cada módulo interpreta su Task de manera autónoma, aplicando los cambios pertinentes en el estado del juego, como por ejemplo la actualización de NPCs, la creación o destrucción de objetos, o la modificación de componentes del entorno.

Gracias a la arquitectura modular del componente GlobalManager, la integración de nuevos módulos en el sistema se realiza de forma sencilla y ordenada. Esta estructura está diseñada para ser altamente extensible, permitiendo adaptar el comportamiento del sistema a nuevos requisitos con un esfuerzo mínimo.

El proceso para añadir un nuevo módulo sigue tres pasos fundamentales:

1. **Definición del módulo y su identificador de tarea (Task ID):** Se crea un nuevo módulo asociado a un identificador específico de tarea. Este identificador será el que

- permitirá al sistema reconocer cuándo debe ejecutarse el módulo correspondiente en función de la respuesta del LLM.
2. **Implementación de la lógica de procesamiento:** Una vez definido el módulo, se implementa su lógica interna. Esta lógica debe estar orientada a procesar de manera adecuada la información contenida en la respuesta asociada a su Task ID, gestionando los datos recibidos según el comportamiento deseado (por ejemplo, actualizar el estado de un NPC).
 3. **Integración en el método ManageAnswer:** Finalmente, el módulo debe añadirse en la cascada de ejecución del método ManageAnswer, que es responsable de recorrer la lista de módulos disponibles cuando se recibe una respuesta del LLM. Esto garantiza que el nuevo módulo será evaluado y ejecutado únicamente cuando el identificador de su tarea esté presente en la lista de Tasks contenida en el prompt procesado.

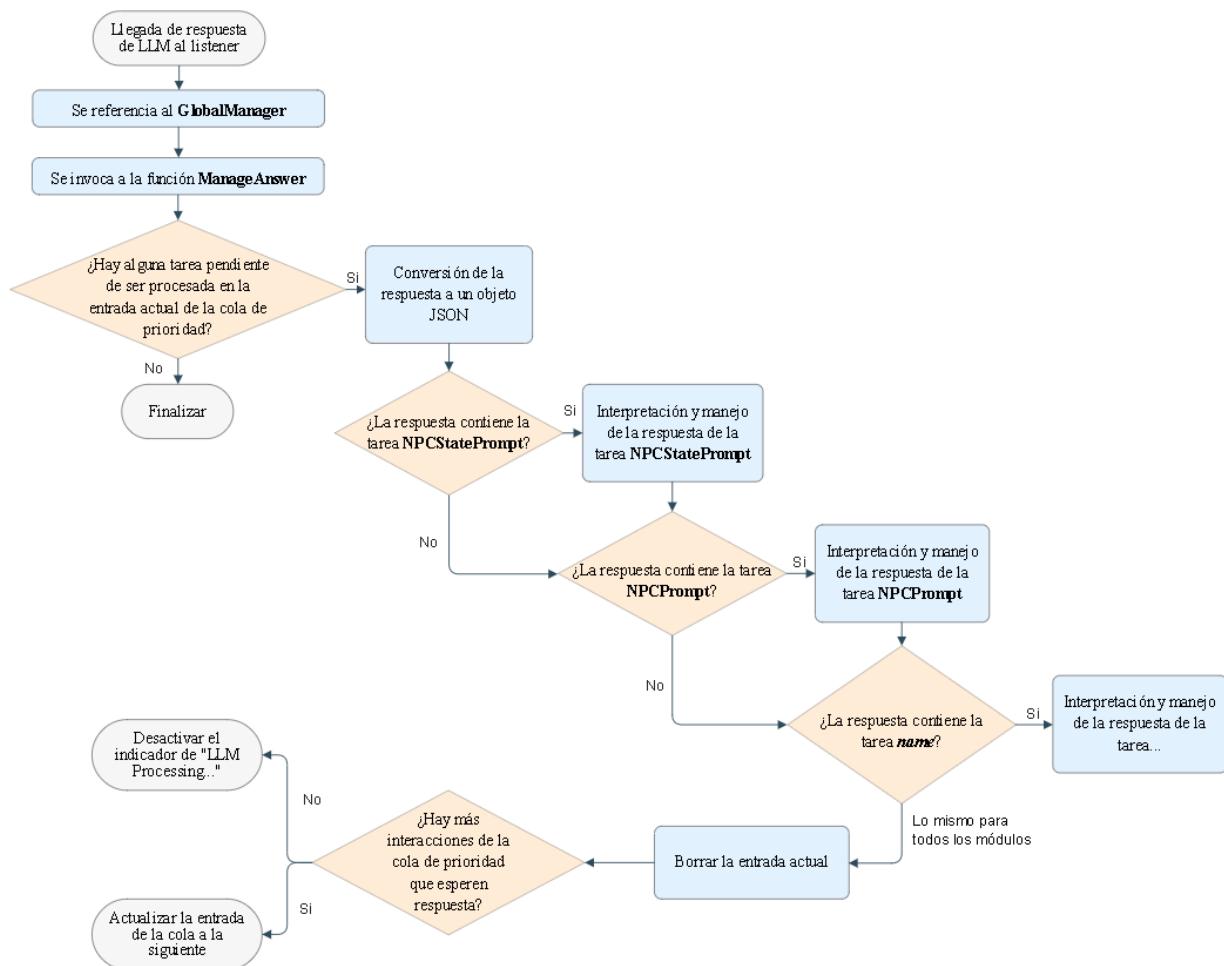


Figura 4.4 Diagrama de flujo del procesamiento de las respuestas del LLM.

4.5.3 Módulos implementados

A continuación, se describen los módulos implementados en el GlobalManager, indicando su función principal:

ChangeNPCState

Este módulo es responsable de procesar la tarea asociada al identificador NPCStatePrompt. Su principal función consiste en recorrer la estructura de datos en formato JSON proporcionada por el LLM, correspondiente a dicha tarea, e identificar los NPCs mencionados en dicha estructura.

Durante la ejecución, el módulo localiza los NPCs especificados en el JSON dentro de la lista de NPCs contenida en la estructura de la entrada actual de la cola de prioridad. Una vez identificados, procede a invocar la función correspondiente de cada NPC encargada de actualizar su estado según la información proporcionada, asegurando que dichos cambios sean coherentes con el contexto narrativo o funcional del entorno virtual.

ChatAction

Este módulo es responsable de procesar la tarea asociada al identificador NPCPrompt. Su principal función consiste en recorrer la estructura de datos en formato JSON proporcionada por el LLM, correspondiente a dicha tarea, e identificar tanto la respuesta del NPC como un posible resumen del historial de interacciones.

Durante la ejecución, el módulo localiza la respuesta y el resumen asociados al NPC especificados en el JSON. Una vez identificados, esta información se transmite a la función correspondiente del NPC —contenido en la estructura de la entrada actual de la cola de prioridad— encargada de actualizar su respuesta —la cual será reproducida mediante el sistema de conversión texto a voz, *Text-To-Speech* en inglés— y de registrar el nuevo historial de interacción.

ReceiverNewState

Este módulo es responsable de procesar la tarea asociada al identificador ReceiverNewState. Su principal función consiste en recorrer la estructura de datos en formato JSON proporcionada por el LLM, correspondiente a dicha tarea, e identificar el nuevo estado de la máquina de estados del objeto receptor mencionado en la estructura.

InstigatorNewState

Este módulo es responsable de procesar la tarea asociada al identificador InstigatorNewState. Su funcionamiento es esencialmente equivalente al del módulo ReceiverNewState, pero afecta al objeto esencial instigador, contenido en la entrada actual de la cola de prioridad.

ActionPrompt

Este módulo es responsable de procesar la tarea asociada al identificador ActionPrompt. Su función principal se limita a recolectar la estructura de datos en formato JSON correspondiente a dicha tarea y transmitirla al componente StateManager, que será el encargado de procesarla y realizar los cambios pertinentes en el sistema.

De este modo, ActionPrompt actúa como intermediario entre la respuesta generada por el LLM y el gestor de estado, asegurando que la información fluya correctamente hacia los módulos encargados de aplicar las modificaciones sobre los elementos del entorno virtual.

UpdateComponents

Este módulo es responsable de procesar la tarea asociada al identificador ComponentsPrompt. Su principal función consiste en recorrer la estructura de datos en formato JSON proporcionada por el LLM, correspondiente a dicha tarea, e identificar los objetos y sus componentes mencionados en esta estructura.

Durante la ejecución, el módulo localiza los objetos especificados en el JSON dentro de la lista de objetos interactivos contenida en la entrada actual de la cola de prioridad. Una vez identificados, procede a invocar la función correspondiente que añade o modifica los componentes que tienen asociados en el JSON.

4.6 Máquinas de Estado

Durante el diseño de las distintas interacciones posibles en el proyecto, así como de las reacciones generadas por el LLM, surgió la necesidad de definir hasta qué punto se debía depender de dicho modelo para gestionar comportamientos complejos, como por ejemplo el movimiento de un NPC.

Tras analizar las implicaciones, se decidió que la mejor opción para garantizar que ciertos elementos del sistema se comportaran de manera precisa y controlada era implementar dichos comportamientos de forma explícita. Por esta razón, se optó por la incorporación de *máquinas de estado*.

Una máquina de estado es una herramienta que se utiliza para controlar el comportamiento de un objeto o sistema a través de un conjunto de *estados* posibles. Cada estado representa una situación específica en la que puede encontrarse el sistema (por ejemplo, "sentado", "moviendo", etc.), y define qué hace el sistema en ese momento.

El uso de máquinas de estado permitió establecer un marco lógico y determinista para regular comportamientos clave, como puede ser el movimiento de los personajes. De esta manera, se logra un equilibrio entre la flexibilidad generativa del LLM y el control estricto necesario en determinadas situaciones del juego.

4.6.1 Implementación con *Logic Driver Lite*

La implementación de las máquinas de estado se realizó utilizando *Logic Driver Lite*³⁶, un plugin gratuito para Unreal Engine que permite crear y gestionar máquinas de estado de manera visual y modular. Este plugin ofrece una interfaz gráfica intuitiva que permite definir los distintos estados y transiciones de manera visual, empleando nodos similares a los utilizados en Blueprints. Esto facilita la creación y mantenimiento de comportamientos complejos, reduce la necesidad de escribir código manualmente y mejora la legibilidad y organización de la lógica del sistema.

Características clave del plugin:

- **Edición visual de máquinas de estado:** Permite crear, modificar y visualizar estados y transiciones directamente desde el editor de Unreal Engine, utilizando nodos similares a los de Blueprints.
- **Soporte para herencia de máquinas y lógica jerárquica:** Aunque la versión Lite tiene limitaciones frente a la versión Pro —alternativa de pago—, permite diseñar máquinas relativamente complejas mediante herencia, facilitando la reutilización y estructuración del comportamiento.
- **Integración con Blueprints:** Logic Driver Lite se integra perfectamente con el sistema de Blueprints, permitiendo vincular los estados con acciones concretas en el juego (como animaciones, movimientos, interacciones, etc).
- **Depuración visual:** El plugin incluye herramientas para monitorear en tiempo real los estados activos y las transiciones realizadas, lo cual resulta de gran utilidad para depurar errores.

³⁶ *Logic Driver Lite*: <https://www.fab.com/listings/e7500c39-5b12-4b88-aac2-12b14c0b1387>

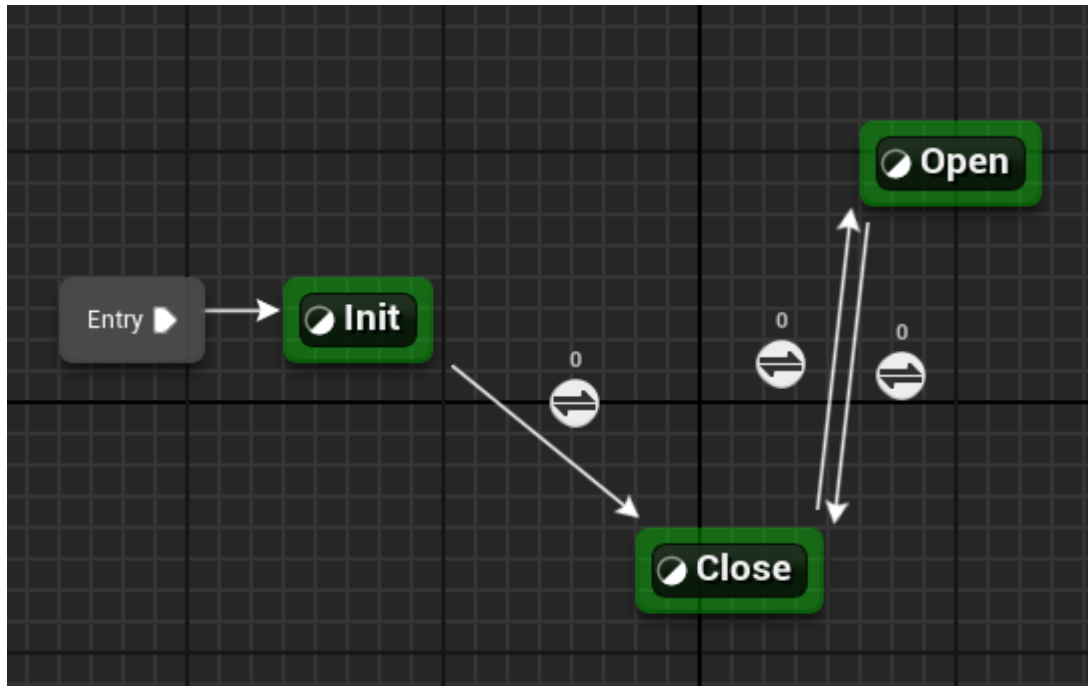


Figura 4.5 Ejemplo de máquina de estado desde Blueprints. Ejemplo del editor visual de una máquina de estados utilizada en el proyecto. En él se observa cómo, tras la llamada inicial a Entry, se accede al nodo estado Init, encargado de inicializar las variables internas necesarias para el funcionamiento posterior. A continuación, se hace transición al nodo estado Close, donde se ejecuta el cierre de la puerta correspondiente. A partir de ese momento, las únicas transiciones posibles se producen entre los nodos estado Close y Open, responsables de cerrar y abrir la puerta, respectivamente.

En estas máquinas, existe un nodo llamado Entry que se llama cuando se crea la máquina de estado. También están los nodos estado que son los que van a contener el código y la lógica. Las conexiones entre estos nodos se representan mediante flechas las cuales contienen las condiciones de transición entre estos estados.

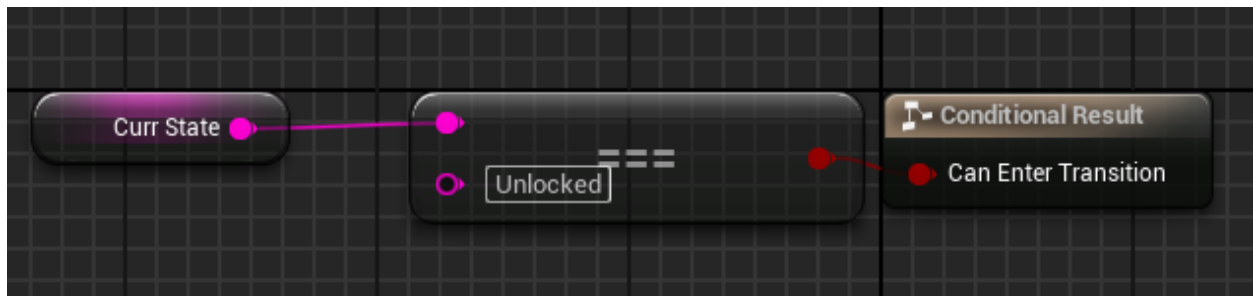


Figura 4.6 Ejemplo de una condición de transición de máquinas de estado asociada al estado Open. En este caso, se verifica si el valor de la variable CurrState (estado actual) es igual a la cadena de texto “Unlocked”. El resultado de esta comprobación determina si la transición al estado Open se realiza o no.

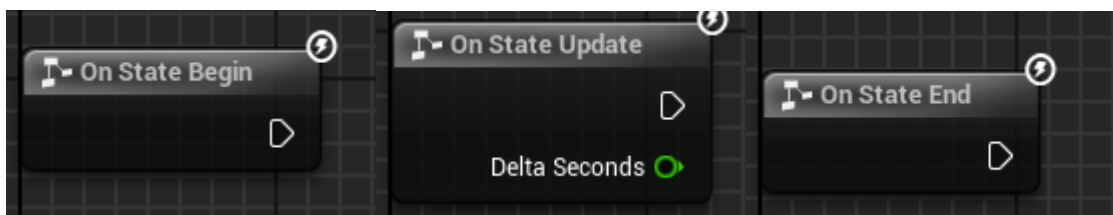


Figura 4.7 Eventos asociados a los estados de una máquina de estados. Un evento es una acción o función que se ejecuta en respuesta a un cambio o condición específica durante la ejecución del sistema. Se incluyen tres tipos por defecto: Evento Begin, activado al ingresar al estado; Evento Update, ejecutando en cada tick del sistema mientras se permanece en el estado (Update se refiere al proceso cíclico mediante el cual el sistema evalúa y actualiza su comportamiento periódicamente); y Evento End, llamado al salir del estado.

4.6.2 Funcionamiento de Máquinas de Estado con un LLM

Para integrar de manera efectiva las máquinas de estado con el modelo de lenguaje, fue necesario diseñar una arquitectura que permitiera al modelo proponer transiciones entre estados de forma comprensible y fácilmente interpretable por el sistema. El objetivo principal era darle al LLM la capacidad de controlar comportamientos específicos sin necesidad de que tuviese que gestionar estructuras complejas, recurriendo en su lugar a un flujo de trabajo simple, robusto y basado en cadenas de texto.

El diseño de estas máquinas de estado se fundamenta en una estructura sencilla pero eficaz, compuesta por los siguientes elementos:

Variables de control:

Cada máquina de estado mantiene internamente dos variables clave:

- CurrState: el nombre del estado en el que se encuentra actualmente.
- LastState: el nombre del estado previo, útil para tomar decisiones contextuales o revertir acciones si es necesario.

Diccionario de estados (StatesList):

Cada máquina incorpora un diccionario que define todos los estados posibles, junto con una breve descripción de su propósito o comportamiento. Este recurso contextual permite al LLM comprender las opciones disponibles y tomar decisiones fundamentadas sobre a qué estado debe realizar una transición un determinado agente.

Transición de estados basada en texto:

Las transiciones entre estados se realizan mediante una comprobación directa del valor de la variable CurrState. Cuando el LLM devuelve un nuevo estado, se comprueba si existe dentro de la lista de los estados posibles. En ese caso, se establece la variable CurrState con el nuevo estado. El sistema detecta automáticamente el cambio y pasa al nuevo estado ejecutando la lógica interna asociada a este. En caso contrario, se mantiene el estado actual.

4.7 Gestión dinámica de objetos interactivos

Todos los objetos interactivos de la escena están representados mediante el actor *InteractiveObj*. Este actor contiene una malla, cuya forma por defecto es un cubo. También contiene el componente *BP_ObjectInfo* que permite *serializar* la información del objeto, conteniendo su nombre y descripción además de otra información relevante. La serialización consiste en convertir un objeto o una estructura de datos en un formato que pueda ser transmitido o almacenado.

Añadir objetos no esenciales a la escena resulta sencillo: se crea un *InteractiveObj*, asignándole un nombre y especificando una descripción, además de cambiar la malla a una que se ajuste al objeto. Una descripción detallada permite al LLM decidir mejor el uso de los objetos y las consecuencias que puede tener cada interacción, dando lugar a una gran variedad de posibilidades jugando únicamente con las descripciones.

El manejo de los objetos no esenciales se realiza mediante un sistema simple: se pueden añadir nuevos objetos a la escena, modificar los ya existentes o borrarlos. Cuando el jugador realiza una interacción entre 2 objetos no esenciales, ambos se serializan y se mandan al LLM con el *ActionPrompt* del *PromptManager*, con el objetivo de informar al modelo de la interacción y que pueda decidir el resultado de la misma. Posteriormente, la respuesta se *deserializa* para que los

cambios tengan consecuencias en el juego. La deserialización consiste en el proceso contrario a la serialización: reconstruir el objeto con la información obtenida.

4.7.1 StateManager

El actor *StateManager* guarda un mapa de strings a *InteractiveObj*, permitiendo el fácil acceso por nombre a cada objeto interactivo. Con esto también se ve la importancia real del identificador único de los objetos, para evitar sobrescribirlos. Dicho actor es el encargado de recoger la interacción realizada entre dos objetos, u objeto y NPC, y crear el prompt mediante el uso del *PromptManager*. Cuando llega una respuesta del *ActionPrompt*, el *GlobalManager* le pasa la información al *StateManager* y este lo guarda en un JSON. Posteriormente, procesa cada una de las posibilidades, ya sea crear, modificar o destruir algún objeto no esencial.

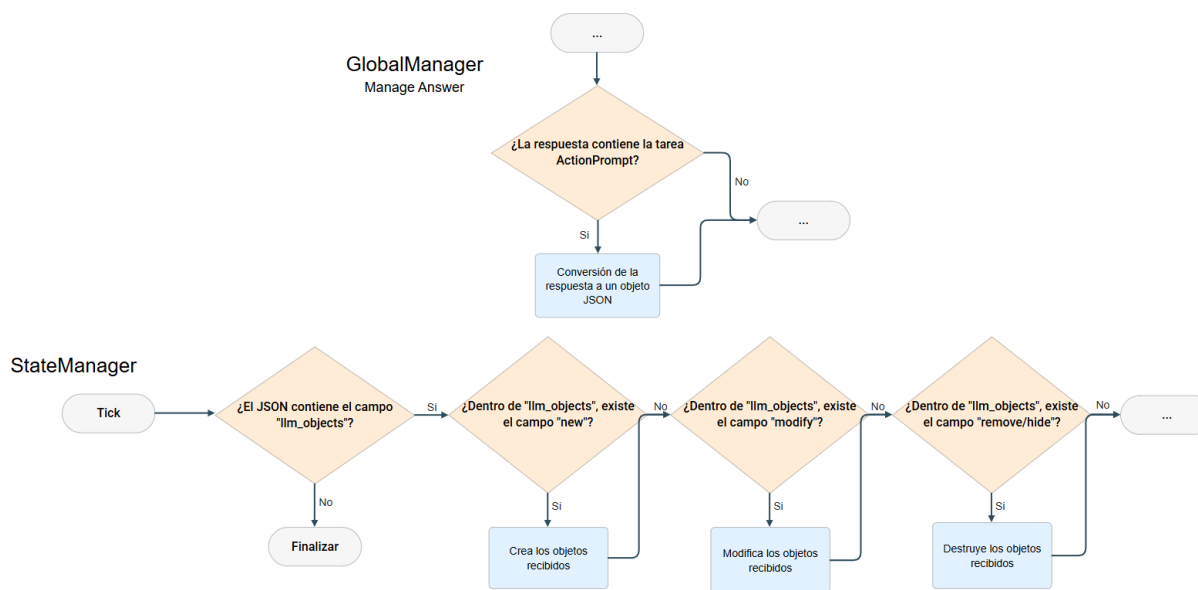


Figura 4.8 Diagrama de flujo del procesamiento de la respuesta de una interacción entre objetos no esenciales. Se recoge la información en un objeto JSON. El *StateManager* comprueba en su tick si la respuesta contiene cada uno de los campos correspondientes, y actúa en los casos pertinentes.

4.7.2 Recoger la acción realizada y construir el mensaje

El script *BP_PlayerInfo* se encarga de detectar el input del jugador. Si el jugador interactúa con otro elemento del entorno, el sistema identifica el tipo de objetivo —ya sea un NPC o un objeto interactivo— y procede a serializar la información relevante del instigador y el receptor. A continuación, el mensaje estructurado se transfiere al *StateManager*, que lo envía al *PromptManager*, indicando además qué Tasks deben activarse en función del contexto de la acción.

4.7.3 Aplicar los cambios en la escena

Una vez recibida la respuesta del modelo de lenguaje, esta se deserializa, permitiendo reconstruir objetos o aplicar modificaciones a partir del contenido generado en la respuesta, para actualizar el estado de la escena en función de las consecuencias decididas por el LLM. El modelo responde con un bloque estructurado de la siguiente forma:

```
{
  "llm_objects":{
    "new": [ ], // new contiene los objetos nuevos
    "modify": [ ], // objetos existentes modificados
    "remove/hide": [ ] // objetos que deben ser borrados
  }
}
```

Cada array representa un tipo de reacción:

- "new": creación de nuevos objetos
- "modify": modificación de objetos ya existentes
- "remove/hide": eliminación de objetos

Dentro de cada array se introducen objetos en formato JSON con la siguiente estructura general:

```
{
  "name" : "Nombre del objeto",
  "summary": "Descripción y resumen de los cambios anteriores",
  "creator": "Nombre del objeto que lo ha creado",
  "components": [ ]
}
```

El StateManager evalúa la existencia de elementos en cada uno de los bloques —new, modify, remove/hide— y, en caso afirmativo, invoca los métodos responsables de aplicar cada tipo de cambio.

Creación de objetos (new)

Si el array new contiene elementos, se invoca la función de creación de objetos interactivos. Cada objeto nuevo se instancia siguiendo estas reglas:

Nombre: Es el identificador único del objeto dentro de la escena. Para garantizar su unicidad, el sistema implementa un proceso de validación durante la creación de nuevos objetos. Al recibir la petición de instanciar un nuevo objeto con un nombre determinado (por ejemplo, "Coche"), se recorre toda la lista de objetos actualmente presentes en la escena y se analiza si existen otros nombres que coincidan total o parcialmente, como "Coche", "Coche_3" o "Coche_8". Si el nombre original ("Coche") no está en uso, se asigna directamente. En caso contrario, el sistema identifica todos los sufijos numéricos existentes

asociados a ese nombre base y selecciona el primer índice libre disponible (por ejemplo, "Coche_4" si los anteriores ya están ocupados). Este control exhaustivo evita conflictos en tiempo de ejecución y garantiza que todos los objetos puedan ser accedidos de forma inequívoca por el StateManager.

Descripción: Contiene la descripción inicial del objeto. Tras realizar una interacción, este campo se actualiza con nueva información. Esto puede provocar que las descripciones sean muy largas, así que se pide constantemente que mantenga resumida la información.

Posición: Cuando se quiere crear un objeto, el campo *creator* de la respuesta señala cuál de los dos elementos implicados en la interacción —ya sea el instigador o el receptor— debe actuar como referencia espacial para la ubicación del objeto generado.

Es decir, el LLM, en función del contexto de la interacción, decide cuál de los dos participantes tiene más sentido como origen para el nuevo objeto. Por ejemplo, si un líquido se vierte sobre una superficie, la superficie podría ser el creador del charco resultante.

El sistema realiza una serie de comprobaciones para ubicar el nuevo objeto en una posición adyacente libre. Primero se comprueba si tiene espacio delante, a su izquierda, a su derecha o arriba, mediante varios *SphereCast*. Un *SphereCast* es un método de comprobación de colisiones dado un radio específico en el tick en el que se utiliza. Si hubiese espacio, genera el nuevo objeto en esa posición. En caso de que no hubiese, se realiza la misma comprobación con esferas más pequeñas, por lo tanto, el objeto que se generará será más pequeño. Este proceso se repite hasta encontrar una posición válida, o en su defecto, hasta un número máximo de iteraciones. Si no puede generarlo, lo intenta crear en la posición del objeto creador. Como último recurso, se crea en la posición el StateManager.

Modificación de objetos (modify)

El array *modify* recoge los objetos ya existentes en la escena que deben actualizarse tras una interacción. Solo pueden incluirse aquí los objetos directamente implicados en la interacción —instigador o receptor— y no se permite modificar terceros objetos ajenos al evento.

Las modificaciones pueden afectar a:

- La descripción del objeto: se actualiza para reflejar los cambios producidos en su estado o condición (por ejemplo, de “jarra llena” a “jarra vacía”).
- Sus componentes internos: pueden añadirse o modificarse para representar nuevas funcionalidades, estados físicos o mecánicos.

Cabe destacar que el nombre del objeto no puede modificarse en ninguna circunstancia, ya que actúa como identificador único dentro del StateManager. Alterarlo comprometería la integridad de las referencias internas del sistema.

3. Eliminación de objetos (remove/hide)

El array remove/hide indica qué objetos deben desaparecer de la escena como consecuencia de la interacción. Al igual que en el caso anterior, únicamente pueden eliminarse el instigador o el receptor. Esto garantiza que el modelo no afecte a objetos no relacionados directamente con la acción del jugador.

4.8 Sistema de componentes

El sistema presentado hasta ahora —basado principalmente en el uso de máquinas de estado y la gestión dinámica de objetos interactivos— no permite crear en tiempo de ejecución objetos con comportamientos únicos y la modificación de su comportamiento como resultado de una interacción. Esto se debe a que los objetos que incorporan máquinas de estado están predefinidos antes de la fase de ejecución.

4.8.1 Diseño de la composición de funcionalidades

Como primera aproximación a este problema —de manera que se pueda aumentar las mecánicas que tiene el juego—, fue el uso de jerarquía de clases para definir cómo se comportan los distintos objetos del juego. Los objetos tendrían a disposición diferentes clases de las que pueden heredar para obtener nuevas funcionalidades. Por ejemplo, si existiera la jerarquía de clases: Actor → InteractiveObject → MovableObject → Vehicle se podría generar un coche o una moto de la clase Vehicle, obteniendo todas sus características de su clase, como montarse en ellos o su movimiento, y las de sus clases padre.

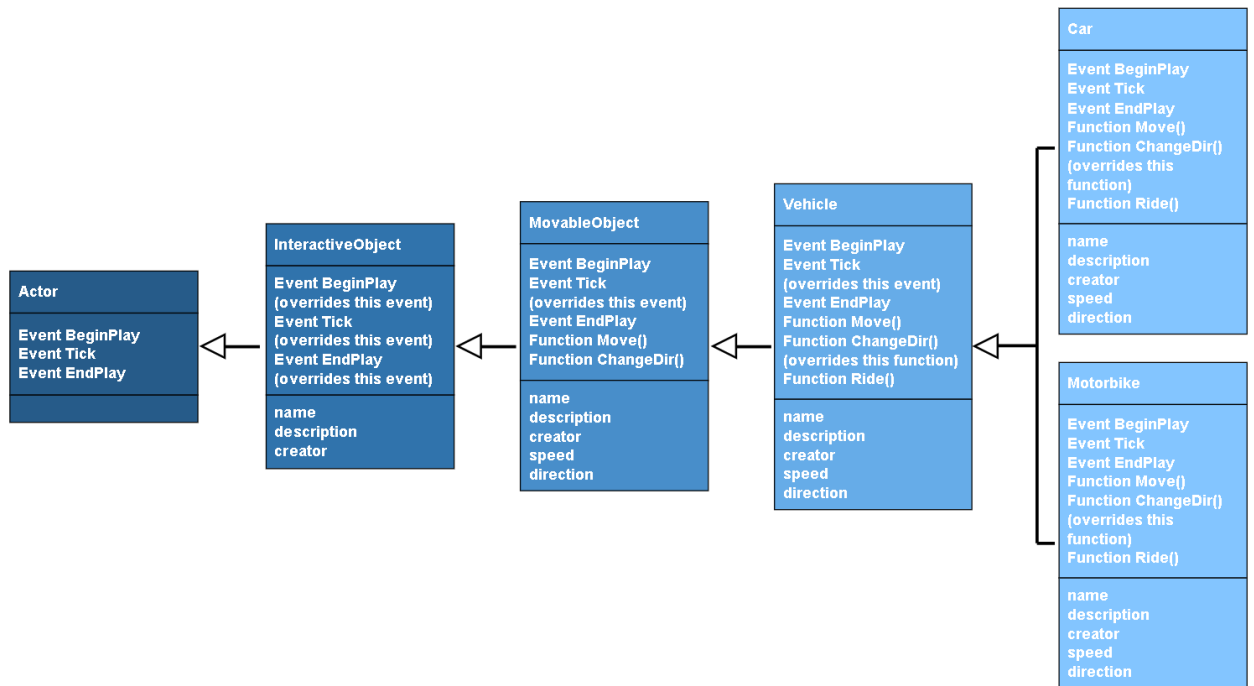


Figura 4.9 UML de ejemplo de herencia. Un UML o Unified Modeling Language (Lenguaje de Modelado Unificado) es un lenguaje que permite construir diagramas, enfocado para sistemas software. En este diagrama se puede apreciar las clases Car o Motorbike que heredan de Vehicle. Vehicle de MovableObject y este de Actor. Con el LLM, no haría falta definir las clases Car o Motorbike. Si decide crear un coche, buscaría la clase más parecida semánticamente, por ejemplo, Vehicle, y heredaría su funcionamiento.

Al profundizar en este concepto, se descartó para utilizar un acercamiento más modular. Para la composición de comportamientos diferentes en videojuegos, varios motores de videojuegos, como Unity o Unreal, utilizan el patrón de diseño conocido como *sistema de entidades-componentes*³⁷. De esta manera, un objeto puede incorporar varios componentes. Cada componente aporta una funcionalidad diferente. El acercamiento no limita el comportamiento a la funcionalidad heredada de una clase. Por lo tanto, se creó el componente *LLMInstantiableComponent*, que funciona como una interfaz para los nuevos componentes que se quieran crear.

³⁷ Sistema de entidades y componentes: https://en.wikipedia.org/wiki/Entity_component_system

4.8.2 Estructura y uso de los componentes

La interfaz tiene utilidades para deserializar y serializar un componente a JSON. Cada componente sigue la siguiente estructura:

```
{
  "name": "Nombre del componente",
  "additional_info": {} // objeto que contiene las propiedades
                        componente en cuestión
}
```

El *StateManager* contiene cada uno de los componentes que van a ser manejados por el LLM, pero desactivados. De esta manera, puede serializar cada uno de ellos para formar el prompt relacionado con los componentes. Por otro lado, cada componente contiene una variable de descripción, que contiene un texto de cómo y en que contexto se debe usar. Esta información también se añade al prompt para ayudar al LLM a la hora de decidir que componente usar.

En el momento de serializar cada componente, se rellena el campo *additional_info* con la función *UpdateVariables*. Esta función se diseñó para que se guardasen aquellas variables del componente que se quieran enviar al LLM con sus valores actuales en un objeto JSON —que es el que se muestra en el campo *additional_info*—. En el caso de ejemplo de un posible componente de vida, podría sobrescribir esta función para guardar la vida actual y la vida máxima, con sus respectivos valores actuales en el JSON.

Al recibir la respuesta, cada objeto serializado contiene un campo *components*, que es un array de objetos JSON. En nuestro caso, cada objeto JSON es uno de nuestros componentes. El LLM devuelve esa estructura añadiendo aquellos componentes que decida crear o cambiar y el *StateManager* lo procesa.

El *StateManager* recorre cada uno de los componentes recibidos de cada objeto. En caso de que el objeto no lo contenga previamente, se le añade un nuevo componente de la clase especificada. Al empezar la ejecución, se registra un mapa de strings y referencias a la clase *LLMInstantiableComponent* y sus clases derivadas. A la hora de añadir uno nuevo, se busca si el nombre existe en el mapa y —en caso de existir— se utiliza esa referencia a la clase para instanciar un nuevo componente de esta.

Si el objeto ya tiene el componente, se actualiza su estado. En ambos casos, el componente se deserializa y cada uno puede definir que hacer al recibir una respuesta. Normalmente se actualizan aquellas variables recibidas y se realiza la lógica correspondiente. Por ejemplo, en el caso mencionado anteriormente del componente de vida, se recibiría la vida actual y, en caso de llegar a 0, se eliminaría el objeto. También, se puede distinguir si se ha deserializado por dos motivos: se ha creado el componente o se ha actualizado. Esto permite que la primera vez que se crea, se pueda ejecutar una lógica diferente. Un ejemplo práctico sería la vida máxima, en caso de que no se quiera cambiar una vez establecida.

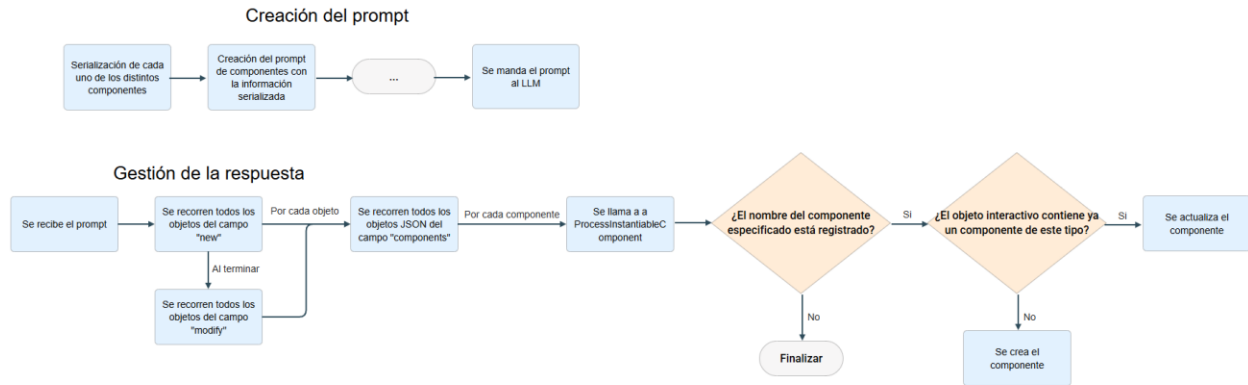


Figura 4.10 Diagrama de flujo de la respuesta del LLM acerca de los componentes.

4.8.3 Casos de uso

Habiendo definido ya la estructura, se añadieron dos componentes. El primero es un componente que permite cambiar la malla de un objeto durante su creación o modificación, eligiendo una entre un surtido de mallas disponibles. El segundo permite reproducir un sonido a partir de una lista de sonidos disponibles. De esta manera, puede escoger aquella malla o sonido que más se asemeje a la interacción realizada, dando una sensación de inmersión mayor en la experiencia.

4.9 Reconocimiento de Voz

Para la implementación de este componente del sistema se ha utilizado *Whisper*¹, una inteligencia artificial de código abierto para *Automatic Speech Recognition* (ASR) de OpenAI. La razón principal por la cual se ha utilizado este modelo es por ser gratuito, además de contar con una API bien documentada, sobre la que existen múltiples recursos disponibles en la web. Prácticamente todos los modelos que se pudieron comparar con Whisper necesitaban de algún tipo de suscripción o prueba gratuita donde datos personales eran requeridos. Por eso fueron descartadas como opciones viables y se escogió la inteligencia artificial de OpenAI como la herramienta a la que confiar la transcripción.

Existen diferentes tamaños de modelo, donde un mayor tamaño implica un rendimiento superior, aunque también conlleva un aumento significativo en el coste de memoria utilizado y una reducción en la velocidad. Pueden ser multilingüaje —con capacidad de procesar múltiples idiomas simultáneamente— o únicamente en inglés, mostrando un desempeño limitado en otras lenguas. La *tabla 4.11* muestra una comparativa de los modelos listados, según la información disponible en el repositorio oficial de Whisper.

Tamaño (Parámetros)	Memoria de Vídeo Requerida	Velocidad Relativa	Idiomas
tiny (39M)	~1GB	~10x	Inglés/Múltiples
base (74M)	~1GB	~7x	Inglés/Múltiples
small (244M)	~2GB	~4x	Inglés/Múltiples
medium (769M)	~5GB	~2x	Inglés/Múltiples
large (1550M)	~10GB	1x	Solo Múltiples
turbo (809M)	~6GB	~8x	Solo Múltiples

Tabla 4.1111 Modelos disponibles de la IA Whisper para procesamiento de voz. Esta tabla es una réplica de la disponible en el repositorio oficial de GitHub de Whisper, tan solo que ha sido traducida al castellano, a excepción de los nombres de sus modelos.

En un escenario ideal, el trabajo se presta a utilizar el modelo *turbo*, ya que combina una alta velocidad de procesamiento con un nivel de precisión inigualable por los otros modelos. Sin embargo, como se ha mencionado en varias ocasiones, este trabajo se ejecuta localmente en un único equipo. El uso de memoria que requiere el modelo *turbo* supera a la VRAM total de la que se dispone. Por ello, se terminó decidiendo utilizar el modelo *base*, el cual tiene una precisión bastante mayor que el *tiny* y su consumo en memoria es asequible.

Así mismo, para aumentar la precisión del modelo, desde OpenAI se recomienda utilizar únicamente la versión inglesa, ya que las pruebas realizadas en castellano daban resultados más bien pobres, como palabras mal transcritas, cortes de frase indebidos y falta de términos concretos. Por lo tanto, se restringió el idioma para intentar mantener la mayor precisión posible dentro de los recursos disponibles.

En un primer momento, se intentó conectar con Whisper utilizando su librería en C++, dado que Unreal Engine opera sobre este lenguaje. Integrar las librerías necesarias en el motor utilizando como base el repositorio *whisper-cpp*³⁸ pareció la mejor idea, para mantener el sistema de diálogos dentro de la aplicación, evitando tener que comunicarse con otra aplicación externa. La implementación y los ejemplos básicos incluidos en Whisper son realmente impresionantes y completamente funcionales. Sin embargo, al intentar integrar las librerías utilizadas por dichos

³⁸ *whisper-cpp*: <https://github.com/ggml-org/whisper.cpp>

ejemplos en Unreal Engine, se encontraron múltiples complicaciones. La limitada experiencia y la escasa documentación disponible en relación con la incorporación de librerías de terceros dificultaron considerablemente su implementación. Aunque finalmente se logró incluir las librerías, Unreal detectó numerosos errores al compilar los archivos fuente de Whisper. Por este motivo, se optó por descartar este enfoque dentro del motor.

Por lo tanto, se optó por la utilización de la API *whisper*, desarrollada para Python, que rápidamente se supo utilizar para hacer que el modelo transcribiese audios grabados en disco. Posteriormente, se incluyeron una serie de módulos destinados a la grabación de audio y su procesamiento de forma asíncrona. Además, otra ventaja presentada por esta alternativa es que ya se había utilizado este *pipeline* con el subsistema desarrollado para la comunicación con el modelo de lenguaje, que también está basada en Python (*apartado 4.3*).

El resultado de este ejercicio es un archivo *.py* (extensión de archivos Python) que actúa como un servidor. Cuando recibe un cliente, espera a que este le dé una señal específica para hablar y, entonces, comienza a grabar su voz utilizando el micrófono que el sistema operativo tenga por activo en ese momento. Cuando el cliente envía otra señal que indique fin de grabación, el script procesará la última información grabada, concatenándola con la procesada hasta entonces y enviando el resultado al cliente. Este acercamiento también permite ejecutar el script en un ordenador remoto en caso de disponerlo. De este modo, se reduce la carga sobre la GPU, liberando espacio y procesamiento destinado al modelo largo de lenguaje encargado de la simulación.

A continuación, se presenta un listado de los módulos utilizados, junto con su función:

- **socket:** Para establecer comunicación mediante el uso de sockets con el motor, siendo este script el servidor al que el motor debe conectarse.
- **struct:** Utilizado para empaquetar los datos a enviar por el socket en un paquete de bytes.
- **numpy:** Para el tratamiento del audio grabado.
- **speech_recognition:** Utilizado para el acceso al micrófono y para grabación de entrada de audio. También se utilizó porque tiene una buena implementación para entender cuándo una frase ha terminado.
- **whisper:** Ganar acceso a los modelos para transcripción de audio a texto.
- **torch:** Para que el modelo de reconocimiento de voz utilice la tarjeta gráfica NVIDIA, si es posible.
- **datetime:** Para cálculos de cuándo ha comenzado/terminado una frase.
- **asyncio:** En un principio no se utilizó un acercamiento multihilo, pero se vio necesario utilizar procesos asíncronos para poder recibir entrada desde el *socket* a la vez que el programa continuaba grabando la voz.

- **pyaudio:** Para encontrar el micrófono activo.

Uno de los *sistemas* mencionados en el *apartado 4.1* que conforman la escena de la aplicación es el *SpeechToTextSubsystem*, un objeto sin representación visual encargado de disparar aquellos eventos relacionados con el reconocimiento de voz. Se encarga principalmente de cuatro funciones:

- **Envío de *flag* (señal) de inicio de grabación:** Se envía al comenzar a pulsar la tecla ‘v’. Permite al script de Python comenzar a grabar.
- **Envío de *flag* de fin de grabación:** Se envía al soltar la tecla ‘v’. Informa al script de Python que debe dejar de grabar.
- **Recibir el texto transcrito:** Se recibe el texto que se está transcribiendo poco a poco. Este no recibe el texto completo, si no partes de la frase que ya han sido procesadas.
- **Recibir *flag* de fin de frase:** Se recibe el final de la frase grabada y una *flag* indicando que la frase ya ha terminado. Como se recibe esta *flag*, se intenta lanzar un prompt al LLM de tipo conversación, donde la frase completa es lo que el jugador le dice a un NPC, en caso de que haya uno cerca capaz de escucharle.

Las *flags* de grabación mencionadas no formaban parte del diseño inicial; sin embargo, fue necesario incluirlas, ya que mantener el micrófono en uso de forma continua no es una práctica recomendable. Además, la ejecución constante del modelo para conversaciones que podrían no ser relevantes para el juego incrementa innecesariamente el coste computacional. Por ese motivo, se optó por incorporar dichas *flags* de inicio y fin de grabación, las cuales, en este caso, se activan mediante la entrada del usuario a través del teclado.

El método llamado durante la inicialización del subsistema ha sido sobrescrito para realizar una llamada al sistema operativo con el fin de ejecutar el script de Python que representa el servidor de reconocimiento de voz. Este método esperará hasta haber establecido la conexión con el servidor para continuar la ejecución del juego.

Finalmente, el resultado de la implementación puede resumirse en la *figura 4.11*.

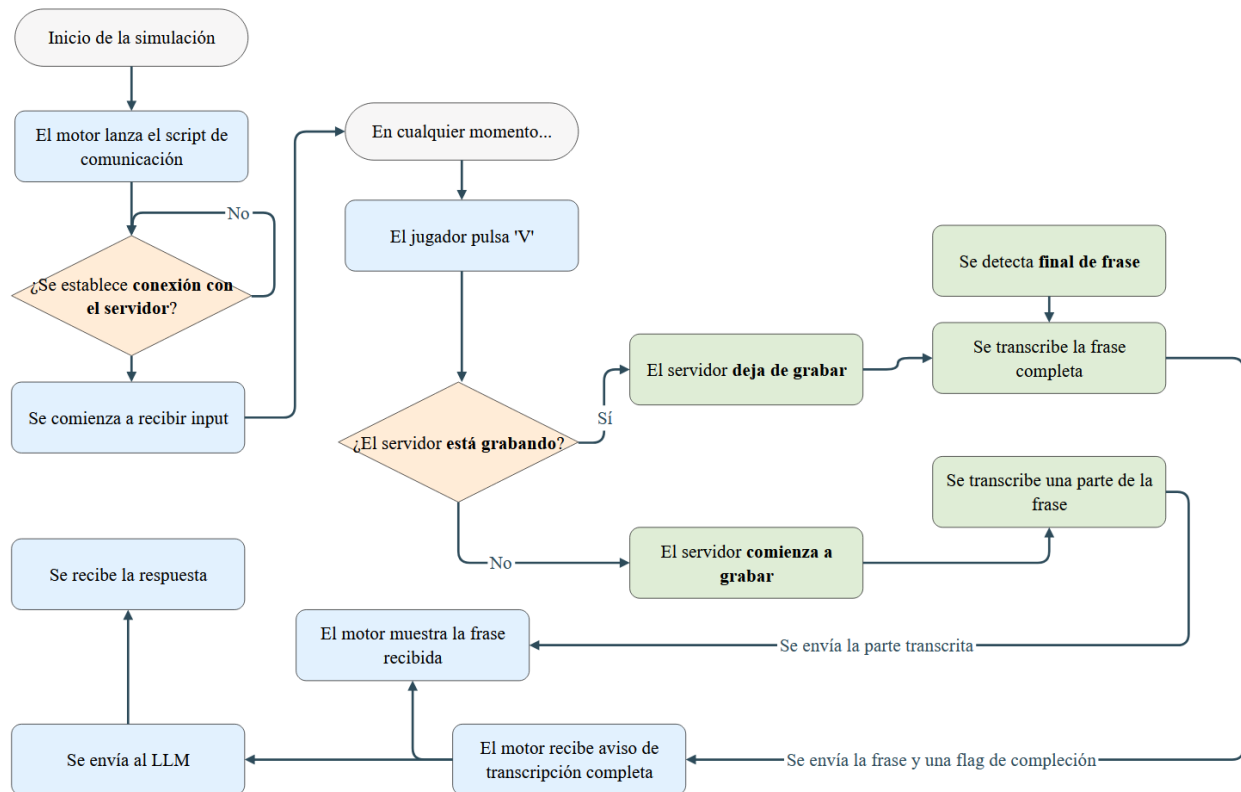


Figura 4.11 Diagrama de flujo del funcionamiento del sistema de reconocimiento automático de voz.

En la utilización de este sistema, es recomendado hablar de manera alta y clara, ya que el margen de error del modelo es relativamente alto cuando la vocalización no es la esperada por ser un modelo con —en comparación con los demás— pocos datos de entrenamiento. Esto, junto a la utilización del inglés como idioma de comunicación único, pueden provocar sesgos en los resultados de las interacciones, pues es posible que la soltura con el idioma no sea la adecuada para expresarse como un usuario lo haría en su idioma natal.

Concluyendo, se ha desarrollado un script de comunicación por reconocimiento de voz asíncrono que es capaz de enviar la información transcrita utilizando una comunicación mediante sockets con un cliente.

4.10 Texto a Voz

Para permitir la comunicación entre los NPCs y el usuario, el sistema emplea un plugin *TextToSpeech* integrado de forma nativa en Unreal Engine. Cuando se recibe la respuesta del LLM tras una interacción con un NPC, y si se ha generado diálogo, dicha respuesta se convierte a voz mediante este plugin.

Este plugin presenta una salida significativamente inferior en comparación con los resultados alcanzados en otros trabajos mencionados en el estado del arte, lo cual se atribuye, una vez más, a limitaciones en recursos disponibles.

Se realizó una extensa investigación inicial con el objetivo de encontrar un modelo de IA gratuito capaz de sintetizar voz a partir de texto. La mayoría de las aplicaciones o servicios que ofrecían funcionalidad de *TextToSpeech* requerían algún tipo de suscripción y/o límite de envío de tokens. Por otro lado, las soluciones que evitaban estas restricciones implicaban ejecutar localmente un modelo de síntesis de voz, lo cual supone nuevamente un alto consumo de VRAM. La aceptación de dicha solución implicaría la ejecución simultánea de tres modelos de inteligencia artificial pesados en la GPU local. Dado que el foco principal es ofrecer una experiencia reactiva y con el menor tiempo de respuesta posible, esta solución fue finalmente descartada y se optó por el plugin sencillo que Unreal Engine incorpora de forma nativa. Asimismo, esta decisión permite mantener una solución más limpia y totalmente integrada en el motor, sin necesidad de conectar con programas externos.

4.11 Prompt Engineering

Durante el desarrollo del sistema, se identificó un aspecto crítico en la comunicación con el LLM: su alta sensibilidad al diseño del prompt. Cambios aparentemente insignificantes en la entrada —como el orden de los elementos, la redacción de las instrucciones, o incluso el uso de mayúsculas y signos de puntuación— podían provocar variaciones importantes en la salida generada por el modelo, afectando directamente al comportamiento del sistema.

Un prompt correctamente diseñado no solo mejora la calidad de la respuesta generada por el LLM, sino que también incrementa la fiabilidad, estabilidad y reproducibilidad del sistema. Dado que muchas de las decisiones y comportamientos en el entorno virtual dependen directamente de la información generada por el modelo, incluso una pequeña ambigüedad puede dar lugar a resultados erróneos, incoherentes o inesperados.

Conscientes de ello, se prestó una especial atención a diseñar y perfeccionar los prompts utilizados por el sistema. A medida que se fueron implementando nuevas funcionalidades, el diseño de los prompts evolucionó de forma constante, ajustándose a las nuevas necesidades y al tipo de información esperada del LLM.

Este proceso iterativo de mejora permitió alcanzar un alto nivel de comprensión por parte del modelo y una alta consistencia en los resultados generados. En definitiva, el cuidado en el diseño del prompt resultó ser tan importante como la implementación técnica de los módulos que procesan sus respuestas.

También, con el objetivo de mantener una mayor coherencia en el desarrollo del sistema y dado que la mayoría de las bases de datos sobre las que se entrenan estos modelos están compuestas principalmente por contenido en inglés, se tomó la decisión de trabajar íntegramente en este idioma, tanto a nivel de prompts como de interfaz y procesamiento textual. Esta elección no solo favorece la uniformidad en la comunicación con el modelo de lenguaje, sino que también optimiza el rendimiento de los módulos de reconocimiento y síntesis de voz (speech-to-text y text-to-speech), los cuales, como se explicó en el 4.9, muestran una precisión y naturalidad significativamente superiores al operar en inglés.

4.11.1 Prompts Usados

A continuación, se muestra un listado de todos los prompts utilizados en el proyecto, junto con las dificultades encontradas y las decisiones tomadas para resolverlas.

Prompt final

Este encabezado forma parte del prompt final enviado al modelo de lenguaje. Su propósito es ofrecer una explicación general sobre la tarea que debe realizar y cómo debe interpretar y procesar el conjunto de Tasks que se le presentan:

```
“You are participating in a multi-step simulation, located in a
prison, where the player is a prisoner and tries to escape. You will
be given several tasks, each marked with TASK followed by a number,
such as TASK1, TASK2, etc.
You MUST return a single JSON object with the following structure:
{
  "TASK1": { /* Your response to TASK1 here / },
  "TASK2": { / Your response to TASK2 here */ },
  ...
}
Each task's result must be a valid JSON object embedded under its
corresponding TASK key. DO NOT merge tasks together.
Follow the expected schemes for each task. Use the output of earlier
tasks to help inform or complete later tasks.
Do NOT invent keys or metadata that are not requested.
Here are the tasks you must complete:”
```

Este fragmento corresponde a la parte final del prompt, donde se refuerzan las instrucciones sobre la estructura y el formato de la respuesta que el modelo debe generar, así como la información específica que debe contener:

Remember I want you to create a JSON that contains all the specific JSON that each task asks for. Each task will be marked with the word TASK followed by a number so it is easier for you to locate them.

Example:

```
{ "TASK1":{the JSON that answers that task}, "TASK2":{the JSON that answers that task}, "TASK3":{the JSON that answers that task},...}. Each answer must have the same tag as the original task, for example the answer of the "TASK1" should be a JSON with the tag "TASK1". Do NOT include anything that is NOT a task. You must return the answer of each TASK, DO NOT leave any TASK without answer.
```

Durante el diseño de este prompt, se identificó la importancia de establecer de manera clara y explícita las instrucciones al inicio. No obstante, se comprobó que resultaba igualmente crucial repetir dichas instrucciones al final, especialmente en contextos donde se presentan múltiples tareas o bloques extensos de información adicional.

En escenarios complejos como en el desarrollo de este proyecto, el prompt puede crecer considerablemente en longitud, lo que incrementa el riesgo de que el modelo pierda de vista la instrucción principal o no respete completamente el formato deseado. Por este motivo, se optó por reforzar la instrucción inicial al final del prompt, asegurando así una mayor consistencia en las respuestas generadas y una alineación más precisa con los objetivos esperados.

SignalPrompt

El SignalPrompt contiene toda la información adicional relacionada con las señales que se encuentren activas en el entorno del juego en un momento determinado. Cada señal gestiona de forma individual y autónoma la generación de su propia información contextual, la cual es posteriormente enviada al PromptManager para ser integrada en el prompt correspondiente.

Este sistema permite que las señales aporten datos relevantes al modelo de lenguaje sin necesidad de intervención externa, garantizando así que el contexto proporcionado en cada interacción sea lo más preciso y actualizado posible. La información incluida puede variar en función del tipo de señal y de su lógica interna, adaptándose dinámicamente a las necesidades de la simulación.

Gracias a esta arquitectura modular, el SignalPrompt contribuye a enriquecer el proceso de toma de decisiones del LLM sin comprometer la escalabilidad del sistema, ya que permite incorporar nuevas señales o modificar su comportamiento sin alterar el funcionamiento general del motor de generación de prompts.

El SignalPrompt comienza de la siguiente forma:

```
Additional information:{*Información de cada señal*}
```

SceneInfoPrompt

El *SceneInfoPrompt* contiene toda la información relevante de la escena que puede resultar útil para el LLM en la toma de decisiones. Este prompt incluye:

- Una lista con los nombres de todos los objetos interactivos presentes en la escena.
- Una lista con los nombres de los NPCs, junto con una breve descripción de cada uno de ellos.
- El nombre del jugador, para poder referenciarlo cuando se necesite.

Este conjunto de datos proporciona al LLM un conocimiento global del estado actual de la escena, permitiéndole interpretar de forma precisa el contexto y actuar en consecuencia.

```
SCENE: {This is NOT a task, DO NOT return this as a new task. Here is
some information about all the elements of the "Scene". Here is the
list:
{
  "InteractiveObjects": {*Lista de nombres de cada objeto
                        interactivo*},
  "npcs": {*Lista de NPCs activos incluyendo nombre y descripción*},
  "Player": "Player"
}
```

Durante la creación del prompt, se consideró inicialmente incluir, junto al nombre, una descripción detallada de cada objeto interactivo presente en el entorno. Sin embargo, a medida que el número de objetos fue aumentando, esta estrategia provocó un incremento significativo en la longitud del prompt, lo cual afectaba negativamente al rendimiento del modelo.

Como solución, se optó por simplificar la lista de objetos, limitándola exclusivamente al nombre de cada uno. Esta decisión permitió reducir el tamaño del prompt, mantener la claridad de las instrucciones y mejorar la capacidad de procesamiento del modelo, sin comprometer la funcionalidad general del sistema.

NPCPrompt

El *NPCPrompt* se encarga de definir el Task relacionado con la generación de una respuesta por parte del NPC tras una interacción. Esta tarea está formulada de forma que el LLM pueda generar un diálogo coherente y contextualizado.

En función del número de interacciones previas registradas con el NPC, esta tarea puede incluir, además, la solicitud de un resumen del historial del personaje, con el objetivo de controlar el crecimiento de este.

El *NPCPrompt* incluye los siguientes elementos:

- El contexto del jugador.

- El contexto del NPC, incluyendo información relevante sobre su personalidad, o rol en la historia.
- El historial de interacciones y conversaciones del NPC, que recoge interacciones anteriores con el jugador u otros elementos del entorno.
- El mensaje de la interacción actual, es decir, la entrada directa que ha motivado la respuesta del NPC.

Este diseño permite que el modelo genere respuestas más naturales, coherentes con el desarrollo narrativo y adaptadas al estado dinámico del mundo del juego.

In this task, you will be asked to roll-play as your character while talking to the player. Remember to stay in character and respond naturally. When answering, remember previous interactions (in the history field), and use them if necessary to make the interaction more natural.

Player context: **Contexto del jugador en la simulación**

Your context: **Contexto del NPC en la simulación**

History: *{*Historial de interacciones y conversaciones del NPC*}*

This is the player's new message: *{*Mensaje de la nueva interacción*}*

I want you to answer two things. First: Your character's response to the player's message, taking into account the "History." REMEMBER to answer in first person roleplaying as your character. DO NOT answer like "Name: answer", just give the answer in first person WITHOUT saying who is speaking. Remember to stick to your character, context and history to make the answer more believable.

La segunda parte del prompt varía en función de si es necesario incluir un resumen del historial del NPC o no.

Con historial:

Second: A text that summarizes the entire info in the "History" including the new conversation.

I want you to respond with this exact JSON format: {"Answer": "your response in first person taking into account the "History", "Summary": "A text that summarizes the entire info in the "History" including the new conversation"}. Here you have an example of a response following the desired JSON format (use them as a template): {"Answer": "Yes, my favorite color is pink, but I also love blue. What's yours?", "Summary": "A text that summarizes the entire info in the "History" including the new conversation"}. Make sure that the format is exactly the one I ask for at all times without adding extra spaces, and above all before sending your response check it several times. This is only an example of the format, and the content should not be used as a guide for your response, the content of the answer should be natural and logical based on the context.

Sin historial:

Second: the word "empty".

I want you to respond with this exact JSON format: {"Answer": "your response in first person taking into account the "History", "Summary": "the word "empty"}. Here you have an example of a response following the desired JSON format (use them as a template): {"Answer": "Yes, my favorite color is pink but I also love blue. What's yours?", "Summary": "empty"}. Make sure that the format is exactly the one I ask for at all times without adding extra spaces, and above all before sending your response check it several times. This is only an example of the format, and the content should not be used as a guide for your response, the content of the answer should be natural and logical based on the context.

NPCStatePrompt

Este prompt tiene como objetivo principal solicitar al LLM la selección del nuevo estado (*NewState*) de un NPC tras una determinada interacción o evento dentro del entorno del juego. La tarea consiste en elegir uno de los estados disponibles que se encuentran definidos en la lista específica *ListOfStates* incluida en el propio prompt. Dicha lista no solo contiene los nombres de los posibles estados, sino también una breve descripción de cada uno de ellos, con el fin de proporcionar al LLM una comprensión precisa del significado y propósito de cada estado antes de tomar una decisión.

Se recuerda al modelo que esta simulación forma parte de un entorno de juego, por lo que se fomenta la creatividad en la toma de decisiones, siempre dentro de los límites marcados por el sistema.

La estructura del prompt puede variar ligeramente en función del número de NPCs implicados. En el caso de que solo exista un NPC afectado, se proporciona información adicional, como su nombre, su *CurrentState* y su *LastState*, con el fin de ofrecer un contexto más preciso al LLM. El resto de información relevante sobre el mundo se obtiene a través del *SceneInfoPrompt*, que se incluye siempre que la interacción involucra a algún NPC.

Un único NPC:

In this task, you will be asked to choose the NPC's "NewState". You must choose one of the states from the list "ListOfStates" of this TASK. DO NOT use any state from other lists from other TASKS and DO NOT create any state that DOES NOT exist in the list. Then return a Json with this format: {"NPC_Name":{"NewState": the state that you choose (for example "Idle")}}. This is an example of the format you must follow and it should not influence your real answer, the state you pick must depend on the current information and context you have. Take into account the result of the action performed in order to choose the new state, and make sure that the name of the state chosen

is correctly written. Here you have information about the "LastState" and the "CurrentState". REMEMBER THIS IS A GAME SIMULATION SO ENCOURAGE CREATIVITY.

Your name is: *Nombre del NPC*

Here you have the "ListOfStates"

```
{
  "LastState": *Estado anterior de la máquina de estado del NPC*
  , "CurrentState": *Estado actual de la máquina de estado del NPC*
  , "ListOfStates": {*Lista de estados incluyendo nombre y descripción del estado*}
}
```

Varios NPCs implicados:

In this task, you will be asked to choose the "NewState" for all the NPCs. You must choose one of the states from the list "ListOfStates" of this TASK. DO NOT use any state from other lists from other TASKS and DO NOT create any state that DOES NOT exist in the list. Then return a Json with this format: {"NPC_Name":{"NewState": the state that you choose (for example "Idle")}}. This is an example of the format you must follow and it should not influence your real answer, the state you pick must depend on the current information and context you have. Take into account the result of the action performed in order to choose the new state, and make sure that the name of the state chosen is correctly written. REMEMBER THIS IS A GAME SIMULATION SO ENCOURAGE CREATIVITY.

Here you have the "ListOfStates":

```
"ListOfStates": {*Lista de estados incluyendo nombre y descripción del estado*}
```

Durante la creación del prompt, se identificaron varias problemáticas relacionadas con la selección de estados por parte del LLM. En primer lugar, el modelo tendía a inventar estados no incluidos en la lista proporcionada, lo que generaba incoherencias con el sistema. Más adelante, al incorporar múltiples tareas de selección de estados dentro de un mismo prompt final —cada una con su propia lista de estados— surgió una nueva dificultad: el modelo comenzaba a mezclar estados entre tareas, seleccionando opciones de listas ajenas a la que correspondía. Además, se observó que, aunque cumpliera con la selección correcta, las respuestas resultaban a veces repetitivas o poco creativas.

Para abordar estas situaciones, se aplicaron varias soluciones. Primero, se reforzó en el prompt la instrucción de que solo podían utilizarse estados explícitamente listados, evitando así la invención de nuevos. Segundo, se especificó de forma clara que el modelo debía elegir únicamente entre los estados asociados a la tarea actual, con el fin de evitar confusiones entre listas. Finalmente, se incluyó una indicación adicional para fomentar la creatividad dentro del conjunto de opciones válidas, promoviendo así respuestas más variadas y ricas en matices sin comprometer la coherencia del sistema.

ActionPrompt

El *ActionPrompt* se plantea como una tarea encargada de la creación, modificación o eliminación de objetos no esenciales. Se activa cuando se produce una interacción entre la mano del jugador y un objeto no esencial, entre dos objetos no esenciales, o entre un objeto no esencial y un NPC. Para que el modelo comprenda adecuadamente el contexto de la acción, se le proporciona información tanto del instigador como del receptor.

Esta primera es la introducción en caso de ser **una interacción con un NPC**:

In this task, something (Instigator) is interacting with an NPC (Receiver).

The NPC reacts to the interaction causing physical changes in the environment or in the object itself.

The NPC is NEVER part of the JSON result. They are not affected directly, only react.

The Instigator is the ONLY object that can be modified or destroyed but if there is a new object, it HAS to be created.

Esta es la introducción en caso de **no tener NPC implicado**:

In this task I will give you an action that has happened, and you have to return the following JSON structure, filling the arrays new, modify and destroy with objects that reflect the consequences of the action.

Ambos comparten la siguiente parte:

This is the ACTION PERFORMED: *Acción que se ha llevado a cabo*

"remove/hide": Every object that is no longer visible in the scene must be in this array. Keep the original name and summary to describe what happened. Again, a removed object can create a new object if you consider that it is necessary. Prioritize the "remove/hide" field rather than "modify", when an object disappears from the scene due to any reason.

"modify": The instigator and the receiver of the action must be here, if they are not in another array. If an object changes or is altered in some way, always CHANGE AND UPDATE THE SUMMARY (even if it is a small change) while keeping its original name. You must never return with the same value we sent to you. The summary MUST HAVE A DETAILED DESCRIPTION of the modification due to the interaction, but you should KEEP IT SHORT. Here you must also specify the creator name. If the creator has not change do not modify the value. Remind that an object can be modified and still create a new object.

"new": Include only COMPLETELY NEW or DERIVED OBJECTS from the instigator or the receiver. It should generate always PHYSICAL and TANGIBLE objects, that can be used later on, created as a result of

the action. Adding items makes the game more dynamic (if it makes sense after the interaction). The summary must have a detailed description of the new object.

Examples: any object created as a result of the action. In this case you must indicate the name of the object that is responsible for the creation of the new object, using the "name" parameter of the instigator or receiver object. For example: "creator": "NameOfTheCreator"

"action": Write a description of the interaction that occurred between the two objects. The action must be represented in the objects too, modifying their summary or creating/destroying new items.

No object can appear in more than one array, for example, if an object is in the "new" array, it cannot appear in "modify" or "remove/hide".

Fill out all applicable arrays and keep them even if they are empty. So, you must fill the arrays and return ONLY this structure, do NOT include any extra metadata: { "llm_objects": { "remove/hide": [], "new": [], "modify": [], "action": {} } }.

Se realizaron varios ajustes para que el prompt funcionara correctamente. En un inicio, el ActionPrompt se acompañaba del SceneInfoPrompt, pero este resultó irrelevante y provocaba que el modelo modificara objetos no relacionados con la interacción.

Además, se renombró el array 'destroy' a 'remove/hide', ya que el modelo rara vez seleccionaba esta opción debido a lo determinista de su nombre. También se modificó el orden en que se presentan los arrays 'new', 'modify' y 'remove/hide'. Originalmente aparecían en ese mismo orden, pero se observó que el modelo tendía a dar más peso al primer elemento mencionado. Por ello, se colocó primero 'remove/hide' para aumentar su uso, seguido de 'modify', dada su importancia en la reacción de los objetos tras una interacción. Finalmente, se ubicó 'new', cuya funcionalidad no se vio afectada por el cambio de orden.

Otro cambio relevante fue la eliminación de ejemplos concretos. Anteriormente, se ofrecían ejemplos específicos, como una interacción entre un árbol y un hacha, lo cual hacía que el modelo incluyera sistemáticamente esos elementos en distintas interacciones que no tenían nada que ver con ello. También se proporcionaban ejemplos para el array 'new', como escombros, fragmentos o piezas, lo que resultaba en la generación recurrente de escombros independientemente del contexto y la acción. Al eliminar estos ejemplos, se logró un comportamiento más generalizado y adaptable por parte del modelo.

OriginalObjectStatePrompt

El *OriginalObjectStatePrompt* cumple una función análoga a la del *NPCStatePrompt*, pero aplicada exclusivamente a objetos esenciales del entorno del juego. Su objetivo principal es solicitar al LLM la selección del nuevo estado (*NewState*) de un objeto esencial tras una interacción concreta.

La tarea consiste en elegir uno de los estados definidos en la lista específica *ListOfStates*, incluida dentro del propio prompt. Esta lista contiene tanto los nombres de los posibles estados como una breve descripción de cada uno, lo que permite al LLM comprender el comportamiento asociado a cada estado antes de realizar la selección.

A diferencia del *NPCStatePrompt*, este prompt está diseñado para afectar únicamente a un objeto a la vez. Por tanto, además de la lista de estados, siempre se proporciona información contextual específica sobre el objeto en cuestión, incluyendo su nombre y su *LastState*, para que el modelo pueda tomar una decisión informada y coherente con su evolución previa.

Además del estado seleccionado, el modelo debe devolver una justificación breve que explique por qué se ha elegido ese nuevo estado en función del contexto y una descripción de la interacción que ha tenido lugar entre el objeto esencial y otro elemento del entorno.

Se hace especial énfasis en que el LLM no debe seleccionar estados ajenos a la lista proporcionada, ni crear estados nuevos no definidos previamente. El comportamiento del objeto debe basarse exclusivamente en la información facilitada en el prompt y en el resultado de la acción realizada.

```
In this task, you will be asked to choose
(*NombreDelObjetoEnCuestion*) "NewState". You must choose one of the
states from the list "ListOfStates" of this TASK. DO NOT use any
state from other lists from other TASKS and DO NOT create any state
that DOES NOT exist in the list. Then return a Json with this format:
```

```
{
  "NewState": the state that you choose, for example "Idle",
  "Explanation": the explanation of why did you choose that new
                  state,
  "action": A description of the interaction that occurred between
             the two objects.
}.
```

```
Take into account the result of the "ACTION PERFORMED" and context,
in order to choose the new state. Here you have information about the
"LastState" and also the "ListOfStates":
```

```
{
  "LastState": /*El estado en el que está la máquina de estados antes
```

```

        de ser actualizado al nuevo*/,
    "ListOfStates": {*Lista de estados incluyendo nombre y descripción
                     del estado*}
}

```

ComponentsPrompt

El *ComponentsPrompt* se define como una tarea autónoma, ya que no se combina con otras debido a su posible extensión. Esta tarea incluye la interacción realizada, una lista de todos los componentes posibles serializados con sus respectivos nombres y variables, un ejemplo de la estructura de un componente, y una explicación individual sobre el propósito y uso de cada componente.

This is an individual task, so you must return only one task.
The following interaction has happened:

This are the objects that were added or modified after the
interaction: {*Objetos añadidos o modificados en la interacción
anterior*}

You have to return the following structure: {"update_objects":[]}

So, you have to fill "update_objects" array with the objects involved
in the interaction. Also, each object must be filled their
"components" array with ONLY the following components, if they are
related to the object. DO NOT ADD ANY OTHER COMPONENT, ONLY THESE
ONES: [/*Componentes posibles a elegir*/]

Example of object structure:

```

{
  "name": "object name",
  "summary": "summary of the object",
  "components":[
    {"name": "component name",
      "additional_info":{"key": "value"}
    }
  ]
}

```

These are the explanations about how should be used the components,
and this should not influence your answer. THIS IS IMPORTANT because
it will make it easier for you to choose when should they be added:
{*Explicación de cada componente*}

El principal problema es que las explicaciones asociadas a cada componente no siempre son tenidas en cuenta por el modelo, por lo que el componente serializado debe ser lo más claro y autosuficiente posible. Se prefiere, por tanto, incluir cierta información directamente dentro del

propio componente en lugar de depender de su descripción externa. Por ejemplo, en el componente encargado de asignar mallas a los objetos, las posibles mallas se incluyen como una variable tipo string dentro del componente, la cual se serializa. Esto mejora la eficacia y claridad en su funcionamiento.

4.12 Adecuación de respuestas al formato requerido

Para mejorar la calidad de las respuestas, evitando posibles alucinaciones —especialmente cuando se trata de modelos más pequeños—, se optó por un acercamiento que corrigiese los errores de formato de la respuesta obtenida. Para ello, existen múltiples técnicas recomendadas actualmente descritas en *el capítulo 2*. Se decidió que la implementación de un sistema de Generación Aumentada por Recuperación (o RAG, por sus siglas en inglés de Retrieval Augmented Generation), junto al de reflexión, resultarían en respuestas con mayor coherencia en el entorno en el que se ejecuta el LLM. Es tremendamente importante eliminar dicha inestabilidad en proyectos interactivos, donde el jugador siempre está esperando una respuesta del sistema a todas sus acciones.

4.12.1 Retrieval Augmented Generation

Uno de los fallos en el diseño de este tipo de modelos es la necesidad de una red extensa de entrenamiento para poder aprender a partir de nueva información. El coste monetario y computacional requerido es extremadamente elevado para un proyecto de corta duración y desarrollado por un equipo pequeño, como ha sido el de este trabajo. Es por ello que se han ideado otras técnicas consideradas estado del arte, capaces de añadir información adicional al modelo sin tener que ser reentrenado.

Arquitectura

Durante el apartado del estado de la cuestión se detalla el funcionamiento básico de esta técnica, que ha sido seguido paso a paso para conseguir los mejores resultados posibles. A continuación, se describirán mínimamente los pasos de nuevo además de añadir información acerca de la implementación general:

- **Entorno de desarrollo:** Antes de definir realmente los pasos seguidos, se debe hablar de qué herramientas se han utilizado para el desarrollo de este sistema. De igual forma que en otros puntos de este trabajo, Python fue el lenguaje de programación escogido, por las múltiples ventajas ya evidenciadas anteriormente. La librería que ha supuesto

el pilar principal del script desarrollado ha sido *Langchain*³⁹, que es un conjunto de herramientas que facilitan la inclusión y ejecución de modelos largos de lenguaje en cualquier tipo de aplicación, ofreciendo frameworks estables y de fácil utilización, además de funcionalidades básicas, como la creación de un chatbot, por ejemplo, que realmente aceleran el proceso de creación de dichas aplicaciones. Asimismo, muchas otras librerías han acompañado el desarrollo del RAG implementado para este proyecto. Se detallarán en las secciones correspondientes donde se hayan utilizado.

- **Formación de la base de datos:** Existieron 2 tipos de bases de datos utilizadas, donde su contenido difería mucho entre sí. Cada una de ellas respondía a una necesidad diferente y fueron creadas con meros fines de prueba, para ver cuál era el tipo de datos que haría que el modelo ajustase mejor su respuesta al formato requerido. A continuación se listan los distintos tipos utilizados y en los diferentes formatos que fueron implementados:
 - **Información en escena, formateada:** Utilizando el formato YAML, se definieron los objetos que formaban parte de la escena en un archivo, donde se rellenaban los siguientes campos: nombre, descripción y componentes; cada uno de estos con sus atributos correspondientes. En otro archivo se daba información de qué componentes podían tener los objetos generados, y por tanto, solo los únicos que se podían añadir y/o modificar a objetos nuevos y/o existentes.
 - **Información en escena, lenguaje natural:** La información contenida en esta base de datos es muy similar a la anterior, tan solo que su formato resulta automáticamente legible. Se detallan en frases separadas por dos saltos de línea cada uno de los objetos en escena, con su nombre, descripción, y sus componentes asociados. Este formato permite que el LLM sea capaz de entender más fácilmente el texto escrito.
 - **Respuestas de ejemplo, formateadas:** Las respuestas con sentido que habían sido generadas por el LLM en formato JSON anteriormente ya formaban de por sí una base de datos interesante, en las que el modelo podría observar los tipos de interacciones llevadas a cabo y el formato de las mismas, teniendo ejemplos de este para evitar errores tipográficos comunes. Esta base de datos fue relativamente sencilla de generar, tan solo se tuvo que probar el modelo con

³⁹ *Langchain*: <https://www.langchain.com/>

varios escenarios diferentes y guardar sus respuestas más coherentes en diferentes archivos al terminar la ejecución de la petición.

- **Respuestas de ejemplo, lenguaje natural:** El resultado de las interacciones del jugador con su entorno se transcribieron a un lenguaje entendible por el modelo automáticamente, sin tener que razonar sobre él. Se explican qué dos objetos han interactuado y cuál fue el resultado final de su interacción, específicamente eliminación, creación o modificación de objetos además de descripciones detalladas. Utilizando otros modelos de inteligencia artificial, se pudo transcribir fácilmente las respuestas formateadas a dicho lenguaje natural, evitando gastar excesivo tiempo en realizarlas a mano.

Se encuentran varios ejemplos en el *apéndice* de algunos los archivos de las bases de datos mencionadas.

- **Partición de los datos en Chunks:** Cada uno de los formatos mencionados anteriormente utiliza una partición distinta, ya que son radicalmente diferentes:
 - **Lenguaje natural:** Cuando se hablan de archivos en formato natural, se utiliza una partición de los datos cada dos saltos de línea, tratando de aislar la información en párrafos que contengan trozos relevantes para el modelo. La implementación de dicho *chunker* —el encargado de partir un documento en chunks de información relevante— fue delegada a la librería *langchain_text_splitters*⁴⁰, concretamente a su clase *CharacterTextSplitter*⁴¹, a la que se le puede dar un carácter especial que buscará en el documento para ejercer la separación de los diferentes chunks.
 - **JSON:** Esta partición también se delegó a la librería mencionada en el apartado anterior, pero en este caso a su clase *RecursiveJsonSplitter*⁴², que parte el texto leído en JSON en diferentes chunks dependiendo de la jerarquía establecida. Cada objeto se parte en un chunk individual, llegando a un alto nivel de especificación, pudiendo saber la ubicación de dichos objetos en la jerarquía.

⁴⁰ *langchain_text_splitters*: https://python.langchain.com/docs/concepts/text_splitters/

⁴¹ *CharacterTextSplitter*: https://python.langchain.com/api_reference/CharacterTextSplitter.html

⁴² *RecursiveJSONSplitter*: https://python.langchain.com/docs/how_to/recursive_json_splitter/

- **YAML:** Se realizó una implementación propia para este chunker, utilizando la librería *yaml*⁴³ de Python para facilitar el acceso a los datos. La clase desarrollada fue llamada *YAMLSectionSplitter*, ya que su objetivo era —dada una clave de una sección del documento— partir cada una de las entradas de este en un chunk diferente, manteniendo sus datos en el mismo formato y facilitando el acceso a él. Al principio de cada ítem, se especifica la clave de la sección a la que pertenecen, para dar mayor contexto.

También hay una separación explícita si existe información a través de diferentes archivos, suponiendo cada uno de ellos un chunk como mínimo. Dicha separación es común para todos los formatos utilizados. Adicionalmente, se realiza un corte en un chunk al superar un cierto límite de caracteres, a no ser que no se puedan generar chunks más pequeños al tamaño mínimo, dado que no existe un separador anteriormente.

- **Creación de los Embeddings:** Para la creación de los embeddings del texto recibido se ha utilizado la API *HuggingFaceInterfaceAPIEmbeddings*⁴⁴, que es ofrecida por el sitio web *HuggingFace*, cuyo objetivo es dar alojamiento en repositorios en la nube a miles de modelos de machine learning. Esta página contiene miles de modelos entrenados para diferentes propósitos, además de una implementación en Python que permite acceder a una base de datos desde la que ejecutar o mandar peticiones a diferentes modelos. En este caso, se utilizó para comunicarse con un modelo de inteligencia artificial que como salida devuelve los embeddings del texto recibido. El modelo utilizado fue el siguiente: *BAI/bge-base-en-v1.5*⁴⁵.
- **Almacenamiento de los embeddings:** Utilizando la librería *Chroma*⁴⁶, que ofrece herramientas de fácil utilización para la creación de vector stores. Este es el tipo de base de datos necesaria para almacenar algo como son los embeddings del modelo usado, ya que almacenan vectores numéricos de longitud fija e implementan funciones de búsqueda optimizadas para encontrar entradas parecidas eficientemente. Se creó la vector store a través de todos los documentos generados durante el proceso de

⁴³ *yaml*: <https://pyyaml.org/wiki/PyYAMLDocumentation>

⁴⁴ *HuggingFaceInterfaceAPIEmbeddings* https://huggingface.co/docs/huggingface_hub/v0.14.1/en/guides/inference

⁴⁵ *BAI/bge-base-en-v1.5*: <https://huggingface.co/BAI/bge-base-en-v1.5>

⁴⁶ *Chroma*: <https://docs.trychroma.com/docs/overview/getting-started>

chunking, además de especificarle a Chroma el modelo de obtención de embeddings que se estaba utilizando.

- **Inclusión de datos relevantes en el prompt:** Para la generación de la respuesta se utilizó la conocida implementación de una cadena pregunta-respuesta (*QA Chain*) que ofrece Langchain en una de sus múltiples librerías. El funcionamiento de esta es sencillo: se le especifica al modelo de lenguaje un formato en el que recibirá la pregunta del usuario —con contexto e información adicional si así se quisiera— y se le añade un *retriever*, que no es más que un buscador dentro de la vector store mencionada anteriormente. Lo que hace dicho retriever es buscar similitudes entre los embeddings del prompt del usuario y los chunks contenidos en su base de datos. Aquellos elementos considerados relevantes se devolverán y formarán parte de la entrada enviada al modelo, para que conozca nuevo contexto adicional que pueda añadir a su respuesta. Además de incorporar el retriever, también se le dio al LLM una base estructurada en la que generar su contenido en formato JSON. Esto se hizo gracias a haber utilizado como ejecutores de los modelos las siguientes clases: *ChatOllama*⁴⁷ y *ChatGoogleGenerativeAI*⁴⁸. Estas permiten ejecutar el modelo especificado por el desarrollador con extrema facilidad, tan solo llamando a una función para invocar su respuesta. Y, como se menciona, también permitió utilizar un output estructurado para que se adecuara lo máximo posible al formato requerido. Esta opción se fue activando y desactivando durante las pruebas para ver cómo afectaba a los resultados, contenidos en el siguiente apartado.

Pruebas y Resultados

La realización de pruebas de RAG en este contexto se llevaron a cabo bajo las siguientes condiciones:

- Las pruebas se realizaron fuera del entorno interactivo, teniendo únicamente el script de Python ejecutándose en local y comunicándose con él mediante una terminal.
- Se probaron las cuatro bases de datos especificadas anteriormente.
- Se probaron los siguientes dos modelos: Mistral:7B (*ChatOllama*) y Gemini-1.5-pro-001⁴⁹ (*ChatGoogleGenerativeAI*).

⁴⁷ *ChatOllama*: https://python.langchain.com/api_reference/ollama/llms/langchain_ollama.llms.OllamaLLM.html

⁴⁸ *ChatGoogleGenerativeAI*: https://python.langchain.com/docs/integrations/chat/google_generative_ai/

- Todas ellas utilizaron la misma interacción como prompt base: Una llave de un coche interactúa con un coche, en un contexto de una prisión donde el jugador quiere escapar de la misma.

Se probaron diferentes formas de utilizar el prompt, que pueden encontrarse en el *apéndice*. Entre ellas:

- Haciendo que módulos que siempre recibía el modelo formasen parte de su contexto de sistema, tan solo especificándoselas una vez.
- Reestructurando la información y añadiendo más o menos contexto dependiendo de la prueba, haciendo énfasis en diferentes frases y cambiando el formato de algunas partes.
- Prescindiendo de la mayor parte del cuerpo del prompt original utilizado en Unreal Engine y utilizando tan solo el formato especificado a través de la API y las instrucciones dadas en el prompt de sistema.

Todos los outputs generados por el modelo de lenguaje fueron guardados en diferentes archivos, consiguiendo hasta un total de 50 respuestas diferentes. Se pueden encontrar algunas de ellas en el *apéndice*.

Los criterios del análisis realizado son los siguientes:

- Que la respuesta se encuentre en formato JSON, sin fallos tipográficos.
- Que la respuesta solo incluya dicho JSON.
- Que no alucine nuevas propiedades de los objetos generados, ni tampoco nuevos objetos.
- Que los resultados de la interacción sean coherentes, dando lugar a acciones que se sientan naturales o posibles.

Por desgracia, el análisis no consiguió que ninguna de las 50 respuestas generadas funcionase en todos los apartados a la vez. Se puede ver a continuación un resumen:

- Ninguna de ellas tuvo errores sintácticos en el formato JSON.
- 32 de las respuestas generadas no seguían el formato especificado, inventándose gran parte de los campos del JSON. Algunas de ellas incluían texto adicional, como mensajes de “Aquí tienes el resultado de la tarea pedida: (...)”.
- Casi todas las respuestas restantes (17) añadían componentes de comportamiento completamente inexistentes a los objetos generados/modificados, resultando inviables para la implementación en el entorno de Unreal Engine.
- Tan solo 1 respuesta fue capaz de generar todos los campos bien, exceptuando la inclusión de una frase antes y después del JSON; y que la interacción generada por el

LLM era acerca de un árbol y un coche, cuando la especificada en el prompt fue entre una llave de un coche y un coche.

Como fue anteriormente mencionado, en el *apéndice* se encuentran ejemplos de las respuestas generadas, para que se comprendan las alucinaciones del modelo y por qué la viabilidad de las respuestas era más bien nula.

Tras ver los resultados de las pruebas, se decidió descartar completamente la inclusión de RAG, ya que las respuestas generadas hasta el momento sin utilizar esta técnica cumplían mucho mejor con las tareas especificadas, a pesar de que no contaban con tanto contexto como lo hacía el modelo con generación aumentada.

Se cree que una investigación en mayor profundidad de esta técnica y una generación de una base de datos más robusta ayudaría a aumentar la generación. Sin embargo, los resultados de las pruebas realizadas desestiman mucho la capacidad que pueda tener RAG en el contexto de este trabajo. Toda la información adicional que el modelo necesite recordar/aprender en tiempo de ejecución puede ser pasada en forma de historial de cada uno de los agentes implicados en la interacción, además de un historial de acciones general, que ya ha dado mejores resultados que los especificados en esta sección.

4.12.2 Reflexion

Con el objetivo de mejorar la fiabilidad estructural y semántica de las respuestas generadas por el modelo de lenguaje, se implementó un sistema experimental basado en el principio de reflexión. Este enfoque, inspirado en trabajos recientes del ámbito de la inteligencia artificial generativa, propone que el propio modelo actúe como evaluador de su salida, validando y corrigiendo su respuesta original mediante una segunda iteración razonada.

El funcionamiento general del sistema de reflexión se estructuró en dos fases consecutivas:

1. **Generación inicial de la respuesta:** el modelo produce un primer bloque de salida en formato JSON, en base a los datos de entrada, contexto y reglas definidas por el sistema.
2. **Validación reflexiva:** se lanza una segunda consulta al modelo en la que se le solicita comprobar su propia respuesta según una serie de criterios formales y de coherencia.

Durante esta fase de validación, el modelo debía aplicar los siguientes controles:

- **Validación del formato y del tipado:** se verificaba que la estructura del JSON coincidiera con el esquema exigido por el sistema (como los campos `new`, `modify`, `remove`/`hide`, etc.) y que los valores asignados fueran consistentes con los tipos de datos esperados.

- **Análisis de coherencia interna:** se evaluaba si los cambios propuestos por el modelo eran compatibles entre sí (por ejemplo, evitar que un mismo objeto sea creado y eliminado en la misma interacción, o que se modifique un objeto inexistente).
- **Cumplimiento de reglas del sistema:** se comprobaba que las acciones generadas respetaran las restricciones del entorno (como el número mínimo de objetos creados simultáneamente, o la no modificación de objetos estructurales esenciales).

Resultados y análisis

A pesar de su planteamiento prometedor, el sistema de reflexión no resultó viable en el contexto específico de este proyecto, por las siguientes razones técnicas:

- **Aumento de la latencia:** al requerir una segunda iteración, el tiempo de respuesta se aumentó notablemente. Esto resultaba especialmente problemático al trabajar con modelos ligeros en local, donde incluso una sola iteración ya requería tiempos considerables.
- **Limitaciones en el razonamiento estructural:** muchos modelos ligeros carecen de la capacidad suficiente para analizar estructuras complejas como JSON con precisión total. Esto derivaba en validaciones erróneas, donde el modelo corregía elementos innecesarios o incluso rompía una estructura válida.
- **Inestabilidad semántica:** durante la segunda inferencia, el modelo no solo evaluaba, sino que frecuentemente regeneraba el contenido completo con ligeras variaciones, introduciendo nuevas alucinaciones o errores que no estaban presentes en la respuesta original.
- **Imprevisibilidad en la autovalidación:** aunque el sistema incluía instrucciones claras para que el modelo actuara como validador, la respuesta era a menudo inconsistente, variando significativamente incluso con el mismo input, lo que dificultaba la trazabilidad y la fiabilidad del sistema.

Conclusión

La técnica de reflexión, aunque conceptualmente sólida y respaldada por literatura reciente, exige modelos con mayor capacidad de razonamiento estructurado y mayor contexto en memoria para ofrecer resultados útiles y coherentes. En el caso de modelos ligeros ejecutados localmente, la técnica no solo no aportó mejoras significativas, sino que agravó los problemas de estabilidad, latencia y control de salida.

Por ello, se optó por desechar este enfoque en favor de estrategias más eficientes y deterministas, como la validación directa de las salidas del modelo mediante lógica de control propia —en Unreal— y un diseño de *prompts* más robusto. De este modo se logró mantener la integridad del sistema sin depender de una segunda iteración adicional.

4.13 Interfaz de usuario

En este apartado se describen los diferentes elementos que componen la interfaz de usuario implementada. La interfaz de usuario es el conjunto de elementos visuales e interactivos que permiten la comunicación entre el usuario y la aplicación, facilitando la interacción con el entorno virtual de forma intuitiva y eficiente.

Los elementos principales desarrollados son los siguientes:

- **Ventana de entrada para el diálogo con NPCs:** se trata de una interfaz situada en la esquina inferior izquierda de la pantalla, destinada a la introducción de mensajes de texto por parte del usuario para establecer conversaciones con los NPCs. Esta ventana actúa como una alternativa al sistema de reconocimiento de voz, permitiendo al usuario comunicarse mediante texto escrito en situaciones donde el uso del micrófono no sea viable o conveniente.

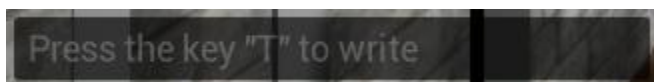


Figura 4.12 Ventana de entrada para el diálogo con NPCs

- **Ventana de entrada para interacciones en modo manual:** esta interfaz permite al usuario introducir manualmente el tipo de interacción que se está llevando a cabo cuando el sistema se encuentra en modo manual. Se activa automáticamente al interactuar con un elemento del entorno, apareciendo en el centro de la pantalla. Una vez que se introduce y envía el tipo de interacción, la ventana se cierra automáticamente, permitiendo al usuario continuar con la experiencia sin interrupciones.

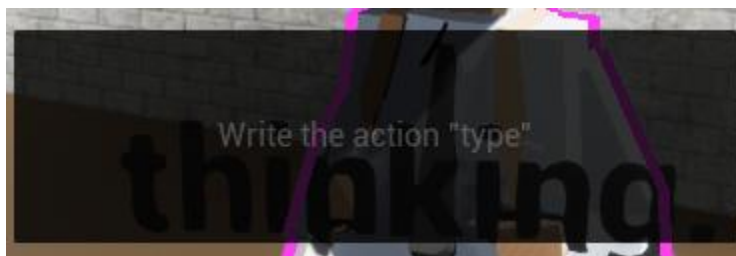


Figura 4.13 Ventana de entrada para interacciones en modo manual.

- **Indicador de procesamiento del modelo de lenguaje:** se trata de una señal visual ubicada en la esquina superior derecha de la pantalla, cuyo propósito es informar al usuario de que el modelo de lenguaje está procesando una solicitud anterior. Este indicador aparece automáticamente en el momento en que se envía un mensaje al LLM, y permanece visible hasta que se han recibido todas las respuestas pendientes de procesar.

LLM processing interaction

Figura 4.14 Indicador del procesamiento del modelo de lenguaje.

- **Iluminación de nombres de NPCs en rango de escucha:** cuando el usuario se encuentra dentro del rango de audición de un NPC, el nombre de este se ilumina como señal de que puede escuchar al usuario. Este indicador visual aparece sobre la cabeza del NPC.



Figura 4.15 Iluminación de nombres de NPCs en rango de escucha.

- **Rebordes brillantes en elementos interactivos:** al apuntar con el cursor hacia elementos que se encuentran dentro del rango de interacción, estos se destacan mediante un reborde brillante. Este efecto visual tiene como objetivo facilitar la identificación de los elementos con los que el usuario puede interactuar. Para mejorar la diferenciación, se ha implementado una diferenciación de colores según el tipo de elemento:
 - **NPCs:** se resaltan con un borde de color magenta.
 - **Objetos interactivos sujetables:** se resaltan con un borde de color cian claro.
 - **Objetos interactivos no sujetables:** se resaltan con un borde de color azul.



Figura 4.16 Borde de objetos interactivos sujetables.



Figura 4.17 Borde de objetos interactivos no sujetables.



Figura 4.18 Borde de NPCs.

- **Ventana de descripción de objetos:** esta interfaz muestra información relevante sobre los objetos presentes en el entorno, permitiendo al usuario comprender su función, características o posibles usos sin necesidad de realizar una interacción directa. La ventana aparece automáticamente cuando el usuario apunta hacia un objeto que se encuentra dentro de su rango de interacción, posicionándose a la izquierda de este.

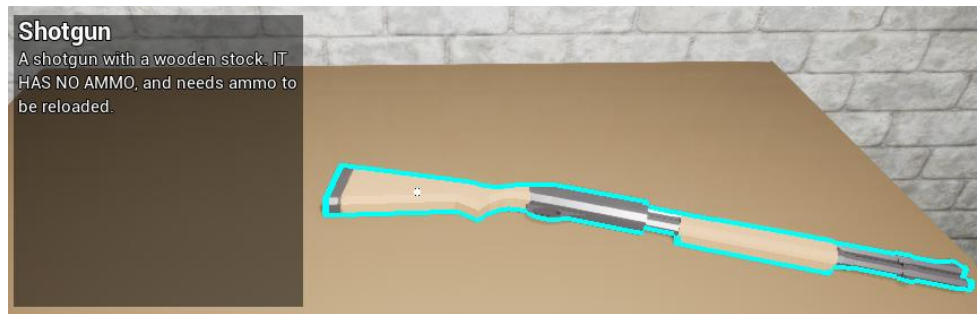


Figura 4.19 Ventana de descripción de objetos

- **Interfaz de elementos en proceso de interacción:** cuando un elemento del entorno está involucrado en una interacción que está siendo procesada, se activa un indicador visual que informa al usuario de este estado. En el caso de los NPCs, se muestra el texto "Thinking..." delante de ellos, indicando que están generando una respuesta. Por otro lado, si se trata de un objeto, aparece un icono de carga en forma de pequeño círculo azul en la esquina superior derecha de su ventana de descripción. Ambos indicadores desaparecen automáticamente una vez que se ha recibido la respuesta correspondiente, lo que permite al usuario identificar en todo momento qué elementos están activos en un proceso de interacción.



Figura 4.20 Interfaz de NPCs en proceso de interacción.

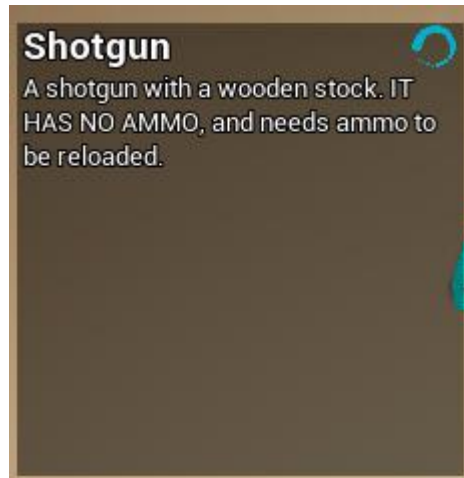


Figura 4.21 Interfaz de objetos interactivos en proceso de interacción.

Asimismo, la interfaz muestra de forma constante información adicional para el usuario, como el modo de interacción activo (manual o automático), el nombre del objeto actualmente equipado —en caso de que haya uno— y un resumen de los controles disponibles.

En conjunto, la interfaz ha sido diseñada para ofrecer una experiencia de usuario clara, accesible y alineada con las mecánicas del sistema, contribuyendo a una mayor inmersión y facilitando una interacción más fluida y eficiente dentro del entorno virtual.



Figura 4.22 Conjunto de elementos del interfaz.

Capítulo 5 - Diseño de experimentos

Tras finalizar con la implementación, se hicieron pruebas con distintos usuarios para comprobar si el procesamiento constante de interacciones mediante el LLM resulta útil, tanto para facilitar el diseño y desarrollo de un videojuego, así como para permitir a los usuarios tener un grado de libertad superior al habitual en los videojuegos convencionales.

Es de extrema importancia la realización de este tipo de experimentos, donde es posible demostrar el potencial de la herramienta desarrollada. Conocer los fallos, y puntos fuertes que los jugadores pueden percibir es muy relevante para marcar un futuro ciclo de desarrollo, tratando de solucionar todo aquello que haya formado brechas con el usuario y puliendo los apartados más robustos. En *UX* (experiencia de usuario), se definen como brechas aquellas diferencias entre los modelos mentales de los usuarios y de los desarrolladores a la hora de realizar una acción.

En este capítulo se revisan en detalle las implementaciones realizadas específicamente para la realización de dichas pruebas, que se basaron principalmente en la creación de un escenario de ejemplo para que los jugadores obtuvieran un contexto concreto. Posteriormente se exponen los diferentes apartados en los que consisten las pruebas en sí, razonando su diseño y explicando el rol de los experimentadores durante las mismas.

5.1 Diseño de nivel para el entorno de pruebas

Con el objetivo de evaluar de manera efectiva la herramienta con usuarios reales, se diseñó un nivel en el que las interacciones tuvieran un papel central en el desarrollo de la partida. Este nivel sirvió como banco de pruebas para comprobar tanto la usabilidad de la herramienta como su capacidad para materializar ideas de diseño de forma ágil.

Gracias al uso de la herramienta, el proceso de creación de la escena y la implementación de las ideas diseñadas resultó notablemente más rápido en comparación con los tiempos habituales de desarrollo en la industria del videojuego, donde estos procesos pueden extenderse durante meses o incluso años, dependiendo de la envergadura del proyecto.

Una vez generado el nivel, los propios diseñadores lo utilizaron para realizar pruebas internas, identificar comportamientos no deseados y corregirlos. Asimismo, se incorporaron nuevos objetos a la escena con el fin de aumentar la variedad de opciones disponibles y enriquecer la experiencia de usuario.

5.1.1 Descripción de la escena

El entorno de pruebas desarrollado para el proyecto tiene lugar en un centro penitenciario ficticio, donde el jugador encarna a un recluso que cumple condena. La escena se divide en dos zonas claramente diferenciadas: el ala de las celdas y el ala principal, ambas separadas por una puerta electrónica que solo puede abrirse mediante un código de acceso.

El ala de las celdas alberga, entre otros elementos, la celda del jugador y la de Antonio, un NPC que también se encuentra cumpliendo pena en el mismo bloque. La celda del jugador está equipada con varios elementos interactivos que permiten distintas acciones: una puerta de barrotes cerrada, una cama y una lima cuidadosamente oculta bajo esta última. La celda de Antonio también cuenta con una puerta de barrotes interactiva, y en su interior, además de una cama, uno de los muros puede ser manipulado, permitiendo su uso en determinadas dinámicas narrativas o de juego.



Figura 5.1 Vista inicial de la escena.



Figura 5.2 Celda inicial. Lugar de inicio del juego



Figura 5.3 Celda de Antonio, contigua a la inicial.

El resto de las celdas en esta ala no están en uso como espacios habitacionales, sino que funcionan como almacenes. A diferencia de las celdas ocupadas, estas no están cerradas, y en su interior se encuentran diferentes tipos de cajas y contenedores con los que se puede interactuar. En una de estas celdas abandonadas, bajo una de las cajas, se oculta una copia del código necesario para desbloquear la puerta electrónica que da acceso al ala principal del edificio, lo cual representa un objetivo clave para el jugador.



Figura 5.4 Celdas en frente de la del jugador.

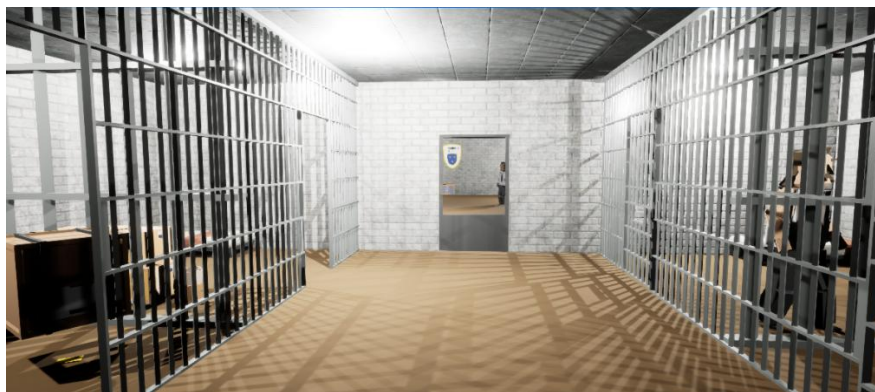


Figura 5.5 Pasillo inicial.

La segunda gran zona del escenario es el ala principal, que a su vez se compone de dos módulos: el primero de ellos, de mayor tamaño, actúa como zona de recepción; el segundo, conectado al primero, está destinado a funciones de armería y taller. Esta zona está vigilada por dos agentes de seguridad, Bob y Manuel, ambos personajes no jugadores que patrullan las inmediaciones de la entrada y suponen una amenaza directa para cualquier intento de fuga o interacción indebida por parte del jugador.



Figura 5.6 Recepción y armería.

La recepción del edificio cuenta con una variedad de objetos interactivos que enriquecen la experiencia inmersiva. Entre ellos se encuentran sillas, una televisión, una máquina expendedora, varios libros dispersos, una cámara de seguridad, un reloj de pared o una alarma de emergencia. Todos estos objetos están diseñados para poder ser usados o examinados por el jugador en diferentes momentos, aportando tanto funcionalidad como contexto narrativo. Además, la recepción incluye una puerta de salida reforzada, que representa uno de los principales obstáculos que el jugador deberá superar para completar su objetivo.



Figura 5.7 Recepción y puerta de salida. La puerta ubicada en el fondo a la derecha de la figura es la puerta reforzada de salida, por la que el jugador puede intentar escapar.

Por su parte, la armería, aunque de dimensiones más reducidas, concentra una importante cantidad de objetos de interés. Se incluyen tanto elementos no sujetables —como una caja de herramientas, varias cajoneras, taquillas de armas y una pila de cajas de madera con contenido variable— como elementos de carácter sujetable. Entre estos últimos destacan dos cajas de munición para escopeta, una caja de munición para rifle, un maletín lleno de dinero y una escopeta descargada. Como elemento crucial para el desarrollo de la historia, se encuentra también una tarjeta de seguridad, escondida estratégicamente bajo una mesa, que resulta imprescindible para abrir la puerta principal reforzada del edificio.



Figura 5.8 Armería.

Este entorno ofrece un espacio de pruebas ideal para validar tanto el sistema de interacción como la integración narrativa con el modelo de lenguaje, permitiendo evaluar el comportamiento del jugador, las decisiones del LLM y el impacto de estas interacciones en un contexto complejo, controlado y altamente interactivo.

5.1.2 Flujo esperado

La escena anteriormente descrita se diseñó con un flujo de juego premeditado donde el jugador tendría que superar varios objetivos para alcanzar un fin. El objetivo final del jugador será escapar de la cárcel y para ello se diseñaron dos métodos posibles. Aunque el objetivo final sea el mismo, el objetivo del diseño planteado es ofrecer las herramientas y medios necesarios para que cada usuario pueda superar el nivel de la manera que considere más oportuna. A continuación, se detallan las dos maneras diseñadas por las cuales el jugador puede escapar:

Manera 1

La primera ruta de escape consiste en abrir la puerta de seguridad situada en la sala principal del edificio. Esta puerta está blindada, por lo que no puede ser forzada ni destruida. Para abrirla, el jugador necesita una tarjeta de seguridad. Existen dos copias: una se encuentra dentro de la armería, y otra está en posesión del guardia Manuel. Alternativamente, cualquier guardia puede abrir la puerta si interactúa directamente con ella.



Figura 5.9 Tarjeta para la puerta de salida.

Para llegar a la salida, el jugador debe primero cruzar la puerta electrónica que separa el ala de las celdas del ala principal, lo cual requiere introducir el código secreto que está en la celda vacía enfrente de la del jugador. Este paso es obligatorio en esta ruta si se opta por obtener la tarjeta.



Figura 5.10 Código para la puerta electrónica.

Los guardias presentan diferentes posibilidades de interacción. Manuel es un joven comprometido con su trabajo, difícil de persuadir debido a su sentido del deber, aunque podría ceder si se siente intimidado, dado su carácter temeroso. Por otro lado, Bob, un veterano frustrado por su bajo salario, puede ser sobornado si el jugador obtiene y usa esa información estratégicamente.

Una vez conseguida la tarjeta —ya sea mediante interacción con los guardias o encontrándola en la armería—, el jugador podrá abrir la puerta principal y escapar. Esta ruta ofrece una solución directa y reduce el riesgo de quedar atrapado en un estado de *softlock*. El *softlock* es una situación en la que el jugador no puede progresar, aunque el juego siga funcionando.

Manera 2

El segundo método de escape consiste en descubrir que uno de los muros de la celda de Antonio está erosionado debido al paso del tiempo y numerosos intentos pasados de fuga. Para descubrir esto, el jugador debe primero abrir la puerta de su celda y la de su convicto vecino. Una forma de hacer esto sería haciendo uso de la lima presente en la celda del jugador para limar las cerraduras y abrir las puertas. Alternativamente, el jugador puede convencer a Antonio de abrir las dos puertas. Una vez fuera, si el jugador se percató del estado del muro, necesitará una herramienta contundente para poder destruirlo por completo. Herramientas como una pala o pico deberían ser suficientes.

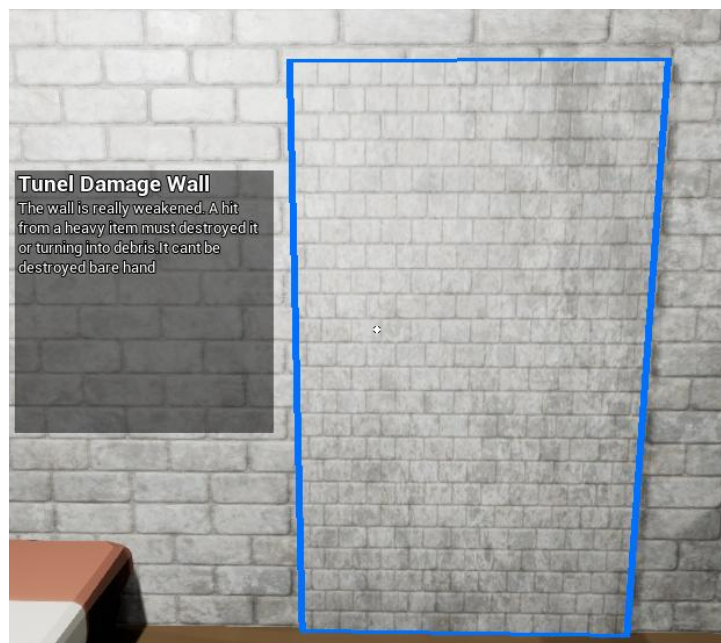


Figura 5.11 Muro frágil que permite escapar de la cárcel.

El jugador puede probar suerte buscando en una de las cajas ubicada en una de las celdas adyacentes, la cual contiene herramientas. Sin embargo, la opción más segura es acceder a la armería donde además de la caja de herramientas, hay una caja de madera que contiene un pico de minería. Para poder llegar a la armería, es necesario el código que permite abrir la cerradura electrónica de la puerta que separa las dos alas del edificio.

En la armería se pueden conseguir varias herramientas que pueden usarse para derribar el muro. Opcionalmente, el jugador puede probar a usar armas de fuego como alternativa, o buscar en alguna de las taquillas con el objetivo de encontrar otro objeto que pueda resultar útil.

Además de todas las condiciones de victoria explicadas anteriormente, el juego también incluye dos situaciones de derrota:

Derrota 1

Durante la simulación, se puede dar el caso de que los policías, ante una emergencia, decidan apresar al jugador, lo que termina el juego inmediatamente y se considera una condición de derrota. Este caso se puede dar si, por ejemplo, el jugador falla en un intento de intimidación o si los policías perciben algún tipo de comportamiento sospechoso.

Derrota 2

El otro caso es que Antonio decida atacar al jugador.

5.1.3 Diseño de comportamientos mediante máquinas de estado

En nuestro diseño hemos hecho uso de máquinas de estado —como se explicó en la sección 4.6— para controlar el comportamiento de ciertos elementos clave del sistema, cuya lógica de funcionamiento no debe depender exclusivamente del modelo de lenguaje. Esta estrategia permite garantizar un control más determinista, predecible y seguro en situaciones donde se requiere una ejecución estricta de reglas o condiciones. En concreto, se han implementado máquinas de estado para gestionar el comportamiento de los NPCs y de los diversos sistemas de puertas del entorno, incluyendo las puertas de las celdas, la puerta eléctrica con código de acceso y la puerta reforzada de salida.

NPCStateMachine

Para el diseño del comportamiento de los NPCs, se desarrolló una máquina de estados denominada *NPCStateMachine*, adaptada al contexto específico de personajes dentro de una prisión. Partiendo de este entorno, se definieron una serie de comportamientos básicos que cubren las principales acciones esperadas de los NPCs en distintas situaciones. Los estados considerados en esta máquina son: *Idle* (reposo o espera sin interacción), *MoveRandom* (movimiento autónomo siguiendo rutas aleatorias), *FollowTarget* (mantenerse cerca de un actor específico, como otro personaje), *InteractWithObject* (por ejemplo, aproximarse a una puerta para abrirla), *ChaseTarget* (respuesta hostil ante una amenaza), y *SeriouslyInjured* (estado terminal que desactiva al NPC). Esta estructuración mediante estados permite una gestión lógica y escalable del comportamiento de los NPCs, garantizando coherencia en sus acciones y facilitando su integración con otros módulos del sistema.

A continuación, se describen todos los estados que son comunicados al LLM. Se detalla primero su implementación y, como conclusión, el mensaje enviado al LLM en el que se describe el mismo estado.

Idle State

El estado *Idle* representa una situación de reposo total del personaje, durante la cual permanece estático en su posición sin realizar ninguna acción. Al entrar en este estado, se ejecuta una instrucción de detención del movimiento en el evento *Begin*, con el objetivo de asegurar que el personaje se detenga por completo en caso de que se encontrara en movimiento previamente. Este estado es fundamental como punto neutro, y garantiza que el agente no interfiera por su cuenta con el entorno mientras se encuentra inactivo. También es el estado asignado al NPC cuando este participa en una conversación con otro agente.

Descripción enviada al modelo de lenguaje: *Your character stays static in place without doing anything at all.*

MoveRandom State

El estado MoveRandom gestiona el desplazamiento aleatorio del personaje dentro de una zona navegable del entorno. Su propósito es simular un comportamiento de deambulación sin un objetivo concreto.

Durante el evento Update, se comprueba si el personaje está actualmente en movimiento. Si no lo está, se selecciona un punto aleatorio dentro de un rango predefinido y el personaje comienza a desplazarse hacia esa ubicación. Una vez alcanza el destino, el personaje permanece en espera durante un intervalo de tiempo aleatorio —dentro de un rango establecido en segundos— tras lo cual se reinicia su movimiento. Esto provoca que en la siguiente iteración del evento Update se seleccione un nuevo punto de destino, generando así un ciclo continuo de movimiento aleatorio.

Además, el sistema contempla situaciones en las que el personaje quede bloqueado o no consiga llegar al destino en un tiempo razonable. En esos casos, también se reinicia su movimiento, forzando la elección de un nuevo punto en la siguiente actualización. Esta lógica evita que el personaje permanezca inactivo indefinidamente y garantiza un comportamiento fluido y dinámico en el entorno.

Descripción enviada al modelo de lenguaje: *Your character moves through random different areas of the stage with no specific target or intention.*

FollowTarget State

El estado FollowTarget permite que el NPC siga de forma continua a un objetivo definido dentro de la escena. Al activarse este estado, en el evento Begin se realiza una búsqueda del objeto identificado como *Target*, cuyo nombre ha sido previamente proporcionado por el LLM. Una vez localizado, dicho objeto se asigna como objetivo de seguimiento.

Durante el evento *Update*, el sistema actualiza constantemente la posición del objetivo y dirige al NPC hacia ella. Para evitar colisiones o solapamientos no deseados, se aplica un pequeño margen de distancia, lo que garantiza que el personaje se mantenga cerca del objetivo sin invadir su espacio exacto. Este comportamiento se mantiene indefinidamente mientras el estado esté activo, lo que permite simular tareas de escolta, vigilancia o seguimiento autónomo de cualquier entidad dinámica dentro del entorno.

Descripción enviada al modelo de lenguaje: *Your character follows a specific moving target none stop (Requires defining the target name from the "Scene" list along with the stat example: "FollowTarget(Dog)").*

InteractWithObject State

El estado InteractWithObject permite al NPC desplazarse hacia un objetivo específico con el fin de interactuar con él. Al activarse este estado, en el evento Begin se lleva a cabo una

búsqueda en la escena del objeto designado como *Target*, cuyo nombre ha sido previamente indicado por el LLM. Una vez localizado, dicho objeto se asigna como objetivo.

Durante el evento *Update*, el sistema actualiza continuamente la posición del objetivo y guía al NPC hacia él. Al alcanzar la posición del objeto, el NPC interactúa con el objeto designado. Este estado permite integrar al NPC de forma dinámica y contextual dentro del entorno simulado, dotándolo de la capacidad de reaccionar con elementos específicos del escenario.

Descripción enviada al modelo de lenguaje: *Your character interacts to a specific target in some way (Requires defining the target name from the "Scene" list along with the state, example: "InteractWithObject(Table")) This state should be picked if you want to interact with an object or search for an object. It lets the NPC interact with the simulation.*

ChaseTarget State

El estado *ChaseTarget* combina elementos de los estados *FollowTarget* e *InteractWithObject*, y está diseñado para que el NPC persiga de forma indefinida a un objetivo con la intención de realizar una interacción de carácter agresivo o violento una vez lo alcanza. A diferencia del estado *InteractWithObject*, en el que el tipo de interacción es libre y dependiente del contexto, en este caso la intención hostil está implícita en el diseño del comportamiento.

Al activarse este estado, en el evento *Begin* se realiza una búsqueda en la escena del objeto identificado como *Target*, cuyo nombre ha sido previamente definido por el LLM. Una vez encontrado, se establece como objetivo a perseguir. Durante el evento *Update*, el sistema actualiza de forma continua la posición del objetivo y dirige al NPC hacia él.

Cuando el NPC alcanza al objetivo, se ejecuta una lógica condicional. Si el objetivo es el jugador, se activa una secuencia de diálogo en la que se notifica el estado de fin de juego y se fuerza la transición a la escena de menú principal. En caso de que el objetivo no sea el jugador, se genera una interacción entre ambos NPCs, cuyo efecto dependerá del contexto en el que ocurra.

Este estado permite representar comportamientos de carácter hostil dentro del sistema, manteniendo el control estructurado a través de la máquina de estados y sin requerir lógica compleja desde el LLM.

Descripción enviada al modelo de lenguaje: *You start chasing the target to tie him up or with aggressive intentions (Requires defining the target name from the "Scene" list along with the state, example: "ChaseTarget(Prisoner)").*

SeriouslyInjured State

El estado SeriouslyInjured representa una condición crítica en la que el personaje sufre una *muerte funcional* dentro del entorno, simulada mediante una caída descontrolada al suelo en forma de *ragdoll*. Este término hace referencia a una técnica de simulación física ampliamente utilizada en videojuegos, donde el cuerpo del personaje deja de estar animado de forma controlada y pasa a comportarse como una estructura articulada sometida únicamente a las leyes de la física, similar a una marioneta sin rigidez. Al activarse este estado, en el evento Begin se ejecutan varias acciones que desactivan completamente el comportamiento activo del personaje: se detiene cualquier movimiento que pudiera estar en curso, se elimina al personaje de todas las listas del sistema que lo referencien —por ejemplo, listas de NPCs activos o interactivos— y se marca explícitamente como *muerto* para evitar que otros módulos lo sigan considerando disponible.

Paralelamente, se desactiva su capacidad de diálogo y se anulan los eventos de colisión convencionales —por ejemplo, detección por parte de otros NPCs o activación de *triggers*—.

Este estado garantiza una transición clara y visualmente creíble hacia la inactividad total del personaje, integrándose de manera coherente con el resto del sistema de máquinas de estado.

Se descartó la opción de referenciar este estado de forma explícita como un estado de *muerte*, a pesar de que funcionalmente cumple con ese propósito. Esta decisión responde al hecho de que el LLM tendía a evitar dicho término, salvo en situaciones extremas, debido a que su uso puede entrar en ligera contradicción con ciertos principios morales integrados en su entrenamiento.

Descripción enviada al modelo de lenguaje: *Your character suffers a serious physical injury that affects their condition.*

DoorStateMachine

Para crear el comportamiento interactivo de las puertas dentro del entorno del juego, se implementó una máquina de estados denominada DoorStateMachine, con el objetivo de gestionar de forma clara y controlada el acceso de los personajes por los diferentes entornos del escenario. Esta máquina consta de dos estados principales que determinan la condición operativa de cada puerta: *Locked* (estado bloqueado que impide la apertura de la puerta) y *Unlocked* (el estado, donde la puerta se encuentra desbloqueada y puede ser abierta).

La lista de estados y descripciones contiene:

Locked State

El estado Locked se encarga de asegurar que la puerta permanezca cerrada o de iniciar su cierre en caso de que se encuentre abierta. Al activarse este estado, durante el evento Begin, se determina el ángulo de rotación necesario para alcanzar la posición de cierre completo. Además, se actualiza la descripción de la puerta añadiendo una anotación informativa que indica su nuevo estado: *Now the door is locked.*

Posteriormente, en el evento Update, la puerta realiza una rotación progresiva sobre su propio eje a una velocidad predefinida, con el objetivo de alcanzar la posición completamente cerrada. Esta transición se lleva a cabo de forma suave, simulando un movimiento realista de cierre.

Descripción para el modelo de lenguaje: *The door stays locked.*

Unlocked State

El estado Unlocked tiene como finalidad permitir la apertura de la puerta o iniciar su apertura en caso de que se encuentre cerrada. Al activarse este estado, durante el evento Begin, se calcula el ángulo necesario para que la puerta se abra completamente. Asimismo, se actualiza la descripción de la puerta añadiendo una anotación que refleja el nuevo estado: *Now the door is unlocked.*

Durante el evento Update, la puerta comienza a rotar progresivamente sobre su propio eje a una velocidad determinada, con el objetivo de alcanzar la posición completamente abierta. Este proceso se ejecuta de forma continua hasta que la rotación cumple con el ángulo especificado, simulando así una apertura fluida y realista.

Descripción para el modelo de lenguaje: *The door stays unlocked.*

Tal como se explicó previamente, la decisión sobre el estado al que deben realizar una transición estos elementos es gestionada directamente por el LLM. La simplicidad de esta arquitectura facilita su integración con otros sistemas del juego, como los comportamientos de los NPC —por ejemplo, en el estado InteractWithObject— o la lógica asociada a las misiones del jugador, sin comprometer la escalabilidad ni la mantenibilidad del sistema.

La verdadera potencia de este enfoque, que podría parecer simple a primera vista, reside en la riqueza semántica de las descripciones asignadas a cada puerta. Estas descripciones no amplían directamente el comportamiento de la puerta, sino que actúan como fuente de conocimiento para el LLM, explicando con detalle el funcionamiento específico de cada una. Es decir, cada tipo de puerta cuenta con una descripción que incluye una serie de características concretas que definen sus condiciones particulares de apertura y cierre.

A partir de esta información, el LLM es capaz de tomar decisiones coherentes con el diseño previsto, garantizando que el sistema se comporte de acuerdo con la lógica establecida. Esta

filosofía de diseño permite que el comportamiento esperado de cada puerta resida en su descripción, de manera que, si se desea modificar dicho comportamiento o añadir uno nuevo, el desarrollador solo necesita ajustar la descripción correspondiente desde el editor, sin necesidad de modificar el código interno del sistema.

A continuación, se presentan tres casos concretos utilizados en el nivel desarrollado:

Cell Doors – Figura 5.12:

This is the player's/Antonio's cell door. This door can be locked/unlocked using the object: ("CellKey"). It can also be unlocked easily if it gets damaged or the lock malfunctions. It can also be unlocked easily with sharpened items. It can be forced opened.

Electric Door – Figura 5.13

This door can be locked/unlocked using the object: ("Secret code"). It is a reinforced door, but it can also be opened if it is really damaged or if a short circuit happens.

Exit Door – Figura 5.14

This Door can only be unlocked with the ("Exit Card") or if a prison guard interacts with it in any way. This door is INDESTRUCTIBLE and CAN NOT suffer any damage.

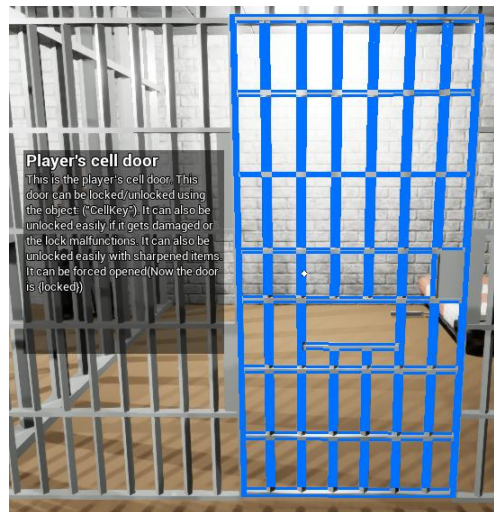


Figura 5.12 Puerta de la celda del jugador



Figura 5.13 Puerta electrónica

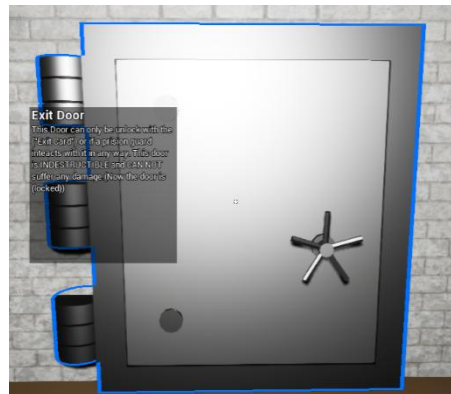


Figura 5.14 Puerta final

5.2 Diseño de las pruebas

Una vez se preparó el escenario, se empezó el diseño de los cuestionarios, observaciones y entrevistas que se harían a los usuarios que fueran a participar en las sesiones de prueba de la herramienta. Con el fin de obtener información relevante de distintas fuentes, se utilizaron métodos de evaluación distintos [33]: la observación como método objetivo y cualitativo, las entrevistas, subjetivas y cualitativas y los cuestionarios son subjetivos y cuantitativos. De esta manera, se buscó conseguir información contrastable mediante las distintas técnicas, para poder validar correctamente los resultados obtenidos y evaluar tanto el correcto funcionamiento de la herramienta como, por consiguiente, la utilidad de los modelos largos de lenguaje en videojuegos.

Para crear y distribuir los cuestionarios se usó *Google Forms*⁵⁰, un software de administración de encuestas web gratuito, desarrollado por Google, que permite crear y compartir formularios de manera fácil y rápida. Los resultados de la encuesta se actualizan al momento y ofrecen herramientas de visualización de los datos obtenidos.

5.2.1 Objetivo

Se establecieron los siguientes objetivos:

Comprobar si el juego da libertad al jugador para hacer lo que quiera en el escenario

Principalmente, se definieron 3 preguntas de investigación para responder este apartado:

- *¿Ha podido completar el juego de una manera diferente a la establecida inicialmente?*
 - Es importante saber si cada partida es realmente diferente, puesto que el interés principal de la herramienta es ofrecer la libertad narrativa a los usuarios para que puedan crear su propia historia. La manera de terminar el juego es común en todas las pruebas: conseguir escapar de la cárcel. No obstante, lo importante es tener en cuenta cómo se alcanzan los objetivos y si difieren del diseñado.
- *¿Todas las acciones del jugador tienen un resultado notable en la escena?*
 - Es decir, se quieren comprobar si cada interacción que se hace aporta algo nuevo a la escena, posibilitando nuevas interacciones. Esta pregunta es fundamental para la construcción de diferentes narrativas.
- *¿Las interacciones con los NPCs afectan al objetivo final?*
 - Como ya se mencionó, los LLMs dan muy buenos resultados para la generación de diálogos. No obstante, para que el jugador se sienta libre, se le quiere proporcionar la mayor inmersión posible, permitiendo a los NPCs que interactúen con el entorno o, dependiendo del rol, ayuden o compliquen la escapada de la cárcel, ya sea por dialogo o por una interacción en concreto.

La experiencia de juego es positiva para el usuario

Para este objetivo, se quiere medir el comportamiento más técnico del LLM y comprobar si su utilidad es viable y aceptable.

- *¿El tiempo entre interacciones es una molestia para los usuarios?*

⁵⁰ Google Forms: <https://docs.google.com/forms/u/0/?tgif=d>

- Los LLMs no se caracterizan por su rapidez —sobre todo si se habla de modelos ejecutados localmente—, siendo un claro obstáculo para ser implementado dentro de un videojuego. Abarcando diferentes perfiles de jugadores, se pretende comprobar si el tiempo de cada interacción es aceptable o compromete la experiencia de juego.
- *¿El usuario esperaba el resultado de sus interacciones?*
 - Cuando un jugador encuentra una brecha, se rompe la inmersión dentro del juego, causando una reacción negativa que puede terminar en el abandono del mismo. Cuando el comportamiento está predefinido por los desarrolladores, este problema es detectable mediante pruebas con usuarios. En este caso, este problema sucede entre tres modelos mentales, el de los desarrolladores, el de los usuarios y el del LLM, que es el que decide la acción. Se pretende buscar posibles brechas entre los tres modelos.

5.2.2 Cuestionario inicial

Antes de comenzar con el resto de los métodos mencionados, se pidió a los usuarios que rellenasen un cuestionario para mejor categorizar su perfil y conocer su experiencia con juegos similares al presentado en las pruebas. Además de sus conocimientos sobre otras experiencias que incorporen algún tipo de modelo de lenguaje en alguna de sus mecánicas. Esto ayuda a entender mejor cada prueba y valorarla según su contexto.

El cuestionario tiene una duración aproximada de 1 minuto y está compuesto por las siguientes preguntas:

- *¿Cuántas horas aproximadamente le dedicas a jugar a videojuegos semanalmente?*
 - 0, no juego a videojuegos
 - De 1 a 5 horas
 - De 6 a 10 horas
 - Más de 10 horas
- *¿Cómo de familiar eres con los juegos tipo sandbox?*

El juego libre (en inglés sandbox) es un estilo de videojuego caracterizado por dar al jugador un alto grado de libertad para ser creativo a la hora de completar tareas hacia un objetivo dentro del juego, o simplemente para jugar sin restricciones. Juegos como Minecraft o Los Sims entran en esta categoría.

- Nada familiar
- He jugado a algunos juegos

- Soy muy familiar con este género
- ¿Cómo de familiar eres con los juegos de rol (videojuegos o de mesa)?

Un juego de rol (RPG, que significa role-playing game), es un tipo de juego en el cual los participantes adoptan e interpretan diferentes roles o personajes. Juegos como Dungeons & Dragons o Fallout son ejemplos de este género.

- Nada familiar
- He jugado a algunos juegos
- Soy muy familiar con este género

5.2.3 Observación

Dura entre 15 y 20 minutos. Al empezar esta fase, se explican los controles —los cuales permanecen siempre visibles en la pantalla—, las mecánicas existentes con los elementos de la escena y el objetivo del juego. El observador no debe intervenir en la prueba, a no ser que ocurra uno de los siguientes sucesos:

- *Softlock*: bloqueo que no permite completar el juego, a pesar de poder seguir jugando. Se soluciona reiniciando la prueba.
- El jugador no comprende las mecánicas pese a haber sido explicadas: el objetivo de las pruebas es probar la herramienta, no validar su facilidad de ser usada.
- En el caso de que se produzca una brecha de uso, el observador puede decidir intervenir para clarificar el suceso.

Tras haber definido la fase, se creó una plantilla de observación para apuntar aquellos sucesos interesantes que pudiesen aportar información y ayudar a resolver las preguntas de investigación. La plantilla se verá en la *tabla 5.1*

Nombre:	
Fecha:	
Observación	Resultado
¿Cómo ha escapado el jugador de la cárcel?	
Mientras el jugador espera un resultado de una interacción, ¿tiene alguna reacción negativa?	
¿El jugador repite alguna interacción?	
¿El jugador hace una acción condicionada por una conversación con un NPC?	
¿El jugador utiliza objetos generados por una interacción?	
¿El jugador ha realizado alguna interacción compleja?	
¿Ha ocurrido algún <i>softlock</i> ? ¿Por qué?	

Tabla 5.1 Plantilla de observación para las pruebas de usuario.

5.2.4 Entrevista

Esta fase se estimó que duraría aproximadamente 5 minutos. En ella, se entrevistó a los usuarios sobre su experiencia. Esta entrevista no es completamente guiada, sino que se basa principalmente en la actuación del usuario en la escena. En caso de que la observación no fuese suficiente para obtener información sobre alguna pregunta de investigación, se dirige la conversación para tratar dichos temas. Principalmente se intenta animar a los usuarios a hablar y compartir su opinión acerca de los sucesos que han ocurrido a lo largo de la observación. No obstante, para fomentar su participación, se plantearon algunas preguntas iniciales:

- ¿Qué te parecieron las respuestas del LLM?
- ¿Hay alguna interacción que no hayas entendido o te haya resultado confusa?
- ¿Qué te han parecido los tiempos de espera de las respuestas?
- ¿La generación de mallas o sonidos ha sido adecuada?

5.2.5 Cuestionario final

Por último, los usuarios recibieron un último cuestionario más enfocado a conocer su opinión sobre la experiencia. Algunas preguntas fueron realizadas anteriormente en la entrevista, lo que permite validar las respuestas de los usuarios y comprobar que mantengan una coherencia.

Para las preguntas se utilizó una escala de Likert de 5 puntos por su carácter intuitivo y fácil de comprender, incluso para usuarios sin experiencia previa. Su número limitado de opciones evita la sobrecarga cognitiva, agiliza el proceso de respuesta y reduce la fatiga, manteniendo un buen equilibrio entre simplicidad y precisión en la recogida de datos.

Es importante destacar que, dependiendo de las acciones de cada usuario, se puede dar el caso de que no puedan responder alguna de las preguntas porque no han visto o vivido alguna experiencia. En esas preguntas, se especifica a los usuarios que la dejen en blanco.

El cuestionario está formado por las siguientes preguntas:

- *¿Cómo de correctas te han parecido las reacciones del LLM teniendo en cuenta el contexto en el que las realizaste?*
- *¿Consideras que las respuestas del LLM a los eventos han sido creativas?*
- *¿Cómo de buenas consideras que han sido las respuestas a los diálogos con NPCs? Teniendo en cuenta el contexto y conversaciones previas. Un personaje no jugador o personaje no jugable es un término a menudo abreviado con la sigla PNJ (o NPC por sus siglas en inglés, de non-player character) que es un personaje controlado por el ordenador. En nuestro caso, personajes como Bob o Manuel son ejemplos de NPCs. Si no has hablado con ningún NPC durante la partida, no respondas*
- *¿Cómo de correctas te han parecido las mallas que el LLM haya podido generar durante tu experiencia? Si el LLM no ha generado ninguna malla, no respondas. Una malla es el modelo 3D que representa a una entidad en la escena.*
- *¿Cómo de correctas te han parecido los sonidos que el LLM haya podido generar durante tu experiencia? Si el LLM no ha generado ningún sonido, no respondas.*
- *¿Cómo de inmerso te has sentido durante la experiencia?*
- *¿Cómo de adecuado has percibido el tiempo de espera del LLM al iniciar una interacción?*
- *¿Las mecánicas presentadas en el juego te han permitido realizar todas las acciones que querías?*
- *¿Las reacciones del LLM a tus acciones se ajustaban a tus expectativas?*
- *¿Cómo de divertida valorarías tu experiencia?*

- *Opcionalmente, agradecemos cualquier comentario, opinión o crítica que tengas sobre tu experiencia.*

Capítulo 6 - Análisis de resultados

Las pruebas realizadas según lo especificado anteriormente tuvieron un total de 24 sujetos, la cual resulta una muestra suficientemente amplia que generó una gran cantidad de resultados para el análisis de la herramienta desarrollada, mostrando también diferencias suficientes en los mismos que demuestran la validez de estas.

En este capítulo se detalla extensamente los resultados obtenidos tras el análisis realizado, seccionándolo en apartados según las preguntas de investigación descritas en el *capítulo 5*, donde se hablan de resultados objetivos utilizando los datos sacados directamente de los formularios, entrevistas y observaciones realizadas durante las pruebas. Asimismo, se detallan tanto los problemas sucedidos como sus posibles soluciones extensamente.

Perfiles de usuario según cuestionario inicial

Los resultados obtenidos del cuestionario inicial no muestran un perfil predominante, sino varios diferentes. Por lo tanto, a la hora de hacer los análisis sobre cada pregunta de investigación, se tienen en cuenta cada uno de los perfiles para entender mejor a los usuarios.

Los resultados se muestran en las figuras 6.1, 6.2 y 6.3.

¿Cuántas horas aproximadamente le dedicas a jugar a videojuegos semanalmente?

24 respuestas

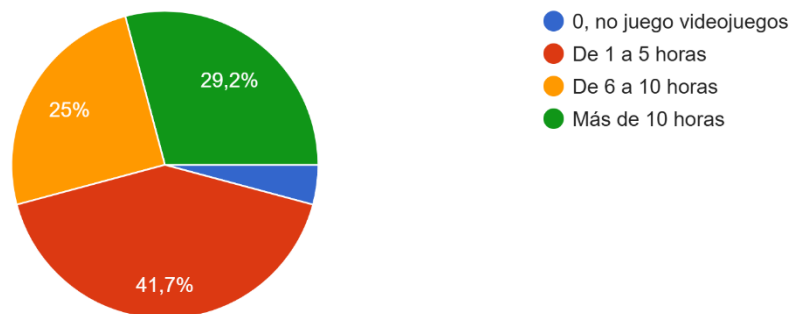


Figura 6.1 Resultados del cuestionario inicial acerca de las horas jugadas semanalmente.

Se observa que un 4.1% (1) de los usuarios no juegan videojuegos; un 41.7% (10) dedica de 1 a 5 horas jugando semanalmente; un 25% (6) dedica de 6 a 10 horas semanalmente; y el 29.2% (7) restante juega más de 10 horas semanales.

¿Cómo de familiar es el jugador con los juegos tipo “sandbox”? El juego libre (en inglés sandbox) es un estilo de videojuego caracterizado por dar al...omo Minecraft o los Sims entran en esta categoría.

24 respuestas

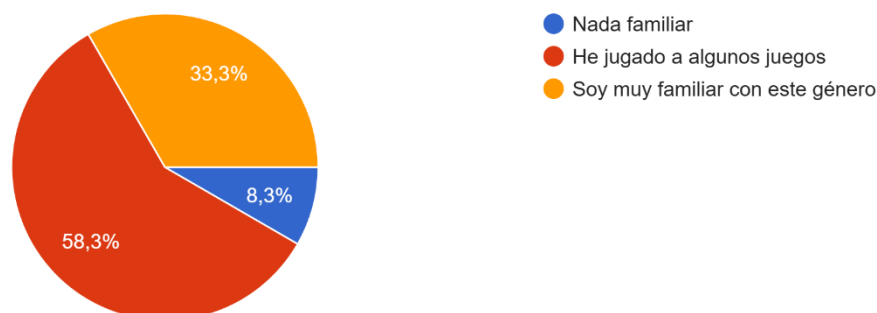


Figura 6.2 Resultados del cuestionario inicial acerca de la familiaridad con juegos sandbox.

Un 8.3% (2) de los usuarios resultó no estar familiarizado con videojuegos de tipo sandbox, mientras que un 58.3% (13) muestran conocimiento medio acerca del género, y el 33.3% (8) restante de usuarios tiene una familiaridad elevada con estos videojuegos.

¿Cómo de familiar es el jugador con los juegos de rol (videojuegos o de mesa)? Un juego de rol (RPG en inglés RPG, que significa "role-playing ga...and Dragons o Fallout son ejemplos de este género.

24 respuestas

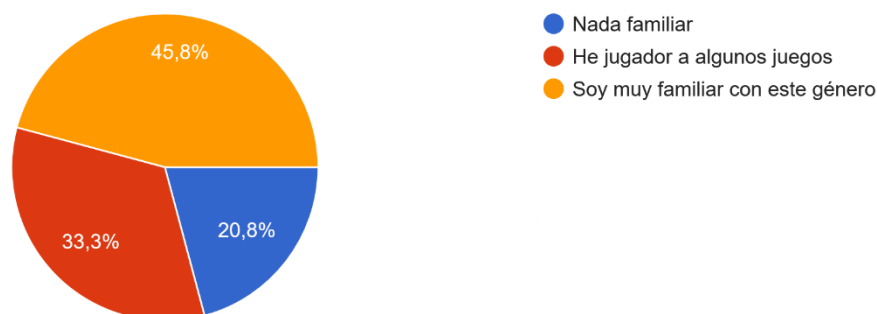


Figura 6.3 Resultados del cuestionario inicial acerca de la familiaridad con juegos de rol.

Con respecto a juegos de rol, los usuarios mostraron ser menos familiares: un 20.8% (5) de los usuarios no resultaron familiares con el género, un 33.3% (8) mostraron cierto conocimiento, y el 45.8% restante resultó ser especialmente familiar con él.

Como conclusión, se puede observar que cerca de la mitad de los usuarios no dedican muchas horas semanalmente a los videojuegos, que la mayoría son familiares con algunos juegos sandbox y que prácticamente la mitad son muy familiares con juegos de rol. No obstante, al tener relativamente pocos usuarios, el resto de los porcentajes son suficientemente significativos, no pudiendo crear un perfil común entre todos los usuarios.

¿Ha podido completar el juego de una manera diferente a la establecida inicialmente?

Durante la fase de observación, se pudo observar las distintas partidas de los usuarios, sacando las medidas mostradas en la *figura 6.4*.

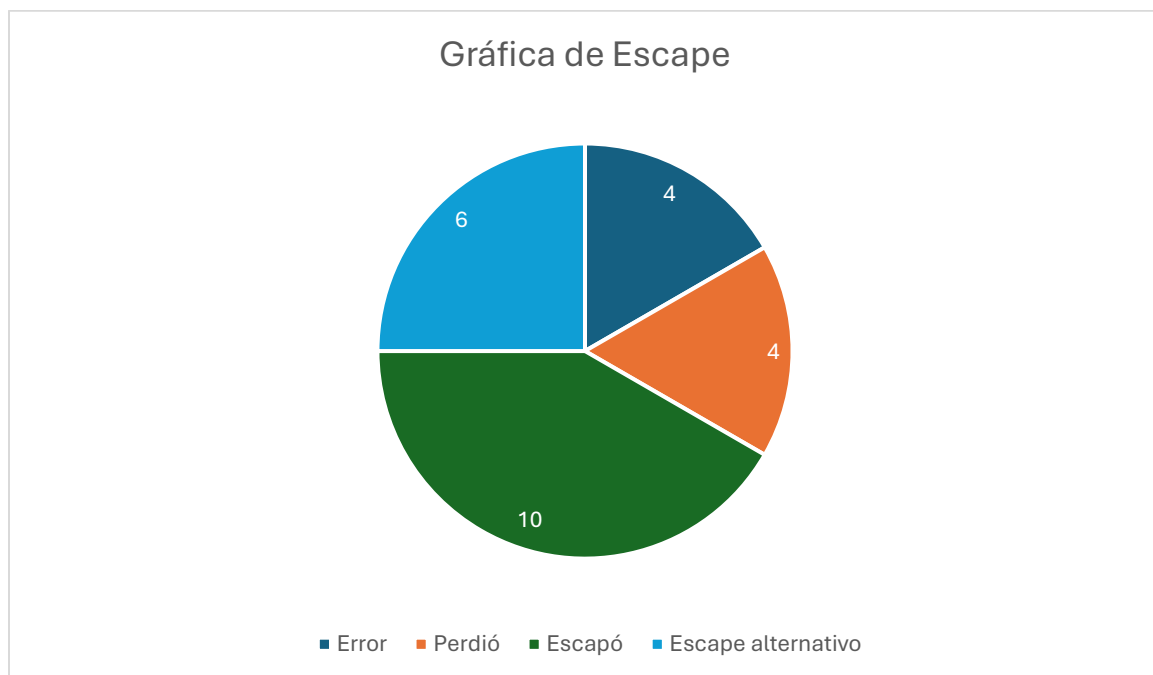


Figura 6.4 Gráfica de escape de la cárcel por parte de los usuarios.

Un cuarto de la muestra consiguió escapar obteniendo un final alternativo. Mientras tanto, se observó como 8 de los usuarios no fueron capaces de completar el objetivo de la prueba, 4 de ellos por algún error durante la ejecución de las mismas. La moda apreciada fue escapar de la cárcel a través de rutas convencionales, con 10 de los usuarios en este apartado.

La baja muestra de finales alternativos conseguidos demuestra las limitaciones físicas que tiene el juego, independientemente del uso del LLM, debido a que la manera de ser completado es escapando por dos lugares concretos. Para aumentar las maneras de escapar, se planteó añadir un sistema de misiones gestionado por el LLM que permita alterar el objetivo principal según las acciones del jugador. Además, las descripciones tan explícitas de objetos como la puerta final o el muro dañado de la celda incitan a los usuarios a realizar una seguir una ruta narrativa en concreto, influenciando sus interacciones. Por tanto, se pensó realizar una descripción pública y otra privada

de los objetos, siendo visible únicamente la descripción pública y reservando la privada para guiar las respuestas del modelo. Estas soluciones buscan limitar la información esencial para completar el objetivo actual y fomentar al jugador a pensar en métodos alternativos para superar los obstáculos presentados.

Es importante destacar que, aunque 8 personas no escaparon, 4 de ellas se quedaron atascados por las respuestas del LLM. Los motivos en concreto fueron:

- Los NPCs no tienen sentidos y les falta el contexto de aquellas cosas que suceden. Un ejemplo concreto es que no se dan cuenta cuando se han abierto o cerrado puertas, de manera que solo son conscientes si se les comunica mediante el dialogo o si han sido ellos los que han realizado la acción.
 - Se plantea añadir sentidos a los NPCs para que se guarde aquellos objetos o interacciones que detecte en su contexto. Por otro lado, con un modelo rápido como Gemini se puede mandar un prompt para que el NPC decida que hacer tras ver una interacción.
- Este apartado está ligado al anterior. Los NPCs no tienen contexto de las cosas que pasan a su alrededor. Si en el historial del NPC se registra una conversación que no ha tenido una respuesta correcta, el historial quedará corrupto. Por ejemplo, si se le da un maletín con dinero a un guardia para sobornarlo y salir, y el LLM decide borrarlo de la escena porque lo tiene el guardia, pero este responde que le va a echar un vistazo, no que lo tenga consigo. Cuando se le dice que ya se le ha dado el dinero, responde que es mentira, rompiendo con la coherencia del juego.
 - Se soluciona con el mismo cambio planteado en el apartado anterior.
- Cuando un guardia es convencido para abrir la puerta, no siempre la abre. El LLM es el que decide el resultado de cada interacción y puede darse el caso de que una interacción diseñada para llevarse a cabo con éxito termine fallando por elección del modelo.
 - Aunque estos sucesos son inevitables cuando se usa un modelo para guiar las acciones del juego, una descripción más detallada de todos los objetos podría ayudar a obtener resultados más predecibles.
- Respecto al punto anterior, el problema puede dar lugar a situaciones donde el guardia intenta abrir la puerta, no lo consigue y, como resultado, intenta volver a abrirla, generando un bucle que solo se puede cortar si el NPC decide dejar de intentar abrir la puerta.

- Se planteó evitar que el NPC pueda repetir el mismo estado que la interacción anterior si el objeto interactuado es el mismo. Este cambio se llevó a cabo después de realizar las pruebas de usuario.

Los otros cuatro participantes perdieron el juego, ya que no encontraron ningún final posible. Se analizaron sus respuestas al cuestionario inicial para identificar sus perfiles como jugador, siendo dos de ellos jugadores bastante casuales y dos jugadores con experiencia previa. Por este motivo, no se pudo sacar una correlación objetiva de acerca de este motivo.

De los 24 participantes, un 25% del total consiguieron escapar de una manera alternativa a la diseñada. El 41.67 % escapó siguiendo alguno de los dos métodos de escape diseñados. 16.7% de la muestra perdió al ser atrapado por alguno de los guardias y únicamente el 16.7% restante no pudo terminar siguiendo la ruta que tenían en mente, debido a alguno de los errores provocados por el modelo mencionados anteriormente. Esto implica que un cuarto de la muestra total pudo terminar exitosamente el juego con métodos alternativos. Además, cabe destacar que no hubo dos resultados iguales dentro de este grupo. Aunque el 16.7% de la muestra fracasó en el intento de escape, todos los miembros de este grupo perdieron por motivos diferentes, lo que implica una mayor variedad de finales. Con estos datos se puede concluir que, en total un 41.67% de la muestra obtuvo finales de juego diferentes a los diseñados. Se teoriza que el 41.67% que obtuvo finales siguiendo los flujos de juego diseñados se deben a lo explícitas que resultan algunas descripciones del juego, que desincentivan a los jugadores a planificar vías de escape alternativas. Siguiendo la solución anteriormente propuesta, se podría dividir el contenido de las descripciones en dos partes. Además, se podrían eliminar algunos objetos de la escena que facilitan en gran medida obtener alguno de los finales diseñados. Se puede entonces teorizar que, a partir de los datos obtenidos en las pruebas y haciendo los arreglos oportunos, la herramienta permitiría a la gran mayoría de los jugadores obtener experiencias de juego y finales completamente distintos, cumpliendo así el objetivo de investigación.

¿Todas las acciones del jugador tienen un resultado notable en la escena?

Para responder a esta pregunta se pretende investigar si las interacciones del jugador con elementos de la escena han generado resultados útiles para la progresión del juego, considerando también la creatividad de las respuestas ofrecidas.

Como respuesta a la primera pregunta, los datos anteriormente presentados permiten reafirmar que las interacciones durante la partida generan reacciones durante su desarrollo, capaces de alterar el final del juego. Las observaciones de usuarios corroboran este punto. Muchos usuarios se centraron en las interacciones verbales con NPCs con las que consiguieron alcanzar estos finales alternativos. Algunos ejemplos de estos casos son: un jugador que convenció al policía Manuel de que iba a romper aguas con lo que consiguió que le abriera la puerta final. Varios usuarios consiguieron sobornar a un guardia para que le abriera la puerta.

Las interacciones con objetos de la escena permiten a los usuarios obtener resultados coherentes según el contexto, pero en la mayoría de los casos, los objetos resultantes de la interacción no resultan útiles para la progresión. Un ejemplo observado es de un usuario que consigue mediante interacciones obtener papel y pólvora de las balas, juntando ambos elementos, obtiene un objeto definido como altamente inflamable. No obstante, no consigue realizar una interacción más compleja a partir de dicho objeto. Como se verá más adelante, la carencia de impacto de las interacciones con objetos puede deberse a las limitaciones del sistema de interacción automática que incorpora el proyecto, el cual fue el sistema de interacción utilizado para las pruebas con usuarios.

Sobre la interacción con elementos de la escena también se observó que todos los sujetos repitieron en algún momento alguna de sus interacciones con el entorno a causa de una reacción indeseada, provocando que se repita la interacción hasta obtener un resultado más acorde al esperado.

Por otro lado, se pudo observar que todos los usuarios tendían a utilizar aquellos objetos generados a partir de otra interacción, dando lugar a nuevos objetos en la escena. Esta información es altamente relevante, puesto que genera que cada partida pueda ser diferente, dependiendo de las elecciones de los usuarios.

Partiendo de estos resultados y los obtenidos a la pregunta de investigación anterior, se puede asumir que la herramienta permite obtener respuestas a las interacciones que provoquen alteraciones en mayor o menor escala en el flujo de juego. La mayoría de las alteraciones observadas ocurren a raíz de interacciones verbales con personajes no jugadores. Para favorecer que haya más interacciones relevantes con objetos se propone añadir más objetos a la escena, así como modificar el prompt relativo a la creación de objetos, para favorecer que se creen más elementos como resultado de las interacciones entre objetos. Con estas soluciones se permitiría aumentar la variedad de objetos presentes durante una partida, aumentando drásticamente las posibilidades de interacción.

¿Las interacciones con los NPCs afectan al objetivo final?

A través de la observación, se comprobó que todos los usuarios interactuaban constantemente con NPCs. Un octavo (3 personas) de ellos estuvieron prácticamente la totalidad del tiempo de las pruebas hablando con ellos.

Se analizaron los comportamientos de cada usuario y se llegó a la conclusión de que todos los usuarios realizaron acciones condicionadas por alguna conversación con un NPC, ya sea abrir alguna puerta o buscar algún elemento de la escena. A pesar de que estas interacciones son útiles para el desarrollo de la narrativa, no afectan directamente al objetivo final. A continuación, se enumeran aquellas que sí fueron realmente importantes, cambiando el final alcanzado:

- El usuario enseña un maletín con mucho dinero a Antonio y deciden llevárselo y escapar juntos.
- El usuario, tras no conseguir convencer a los policías para que le abran la puerta, los mató y escapó junto con Antonio de la cárcel.
- El usuario convence a un policía de que está embarazada y va a romper aguas, consiguiendo que le abra la puerta de la cárcel.
- Antonio le habla sobre el agujero que ha intentado hacer en la pared. El usuario destruye la pared con un destornillador y escapa.
- El usuario sobornó a Bob para escapar.
- Antonio intenta convencer al usuario para que incapacite a un guardia. Posteriormente, el usuario convenció a Antonio para que lo hiciese él. El policía no murió con la interacción. Como resultado, pulsó la alarma, que alertó al otro policía y ambos persiguieron a los reclusos. No obstante, consiguieron escapar juntos por la puerta principal con la tarjeta.

La amplia variedad de opciones y las maneras en que estas afectan a la narrativa es altamente notable. Los personajes no jugadores aportan gran variedad de posibilidades diferentes, enriqueciendo los sucesos y generando escenarios únicos, lo que provocó que durante las pruebas no se observase la repetición de ninguno de los finales alternativos obtenidos.

¿El tiempo entre interacciones es una molestia para el usuario?

¿Cómo de adecuado has percibido el tiempo de espera del LLM al iniciar una interacción?

24 respuestas

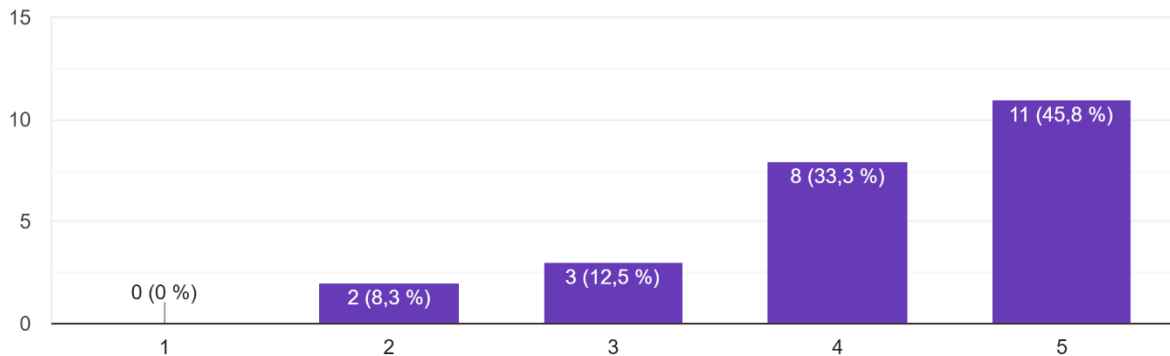


Figura 6.5 Evaluación de los usuarios sobre el tiempo de respuesta de una interacción.

Como se puede observar en la *figura 6.5* resultante de la pregunta acerca de los tiempos de espera. El grupo más mayoritario de probadores, que representa un 45.8% de la muestra considera que el tiempo de espera entre interacciones es muy adecuado. De igual manera, un 33.3% de la muestra considera que el tiempo de espera es adecuado. Del restante, el 12.5% considera que los tiempos de espera podrían ser mejores y una minoría del 8.3% considera negativos los tiempos de espera para la experiencia de juego. En total un 79.1%, más de tres cuartos de la muestra, consideran adecuados los tiempos de espera. La media de los datos es 4.17 y su desviación estándar muestral es de 0.96, dando lugar a un coeficiente de variación de Pearson del 23.02%. Se aprecia que las respuestas tienen una dispersión algo alta, pero la valoración general de los usuarios es positiva.

De las respuestas negativas o neutras, se observa que la mayoría de los perfiles de los usuarios son personas que juegan más de 10 horas a la semana y con experiencia previa en otros juegos. Se asume que son perfiles acostumbrados a la inmediatez dentro de este medio, por tanto, se entienden las respuestas obtenidas. Durante la fase de observación, no se pudieron apreciar reacciones negativas respecto a los tiempos de espera. No obstante, se pudo observar un comportamiento recurrente en algunos usuarios. Concretamente, se registraron 6 usuarios de la muestra que no esperaban a observar la respuesta generada por el LLM, sino que se movían a otro lugar y luego encontraban nuevos objetos generados de interacciones pasadas. Si bien es cierto que gran parte consideraron adecuados los tiempos de respuesta, es importante tener en cuenta que se están utilizando modelos largos de lenguaje, que aumentan notablemente la latencia entre una acción y su respuesta, haciendo las interacciones más tediosas que en juegos tradicionales, de naturaleza más inmediata.

Todas las pruebas se realizaron usando el modelo largo de lenguaje Gemini por su escaso tiempo de respuesta, especialmente comparado con los modelos ejecutados localmente. Teniendo en cuenta los datos obtenidos, se podría afirmar que el tiempo de espera para obtener respuestas a las acciones es correcto si se usa la herramienta con un LLM remoto. Como se demostró anteriormente, la diferencia de latencia entre los modelos remotos y locales hace que estos últimos sean prácticamente inviables para este tipo de experiencias.

¿El usuario se esperaba el resultado de sus interacciones?

Para poder responder a esta pregunta en su totalidad se van a abordar varios temas que permitan sacar una conclusión más completa. En primer lugar, es importante comprobar que las respuestas del LLM son correctas y adecuadas para el contexto de la interacción. Si no fuera el caso, la pregunta de investigación quedaría invalidada.

¿Cómo de correctas te han parecido las reacciones del LLM teniendo en cuenta el contexto en el que las realizaste?

24 respuestas

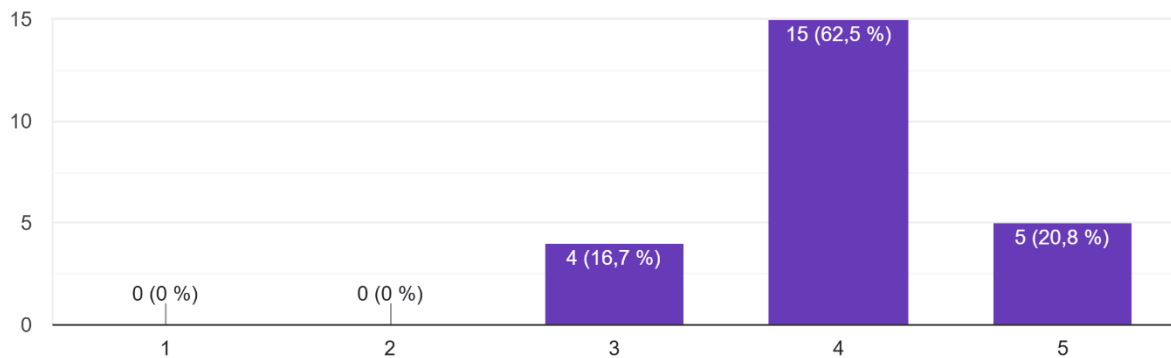


Figura 6.6 Evaluación de los usuarios sobre las reacciones del LLM.

Utilizando el cuestionario como base, se puede observar que los usuarios consideran que las reacciones son en general coherentes, siendo un 20.8% de usuarios los que creen que las reacciones son muy adecuadas dado el contexto de la interacción y un 62.5% que cree que las respuestas son adecuadas. Salvo algunas interacciones con resultados poco coherentes, las cuales se pudieron percibir durante el periodo de observación, la mayoría de las interacciones presentan un resultado afín al contexto e interacción realizada. Esta hipótesis está respaldada por los datos recogidos en el cuestionario, donde ningún usuario afirma haber tenido una reacción poco adecuada. El resultado permite afirmar que la mayoría de las interacciones fueron coherentes, con un 83.3% de respuestas positivas en el cuestionario y un 16.7% de respuestas neutrales. La media obtenida con estos datos es de 4.04, con desviación estándar muestral de 0.62 y coeficiente de variación del 15.35%. Puesto que la variación observada es baja, se puede afirmar que la mayoría de los usuarios coincidieron en su valoración positiva.

Por otro lado, se valoró que las respuestas de los NPCs fueran coherentes tanto con el desarrollo de la conversación con el jugador como con el contexto en el que se producen. Con lo que se obtuvo los siguientes resultados:

¿Cómo de buenas consideras que han sido las respuestas a los diálogos con NPC? Teniendo en cuenta el contexto y conversaciones previas. Un p...o con ningún NPC durante la partida, no respondas

23 respuestas

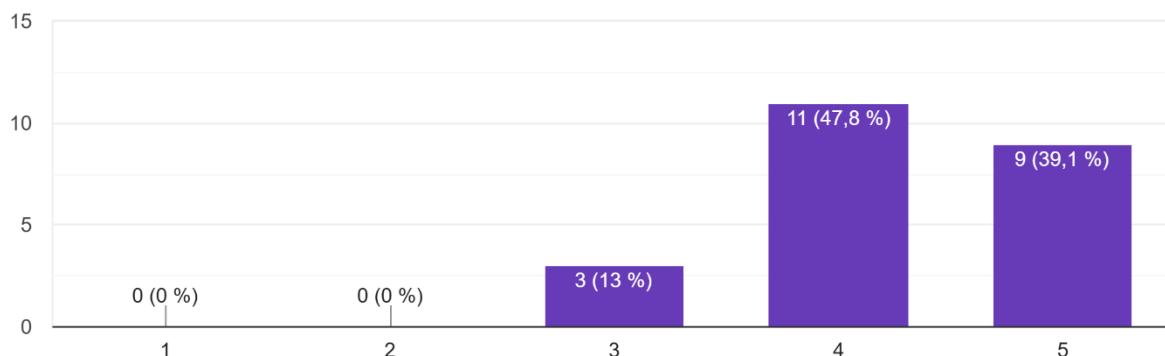


Figura 6.7 Evaluación de los usuarios sobre los diálogos con los NPCs.

Se observa que prácticamente todos los usuarios han interactuado con algún NPC durante su partida, excepto uno. Del grupo de usuarios que han probado esa mecánica, un 86.9% de los usuarios consideran que las respuestas fueron adecuadas y solo un 13% aproximadamente han mostrado una opinión neutral al respecto. Esta neutralidad puede deberse en parte por los resultados que se pudieron sacar de la fase de observación, por los problemas de comportamiento ya mencionados de los NPCs. Teniendo en cuenta que con estos datos se obtiene una media de 4.26, desviación estándar muestral de 0.69 y coeficiente de variación de 16.20%, se concluye que las respuestas de los NPCs son en su mayoría buenas, según la opinión de los sujetos de pruebas.

Durante la observación y posteriormente en entrevista, dos usuarios admiten haber intentado sacar a los NPC de sus roles preestablecidos, sin éxito alguno. Esto es una buena demostración de la robustez de la herramienta. Por tanto, a pesar de las alucinaciones puntuales generadas por la naturaleza del modelo, las limitaciones de la herramienta y la falta de sentidos en los NPC, partiendo de los datos recogidos, se ha considerado que las respuestas de los personajes no jugadores son de calidad, consistentes con el rol del personaje y se adecúan al contexto.

Como siguiente punto de investigación, se quiere estudiar la inmersión del jugador durante la experiencia. Una reacción poco predecible implica en muchos casos que el resultado de una interacción no se adecuaba a las expectativas del jugador y al estado del escenario provocando una ruptura en la inmersión de juego. Por este motivo se considera importante estudiar los resultados y evaluar dos comportamientos concretos que pueden afectar a la inmersión.

¿Cómo de correctas te han parecido las mallas que el LLM haya podido generar durante tu experiencia? Si el LLM no ha generado ninguna mal...elo 3D que representa a una entidad en la escena.

21 respuestas

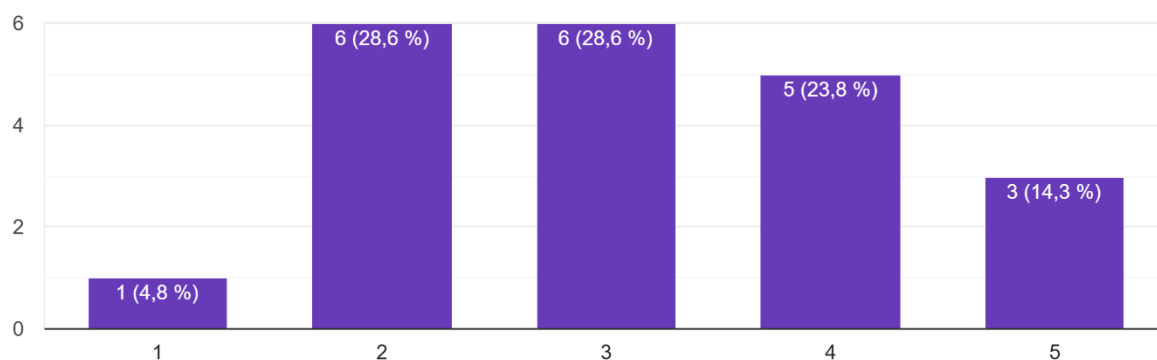


Figura 6.8 Evaluación de los usuarios sobre la generación de mallas.

¿Cómo de correctas te han parecido los sonidos que el LLM haya podido generar durante tu experiencia? Si el LLM no ha generado ningún sonido no respondas.

22 respuestas

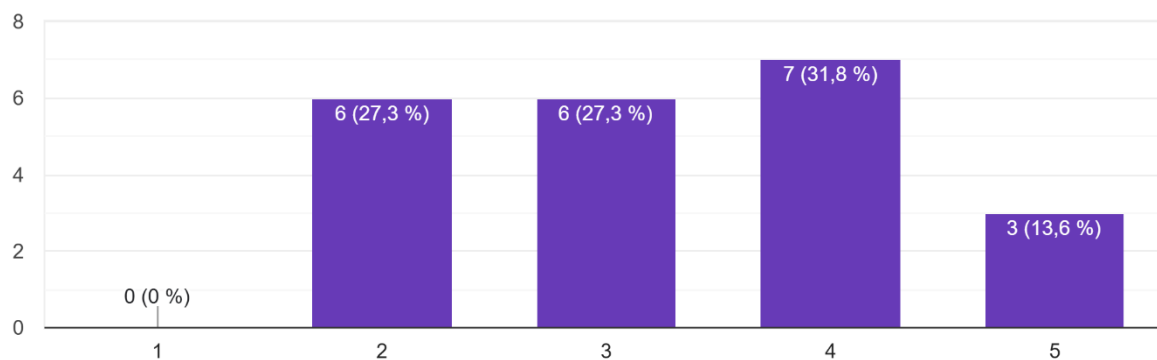


Figura 6.9 Evaluación de los usuarios sobre la generación de sonidos.

En las *figuras 6.8 y 6.9* se puede observar mucha más disparidad de opiniones. Aunque no todos los usuarios han podido experimentar la generación de mallas o sonidos como resultado de sus interacciones —y por lo tanto se les especificó que no respondiesen—, de entre todos los usuarios que sí pudieron experimentar esta mecánica, un 33.4% de los usuarios consideran que la generación de mallas no fue correcta según el contexto y un 27.3% tampoco considera adecuados

los sonidos generados. De la misma forma, un 38.1% y un 45.4% de los usuarios votaron positivamente la generación de mallas y sonidos respectivamente. Los resultados correspondientes a las valoraciones neutras fueron de 28.6% y 27.3%.

Se calcularon datos estadísticos tanto para los datos de las mallas como para los de los sonidos, obteniendo medias de 3.14 y 3.32 respectivamente, desviaciones estándar muestrales de 1.15 y 1.04 y coeficientes de variación de 36.62% y 31.33%. Se observa claramente que las medias son bastante bajas, comparadas con el resto de los apartados. Además, la variabilidad de los datos es demasiado elevada. Se proponen varias soluciones para resolver estas deficiencias y mejorar la experiencia de los usuarios. Por una parte, se podría editar la descripción del componente encargado de cambiar la malla de los objetos, para que favorezca su creación y se utilice siempre que se quiera crear o modificar un objeto de la escena. Con esta primera solución, se reducen los casos en los que el componente no se utiliza y el objeto generado se queda sin malla personalizada. En segundo lugar, para mejorar la calidad de las mallas generadas, se propone aumentar el catálogo de mallas que puede elegir el modelo a la hora de atribuir un modelo tridimensional a un objeto. Finalmente, para evitar la generación de mallas disonantes con el entorno, se propone eliminar de la lista de mallas disponibles, aquellas que representen primitivas básicas (cubo, cilindro, cono, etc). Esta medida también evita que el LLM abuse de estos elementos en situaciones ambiguas.

Después del análisis de estos puntos más concretos, se consideró importante conocer si el conjunto de acciones y reacciones tuvieron un impacto en la inmersión general de los usuarios a lo largo de la partida.

¿Cómo de inmerso te has sentido durante la experiencia?

24 respuestas

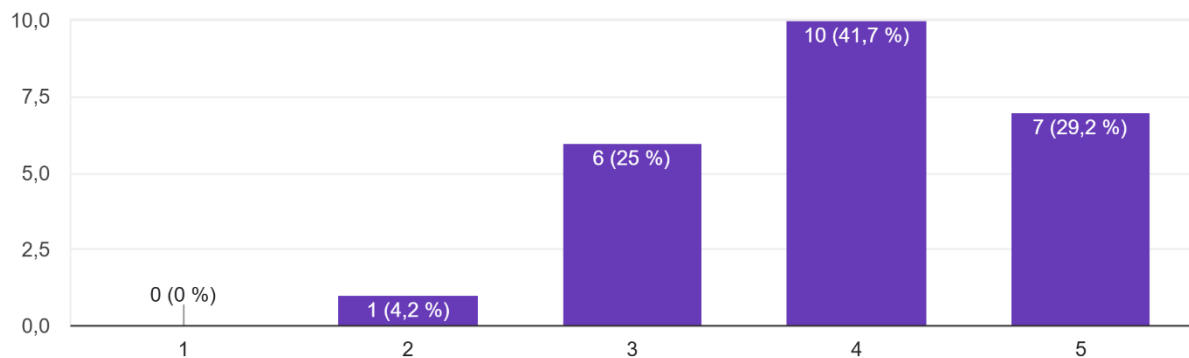


Figura 6.10 Evaluación de los usuarios sobre la inmersión durante la prueba.

En la *figura 6.10* se puede observar que un 70.9% de los votos fueron positivos. No obstante, un 25% votó neutro, y únicamente un 4.2% tuvo una experiencia poco inmersiva. La

media es de 3.96, desviación estándar muestral de 0.86 y coeficiente de variación del 21.72%. Aunque se aprecia algo de variabilidad en los datos, mayoritariamente se valora positivo. Por otro lado, se cree que el conjunto de soluciones propuestas anteriormente podría ser suficiente para mejorar la experiencia de aquellos usuarios con opiniones negativas o neutrales. No obstante, además de los problemas anteriormente mencionados, un motivo que puede empeorar la inmersión de jugadores es la limitación de interacciones de la herramienta. A continuación, se evalúa si las mecánicas presentadas en el juego permiten a los jugadores realizar las interacciones deseadas:

¿Las mecánicas presentadas en el juego te han permitido realizar todas las acciones que querías?

24 respuestas

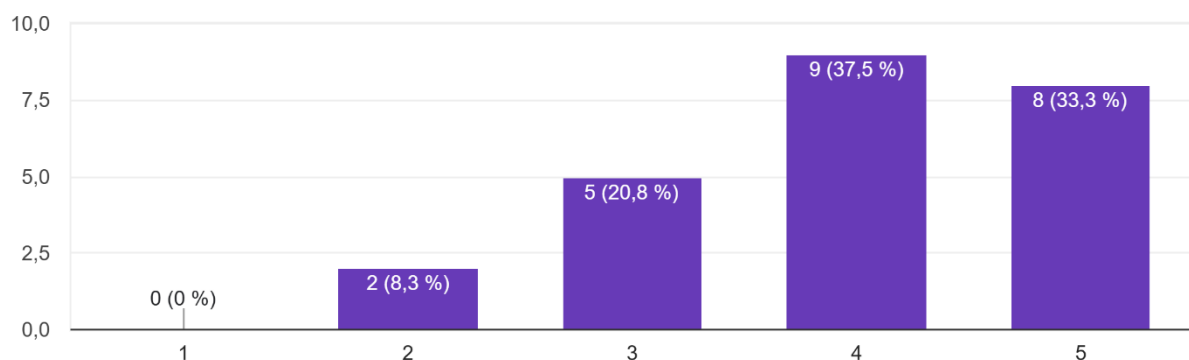


Figura 6.11 Evaluación de los usuarios sobre las mecánicas de juego.

Se analiza la *figura 6.11*. La media de este apartado es de 3.96, la desviación estándar muestral de 0.95 y coeficiente de variación de 23.99%. Se puede observar que la valoración media es positiva, aunque con una variabilidad algo alta. El 70.8% pudieron realizar prácticamente todas las interacciones que querían. El 29.2% restante no consideran haber podido realizar tantas interacciones como les gustaría. Además, se pudo observar que la gran mayoría de estos son usuarios con experiencia en este tipo de juegos, gracias al cuestionario inicial. Se analizaron las pruebas de estos usuarios para comprender mejor la situación. Tras un estudio de sus casos particulares se detectó un problema común: las intenciones de los usuarios no se corresponden con las acciones que decide el LLM, especialmente en casos donde se tratan temas sensibles. Un ejemplo recurrente entre los usuarios de este tipo de situaciones ocurre al usar la escopeta con los guardias, el LLM decide cual va a ser la interacción y asume que se está intentando intimidar a los guardias, no dispararles. Durante la entrevista, se les preguntó por sus intenciones y afirmaron intentar disparar a los policías.

Este problema se debe por dos motivos principales, en primer lugar, la ambigüedad de la interacción automática provoca que los resultados a una interacción sean impredecibles y en

muchos casos, distintos a los esperados por los usuarios. Ante la previsión de que ocurriesen estos fallos de interacción, se desarrolló el modo de interacción manual. Con este modo, se permite a los usuarios especificar qué acción quiere realizar mediante texto, tras haber seleccionado el objeto o NPC con el que se quiere interactuar. Esto genera resultados más afines a las intenciones de los usuarios y ofrece una mayor libertad de acción, pero explotar las carencias de contexto de la herramienta, pudiendo realizar acciones que no tienen sentido en el contexto de la acción. Un ejemplo sería interactuar con una puerta con la mano vacía y decir que la interacción es una explosión, lo que puede resultar en la destrucción de la puerta, algo impensable teniendo en cuenta el contexto general y rompiendo el diseño de nivel establecido. Por este motivo, el modo de interacción manual no se usó durante las pruebas de usuario. En segundo lugar, otra limitación de las interacciones se debe al código ético y moral que disponen la mayoría de los modelos de lenguajes pre entrenados, por el cual nunca tienden a tener comportamientos agresivos y evitar temas sensibles o tabú. Para solventar el problema presentado y como trabajo futuro, se podría realizar una validación de la acción especificada en el modo de interacción manual según el contexto de la interacción, todo esto mediante el LLM. Esto permite un mayor grado de libertad para ejecutar interacciones, pero manteniendo la coherencia.

Finalmente, para responder la pregunta de investigación, se pregunta a los usuarios acerca de si se esperaban los resultados de las acciones realizadas:

¿Las reacciones del LLM a tus acciones se ajustaban a tus expectativas?

24 respuestas

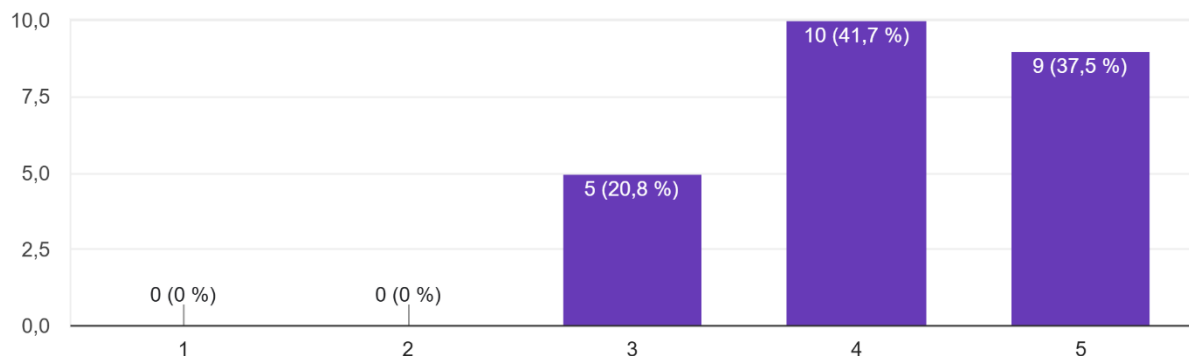


Figura 6.12 Evaluación de los usuarios sobre las reacciones del LLM respecto a sus expectativas.

En la *figura 6.12*, se aprecia una media de 4.17, desviación estándar de 0.76 y coeficiente de variación de 18.23%, observando claramente una dispersión no muy alta de los datos con una media positiva. De hecho, un 79,2% votaron positivamente este apartado. Por otro lado, un 20.8% de los usuarios votaron neutro, lo que lleva a pensar que hubo algunas interacciones que no se

ajustaron muy bien a sus expectativas. Tras la revisión de las pruebas de estos usuarios, se consideró que este resultado está afectado principalmente por el uso del modo automático de las interacciones, generando resultados inesperados.

Se puede concluir que en la gran mayoría de casos se ajustan correctamente a lo que el usuario espera. No obstante, es importante tener en cuenta los errores mencionados y arreglarlos para mejorar la calidad de la respuesta y las posibilidades que ofrece el juego.

6.1.1 Conclusiones

Tras el análisis extenso de los resultados obtenidos con los cuestionarios, contrastados con los resultados de observación y entrevista, se puede afirmar que las pruebas realizadas con usuarios permiten demostrar la eficacia de la herramienta para crear contenido reactivo a las acciones del jugador mediante el uso de LLMs. Las conclusiones obtenidas del estudio son las siguientes:

El uso de la herramienta permite generar escenarios interactivos y reactivos al jugador. Estos escenarios resultan inmersivos en todo momento, especialmente durante las conversaciones con los personajes no jugadores. Estos presentan un comportamiento realista y acorde con el papel que representan. Además, su consistencia y robustez frente a cualquier tipo de interacción verbal hacen que las conversaciones dentro del juego resulten satisfactorias y útiles para el desarrollo de la partida.

En cuanto a la interacción con elementos no animados de la escena, los resultados generales son positivos, pero se observa una mayor disparidad en las opiniones de los probadores, especialmente en lo que respecta a la inmersión y a la reacción de los objetos frente a las interacciones del jugador y otros personajes. Esta ligera inconsistencia se debe principalmente al sistema de interacción utilizado durante las pruebas y su implicación con la naturaleza del comportamiento impredecible de los LLMs. El sistema de interacción automático permite realizar acciones con mayor rapidez y fluidez que el sistema manual, con la contraparte de que no se tiene en cuenta parte del contexto en el que ocurre la acción. Este sistema genera casos de ambigüedad a la hora de hacer interacciones más complejas, que dependen más del estado general de la escena y contextos previos. Debido a esta ambigüedad, en determinados casos, las reacciones generadas por el modelo no se ajustan a las expectativas del usuario en algunos casos, lo que provoca una ruptura de la inmersión y la imposibilidad de realizar interacciones más complejas. Es esta imposibilidad la que ha limitado a algunos jugadores a completar el nivel de maneras más alternativas a las ya establecidas. Como solución para este problema, se propone extender la herramienta para que disponga de una mejor comprensión del entorno en todo momento y del contexto de cada acción realizada. De esta manera, se pretende mejorar los resultados obtenidos por la interacción automática y, en caso de usar la interacción manual, solucionar algunas de las vulnerabilidades que presenta este método de interacción.

En caso de utilizarse un modelo en remoto, los resultados obtenidos de los cuestionarios también permiten concluir que la herramienta presenta un rendimiento y tiempo de respuesta adecuado para su uso en aplicaciones que requieran de constante interacción entre el usuario y la máquina, como lo son los videojuegos.

Se quiere remarcar la pregunta final del cuestionario, donde se pregunta a los usuarios que valoren si la experiencia ha sido divertida o no, con los siguientes resultados:

¿Cómo de divertida valorarías tu experiencia?

24 respuestas

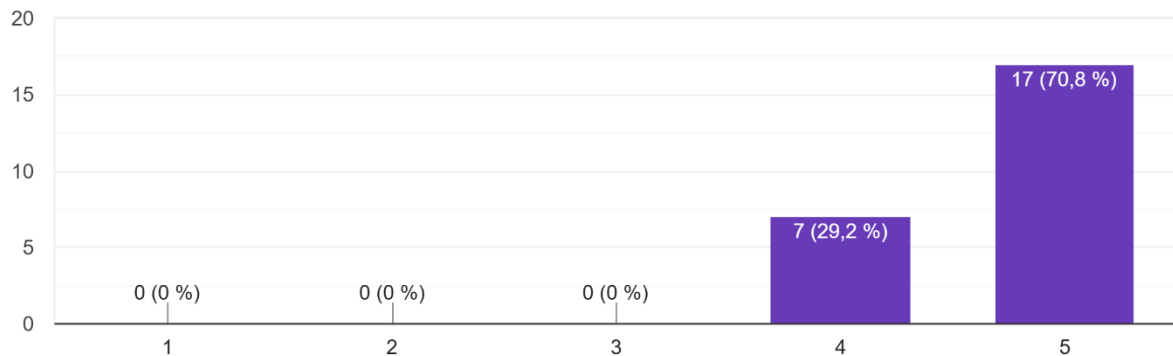


Figura 6.13 Evaluación de los usuarios sobre la diversión en esta experiencia.

Claramente se puede observar que todos los usuarios votaron positivamente la experiencia, con una media de 4.71, una desviación típica de 0.46 y un coeficiente de variación de 9.77%, siendo la menor variabilidad observada en los datos y la media más alta obtenida. De esta manera, se puede confirmar que todos los usuarios se divirtieron durante la experiencia.

Con todo lo anteriormente mencionado se puede concluir que la herramienta presenta buenos resultados en general, lo que consolida su utilidad en el desarrollo y prototipado de escenarios con un enfoque en la reactividad del entorno y de la narrativa. Para obtener mejores resultados, se optó por usar únicamente un modelo largo de lenguaje de altas prestaciones, alojado de manera externa, lo que también permite mejorar el rendimiento general, pero no se descarta la posibilidad de uso de modelos locales en caso de disponer de un equipo con las prestaciones suficientes.

Capítulo 7 - Discusión

A continuación, se presenta una discusión del trabajo realizado, donde se habla de su contexto general resumido, de sus principales ventajas, desventajas y limitaciones, además de presentar nuevas alternativas para posibles escenarios donde aplicar la herramienta desarrollada.

En este trabajo se ha desarrollado una aplicación interactiva en tres dimensiones, donde un jugador es capaz de interactuar con su entorno a través de un simple sistema de interacciones. Estas son evaluadas y resueltas por un modelo largo de lenguaje, al que se le dan una serie de instrucciones específicas que debe seguir para dar una respuesta a cada interacción. Debe seguir un formato concreto de manera estricta, ya que, en caso de error, no se podría transcribir la respuesta dada a acciones directas sobre el juego. Para la implementación de la aplicación se ha utilizado Unreal Engine 5.2.1, que ha resultado especialmente útil por su versatilidad y sistema de código visual robusto. La comunicación con el modelo de lenguaje se ha realizado utilizando C++, estableciendo un canal de envío de información a través de sockets con los scripts realizados en Python para correr el LLM. Las pruebas de usuario han tenido lugar en una versión específica de la aplicación, donde los sujetos debían tratar de escapar de una prisión —que era el contexto dado— para ejemplificar el potencial de la inteligencia artificial, al presentar un escenario con múltiples posibilidades.

Principalmente, la herramienta desarrollada muestra una fuerte alineación con la hipótesis planteada durante la introducción. Tras las pruebas de usuario, se observaron resultados especialmente positivos con respecto a las reacciones esperadas por parte del modelo de lenguaje, siendo estas creativas a la vez que coherentes. Se prueba que la inclusión de este tipo de inteligencia artificial resultó especialmente divertida para muchos de los usuarios —no consiguiendo ninguna nota negativa a este respecto— y logró que cada partida fuese una experiencia única. Por lo tanto, se recalca que estos resultados muestran especial afinidad con la hipótesis. Sin embargo, no es posible dar certificación absoluta a este hecho, ya que, aunque el tamaño de la muestra era adecuado para obtener una estimación general, se vio sesgada por contener una amplia mayoría de usuarios pertenecientes a estudios superiores en ingenierías informáticas o similares. Sin embargo, dentro de la muestra, existía una gran variedad de perfiles de jugadores de videojuegos, de los que se obtuvo resultados positivos en general.

Una de las ventajas principales de la herramienta es su modularidad. Se muestra como una herramienta versátil y que fácilmente se adapta a los diferentes contextos ofrecidos. A través de simples descripciones en lenguaje natural y tras el desarrollo de los sistemas de interacción del jugador, es posible cambiar drásticamente el resultado de una interacción, acercando el desarrollo de experiencias interactivas a un público más generalista que el actual, donde se necesita de experiencia técnica concreta. De esta forma, se ve reducido el tiempo de prototipado de escenas concretas, puesto que la implementación de las interacciones específicas alargaría este proceso.

Implementando inteligencia artificial de esta forma, se puede generar un flujo de interacciones rápidamente, sin necesidad de programar gran parte del mismo. Puede resultar especialmente positivo para diseñadores que quieran realizar pruebas conceptuales, pero no dispongan las habilidades técnicas necesarias.

Su afinidad con el proceso iterativo de desarrollo de videojuegos se ve reforzada cuando se utilizó la herramienta durante las pruebas de usuario que permite identificar cuáles son las interacciones más realizadas y aquellas que se intentaron realizar, pero no estaban planeadas en el diseño del videojuego. Esto permite que se añadan al mismo, viendo cuáles han sido las reacciones a dichas interacciones y generando un flujo de interacción con el entorno adaptado a lo que el jugador le gustaría hacer realmente.

Otra de las ventajas principales es la integración de diálogo natural por parte del jugador con los NPCs de su entorno, sin poder modificar el papel que desempeñan en el mundo. Desarrollando un sistema de restricción de acciones mediante las máquinas de estado mencionadas durante la memoria, se pueden generar interacciones donde el jugador realmente se sienta parte del mundo. La herramienta desarrollada facilita en gran medida este apartado, que cada vez cobra más relevancia en la industria del videojuego actual.

No obstante, a pesar de las múltiples ventajas mencionadas, la herramienta también muestra una serie de limitaciones importantes. Durante el desarrollo del trabajo, el equipo disponible para correr localmente los modelos largos de lenguaje era especialmente pobre, haciendo que el ciclo de desarrollo se ralentizase, generando un cuello de botella en la capacidad real de desarrollo del grupo de trabajo. Se quiso asegurar el buen funcionamiento de la herramienta utilizando modelos tanto en remoto como en local. Esto resultó un punto significativo que causó que la escala del proyecto se viera reducida por múltiples motivos.

Al utilizarse modelos de lenguajes ejecutados localmente, la comunicación resultó ser especialmente lenta, por lo que se optó por la reducción de la escala de los prompts, priorizando la velocidad del modelo y que la aplicación se bloqueara el menor tiempo posible, para evitar grandes brechas de usuario durante las pruebas a realizar. Como consecuencia, los prompts tuvieron que ser unificados en uno, para evitar realizar múltiples peticiones al modelo, borrando también capacidades de memoria de los modelos utilizados por su alta latencia. También se eliminaron partes del proyecto que, sin duda, hubiera mejorado la experiencia de los sujetos. Este es el ejemplo de la falta de contexto de los NPCs con su entorno, haciendo que las interacciones del jugador deban ser explicadas mediante diálogos para que sean realmente capaces de entender el contexto. Anteriormente, se consideró como posible expansión este apartado. Sin embargo, las limitaciones de tiempo han resultado ser un factor restrictivo durante el desarrollo del trabajo. Todos los inconvenientes mencionados se hubiesen evitado fácilmente centrando el desarrollo con LLMs en remoto, aumentando las capacidades técnicas del proyecto.

También, como limitación especialmente vinculada a las pruebas de usuario, es la utilización del inglés como idioma principal de la herramienta. Al estar todas las interacciones escritas en este idioma y todos los sujetos no tenerlo como lengua principal, se genera un importante sesgo, donde algunos probadores presentan dificultades de comprensión del texto recibido, al igual que se dialoga con los NPCs en un formato no del todo correcto. Por suerte, la capacidad del modelo utilizado durante las pruebas para sortear errores gramaticales ayudó especialmente a esto. Como desventaja principal de la utilización del inglés se destaca que ninguno de los usuarios utilizó el sistema de reconocimiento de voz, principalmente por vergüenza al encontrarse rodeado el sujeto de observadores. También, el entorno de realización de las pruebas, donde se producían ecos e individuos externos pasaban por el edificio, hicieron que la implicación del sistema de reconocimiento de voz fuera más compleja, por lo que finalmente ningún usuario optó por utilizarlo, reduciendo la inmersión en la aplicación. A pesar de esto, los usuarios mostraron un alto nivel de implicación en el mundo virtual donde se encontraban.

Adicionalmente, una limitación de las pruebas de usuario realizadas radica en la falta de un grupo de control, que no tenga acceso a la herramienta desarrollada y que permite la comparación entre dos modelos de la aplicación diferentes. De esta forma se podría realizar un análisis más profundo, donde realmente se vea una diferencia entre las interacciones realizadas en un mismo entorno sin un modelo de lenguaje llevando las interacciones del usuario, frente a ese mismo entorno con las capacidades de la inteligencia artificial respaldando el desarrollo narrativo de la misma.

Capítulo 8 - Conclusiones y trabajo futuro

El proyecto tiene como objetivo estudiar la viabilidad del uso de modelos largos de lenguaje para el desarrollo de escenarios tridimensionales interactivos y reactivos en tiempo real, manteniendo un enfoque en la narrativa. Para el desarrollo de la aplicación se utilizó Unreal Engine 5 como motor de desarrollo de la aplicación. En el caso de los modelos largos de lenguaje, se implementaron varias librerías de Python con la funcionalidad necesaria para establecer comunicación con los LLMs y reenviar la respuesta al motor. Con el fin de obtener información relevante para responder a la hipótesis de investigación, se llevaron a cabo numerosas pruebas con usuarios externos, realizando posteriormente un extenso análisis a partir de los resultados obtenidos. Apoyándonos en las conclusiones obtenidas se pretende dar una conclusión más completa que permita responder mejor a los objetivos inicialmente planteados.

El objetivo principal de este trabajo consistía en diseñar e implementar un sistema basado en modelos largos de lenguaje capaz de generar reactividad a las acciones de un usuario en un entorno interactivo. Se implementó una herramienta capaz de establecer una conexión por puertos entre el motor de videojuegos Unreal Engine 5 y un modelo largo de lenguaje. Dicha herramienta se diseñó de manera modular, pudiendo cambiar fácilmente el LLM utilizado, lo que a su vez permitió realizar numerosas pruebas con modelos locales y remotos y así poder hacer un riguroso análisis que nos permitiese elegir el modelo más ajustado para las necesidades de la herramienta. En el motor de videojuegos, se programó un sistema que ofrece una interfaz de comunicación con el modelo, así como una herramienta modular de comandos que permite personalizar ciertos aspectos de la ejecución del LLM. Utilizando principalmente el sistema de programación visual Blueprints, se implementó un gestor de prompts capaz de estructurar los mensajes, apoyado por un proceso de investigación, que permite optimizar la longitud de las peticiones, maximizando la calidad de las respuestas. Adicionalmente, mediante interacciones o señales, el gestor de la escena es capaz de alterar el estado actual del nivel como consecuencia de una interacción. El estado se puede alterar mediante conversaciones con personajes no jugadores, modificando las máquinas de estado que presentan algunos elementos, o eliminando, añadiendo y modificando objetos existentes. La herramienta incluye un sistema de composición de comportamiento, por el cual le permite al modelo extender el comportamiento de los objetos de la escena añadiendo componentes de lógica. Finalmente, con el objetivo de mejorar la inmersión en la comunicación con NPCs, se implementó un sistema de voz a texto, mediante la inclusión de un segundo modelo de inteligencia artificial, especializado para estas tareas. Adicionalmente, para las respuestas de los personajes no jugadores, un sistema de texto a voz permite escuchar las respuestas de estos aportando naturalidad e inmersión a las conversaciones.

Una vez terminado el desarrollo de la herramienta, se procedió con la preparación de las pruebas de usuario, diseñando un nivel de pruebas y distintos métodos de evaluación. Gracias al número de sujetos de prueba que participaron en las pruebas (24), se pudo obtener una muestra lo

suficientemente grande como para extraer las conclusiones necesarias para realizar mejoras a la herramienta y responder a la hipótesis de investigación.

La herramienta presenta una facilidad sin precedentes a la hora de generar una amplia variedad de comportamientos y escenarios mediante la modificación de descripciones en forma de texto, desde el editor. A través de la combinación de descripciones semánticas y modelos de lenguaje, se puede alcanzar una escalabilidad conceptual casi ilimitada, abriendo la puerta a experiencias dinámicas y altamente personalizables dentro de los videojuegos. La facilidad de edición y su flexibilidad permite a los desarrolladores definir, ajustar o ampliar el comportamiento del sistema sin necesidad de modificar el código base, pudiendo crear entornos ricos, reactivos y altamente interactivos con una carga técnica mínima, lo que democratiza el desarrollo de contenido, haciéndolo accesible incluso a perfiles con menor experiencia en programación.

Los resultados obtenidos de las pruebas de usuario y su posterior análisis demuestran que la experiencia diseñada resulta altamente interactiva e inmersiva, con aproximadamente un 71% de opiniones positivas, lo que permite a los usuarios obtener reacciones y resultados altamente variados dependiendo de sus acciones. Del 83.3% de usuarios que consiguieron terminar la experiencia, se observó en la mayoría de las partidas resultados únicos a las acciones del jugador y aproximadamente el 41% de los probadores obtienen finales que no estaban diseñados. Con una valoración positiva del 83.3% y una media de 4.04 puntos sobre 5, el estudio también demuestra la efectividad de la herramienta para atenerse al contexto actual a la hora de reaccionar a interacciones, especialmente en el caso de los objetos. El comportamiento contextual robusto de los personajes no jugadores mejora la inmersión y permite interacciones más complejas. Un 86.9% de los usuarios consideran que las respuestas de NPCs fueron adecuadas, resultando en una valoración de 4.26 sobre 5, lo que respalda el último punto descrito.

El definitiva, el resultado de este trabajo es una herramienta fácilmente extensible y modular, capaz de generar contenido y modificar el estado actual de un nivel mediante interacciones de diferentes tipos. La herramienta permite a los desarrolladores generar de manera eficiente, escenarios con multitud de interacciones, que permiten a los jugadores alterar su estado y obtener resultados diferentes. Es una herramienta adecuada para el desarrollo de juegos donde la narrativa y la alteración del entorno son elementos clave de sus dinámicas. No obstante, la herramienta presenta algunas limitaciones: la simplicidad y restricciones naturales del modo de interacción automática utilizado, provoca en ocasiones resultados impredecibles y menos inmersivos. Por otra parte, la ausencia de un contexto general actualizado para los NPCs durante las conversaciones limita la complejidad de sus acciones más allá del diálogo y tareas sencillas. Estas son las principales deficiencias que limitan el uso de esta herramienta en proyectos de mayor envergadura.

8.1 Trabajo futuro

La sección de trabajo a futuro pretende presentar las principales mejoras del proyecto propuestas, mediante las cuales, se podría extender el trabajo realizado hasta la fecha y solventar las principales deficiencias presentes.

- Se plantea la incorporación de un *quest tracker* gestionado por el LLM, encargado de identificar y actualizar dinámicamente el objetivo principal activo en función del contexto del juego. Este sistema permitiría que —a medida que se produzcan acciones relevantes en el mundo— el estado de las misiones se actualice automáticamente, lo que incluye modificar el objetivo principal, añadir misiones secundarias o ajustar el progreso según convenga.

Una propuesta complementaria consiste en delegar en el propio LLM la responsabilidad de determinar cuándo se ha completado la misión principal o incluso el juego completo, en lugar de depender de una lógica programada estáticamente. Así, tras cada interacción significativa, el modelo evaluaría el estado narrativo general y decidiría si se ha alcanzado el final o si deben continuar los eventos. Este enfoque permitiría una experiencia narrativa más flexible y adaptada al estilo de juego.

- La implementación del proyecto en un entorno de realidad virtual (VR por sus siglas en inglés, de *Virtual Reality*) representa una evolución natural del sistema. Contar con un entorno interactivo como el que ofrece la VR permitiría definir acciones entre elementos de forma más intuitiva y directa, sin necesidad de introducirlas manualmente mediante texto. A través del seguimiento preciso del movimiento y la velocidad de los gestos del jugador, sería posible inferir intenciones y tipos de interacción (como golpear, lanzar, recoger o señalar), lo que enriquecería significativamente el contexto proporcionado al LLM. Esto no solo mejoraría la inmersión del jugador, sino también la calidad y pertinencia de las respuestas generadas.
- La incorporación de un sistema de reconocimiento de gestos como una capa adicional de entrada contextual en el proyecto resultaría interesante. Esto permitiría mejorar la interacción con el LLM mediante gestos, sin depender exclusivamente de un controlador físico, lo que aportaría una experiencia más natural e inmersiva. Además, el reconocimiento gestual podría ayudar a la interpretación de intenciones o acciones del jugador que no se expresan de forma explícita, mejorando así la calidad de las respuestas generadas y la adaptabilidad del sistema.
- Una posible mejora consiste en reorganizar la estructura de la información de los objetos del juego, dividiéndola en dos descripciones diferenciadas: una pública y otra privada. La descripción pública sería visible para el jugador al apuntar o interactuar

con un objeto, ofreciendo información relevante desde una perspectiva diegética o estética. Por otro lado, la descripción privada estaría destinada exclusivamente al LLM y contendría datos más detallados o técnicos que no son visibles en el juego, pero que permiten generar respuestas o comportamientos más coherentes y contextualizados. Esta separación permite optimizar tanto la experiencia del jugador como la calidad de las respuestas generadas por el modelo.

- Se propone un aumento en la complejidad del comportamiento de los NPCs mediante la mejora de sus máquinas de estados, incorporando sistemas de percepción y sentidos más avanzados. El objetivo es dotarlos de una mayor capacidad de reacción autónoma, permitiéndoles ofrecer respuestas más realistas y naturales ante lo que ocurre en el entorno.

Esto implicaría que los NPCs no solo respondan a interacciones directas, sino que también sean capaces de observar eventos que suceden cerca —como interacciones entre otros personajes u objetos— y registrar esta información en su historial interno.

Además, los NPCs podrían decidir de forma autónoma si una situación requiere una reacción, y generar dicha respuesta consultando al LLM. Esto abriría la puerta a interacciones emergentes, donde los personajes reaccionan como agentes verdaderamente conscientes de su entorno, enriqueciendo la dinámica del mundo del juego.

- Actualmente, el sistema de diálogo con los NPCs requiere que el jugador se encuentre dentro de una zona de activación para que estos *escuchen* e interpreten los mensajes, sin tener en cuenta factores contextuales más realistas. Se propone una mejora de este sistema para que la percepción de los NPCs sea más natural y sensible al entorno.

Esto implicaría considerar variables como la distancia entre el jugador y el NPC, las características acústicas del espacio (por ejemplo, si hay obstáculos, paredes o reverberación), así como la tonalidad o volumen del mensaje. Estas interacciones aportarían más riqueza y complejidad a las conversaciones, pudiendo generar respuestas más matizadas y comportamientos diferentes por parte del LLM dependiendo del contexto. De esta manera, un grito podría percibirse como una amenaza o una llamada urgente, mientras que un susurro podría generar desconfianza o confidencialidad. Esta aportación, además de mejorar la inmersión, también ayudaría a mejorar las respuestas de los NPCs.

- Se propone un modo manual más controlado, en el que el LLM no solo interprete la acción introducida por el jugador, sino que también evalúe su validez en función del contexto específico. Para ello, el modelo tendría en cuenta tanto al instigador como al receptor de la interacción, determinando si la interacción propuesta es coherente,

posible o relevante dentro del estado actual del mundo. Este enfoque permitiría filtrar acciones inválidas o inverosímiles.

- La realización de unas pruebas de usuario donde el grupo de control no tenga acceso a la herramienta desarrollada. De esta forma, dicho grupo no podría delegar el resultado de las interacciones a un modelo de inteligencia artificial. Así se podría establecer una comparación exhaustiva entre un flujo de interacción tradicional y un escenario que incorpore la herramienta detallada en este trabajo.

INTRODUCTION

Generative Artificial Intelligence (AI) has gained great international relevance since the public launch of ChatGPT⁵¹, this being one of the most discussed topics daily by thousands or even millions of people unfamiliar with its functioning. This launch has sparked a wave of research and development around the world with the aim of testing the limits of large language models (LLMs) for multiple purposes. It has been observed how, in a matter of months, practically all the technological giants have developed interfaces with access to a generative AI, thus flooding the most used applications with virtual assistants capable of answering general or specific questions regarding the application.

The entertainment sector has not lagged: there are already multiple video games that use natural language generation for interaction with the inhabitants of their virtual environment. An example would be *Suck Up!*⁵², a video game that allows the player to play the role of a vampire in a humble village. As popular legends say, these beings are not able to enter their victims' houses until they are explicitly invited; therefore, the player's goal is to establish conversations with the villagers, trying to lie in order to get into their houses and feed on them. All conversations are carried out by an LLM, which generates thousands of emergent situations. They also allow generating new characters with unique descriptions and appearances. Another prominent example in the industry is *Cygnus Enterprise*⁵³, which uses generative AI for the dialogues with non-player characters (NPCs) of the environment. The most remarkable thing is that these NPCs not only participate in conversations, but they can also understand direct instructions from the player, such as requesting help in specific combat situations or indicating tasks to their allies

The possibilities that these types of models offer the user approach something dreamed of within a video game: the feeling of complete freedom, where actions truly have notable consequences in the virtual environment.

However, current applications have mainly focused on the interaction with NPCs, offering natural, complex, and real-time dialogues. Although this approach is especially attractive, the true potential of LLMs goes beyond simple text generation. Their ability to understand described environments and process large volumes of information raises a key question: *can this technology be applied to other areas within video game development?*

⁵¹ ChatGPT: <https://openai.com/chatgpt/overview/>

⁵² SuckUp!: <https://www.playsuckup.com/>

⁵³ Cygnus Enterprises: https://store.steampowered.com/app/1963520/Cygnus_Enterprises/

Beyond their integration in video games, LLMs are acquiring a fundamental role in research on a global scale. Since the publication of the article *Attention Is All You Need* [1], these models have been applied to multiple contexts, demonstrating their versatility and effectiveness. In said publication, outstanding results were presented in automatic translation tasks—for example, from English to French or German—surpassing previous models in precision and efficiency. Currently, their use is expanding to other areas such as medicine, where Kanithi et al [2] propose a series of tests to evaluate the performance of LLMs in clinical environments. Likewise, Scarlatos et al [3] explore their application in the educational field, proposing them as support tools for the personalized tutoring of students.

In this context, the R+D+I project DARK NITE, PID2023-146308OB-I00, funded by the State Research Agency (code MICIU/AEI/10.13039/501100011033/) and FEDER of the European Union proposes the development of a system capable of generating dynamic narrative structures using one or more LLMs as the main interaction processing engine. These models are integrated into an interactive environment, being able to communicate with characters through natural language and interact with elements of the environment through controllers. In this way, the project seeks to develop tools capable of receiving new objectives defined by a human supervisor, while interacting with the user and dynamically adapting to their actions within the interactive environment. This line of research is especially relevant to the approach proposed in this work.

Motivation

According to the previous context, it was considered that LLMs can become key allies to facilitate the design of reactivity in video games, beyond the field of NPC dialogues. This type of interactions where we have environments so reactive to the whole range of player actions have a very high production cost. A perfect example of this is *Baldur's Gate 3*⁵⁴, which designs an experience with practically unlimited possibilities in its interactions, but its development time and cost—in addition to a large team of experienced personnel—are unattainable for most of the industry, made up of smaller companies, focused on short and/or medium developments.

There still exists a notable gap in the design of experiences that respond meaningfully to the player's actions beyond dialogue, especially in interactive three-dimensional or two-dimensional contexts. In this type of environments, the interaction with objects or systems of the virtual world is usually limited to previously programmed behaviors, which restricts the expressiveness and the sense of agency of the player. Unlike text-based adventures, where the model's control over the experience is almost total, video games with visual and sound components

⁵⁴ Baldur's Gate 3: <https://baldursgate3.game/>

require more structured and adapted responses, which poses new technical challenges, but also very interesting opportunities. The combination between the player's creativity and the generative capacity of the language models guarantees the appearance of dynamic responses, capable of maintaining the user's attention, interest, and immersion.

All this context motivates the development of this work, where the incorporation of artificial intelligence, capable of understanding the context and generating coherent responses, allows to significantly accelerate development times and makes more accessible the complex reactive experiences so much demanded by the video game industry.

Hypothesis

Given that the current design of video games demands increasingly immersive experiences, it is fundamental to investigate how artificial intelligence can facilitate the creation of reactively complex systems for developers. The hypothesis of this work is the following:

The inclusion of artificial intelligence models, with the ability to understand their environment and react to it, improves the experience of design and development of reactivity in video games.

This approach assumes that the agents controlled through artificial intelligence are not limited to executing predefined behaviors, but are capable of dynamically expanding the narrative and their repertoire of actions according to the context of the environment and the player's decisions. Furthermore, this capacity of adaptation is not restricted only to the NPCs, but can be extended to a global artificial intelligence in charge of managing the reactivity of the environment as a whole.

Goals

It is intended to understand what function an LLM can serve in the reactivity in video games. For this, it is proposed to analyze which type of interactions the model can interpret, how it reacts to them, and in what part of the system it must be integrated to maximize its usefulness. This process includes: defining the specific tasks that the LLM will carry out, evaluating its adequacy to the context of use, and verifying whether the decisions made during the design and implementation have produced the expected results.

General objective:

Design and implement a system based on large language models capable of generating reactivity to the actions of a user in an interactive environment.

This research focuses especially on the analysis of the model's behavior when receiving information about the player's interactions with objects or NPCs of the virtual world. The LLM answers must conform to a predefined structure and minimize the so-called hallucinations, that is, errors typical of language models that generate incoherent or invented content.

Specific objectives:

- Develop an interactive application in which the user can explore and act freely within an environment.
- Implement a communication system between the application and an LLM.
- Analyze and select the most appropriate language model depending on the resources and needs of the project.
- Adapt the responses of the model used to the structured format required for its interpretation by the system.
- Develop a voice recognition system that allows the user to interact verbally with the environment.
- Design a test scenario in which multiple users interact with the implemented system.
- Carry out an exhaustive analysis of the results obtained in the tests, considering both the quality of the model's responses and the experience of the participants.

The result of this work will be a detailed analysis of the responses generated by the model to the user's actions. The precision, contextual coherence, and absence of hallucinations will be evaluated, as well as the impact of these factors on the player's experience. The subjective perception of the participants will also be considered, assessing their level of immersion and overall satisfaction.

Finally, the conclusions drawn will be conditioned by the resources available during the development, although the possible improvements that could be achieved with greater access to computational infrastructure will be estimated.

Work plan

The research will be carried out over 9 months. The work plan follows a structure coherent with the specific objectives described in the previous section, since these mark the necessary stages for the implementation of the system and its subsequent analysis. Considering this time window and the defined objectives, the team has established the following organization:

September – November:

During these first months, an exhaustive investigation will be carried out into the best options for communication with a large language model. This stage includes the analysis of different alternatives depending on variables adapted to the resources available for the project. At the same time, a first draft of the system architecture will be developed, where the expected behavior of the language model and the functionalities it must cover will be specified through diagrams. In addition, it will be defined what data the LLM must receive from the environment and how these data will be transformed into appropriate requests to the model.

December – January:

In this phase, the development of the interactive three-dimensional application will proceed, in accordance with the previously designed architecture. The first version of the application will be subject to possible adjustments derived from technical needs during the implementation. The team will build the necessary infrastructure so that the different objects meet the requirements described by the architecture. Simultaneously, the implementation of the automatic speech recognition system will be addressed.

February:

This month will be key for the integration of the application with the language model, thus forming a complete and interactive experience. Work will be done to ensure smooth communication between both components, following the technical specifications of each one.

March:

During March, technical tests of the system will be carried out. Specific scenarios will be designed to verify the correct functioning of all integrated elements, with the objective of detecting possible errors and refining the solution before its final validation.

April:

This stage will be dedicated to the design and execution of tests with users. The system will be presented to an external audience to the development team. Data collection is fundamental for the evaluation of the project, so the meticulous design of the tests —as well as of the final scenario— acquires extreme importance for the subsequent analysis and validation of the results.

May:

Finally, the month of May constitutes the end of this work, where it will be dedicated to the analysis of the data collected during the user tests and to the final writing of the work. The implementation processes, the difficulties encountered, the results obtained, and the modifications made during the project will be documented in detail.

In *figure 8.1*, a Gantt diagram is shown that graphically illustrates the described time planning.



Figure 8.1 Gantt Diagram Represents the temporal sequence and estimated duration of the different tasks contemplated in the work plan

Methodology

For the development of this research, a series of accessible and free tools for all audiences will be used, which facilitates the reproducibility of the results obtained.

The entire development process will be carried out on the *Windows 11*⁵⁵ operating system, since it is the platform with which the team is most familiar. Its versatility, availability of educational resources, extensive documentation, and ease of use allow for agile implementation. Although the result of the project does not depend directly on the operating system, the systems developed will be constrained by the use of specific libraries which are only compatible with this platform.

Regarding the creation of a three-dimensional interactive application, the *Unreal Engine*⁵⁶ video game engine will be used, in its recent 5.2.1 version. This engine stands out for its set of integrated functionalities, its extensive support community, and its visual programming system, which facilitates rapid development while maintaining code readability. Unreal Engine is one of the most used in the professional environment of the video game industry, being the basis of multiple successful games.

⁵⁵ *Windows 11*: <https://www.microsoft.com/en-us/windows/windows-11>

⁵⁶ *Unreal Engine*: <https://www.unrealengine.com/en-US>

For the implementation of communication with a large language model, the *Python*⁵⁷ programming language will be used, due to its compatibility with various libraries and APIs⁵⁸ developed by different entities. These tools allow for agile iterations in the design and integration of LLMs. Among the candidate models to be tested are *DeepSeek*⁵⁹, *LLaMA*⁶⁰, *Mistral*⁶¹, and *Gemini*⁶²; all of them are free to use except for Gemini, which imposes a limit on the number of words sent to the model for free. As a complementary tool to these language models, web interfaces such as the ChatGPT or DeepSeek clients will be used to experiment with different request formats, with the objective of optimizing communication with the models and reducing response times, thus facilitating a more efficient iterative process.

To be able to establish a common repository during development where the entire project can be kept updated for the team members, *GitHub*⁶³ is shown as an ideal tool. This tool facilitates the management and control of software repositories during their implementation. Its operation is based on *Git*⁶⁴, a distributed version control system, which allows the code to be stored locally on all the devices with access to the remote repository. The main advantage of using GitHub is its accessibility, a clean interface and, especially, the team's previous experience in medium-duration projects using this platform as the central axis of collaborative development.

Finally, the coordination of the team will be carried out through biweekly in-person meetings. In case any member cannot attend physically, telematic meetings will be held through the *Discord*⁶⁵ platform, where a server has been created that facilitates the storage and consultation of documentation, links and messages, structured through different thematic channels. Complementarily, the team will use the online platform Trello⁶⁶ for the organization and

⁵⁷ Python: <https://www.python.org/>

⁵⁸ API: mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos

⁵⁹ DeepSeek: <https://www.deepseek.com/en>

⁶⁰ Llama: <https://www.llama.com/>

⁶¹ Mistral: <https://mistral.ai/>

⁶² Gemini: <https://gemini.google.com/>

⁶³ GitHub: <https://github.com/>

⁶⁴ Git: <https://git-scm.com/>

⁶⁵ Discord: <https://discord.com/>

⁶⁶ Trello: <https://trello.com/>

monitoring of tasks, through a virtual board that allows the status of the project and the assignment of responsibilities to be visualized at all times.

Conclusions and future work

The project aims to study the feasibility of using large language models for the development of interactive and reactive three-dimensional scenarios in real time, while maintaining a focus on narrative. For the development of the application, Unreal Engine 5 was used as the engine. In the case of large language models, several Python libraries were implemented with the necessary functionality to establish communication with the LLMs and forward the response to the engine. In order to obtain relevant information to answer the research hypothesis, numerous tests were carried out with external users, followed by an extensive analysis based on the results obtained. Based on the conclusions drawn, the goal is to provide a more complete conclusion that better addresses the initially proposed objectives.

The main objective of this work was to design and implement a system based on large language models capable of generating reactivity to a user's actions in an interactive environment. A tool was implemented that can establish a port-based connection between the Unreal Engine 5 video game engine and a large language model. This tool was designed in a modular way, allowing the LLM to be easily changed, which in turn enabled numerous tests with both local and remote models, allowing for a thorough analysis to determine the model best suited to the tool's needs. In the game engine, a system was programmed that provides an interface for communication with the model, as well as a modular command tool that allows customization of certain aspects of the LLM's execution. Using primarily the visual programming system Blueprints, a prompt manager was implemented, capable of structuring messages—supported by a research process—which helps optimize the length of the requests, maximizing the quality of the responses. Additionally, through interactions or signals, the scene manager is able to alter the current state of the level as a consequence of an interaction. The state can be altered through conversations with non-player characters, by modifying the state machines used by certain elements, or by deleting, adding, or modifying existing objects. The tool includes a behavior composition system that allows the model to extend the behavior of scene objects by adding logic components. Finally, to improve immersion in communication with NPCs, a speech-to-text system was implemented through the inclusion of a second artificial intelligence model specialized in these tasks. Additionally, for the responses of non-player characters, a text-to-speech system allows their replies to be heard, adding naturalness and immersion to the conversations.

Once the tool was developed, user testing was prepared, including the design of a test level and various evaluation methods. Thanks to the number of test subjects who participated in the testing (24), a large enough sample was obtained to extract the necessary conclusions to improve the tool and address the research hypothesis.

The tool offers unprecedented ease in generating a wide variety of behaviors and scenarios through the modification of descriptions in text form, directly from the editor. Through the

combination of semantic descriptions and language models, nearly unlimited conceptual scalability can be achieved, opening the door to dynamic and highly customizable experiences within video games. Its ease of editing and flexibility allows developers to define, adjust, or expand the system's behavior without needing to modify the base code, enabling the creation of rich, reactive, and highly interactive environments with minimal technical load—democratizing content development and making it accessible even to profiles with less programming experience.

The results obtained from the user tests and their subsequent analysis show that the designed experience is highly interactive and immersive, with approximately 71% positive opinions, allowing users to obtain highly varied reactions and results depending on their actions. Of the 83.3% of users who managed to complete the experience, most playthroughs showed unique results based on player actions, and approximately 41% of testers reached endings that were not pre-designed. With a positive rating of 83.3% and an average of 4.04 out of 5 points, the study also demonstrates the effectiveness of the tool in maintaining contextual relevance when reacting to interactions, especially in the case of objects. The robust contextual behavior of non-player characters enhances immersion and allows for more complex interactions. 86.9% of users considered the NPC responses to be appropriate, resulting in a rating of 4.26 out of 5, supporting the last point described.

Ultimately, the result of this work is a modular and easily extensible tool capable of generating content and modifying the current state of a level through various types of interactions. The tool allows developers to efficiently generate scenarios with numerous interactions that let players alter their state and obtain different outcomes. It is a suitable tool for the development of games where narrative and environmental change are key elements of their dynamics. However, the tool does present some limitations: the simplicity and natural restrictions of the automatic interaction mode used can sometimes lead to unpredictable and less immersive results. Moreover, the absence of an updated general context for NPCs during conversations limits the complexity of their actions beyond dialogue and simple tasks. These are the main shortcomings that limit the use of this tool in larger-scale projects.

Future Work

The Future Work section aims to show a proposal of the main upgrades to the project, in order to extend the project and solve the principal existing deficiencies.

- The integration of a quest tracker managed by the LLM is proposed, responsible for identifying and dynamically updating the active mission based on the game's context. This system would allow mission states to evolve automatically as relevant actions occur in the world, including modifying the main objective, adding side quests, or adjusting progress as needed.

An additional proposal is to delegate to the LLM the responsibility of determining when the main mission—or even the entire game—has been completed, instead of relying on statically programmed logic. After each significant interaction, the model would assess the overall narrative state and decide whether the game has reached its conclusion or if events should continue. This approach would enable a more flexible, reactive, and adaptive narrative experience, aligned with the emergent nature of the interactive environment.

- Implementing the project in a virtual reality environment represents a natural evolution of the system. An interactive space like the one offered by VR would allow interactions between elements to be defined in a more intuitive and direct way, without the need to input them manually through text. Thanks to precise tracking of the player's movement and gesture speed, it would be possible to infer intentions and types of interaction (such as hitting, throwing, picking up, or pointing), significantly enriching the context provided to the LLM. This would not only enhance player immersion but also improve the relevance and quality of the generated responses.
- A potential improvement involves reorganizing the structure of information associated with game objects by dividing it into two distinct descriptions: one public and one private. The public description would be visible to the player when pointing at or interacting with an object, providing relevant information from a diegetic or aesthetic perspective. The private description, on the other hand, would be intended exclusively for the LLM and would contain more detailed or technical data that is not visible in-game but enables the generation of more coherent and context-aware responses or behaviors. This separation helps optimize both the player's experience and the quality of the model's output.
- An improvement is proposed to enhance the behavioral complexity of NPCs by enriching their state machines with more advanced perception and sensory systems. The goal is to provide them with greater autonomous reactivity, enabling more realistic and natural responses to events occurring in their environment. This would mean that NPCs are not limited to reacting only to direct interactions but are also capable of observing nearby events —such as interactions between other characters or objects— and recording this information in their internal memory. Furthermore, NPCs could autonomously decide whether a given situation warrants a reaction and generate that response by consulting the LLM, even without direct activation by the player. This would enable emergent interactions, where characters behave like truly aware agents within their environment, enriching the dynamics of the game world.
- Currently, the dialogue system with NPCs requires the player to be within an activation zone for the characters to *hear* and interpret messages, without accounting for more

realistic contextual factors. The proposed improvement aims to make NPC perception more natural and environmentally aware.

This would involve considering variables such as the distance between the player and the NPC, the acoustic properties of the environment (e.g., obstacles, walls, or reverberation), as well as the tone or volume of the message. In this way, NPCs could interpret a shouted phrase, a whisper, or a message spoken from a distance differently. For instance, a shout might be perceived as a threat or an urgent call, whereas a whisper could evoke suspicion or confidentiality, depending on the context.

- A more controlled manual mode is proposed, in which the LLM not only interprets the action entered by the player but also evaluates its validity based on the specific context. To do this, the model would consider both the instigator and the receiver of the action, determining whether the proposed interaction is coherent, feasible, or relevant within the current state of the world. This approach would allow the system to filter out invalid or implausible actions.
- The execution of user tests where the control group does not have access to the developed tool. In this way, that group could not delegate the result of the interactions to an artificial intelligence model. As a result, an exhaustive comparison could be established between a traditional interaction flow and a scenario that incorporates the tool detailed in this work.

CONTRIBUCIONES PERSONALES

Claudia Zarzuela Amor

Claudia estuvo involucrada de forma activa durante todo el desarrollo del proyecto y, al igual que el resto del equipo, participó tanto en el desarrollo técnico como en las reuniones de seguimiento, colaborando de manera constante para asegurar el buen avance del proyecto. Su principal aportación se centró en el diseño inicial, definiendo en detalle los tipos de interacciones requeridas, la estructura general que se le iba a mandar al LLM (verificando constantemente la coherencia de estas realizando pruebas con ChatGPT) y la implementación global en Unreal Engine.

En las primeras fases del proyecto, Claudia participó activamente junto al resto del equipo en la elaboración inicial de los prompts que se le iban a mandar al LLM, estableciendo una estructura clara sobre los objetivos comunes. Al mismo tiempo, ayudó a definir las interacciones entre objetos como la creación, modificación y eliminación. Participó en la elección y comparación de distintos modelos de lenguaje, llevando a cabo pruebas rigurosas en las que enviaba el mismo mensaje siguiendo la estructura predefinida inicialmente, organizada en distintos archivos separados. Hizo las pruebas comparativas entre los modelos de LLaMA 3.1, Granite 3.2 y OLMo. Posteriormente, analizó los resultados junto a sus compañeros para determinar cuál ofrecía un rendimiento más eficaz, sin prever aún las limitaciones que más adelante conducirán a un cambio del modelo.

Una vez elegido el LLM, trabajó en estrecha colaboración con su compañero Roi en el diseño de una estructura adecuada para implementar las interacciones con objetos. Mientras Roi se encargaba de la implementación general del StateManager, la instanciación y registro de los objetos, Claudia se centró en recibir y procesar la respuesta del LLM, deserializándolos correctamente y ejecutando las acciones correspondientes en función de su contenido. Durante esta etapa, ambos colaboraron en la investigación de plugins que permitieran manejar el formato JSON desde Blueprints en Unreal Engine, con el objetivo de facilitar el prototipado rápido de la lógica de interacción.

Claudia se encargó de implementar los métodos de crear, modificar y destruir, siendo el más tedioso el de la creación. Se encargó de que se generasen objetos no solo con el nombre y la descripción, sino que también eligieran su forma. Esta lógica fue diseñada inicialmente como parte del método de creación, aunque más adelante fue refactorizada por Miguel, quien la trasladó a ser un componente. También trabajó en la funcionalidad de instanciación de objetos con posiciones personalizadas, introduciendo las bases para permitir la colocación relativa al objeto que recibía la acción.

Durante esta misma etapa, y tras múltiples pruebas, Claudia propuso un enfoque experimental basado en reflexión con el objetivo de mejorar la calidad, coherencia y robustez de las respuestas generadas. Inspirada en estudios recientes en el campo de IA generativa, esta técnica consistía en permitir que el propio modelo evaluase su salida original y la reescribiera de forma razonada siguiendo unos criterios. El modelo debía verificar el formato JSON y tipos de datos, la coherencia interna y el cumplimiento de las reglas del sistema. Aunque el enfoque comentado en la memoria anteriormente resultaba prometedor, desde el punto de vista conceptual, en la práctica demostró ser inviable en el contexto del proyecto por varias razones técnicas.

Estas dificultades, junto con otros factores, llevaron al equipo a cambiar de modelo LLM. Esta experiencia puso en evidencia la importancia crítica de definir prompts bien estructurados y adaptados al modelo utilizado. A raíz de esto, Claudia decidió llevar a cabo un estudio exhaustivo de prompt engineering. Para ello, recopiló meticulosamente en un documento todas las respuestas generadas por el modelo a partir de diferentes prompts, analizando variaciones, errores, aciertos y patrones recurrentes. Este estudio permitió identificar cuáles eran las formulaciones más eficaces en función del tipo de tarea. El objetivo de este análisis fue ayudar a sus compañeros a comprender cómo pequeñas diferencias en la formulación del prompt podían desencadenar cambios radicales en el comportamiento del modelo. Gracias a esta iniciativa, el equipo pudo optimizar significativamente la calidad y coherencia de las respuestas.

Durante esta etapa, trabajo junto a Raúl y Roi para diseñar y crear los prompts finales que se le mandarían al LLM. Esta tarea resultó especialmente exigente en términos de tiempo y requería múltiples iteraciones, ya que, como se ha mencionado previamente, incluso pequeñas modificaciones en la estructura podían generar respuestas completamente distintas. Después, Claudia se encargó de implementar en el Blueprint creado por Roi del input del jugador para detectar con qué ha interactuado (objeto o NPC) y mandarle a información necesaria al PromptManager creado por Raúl.

Una vez cerrada la implementación global del proyecto, el equipo se dividió en pruebas de usuario y escribir la memoria. Claudia redactó en su completitud el diseño del sistema de interacción con el entorno y comunicación semántica con el LLM, la investigación y elección de los LLMs, además de documentar sus propias aportaciones técnicas al proyecto. Asimismo, desempeñó un rol clave como revisora del contenido redactado por el equipo, realizando correcciones y aportando comentarios.

Tras la finalización del diseño de las pruebas de usuario, Claudia estuvo presente en la mayoría de las sesiones junto al resto de sus compañeros del equipo, asumiendo el rol de observador y reclutando activamente a nuevos participantes dispuestos a contribuir con sus experiencias.

David Rivera Martínez

David ha resultado un miembro activo durante el desarrollo técnico de este trabajo. Su trabajo principal ha estado centrado en: la realización de una extensa revisión del estado del arte relevante para la investigación; la primera parte de la comunicación con el modelo de lenguaje, realizando las que terminaron siendo las primeras pruebas conceptuales de la misma; el intento de inclusión de la técnica RAG en el proyecto para reducir alucinaciones de los modelos; y la gestión de las herramientas de Speech-to-text y Text-to-speech. Adicionalmente, David también mostró especial actividad en los aspectos menos técnicos del proyecto, participando en las reuniones aportando ideas de diseño y participando en la recogida de sujetos para las pruebas de usuario, para su posterior realización y observación por él, entre otros.

Marcó un hito importante durante el comienzo del proyecto, investigando y consiguiendo la primera ejecución de un modelo largo de lenguaje pre entrenado en local, además de desarrollar un script base para realizar una comunicación sencilla por terminal con el modelo. Posteriormente, consiguió utilizar aceleración por GPU para explotar al máximo los recursos disponibles. Para la realización de esta tarea investigó principalmente *llamafile*⁶⁷, un tipo de archivo desarrollado por Mozilla que es capaz de contener un LLM para que sea ejecutado localmente. Dicha herramienta fue sugerida por el tutor al comienzo del proyecto, y marcó gran parte del inicio del mismo. Posteriormente, Miguel remataría esta tarea utilizando una herramienta diferente (Ollama).

David también realizó la primera implementación básica de un socket, que conectaba mensajes desde una terminal ejecutando un script de Python y la herramienta desarrollada en Unreal Engine, utilizando C++ y la librería *winsock2*. El resultado de dicha tarea fue una clase en C++ básica que utilizaba el patrón Singleton, capaz de conectarse a un socket dado, y un script de Python que actuaba como un servidor desde el que mandar y recibir cadenas de texto. Miguel fue el encargado de adaptar esta tarea a las necesidades del proyecto, juntando esta primera implementación con el script de comunicación con el modelo de lenguaje.

Tras varias recomendaciones del tutor, David fue el encargado de implementar la técnica de Retrieval Augmented Generation. Esta técnica se realiza a través de la generación de una base de datos que contenga información relevante para las preguntas realizadas por el usuario. Posteriormente, será guardada en una estructura de datos, tras haber sido convertido su información a embeddings. Se realiza una búsqueda en dicha estructura tratando de encontrar datos parecidos a los que ha introducido el usuario en su pregunta, intentando completar la respuesta. Estos datos son devueltos al modelo como parte del prompt del usuario, haciendo que la inteligencia artificial cuente con nueva información que desconocía anteriormente. Todo este

⁶⁷ Llamafile: <https://github.com/Mozilla-Ocho/llamafile>

pipeline, considerado actualmente estado del arte, fue implementado por David. Confeccionó la base de datos, pasando por hasta 4 iteraciones bien diferenciadas de la misma. Realizó un script de Python, que utiliza modelos de HuggingFace para el proceso de conversión de texto a embeddings y librerías como Chroma para guardar los datos y poder buscar cómodamente por ellos. Utilizando Langchain, realizó la comunicación con los modelos de lenguaje (Mistral:7B y Gemini-pro-v1.5). Utilizó también múltiples versiones de los prompts que sus compañeros habían generado hasta el momento, modificándolos e incluyendo sus datos como parte de datos del sistema que tiene el modelo de base. Al necesitar respuestas en formatos estructurados, también intentó forzar esta estructura utilizando funcionalidades de Langchain. Por desgracia, el resultado final de la implementación de esta técnica resultó de todo menos satisfactoria. Todas las respuestas generadas frente a las pruebas realizadas fueron completamente negativas. Se concluyó que RAG no era una inclusión adecuada para el marco de este proyecto y este trabajo resultó descartado.

Su contribución principal en Unreal Engine fue el desarrollo del sistema de reconocimiento de voz, que realizó a través de un script de Python, que utiliza una estructura similar al realizado por Miguel para la comunicación con el modelo largo de lenguaje. Centró su trabajo en una clase de C++ capaz de comunicarse con dicho script y que pudiera la grabación de la voz del usuario mediante entrada por teclado, evitando así un secuestro permanente de micrófono por parte del script de Python. El resultado de esta tarea fueron los archivos de código mencionados, que funcionaban como una transcripción en tiempo real de aquello que el jugador decía, permitiendo que, una vez terminada una frase, esta fuera comunicada a los NPCs de su alrededor. Sin embargo, el acercamiento inicial era un tanto diferente al resultado finalmente obtenido. En un principio, David quiso acercar la implementación de este sistema únicamente a Unreal Engine, utilizando la librería whisper-cpp. No obstante, la inclusión de dicha librería en el motor causó miles de errores de compilación por la configuración del mismo, por lo que resultó imposible realizar este acercamiento.

También realizó la implementación del sistema de Text-to-speech, utilizando un plugin propio de Unreal Engine, que permitió que los NPCs tuvieran voz dentro de la aplicación desarrollada.

La participación de este integrante en la realización de pruebas de usuario fue crucial para conseguir ampliar la muestra considerablemente. Estuvo presente durante la mayoría de pruebas realizadas, cumpliendo el papel de observador y buscando más sujetos de pruebas que quisieran aportar a los experimentos realizados. Cabe destacar también la participación en la memoria de David, que redactó en su completitud la introducción y estado del arte, además de sus contribuciones técnicas al proyecto. También se constituyó como un revisor activo y constante de todo el contenido redactado por sus compañeros, realizando correcciones y comentarios a diario, asegurándose, al igual que sus compañeros, de que el documento final contase con un formato correcto y entendible.

Miguel Ramírez Castrillo

Miguel ha sido un miembro activo del equipo, colaborando en multitud de tareas desde el principio del desarrollo del proyecto. Asimismo, también participó en todas las reuniones grupales propuestas y organizó algunas de ellas. Su trabajo se centró en la investigación de modelos largos de lenguaje y la implementación de la conexión entre los modelos y el motor de desarrollo.

Durante los comienzos del proyecto, sus tareas se centraron en la investigación de modelos largos de lenguaje. El trabajo de investigación se dividió en tres partes principales, en primer lugar, fue necesario buscar una herramienta que permitiese descargarse modelos pre entrenados y que contase con una librería que permitiese su uso desde código. Una vez elegida la librería Ollama, Miguel participó activamente en la investigación y pruebas para la elección del modelo largo de lenguaje mejor adaptado para la herramienta, concretamente Miguel hizo pruebas comparativas de rendimiento con los modelos de Mistral y DeepSeek (el de 14 billones de parámetros). En último lugar, también investigó sobre la manera de mejorar la eficiencia de ejecución de los modelos, mediante la aceleración por GPU.

Una vez terminada la fase de investigación, su tarea se centró en la implementación de un sistema de comunicación entre el modelo y el motor Unreal Engine 5. Antes de proceder con el desarrollo, realizó una investigación previa para determinar la manera más modular de implementar la conexión. Finalmente, optó por implementar un script de Python que implementase la librería de Ollama para ejecutar los LLMs. Para ejecutar el script y gestionar la comunicación entre el motor y dicho script, implementó en C++ el subsistema de Unreal, que mediante una llamada al sistema, era capaz de lanzar el script en segundo plano y establecer una conexión por puertos directa con el modelo. Además del código encargado de la ejecución y conexión con el script, implementó los despachadores de eventos capaces de notificar cuando se enviaba y recibía una respuesta del LLM.

Con el objetivo de parametrizar la ejecución del script de Python, Miguel extendió desde C++ los ajustes del proyecto de Unreal, lo que permitió incluir los parámetros que controlan la ejecución del script, pudiendo elegir el nombre del script que se quiere lanzar, así como los argumentos y sus respectivos valores con los que se pretende ejecutar. Para admitir estos argumentos adicionales, extendió el script de Python.

Durante el desarrollo, fue el encargado de desarrollar en C++ el sistema de señales, capaz de notificar de un evento a varios elementos de la escena. La clase incorpora varios eventos expuestos en Blueprints, para facilitar su desarrollo sin conocimientos de programación. A partir de la clase padre implementada en C++, Miguel creó una clase hija en Blueprints, a partir de la cual creó la señal para la alarma de emergencia. Se encargó de la implementación de la lógica de esta

última señal, y posteriormente probó su correcto funcionamiento con los personajes no jugadores de la escena.

A partir del sistema de composición de comportamiento desarrollado por uno de sus compañeros, Miguel se encargó de implementar el componente capaz de cargar y cambiar la malla de los objetos de la escena acorde al estado de este. Para la implementación, ideó una manera asíncrona y más eficiente que el método utilizado originalmente para el componente encargado de reproducción de sonidos, por lo que también se encargó de refactorizar parte del código de ese componente. Además, también fue uno de los encargados de la programación del sistema de generación de objetos con posiciones relativas, sus contribuciones a este sistema incluyen mejoras de rendimiento y mejoras a la elección de posición de generación permitiendo obtener resultados más consistentes y adaptados a las propiedades del objeto generado.

Otras aportaciones al proyecto incluyen varias mejoras de claridad visual para el proyecto que incluyen: la implementación de un material parametrizable utilizado para marcar el contorno de los objetos interactivos dentro de la escena, la extensión de la interfaz de usuario incluyendo cuadros de texto con la información del objeto apuntado y un texto indicativo del objeto que tiene equipado el jugador. Además, también contribuyó a mejorar la calidad de las respuestas del modelo ante ciertas interacciones mejorando algunas prompts ya existentes, así como ampliando descripciones de elementos interactivos.

Una vez terminado el desarrollo, se encargó de comprobar que el proyecto se podía empaquetar y compilar, corrigiendo todos los problemas que impedían el correcto funcionamiento del ejecutable.

Miguel fue un miembro activo durante el diseño y ejecución de las pruebas de usuario. Inició su trabajo aportando ideas para el diseño de las pruebas y posteriormente participó activamente en el diseño del nivel donde se llevarían a cabo. Aportó ideas para los cuestionarios de evaluación y fue el miembro encargado de crear los cuestionarios. Una vez iniciadas las pruebas, fue un miembro activo del equipo de evaluación y observación. Al terminar, se encargó de la transcripción de apuntes de observación y entrevistas para su posterior análisis. También fue uno de los encargados de analizar los datos obtenidos de las pruebas y redactar las conclusiones obtenidas.

Durante la redacción de la memoria, Miguel participó junto a sus compañeros activamente aportando contenido a diferentes capítulos. Al igual que sus compañeros, también se encargó de la revisión y corrección del contenido ya redactado.

Raúl Pérez Cogolludo

Raúl estuvo muy presente durante todo el desarrollo del proyecto, colaborando activamente, ayudando a sus compañeros a resolver problemas y asumiendo múltiples tareas en diferentes áreas. Desde las primeras etapas, participó tanto en la investigación y comparación de distintos modelos de lenguaje para el sistema, como en el diseño de la estructura de información que se compartiría con el LLM. Su labor se centró especialmente en el diseño e implementación de las interacciones y del flujo del funcionamiento del sistema, incluyendo la lógica de creación de los prompts, el manejo de las respuestas del LLM y la lógica y comportamientos asociados al sistema de máquinas de estado. Además, participó en el diseño y creación del escenario de juego final utilizado en las pruebas de usuario.

Durante la primera fase del proyecto Raúl se encargó junto al resto de compañeros del diseño de las estructuras de información que se intercambiarían la aplicación y el LLM, para conseguir un flujo de juego coherente y reactivo. También realizó un estudio comparativo entre distintos modelos de lenguaje, evaluando su rendimiento en términos de calidad de respuesta y eficiencia. Hizo pruebas comparativas de rendimiento con los modelos de Mistral, DeepSeek (de 14 billones de parámetros) y DeepSeek (de 32 billones de parámetros).

Después, Raúl se encargó íntegramente del desarrollo de la lógica e implementación del sistema de comunicación entre el jugador y los NPC mediante diálogos. Este sistema quedó encapsulado en la clase *InputWindow*, que contiene la interfaz de entrada para los mensajes del jugador. Posteriormente, David incorporaría a este sistema de comunicación el sistema de SpeechToText y TextToSpeech. Para llevar a cabo esta funcionalidad de comunicación, Raúl desarrolló también la clase *SimulationNPC*, que encapsula toda la lógica de comportamiento y personalidad de los NPC. Este sistema de comunicación constituye una de las interacciones más importantes de la aplicación, ya que es clave para la generación de narrativa y la creación de una experiencia inmersiva.

También, con el objetivo de aportar dinamismo y *vida* al entorno del juego, Raúl diseñó e implementó un sistema de máquinas de estado dirigido por el LLM. Este sistema fue incorporado a los NPCs, logrando que estos tuviesen un comportamiento más complejo, pero al mismo tiempo sencillo de manejar a través de la comunicación con el LLM. Raúl también se encargó de desarrollar los distintos estados que componen el comportamiento de los NPCs, permitiéndoles realizar un conjunto de acciones muy completo, incluyendo la posibilidad de estos poder interactuar tanto con otros personajes como con elementos del entorno. Estas interacciones se comunican al LLM y repercuten directamente en el estado del escenario.

Más tarde incorporó este sistema a los objetos interactivos, permitiendo clasificarlos como esenciales (con comportamiento definido por la máquina de estados) o no esenciales (gestionados directamente por el LLM).

A medida que el proyecto avanzaba, surgió la necesidad de centralizar la lógica de creación de prompts, ya que su distribución descentralizada generaba redundancia y dificultades de mantenimiento. Raúl diseñó e implementó íntegramente un sistema modular para la generación de prompts, encapsulado en la clase `PromptManager`, accesible desde cualquier módulo de la aplicación, sencillo de utilizar y fácilmente extensible.

De forma análoga al problema de la descentralización de la creación de prompts surgió el problema de la descentralización de la recepción de las respuestas del LLM y su correspondiente manejo. Para solventar este problema y de igual forma que el anterior, Raúl se encargó personalmente de crear la clase `GlobalManager` encargada de recibir las respuestas del LLM y —manteniendo la filosofía modular del `PromptManager`— manejar la respuesta aplicando los cambios pertinentes a las diferentes partes de la aplicación que lo necesitasen. Este sistema utiliza una cola de prioridad para determinar el orden en que deben procesarse las peticiones pendientes; una vez una respuesta es tratada, se elimina de la cola. Esto, junto a la lógica del `PromptManager` permitió aumentar significativamente la reutilización del código, su escalabilidad y su eficiencia a nivel de ingeniería.

Junto a las tareas de envío y manejo de mensajes con el LLM, Raúl colaboró con varios de sus compañeros en la redacción, revisión y optimización iterativa de los prompts enviados al LLM, con el objetivo de obtener respuestas de mayor calidad y coherencia.

Raúl también desarrolló el sistema de modo manual, que permite al usuario escribir en una ventana de entrada la interacción que está realizando al interactuar con un elemento. Asimismo, implementó la lógica que permite gestionar interacciones incluso cuando el usuario no tiene un objeto en la mano. Por otro lado, ante la complejidad que representaba la tarea de añadir componentes a los objetos debido a su extensión, Raúl ideó y desarrolló un sistema de doble iteración. Este sistema, al recibir una respuesta que crea o modifica un objeto, envía automáticamente una segunda solicitud al modelo de lenguaje para encargarse de añadir o modificar los componentes de los objetos implicados.

De cara a las pruebas de usuario, Raúl colaboró en el diseño y construcción del escenario de juego, así como en la implementación del bucle principal (transición entre el menú principal, el juego y el retorno al menú al ganar o perder una partida). Además, participó activamente en la primera sesión de pruebas, asegurando su correcta ejecución y el funcionamiento del sistema.

Finalmente, Raúl participó activamente en la redacción de la memoria del proyecto junto a sus compañeros, centrándose especialmente en las secciones relacionadas con las tareas en las que estuvo directamente involucrado. Al igual que el resto del equipo, colaboró en las revisiones continuas del documento, asegurándose de que el contenido estuviera correctamente redactado, estructurado y formateado conforme a los estándares establecidos.

Roi Quintas Diz

Roi tuvo una participación constante en el proyecto, colaborando desde el principio en múltiples áreas. Su trabajo se centró especialmente en el diseño de las interacciones, la implementación general en Unreal Engine y en todo el proceso de pruebas de usuario. Al igual que el resto de sus compañeros de equipo, estuvo presente tanto en el desarrollo como en las reuniones, aportando continuamente para que el proyecto avanzase correctamente y fomentando un ambiente de colaboración positiva dentro del equipo.

Participó desde las primeras etapas junto a todos los integrantes en el diseño inicial de los prompts que se enviarían al modelo largo de lenguaje, creando una base sólida sobre los objetivos del equipo y garantizando que todos tuviesen la misma estructura presente antes de empezar el proyecto. Además, colaboró en el diseño del sistema de interacciones entre objetos, proponiendo ideas junto al resto del equipo que luego se implementarían dentro del proyecto de Unreal Engine, como la creación, modificación y destrucción de objetos o las máquinas de estado, todo ello controlado mediante el LLM.

Mientras algunos de sus compañeros investigaban acerca de los LLMs que se podían utilizar y la conexión mediante Python, Roi empezó el desarrollo de las interacciones en Unreal Engine. Participó junto a Claudia en la investigación acerca de plugins que permitan el manejo del formato JSON mediante blueprints, para facilitar el prototipado rápido de la lógica de las interacciones. Posteriormente, realizó el componente mediante el cual el jugador puede coger y soltar objetos de la escena, que marcó un comienzo en el desarrollo de las interacciones. Además, desarrolló el componente “BP_ObjectInfo” que es contenido por todos los objetos interactivos de la escena y los registra automáticamente en el StateManager. También sirve para guardar la información de los objetos como su nombre y descripción.

Después, Roi realizó la mecánica para interactuar con el objeto al que está siendo apuntado y creó el sistema que genera una interacción en formato JSON entre dos objetos no esenciales. Tras esto, desarrolló la primera comunicación entre Unreal y el LLM en el sistema de blueprints. Finalmente, en esta fase, participó junto a Claudia en el desarrollo de la creación, modificación y destrucción de objetos no esenciales tras recibir una respuesta del LLM, para tener una primera versión funcional de una interacción, y concluyó ayudando con el prompt engineering relacionado a esta tarea.

Además, diseñó y creó el sistema de composición de comportamientos, que está compuesto de múltiples apartados. En primer lugar, participó en el diseño del sistema junto al resto del equipo, cuya idea se planteó inicialmente como una jerarquía de herencia. Sin embargo, se decidió cambiar este enfoque a una arquitectura basada en componentes, ya que permitía combinar comportamientos de manera sencilla y se adaptaba mejor al proyecto.

Una vez diseñado, creó la interfaz para poder desarrollar componentes, que sirve para el uso de forma genérica de los mismos. También se encargó del diseño y desarrollo de la serialización y deserialización de los componentes en formato JSON. Creó el prompt que se manda al LLM para gestionar los componentes, incluyendo el prompt engineering necesario. También desarrolló la deserialización de la respuesta obtenida, creando los nuevos componentes que el LLM devolvió o modificando los valores de los ya existentes, permitiendo que el modelo largo de lenguaje pueda controlar sus variables y, por ende, su funcionamiento. En resumen, apoyó en el diseño del sistema y desarrolló individualmente todo lo involucrado con el mismo. Por último, desarrolló los componentes Health y Projectile como ejemplo de uso, y el componente PlaySound utilizado en la versión final, que permite al LLM escoger un sonido en concreto y reproducirlo en un objeto.

Igualmente, implementó el patrón Observer para gestionar de forma eficiente las respuestas recibidas del LLM, como parte de la refactorización de código y creación del PromptManager que se estaba llevando a cabo. De esta manera, se mejoró considerablemente la organización del código, facilitando su uso posterior. Asimismo, estuvo involucrado constantemente en el pulido de la herramienta, dando lugar a su finalización.

En la etapa de evaluación, Roi participó en el diseño de las pruebas con Raúl y Miguel, que incluye la planificación de un nivel de prueba y la definición de los objetivos y preguntas de investigación. En el diseño del nivel, aportó ideas del objetivo del juego, objetos y NPCs que intervendrían en el mismo. Por otro lado, en la elaboración de las pruebas de usuario, participó tanto en los objetivos descritos y las preguntas de investigación pertinentes como en la plantilla de observación, el proceso de la entrevista y finalmente aportando preguntas en el cuestionario final. A continuación, implementó en Unreal esta escena con todos los objetos y descripciones, y participó en el pulido de esta para poder comenzar las pruebas con usuarios.

Roi estuvo presente durante todas las sesiones de prueba, rellenando las plantillas de observación, prestando atención a los usuarios y posteriormente realizando entrevistas acordes a lo observado, con el fin de recoger información precisa que pudiera contrastarse más adelante. Una vez finalizadas las pruebas, estuvo involucrado en su análisis, interpretando los datos obtenidos de los distintos métodos de evaluación mencionados y sacando conclusiones acerca del buen funcionamiento de la herramienta y posibles áreas de mejora.

Finalmente, Roi participó en la redacción de la memoria del proyecto junto a sus compañeros, centrándose especialmente en aquellas secciones que abarcaban las tareas en las que estuvo involucrado. Al igual que sus compañeros, contribuyó en las continuas revisiones de la memoria, para garantizar que todo esté correctamente redactado y formateado. En resumen, las aportaciones de todos los miembros del equipo, incluyendo a Roi, fueron fundamentales para que el proyecto se completara satisfactoriamente. El compromiso y la colaboración de todos los integrantes garantizaron el correcto desarrollo del trabajo en todas sus fases.

BIBLIOGRAFÍA

- [1] A. Vaswani *et al.*, «Attention Is All You Need», 2 de agosto de 2023, *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.
- [2] P. K. Kanithi *et al.*, «MEDIC: Towards a Comprehensive Framework for Evaluating LLMs in Clinical Applications», 11 de septiembre de 2024, *arXiv*: arXiv:2409.07314. doi: 10.48550/arXiv.2409.07314.
- [3] A. Scarlatos, N. Liu, J. Lee, R. Baraniuk, y A. Lan, «TRAINING LLM-BASED TUTORS TO IMPROVE STUDENT LEARNING OUTCOMES IN DIALOGUES».
- [4] J. Y. Bo, S. Wan, y A. Anderson, «To Rely or Not to Rely? Evaluating Interventions for Appropriate Reliance on Large Language Models», 9 de marzo de 2025, *arXiv*: arXiv:2412.15584. doi: 10.48550/arXiv.2412.15584.
- [5] S. S. Y. Kim, J. W. Vaughan, Q. V. Liao, T. Lombrozo, y O. Russakovsky, «Fostering Appropriate Reliance on Large Language Models: The Role of Explanations, Sources, and Inconsistencies», abr. 2025, pp. 1-19. doi: 10.1145/3706598.3714020.
- [6] J. Chen, X. Wu, T. Lan, y B. Li, «LLMER: Crafting Interactive Extended Reality Worlds with JSON Data Generated by Large Language Models», *IEEE Trans. Vis. Comput. Graph.*, vol. 31, n.º 5, pp. 2715-2724, may 2025, doi: 10.1109/TVCG.2025.3549549.
- [7] Miguel Escarda-Fernández *et al.*, «LLMs on the Fly: Text-to-JSON for Custom API Calling», Accedido: 24 de mayo de 2025. [En línea]. Disponible en: https://ceur-ws.org/Vol-3729/d3_rev.pdf
- [8] X. Tang *et al.*, «Struc-Bench: Are Large Language Models Really Good at Generating Complex Structured Data?», 4 de abril de 2024, *arXiv*: arXiv:2309.08963. doi: 10.48550/arXiv.2309.08963.
- [9] «Computing Machinery and Intelligence (1950)», *Essent. Turing*, pp. 433-464, sep. 2004, doi: 10.1093/oso/9780198250791.003.0017.
- [10] J. et al McCarthy, «A proposal for the Dartmouth Summer Research Project on Artificial Intelligence; 1955».
- [11] J. Weizenbaum, «ELIZA—a computer program for the study of natural language communication between man and machine».
- [12] D. E. Rumelhart, G. E. Hinton, y R. J. Williams, «Learning representations by back-propagating errors», *Nature*, vol. 323, n.º 6088, pp. 533-536, oct. 1986, doi: 10.1038/323533a0.
- [13] D. C. Ciresan, U. Meier, L. M. Gambardella, y J. Schmidhuber, «Deep Big Simple Neural Nets Excel on Handwritten Digit Recognition», *Neural Comput.*, vol. 22, n.º 12, pp. 3207-3220, dic. 2010, doi: 10.1162/NECO_a_00052.

- [14] T. Mikolov, K. Chen, G. Corrado, y J. Dean, «Efficient Estimation of Word Representations in Vector Space», 7 de septiembre de 2013, *arXiv*: arXiv:1301.3781. doi: 10.48550/arXiv.1301.3781.
- [15] I. Sutskever, O. Vinyals, y Q. V. Le, «Sequence to Sequence Learning with Neural Networks».
- [16] D. Bahdanau, K. Cho, y Y. Bengio, «Neural Machine Translation by Jointly Learning to Align and Translate», 19 de mayo de 2016, *arXiv*: arXiv:1409.0473. doi: 10.48550/arXiv.1409.0473.
- [17] R. Ranjan, Shailja Gupta, y Surya Narayan Singh, «A Comprehensive Survey of Bias in LLMs: Current Landscape and Future Directions», Accedido: 22 de mayo de 2025. [En línea]. Disponible en: <https://arxiv.org/pdf/2409.16430>
- [18] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, y A. Chadha, «A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications», 16 de marzo de 2025, *arXiv*: arXiv:2402.07927. doi: 10.48550/arXiv.2402.07927.
- [19] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, y S. Yao, «Reflexion: Language Agents with Verbal Reinforcement Learning», 10 de octubre de 2023, *arXiv*: arXiv:2303.11366. doi: 10.48550/arXiv.2303.11366.
- [20] X. Peng *et al.*, «Player-Driven Emergence in LLM-Driven Game Narrative», 3 de junio de 2024, *arXiv*: arXiv:2404.17027. doi: 10.48550/arXiv.2404.17027.
- [21] Y. Wu *et al.*, «SPRING: Studying the Paper and Reasoning to Play Games», 11 de diciembre de 2023, *arXiv*: arXiv:2305.15486. doi: 10.48550/arXiv.2305.15486.
- [22] Z. Kaiya *et al.*, «Lyfe Agents: Generative agents for low-cost real-time social interactions», 3 de octubre de 2023, *arXiv*: arXiv:2310.02172. doi: 10.48550/arXiv.2310.02172.
- [23] S. Hu *et al.*, «A Survey on Large Language Model-Based Game Agents», 30 de marzo de 2025, *arXiv*: arXiv:2404.02039. doi: 10.48550/arXiv.2404.02039.
- [24] L. M. Csepregi, «The Effect of Context-aware LLM-based NPC Conversations on Player Engagement in Role-playing Video Games».
- [25] G. Todd, S. Earle, M. U. Nasir, M. C. Green, y J. Togelius, «Level Generation Through Large Language Models», abr. 2023, pp. 1-8. doi: 10.1145/3582437.3587211.
- [26] T. Schaul, «A video game description language for model-based or interactive learning», en *2013 IEEE Conference on Computational Intelligence in Games (CIG)*, Niagara Falls, ON, Canada: IEEE, ago. 2013, pp. 1-8. doi: 10.1109/cig.2013.6633610.
- [27] A. Mehta, Y. Kunjadiya, A. Kulkarni, y M. Nagar, «Exploring the viability of Conversational AI for Non-Playable Characters: A comprehensive survey», en *2021 4th International Conference on Recent Trends in Computer Science and Technology (ICRTCST)*, feb. 2022, pp. 96-102. doi: 10.1109/ICRTCST54752.2022.9782047.
- [28] «LLM Evaluation | IBM». Accedido: 22 de mayo de 2025. [En línea]. Disponible en: <https://www.ibm.com/think/insights/llm-evaluation>

APÉNDICES

Apéndice A - Ejemplos de prompts y respuestas

En este apéndice podemos encontrar aquellos ejemplos extensos de los diferentes estados de los prompts que el proyecto ha atravesado, además de las respuestas generadas por el modelo de lenguaje; tanto si estas son erróneas como correctas. En el primer caso, se detallarán cuáles son dichos errores con respecto a las reglas establecidas durante el contenido de la memoria.

RAG

Dividiremos esta sección en tres partes, donde se verán primero los diferentes tipos de bases de datos generadas, a continuación los prompts para el modelo de lenguaje y, para finalizar, las respuestas a estos. El objetivo principal de esta sección es dar ejemplos claros de cuáles fueron las razones de la no utilización de una técnica tan famosa que aumenta, por lo general, la precisión en la salida de los modelos largos de lenguaje. Se verán varios ejemplos de respuesta completamente incoherentes con el objetivo del proyecto.

Bases de datos

Se ejemplifican las bases de datos utilizadas durante el proceso de pruebas de la técnica RAG en el marco de este proyecto.

Información en escena, formateada

```
components:
  - name: "Health"
    summary: "A tall plant with a trunk, branches and leaves"
    additional_info:
      - max_health: "Maximum health value."
        current_health: "Current health status."
      - damage_received: "Amount of health lost due to damage."
        health_healed: "Amount of health restored."

  - name: "Projectile"
    additional_info:
      - speed: "speed of the projectile"
      - generated__ranged_gun: "true or false wether it was generated
by a gun or not"
        gun: "Name of the ranged gun that was used, if any"

objects:
  - name: "Tree"
    summary: "A tall plant with a trunk, branches and leaves"
    components:
```

- name: "Health"
summary: "Keeps track of the durability of the object"
attributes:
max_health: "Maximum health value."
current_health: "Current health status."
damage_received: "Amount of health lost due to damage."
health_healed: "Amount of health restored."
- name: "Axe"
summary: "A sharp-bladed tool used primarily to chop or split wood."
components: []
- name: "CarKey"
summary: "Its used to lock or unlock a Car object, depending on its state"
compontens: []
- name: "Car"
summary: "Its a vehicule capable of transporting multiple people. It needs a key to run."
components: []

Información de escena, en lenguaje natural

A Health component is used to indicate the health and damage of an object. Its attributes are max_health, which tells us the maximum health value for the object, current_health, which represents the current health status of the object, damage_received, which indicates the amount of health lost due to damage, and health_healed, which represents the amount of health restored to the object.

A Projectile component is used to define the properties of a projectile. Its attributes are speed, which indicates the speed at which the projectile moves, generated__ranged_gun, a boolean value (true or false) that tells us whether the projectile was generated by a gun or not, and gun, which is the name of the ranged gun used to generate the projectile, if applicable.

The Tree object is a tall plant with a trunk, branches, and leaves. It has the following component: Health component, which keeps track of the durability of the object. Its attributes are max_health, which tells us the maximum health value for the object (100), current_health, which represents the current health status of the object (100), damage_received, which indicates the amount of health lost due to damage (0), and health_healed, which represents the amount of health restored to the object (0).

The Axe object is a sharp-bladed tool used primarily to chop or split wood. It does not have any components associated with it.

The CarKey object is used to lock or unlock a Car object, depending on its state. It does not have any components associated with it.

The Car object is a vehicle capable of transporting multiple people. It needs a key to run. It does not have any components associated with it.

Respuestas de ejemplo, formateadas

Se incluyen 3 ejemplos relevantes por los motivos descritos antes de cada uno de ellos.

En este caso, se ve una interacción de la que surge un objeto completamente nuevo y se destruye otro como causa de la interacción

```
{ "TASK1": {
  "llm_objects": {
    "new": [
      {
        "name": "Wood_Debris",
        "summary": "It is a small piece of wood created from
the cut made by the axe on the tree.",
        "components": [
          {
            "name": "Health",
            "additional_info": {
              "max_health": 0,
              "current_health": 0,
              "damage_received": 0,
              "health_healed": 0
            }
          }
        ]
      }
    ],
    "modify": [],
    "destroy": [
      {
        "name": "Tree",
        "summary": "It is a damaged tree with a big cut on
it.",
        "components": [
          {
            "name": "Health",
            "additional_info": {
              "max_health": 100,
              "current_health": 50,
              "damage_received": 50,
              "health_healed": 0
            }
          }
        ]
      }
    ]
  }
}
```

```

    ],
    "action": "The axe was used to cut the tree."
  }}
}

```

En el siguiente, se puede observar cómo el LLM ha respondido a una interacción entre dos objetos que no tendría sentido que se destruyeran, sino que uno modificara al otro.

```

{
  "TASK1": {
    "llm_objects": {
      "new": [],
      "modify": [
        {
          "name": "Car",
          "summary": "It is a car that has been unlocked using the CarKey",
          "components": []
        }
      ],
      "destroy": [],
      "action": "The CarKey was used to interact with the Car, resulting in the Car being unlocked."
    }
  }
}

```

Se ve a continuación un ejemplo de una interacción que no resulta en nada específico, ya que no tendría sentido que nada sucediera.

```

{
  "TASK1": {
    "llm_objects": {
      "new": [],
      "modify": [],
      "destroy": [],
      "action": "The CellKey was used to interact with the Tree, but since a key and a tree have no functional interaction, nothing significant happened."
    }
  }
}

```

Respuestas de ejemplo, en lenguaje natural

Se incluyen los mismos tres ejemplos anteriores, pero en lenguaje natural. Se omite en este caso la explicación de su relevancia.

The axe was used to cut the tree, damaging it and creating a piece of wood debris.

The CarKey was used to unlock the Car.

The CellKey was used on the Tree, but nothing happened because they don't interact.

Prompts generados

Como se indicó durante el apartado de RAG en el capítulo 4, se utilizaron varios prompts diferentes, utilizando uno como base, pero modificando palabras o frases clave. Lo que nunca fue modificado fue el siguiente texto introductorio, que era el que siempre daba resultados correctos en las pruebas generales antes de utilizar esta técnica:

You are participating in a multi-step simulation, located in a prison, where the player is a prisoner and tries to escape. You will be given several tasks, each marked with TASK followed by a number, such as TASK1, TASK2, etc.

You MUST return a single JSON object with the following structure:

```
{{
  "TASK1": {{ /* Your response to TASK1 here / }},
  "TASK2": {{ / Your response to TASK2 here */ }},
  ...
}}
```

. Each task's result must be a valid JSON object embedded under its corresponding TASK key. DO NOT merge tasks together.

Follow the expected schemas for each task. Use the output of earlier tasks to help inform or complete later tasks.

Do NOT invent keys or metadata that are not requested
Here are the tasks you must complete.:

Sin embargo, sí que fue cambiada su posicionamiento en la generación del prompt. En la mayoría de los casos, se utilizó como prompt perteneciente al contexto del sistema, que siempre sería recordado por el modelo; mientras que en otras ocasiones fue utilizado como parte de cada uno de los prompts individuales. La generación resultante se mostraba indiferente al posicionamiento de esta parte introductoria.

Las siguientes partes del prompt, al utilizar como base un prompt de interacción de un objeto en la mano del jugador con otro de la escena, constaba de varias partes claramente diferenciadas.

La primera de ellas habla de la introducción, y de cuál es el formato a seguir para que se complete la TASK especificada:

"TASK1": {In this task I will give you the scene described in a json format and an action that has happened, and you have to return the following json structure, filling the arrays new, modify and destroy with objects that reflect the consequences of the action.
"new": Include only completely new, physically separate objects, that can be used later on, created as a result of the action.
Examples: debris, broken parts, tools knocked loose, or any object created due to transformation or destruction.

"modify": If an object changes (is scratched, damaged, dirty, stuck, altered in some way), update its summary while keeping its original name.

"destroy": If the object is fully destroyed, removed, consumed, or irreversibly transformed, include it here. Keep the original name and summary to describe what happened.

"action: Write a description of the interaction that occurred between the two objects.

No object can appear in more than one array, for example, if an object is in the "new" array, it cannot appear in "modify" or "destroy."

Fill out all applicable arrays and keep them even if they are empty. So, you must fill the arrays and return ONLY this structure, do NOT include any extra metadata:

```
{
  "llm_objects":
  {
    "new":[],
    "modify":[],
    "destroy":[] ,
    "action":""
  }
}.
```

Esta es la versión original utilizada, donde se describe ampliamente las tareas a realizar, con restricciones específicas y todo escrito en lenguaje natural. También se incluía un ejemplo en la sección de creación de objetos nuevos y modificados.

La siguiente versión prescindió de muchos de los ejemplos dados, tratando de fijar más la respuesta del LLM y no generar ningún tipo de sesgo. Como consecuencia, la extensión del prompt fue reducida notablemente.

"TASK1": {I will give you the scene described in a json format and an action that has happened, and you have to answer in the following json structure:

"new": New, physically separate objects, that can be used later on, created as a result of the action.

"modify": If an object changes, update its summary while keeping its original name.

"destroy": If the object is fully destroyed, removed, consumed, or irreversibly transformed, include it here. Keep the original name and summary to describe what happened.

"action: Write a description of the interaction that occurred between the two objects.

No object can appear in more than one array."

You must fill the arrays and return ONLY this structure:

```
{
```

```
  "llm_objects":
  {
    "new":[],
    "modify":[],
```

```

        "destroy":[] ,
        "action":""
    }
}.

```

Se optó por este enfoque porque los ejemplos ya iban a estar incluidos en la base de datos del RAG, haciendo que innecesaria la duplicación de dicho contenido.

Otra de las versiones utilizadas contaba con que al modelo se le pasase la estructura de json como parte del prompt, a través de una función propia de Langchain. Esa estructura se define a través de una serie de clases de Python, con una estructura muy concreto:

```

class TaskOutput(RootModel[Dict[str, Task]]):
    root: Dict[str,Task]

class Task(BaseModel):
    llmObject: LLMObjectsList

class LLMObjectsList(BaseModel):
    new: List[LLMObjectOutput] = Field(description="List of new objects
to generate in the simulation")
    modify: List[LLMObjectOutput] = Field(description="List of objects
to modify that already exist in the simulation")
    destroy: List[LLMObjectOutput] = Field(description="List of objects
that already exist in the simulation to destroy")
    action: str = Field(description="A description of what happened")

class LLMObjectOutput(BaseModel):
    name: str = Field(description="The name of the object")
    summary: str = Field(description="Summary of the object")

class ComponentOutput(BaseModel):
    name: str = Field(description="Component name")
    additional_info: Dict[str, Any]
    components: list[ComponentOutput] = Field(description="list of
components to add or modify")

```

Esta estructura es la misma que se le pide en los prompts anteriores, pero definida de una forma diferente. Con la ayuda de Langchain, se pensó que podría mejorar la generación de la respuesta.

Este fue la sección del prompt utilizada acompañado de esta estructura:

```

"TASK1": {I will give you the scene described in a json format and an
action that has happened, and you have to answer in the json structure
that was provided before. No object can appear in more than one array."

```

Como siguiente parte del prompt, podemos ver la descripción de la escena en cuestión. Esta otra de las secciones que nunca varió entre las diferentes iteraciones del prompt.

```

This is the scene and the action performed:
{
    "Instigator":

```

```

{
  "name": "CarKey",
  "summary": "Its a Car Key",
  "components": []
},
"Receiver":
{
  "name": "Car",
  "summary": "It is a car",
  . . "components": []
},
"action": "interaction between CarKey and Car"
}

```

La siguiente sección comentada es donde se describe cómo el LLM debe generar los componentes asociados a cada uno de los objetos. El prompt original es el siguiente:

You SHOULD NOT ADD ANY COMPONENTS IF THEY ARE NOT STRONGLY RELATED to the object.

You can leave the "components" array empty if they do not fit in the object. However, these are the only components you may add (if it makes sense and fulfills their utility), any other would not be supported. You should fill the "components" array of the new or modified object with any of the following json objects:

COMPONENTS YOU SHOULD USE:

```

[
{
  "name": "Health",
  "additional_info":
  {
    "max_health": 100,
    "current_health": 100,
    "damage_received": 0,
    "health_healed": 0
  }
}
]{
  "name": "Projectile",
  "additional_info":
  {
    "speed": 0,
    "Generated by a ranged gun": false,
    "Name of the ranged gun that was used": "Insert name"
  }
}
]

```

Every object must have a components array filled with the jsons objects mentioned before. Try to use them in every interaction that has sense.

Example:

```

{"name": "object name", "summary": "summary of the object",
"components": [{"name": "component name", "additional_info": {"key":
"value"}}]}

```

These are the explanations about how should be used the components, THIS IS IMPORTANT because it will make it easier for you to choose when should they be added:

The component "Health": It must be added/modified in damaged objects. You have to specify how much is the damage received or the health healed in this interaction. If it has not been affected you can ignore both parameters. max_health will only be stored the first time, with default value 100 if you do not put another value. Consider not to deal max_health quantity of damage in an interaction, cause the object will be destroyed.

The component "Projectile": It must be only added to bullets spawned from a ranged gun. It must no be added in other circumstances. Parameter "speed" must be greater than 0.

}

Se siguió un acercamiento parecido al del utilizado con la descripción de la tarea, donde se fueron reduciendo las descripciones y especificaciones más concretas, siendo sustituidas por los datos guardados en la vector database generada.

You SHOULD NOT ADD ANY COMPONENTS IF THEY ARE NOT STRONGLY RELATED to the object.

You can leave the "components" array empty if they do not fit in the object. These are the only components you may add, any other would not be supported.

COMPONENTS YOU MAY USE:

```
[
{
  "name": "Health",
  "additional_info":
  {
    "max_health": 100,
    "current_health": 100,
    "damage_received": 0,
    "health_healed": 0
  }
}{
  "name": "Projectile",
  "additional_info":
  {
    "speed": 0,
    "Generated by a ranged gun": false,
    "Name of the ranged gun that was used": "Insert name"
  }
}
]
```

These are the explanations about how should be used the components:

"Health": It must be added/modified in damaged objects. You have to specify how much is the damage received or the health healed in this interaction. If it has not been affected you can ignore both parameters.

max_health will only be stored the first time, with default value 100 if you do not put another value. Consider not to deal max_health quantity of damage in an interaction, cause the object will be destroyed.
 "Projectile": It must be only added to bullets spawned from a ranged gun. It must no be added in other circumstances. Parameter "speed" must be greater than 0.
 }

Una de las versiones de la base de datos incluía las especificaciones de cada componente y sus casos de uso, por lo que la siguiente versión de esta sección obvió completamente la parte final.

You SHOULD NOT ADD ANY COMPONENTS IF THEY ARE NOT STRONGLY RELATED to the object.
 You can leave the "components" array empty if they do not fit in the object. These are the only components you may add, any other would not be supported.

Para las pruebas realizadas se utilizaron todas las combinaciones posibles de las secciones mencionadas hasta ahora, incluyendo la sustitución de dichas partes por las bases de datos correspondientes.

Finalmente, detallamos el prompt más largo generado.

Here are the tasks you must complete.:

"TASK1": {IO n this task I will give you the scene described in a json format and an action that has happened, and you have to return the following json structure, filling the arrays new, modify and destroy with objects that reflect the consequences of the action.

"new": Include only completely new, physically separate objects, that can be used later on, created as a result of the action.

Examples: debris, broken parts, tools knocked loose, or any object created due to transformation or destruction.

"modify": If an object changes (is scratched, damaged, dirty, stuck, altered in some way), update its summary while keeping its original name.

"destroy": If the object is fully destroyed, removed, consumed, or irreversibly transformed, include it here. Keep the original name and summary to describe what happened.

"action: Write a description of the interaction that occurred between the two objects.

No object can appear in more than one array, for example, if an object is in the "new" array, it cannot appear in "modify" or "destroy."

Fill out all applicable arrays and keep them even if they are empty. So, you must fill the arrays and return ONLY this structure, do NOT include any extra

```
metadata:{  "llm_objects":{      "new":[],      "modify":[],      "destroy":[], "action":  }}
```

This is the scene and the action performed:

```
{
```

```

    "Instigator":
    {
        "name": "CarKey",
        "summary": "Its a Car Key",
        . "components": []
    },
    "Receiver":
    {
        "name": "Car",
        "summary": "It is a car",
        "components": []
    },
    "action": "interaction between CarKey and Car"
}

```

Now, you should fill the json structure provided with what happened between the Instigator and the Receiver.

You SHOULD NOT ADD ANY COMPONENTS IF THEY ARE NOT STRONGLY RELATED to the object.

You can leave the "components" array empty if they do not fit in the object. However, these are the only components you may add (if it makes sense and fullfills their utility), any other would not be supported. You should fill the "components" array of the new or modified object with any of the following json objects:

COMPONENTS YOU SHOULD USE:

```

[
{
    "name": "Health",
    "additional_info":
    {
        "max_health": 100,
        "current_health": 100,
        . "damage_received": 0,
        "health_healed": 0
    }
}]{
    "name": "Projectile",
    "additional_info":
    {
        "speed": 0,
        ..... "Generated by a ranged gun": false,
        ..... "Name of the ranged gun that was used": "Insert name"
    }
}
]

```

Every object must have a components array filled with the jsons objects mentioned before. Try to use them in every interaction that has sense.

Example:

```

{"name": "object name", "summary": "summary of the object",
"components": [{"name": "component name", "additional_info": {"key":
"value"}}]}

```

These are the explanations about how should be used the components, THIS IS IMPORTANT because it will make it easier for you to choose when should they be added:

The component "Health": It must be added/modified in damaged objects. You have to specify how much is the damage received or the health healed in this interaction. If it has not been affected you can ignore both parameters. max_health will only be stored the first time, with default value 100 if you do not put another value. Consider not to deal max_health quantity of damage in an interaction, cause the object will be destroyed.

The component "Projectile": It must be only added to bullets spawned from a ranged gun. It must no be added in other circumstances. Parameter "speed" must be greater than 0.

También el prompt más corto generado:

Here are the tasks you must complete.:

"TASK1": {This is the scene and the action performed:

```
{
  "Instigator":
  {
    "name": "CarKey",
    "summary": "Its a Car Key",
    "components": []
  },
  "Receiver":
  {
    "name": "Car",
    "summary": "It is a car",
    "components": []
  },
  "action:": "interaction between CarKey and Car"
}
```

Now, you should fill the json structure provided with what happened between the Instigator and the Receiver based in the context provided before.

}

Respuestas del modelo

A continuación se muestran las respuestas del modelo utilizando esta técnica. Cabe destacar que no son todas las pruebas realizadas, si no las más relevantes. Toda ellas no cumplen con las características pedidas.

Todas las respuestas generadas respondían a un prompt de interacción entre CarKey y Car, por lo que la siguiente respuesta no es capaz de cumplir este objetivo. Además, alucina ciertos tipos de componentes que nunca fueron determinados en ningún lugar. También, no responde a la TASK correspondiente, ya que la TASK pedida era la de número 1.

```

{
  "TASK2": {
    "llm_objects": {
      "new": [
        {
          "name": "Axe",
          "summary": "The Axe was found in the cell, it can be used to
cut down trees or break objects.",
          "components": [
            {
              "name": "Durability",
              "additional_info": {
                "max_durability": 100,
                "current_durability": 100,
                "durability_used": 0
              }
            },
            {
              "name": "Damage",
              "additional_info": {
                "min_damage": 5,
                "max_damage": 10,
                "average_damage": 7.5
              }
            }
          ]
        },
        {
          "name": "Wood_Debris",
          "summary": "The Wood Debris was created from the cut made by
the Axe on the tree.",
          "components": [
            {
              "name": "Health",
              "additional_info": {
                "max_health": 0,
                "current_health": 0,
                "damage_received": 0,
                "health_healed": 0
              }
            },
            {
              "name": "Usefulness",
              "additional_info": {
                "usefulness_level": 1,
                "possible_uses": ["Fuel for fire", "Bartering item"]
              }
            }
          ]
        }
      ],
      "modify": [],

```

```

    "destroy": []
  },
  "action": "The Prisoner found an Axe in the cell and used it to cut
down a tree, creating Wood Debris."
}
}

```

La siguiente respuesta alucina nuevos campos del JSON que no existían: “Explanation” y “NewState”, además de añadir componentes erróneos y responder a una TASK inexistente con objetos que no fueron mencionados anteriormente.

```

{
  "TASK3": {
    "llm_objects": {
      "new": [],
      "modify": [
        {
          "name": "CellKey",
          "summary": "The CellKey is now in the possession of a fellow
prisoner.",
          "components": [
            {
              "name": "State",
              "additional_info": {
                "current_state": "InUse"
              }
            },
            {
              "name": "Location",
              "additional_info": {
                "location": "Prisoner's Cell"
              }
            }
          ]
        }
      ],
      "destroy": [],
      "action": "The fellow prisoner takes the CellKey from the CellKey,
preparing to use it for their own escape attempt."
    },
    "NewState": "InUse",
    "Explanation": "By giving the CellKey to a fellow prisoner, its
state changes from 'Idle' to 'InUse'."
  }
}

```

La siguiente respuesta es, con diferencia, la que menos sentido tiene de todas. No solo se responden a Tasks nuevas que ni siquiera existen, si no que los objetos no tienen que ver con lo pedido y es capaz de alucinar una historia en la que uno de ellos cobra sentencia y escapa de la cárcel, generando un plan por el camino.

```

{

```

```

"TASK2": {
  "llm_objects": {
    "modify": [
      {
        "name": "Car",
        "summary": "The CellKey managed to find a tool in the prison
and used it to fix the damaged lock on the car.",
        "components": [
          {
            "name": "Lock",
            "additional_info": {
              "status": "Repaired"
            }
          }
        ]
      }
    ],
    "action": "The CellKey successfully repaired the lock on the car,
making it possible to use it."
  },
  "TASK3": {
    "llm_objects": {
      "modify": [],
      "destroy": [],
      "action": "The CellKey managed to find a map of the prison's
layout in the library. The map shows possible escape routes and
locations of guards."
    },
    "TASK4": {
      "llm_objects": {
        "new": [
          {
            "name": "Escape Plan",
            "summary": "The CellKey studied the prison's layout map and
came up with a plan to escape through the ventilation system.",
            "components": []
          }
        ],
        "action": "The CellKey has developed an escape plan using the
information from the prison's layout map."
      },
      "TASK5": {
        "llm_objects": {
          "modify": [],
          "destroy": [],
          "action": "The CellKey gathered materials needed for the
ventilation system escape, such as a makeshift ladder and a breathing
mask."
        }
      }
    }
  }
}

```

```

    },
    "TASK6": {
      "llm_objects": {
        "modify": [
          {
            "name": "Escape Plan",
            "summary": "The CellKey has added the materials needed for the
ventilation system escape to their plan.",
            "components": []
          }
        ],
        "action": "The CellKey has updated their escape plan with the
necessary materials."
      }
    },
    "TASK7": {
      "llm_objects": {
        "destroy": [],
        "action": "The CellKey executed their escape plan by using the
makeshift ladder and breathing mask to navigate through the ventilation
system."
      }
    },
    "TASK8": {
      "llm_objects": {
        "new": [
          {
            "name": "Freedom",
            "summary": "The CellKey successfully escaped from the prison
using the ventilation system.",
            "components": []
          }
        ],
        "action": "The CellKey has achieved their goal and gained
freedom."
      }
    }
  }
}

```

La respuesta generada con más coherencia, aunque todavía errónea por fallar en la elección de objetos, además de incluir el JSON en formato *Markdown*⁶⁸, es la siguiente:

```

```json
{
 "TASK1": {
 "llm_objects": {
 "modify": [],

```

---

<sup>68</sup> <https://www.markdownguide.org/basic-syntax/>

```

"destroy": [],
"action": "You try to use the tree to fix the car, but it's just a
damaged tree and doesn't help in fixing the car."
}
}
}
```

```

Cuando fue introducido el formato YAML como parte de la base de datos, el resultado de las respuestas sufrió un gran sesgo en su propio formato, llegando a generar la siguiente estructura (completamente errónea):

```

```json
{
 "TASK1": {
 "inventory": [
 {
 "name": "Spoon",
 "description": "A basic eating utensil."
 },
 {
 "name": "Blanket",
 "description": "Provides a moderate amount of warmth."
 }
],
 "TASK2": {
 "objects": [
 {
 "name": "Car",
 "description": "Its a vehicule capable of transporting multiple people.
It needs a key to run.",
 "obtainable": false,
 "interactable": true
 },
 {
 "name": "CarKey",
 "description": "Its used to lock or unlock a Car object, depending on
its state",
 "obtainable": false,
 "interactable": true
 },
 {
 "name": "Axe",
 "description": "A sharp-bladed tool used primarily to chop or split
wood.",
 "obtainable": true,
 "interactable": true
 }
]
 },
 "TASK3": {}
 }
}
```

```

```
}...
```

Finalmente, terminaremos esta sección del apéndice con el otro tipo de respuesta generado, donde se mezcla el formato JSON con el YAML, se incluyen frases antes y durante la estructura generada y, aunque bien numeradas, se responden a Tasks adicionales y desconocidas:

Here's how you can structure your response:

```
```json
{
 "TASK1": {
 "Car": {
 // Assume Car keys are stored in a specific location in the
prison
 "location_in_prison": "Tool Shed"
 },
 "Tree": {
 // The tree is healthy because it hasn't been damaged or
healed yet
 "Health Component": {
 "max_health": 100,
 "current_health": 100,
 "damage_received": 0,
 "health_healed": 0
 }
 },
 "CarKey": {
 // The Car key can be found near the tree to add a hint for
the player
 "location_in_prison": "Near Tree"
 },
 "Axe": {
 // Assume Axe is located in the same place as the car key
for simplicity
 "location_in_prison": "Near Tree"
 }
 },
 "TASK2": {
 // Let's assume the player finds the Car Key and uses it to take
the Car
 "Car": {
 "can_be_used": true,
 "current_occupants": 0
 },
 // Using the Axe, the player can damage the tree to weaken its
Health Component
 "Tree": {
 "Health Component": {
 "current_health": 85,
 "damage_received": 15
 }
 }
 }
}
```

```

 }
 },
 // You can continue this pattern for future tasks. For example:
 // "TASK3": {...} etc.
}
...

```

Tras la revisión de las respuestas generadas, se vio la completa inviabilidad de la técnica de RAG para el marco del trabajo realizado.

## Ejemplo de respuesta de DeepSeek

Se quiere evidenciar con este apartado la excesiva cantidad de razonamiento que el modelo presenta, haciendo que la latencia de este aumente considerablemente en comparación a los demás modelos evaluados.

La **petición** realizada al modelo fue la siguiente:

Given these basic shapes:

```

Cone
Cube
Cylinder
Capsule
Plane
Sphere
Torus

```

And this simple json structure:

```

{
 "objects": [
 {
 "name": "",
 "shape": "",
 "summary": ""
 }
]
}

```

Copying the same json structure as before, create a new object that can be found inside a prison cell and select the best shape from the list that could resemble the new object

La **respuesta** del modelo:

Okay, so I need to figure out what shape fits into a prison cell based on the given shapes: Cone, Cube, Cylinder, Capsule, Plane, Sphere, Torus. The example was a Chair with the structure matching the original JSON format.

First, I should think about common objects found in a prison cell and which shapes they might take. Maybe things like tables, beds, storage

containers, or maybe even items like chairs, as shown before. Each of these has to fit into one of the given shapes.

Let me list out possible items: bed, table, chair, lamp, mirror, container, etc. Then match each to a shape:

- Bed: Could be a rectangular prism (like a cuboid), but that's not listed; closest is Cube.
  - Table: Often square or rectangular, so again maybe Cube?
- Wait, the example used a Chair as a cube, so perhaps other items can also use Cube.

But let me check each possible item:

- Chair was Cube. Maybe Bed could be Cylinder? A round bed might fit that.
- Table: If it's a wooden table with legs, maybe Cone (like the leg part) or Sphere if rounded. But more likely a Cuboid shape for the top and Cylinder for the legs. Since Cylinder is an option, perhaps that's better.
- Lampshade could be Cone-shaped.
- Storage container might be Cuboid but not in the list; so maybe Cube isn't perfect either.

Wait, maybe I should think differently: each item should fit into one of the given shapes. The example used a Chair with structure matching JSON, which was Cube for the seat and Plane or Sphere for legs. So perhaps I can use similar logic.

Let me pick an object like a Table. Maybe it has a rectangular top (which could be a Cuboid but not listed) or maybe a square top, so Cube is possible if we're simplifying.

Alternatively, thinking of a more specific shape: a table might have four legs which are Cylinders. So perhaps Cylinder for the legs and Cube for the tabletop.

But since each object has one name, shape, and summary, I need to pick one per item. Let's go with Table as the example.

So Table could be represented as:

```
{
 "name": "Table",
 "shape": "Cuboid",
 "summary": "A flat, rectangular structure typically found in a prison cell."
}
```

But since Cuboid isn't an option, maybe Cube is acceptable if we assume it's square or rounded.

Alternatively, maybe the storage container. If it's cylindrical, like a can, then Cylinder would fit.

Wait, perhaps better to go with an item that naturally fits one of the given shapes without forcing a shape name not in the list. For example:

- Chair: Cube
- Table: Maybe Cuboid but if forced to choose from given options, perhaps Cylinder for legs or Sphere? Wait, maybe a table top is a Circle (Sphere would be 3D sphere). So maybe Cylinder as the top.

But I think the best approach is to pick an item and match it to one of the provided shapes. Let's go with Table.

Table: The top could be a rectangle, which can be approximated by Cube if we're using basic shapes for simplicity in a prison setting where materials might be standard.

So:

```
{
 "name": "Table",
 "shape": "Cube",
 "summary": "A flat, rectangular structure placed horizontally in a
cell."
}
```

But actually, tables are usually made of wood or metal and have four legs. If we model each part separately, maybe:

- Tabletop: Cube
- Legs: Cylinder

But since the instruction is to create a new object within a prison cell, perhaps choosing just one shape per item.

Alternatively, maybe better to pick something else. Let's go with Storage container as an example, which can be Cuboid or maybe Sphere if it's a ball-shaped container.

Wait, Sphere isn't typically used for storage containers; they are usually cylinders or boxes.

So perhaps:

```
{
 "name": "Storage Container",
 "shape": "Cube",
 "summary": "A cubic container used to hold items within a prison
cell."
}
```

But again, maybe more accurate shapes exist. Alternatively, a table with legs as Cylinder and tabletop as another shape, but since each object needs one shape, perhaps just the tabletop.

Alternatively, thinking of a Chair was an example; so let's pick something else, like a Lampshade, which is typically Cone-shaped.

So:

```
{
 "name": "Lampshade",
 "shape": "Cone",
 "summary": "A cone-shaped light fitting mounted in a prison cell."
}
```

That makes sense. Another option could be the mirror on the wall, perhaps Sphere if it's a globe or Cylinder if it's a flat mirror.

So:

```
{
 "name": "Mirror",
 "shape": "Cylinder",
 "summary": "A cylindrical mirror hanging from the ceiling of a prison cell."
}
```

Or maybe a light bulb holder as Cylinder since it's often round.

But I think choosing Lampshade as Cone is more unique and fits well within the list. So that would be my selection.

Final Answer:

```
{
 "name": "Lampshade",
 "shape": "Cone",
 "summary": "A cone-shaped light fitting mounted in a prison cell."
}
```